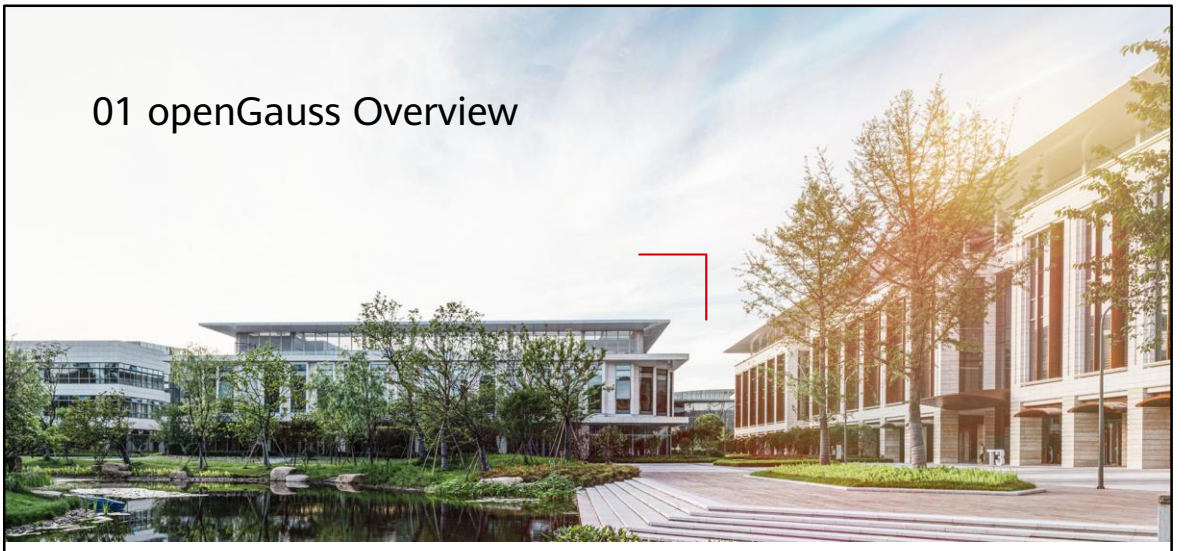
01 openGauss Overview



HUAWEI

Foreword

- Database emerged in the 1960s as a new discipline for automating information management, and has since become an important branch of computer science. Data processing and database technology have found their way into more and more computer applications as such applications have developed. And while a database is a product of data management, data management is the core task involving a database. It involves data classification, organization, encoding, storage, retrieval, and maintenance. This section describes the development and features of the openGauss database.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

Upon completion of this course, you will be able to:

- Understand what a database is, as well as the history behind the development of the technology.
- Be familiar with how the architecture of relational databases has evolved.
- Understand the main applications for relational databases.
- Understand where the openGauss database is positioned, and the history of its development.
- Understand the technical specifications and basic functions of openGauss.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **Database Overview**
2. Introduction to openGauss
3. openGauss Technical Specifications
4. Basic Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

What Is a Database?

- Data is facts or statistics used to represent something for observation or analysis.
- Data can represent carrier of information as symbols, text, numbers, voices, images, or videos. Data and information are inseparable and data is basically a form of information. Data itself is meaningless. Data becomes information only when it is put to use.
- Data in a computer system is expressed as various letters, digits, sounds, graphs, images, and more, which are formed through binary units "1" and "0". Binary is converted into more conventional forms of data with processing.
- Forms
  - **Digital data**: data consisting of discrete symbols, characters, and numerals. For example, statistics or measurement data stored on a computer.
  - **Analog data**: refers to a continuous range of values over an interval. This data takes a physical form, such as video, image, text, or audio.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Symbols that represent things are called data. Data may be represented in various forms: a number, or text, a graph, an image, a sound, a language, or anything really. Data is recorded in a database. For example, a student management database would include information about a student, including the student number, name, gender, age, place of origin, and phone number.
- Information that can be read and used in society and production is extracted from data through processing. It is for this information that we have data. Unprocessed data is akin to a raw material, and its value lies in its ability to record facts about the real world.

Database

A database is an, often large, collection of organized data that is stored in a computer, often for a long time, and with the ability to be accessed. Data in a database is organized, described, and stored according to a data model. The data in a database is efficient, relatively independent, and scalable, and can be accessed by various users.



Permanent

Organized

Sharable

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- A database is a collection of organized and accessible data stored in a computer, generally for a long time. Generally speaking, a database is a place to store data, just as a refrigerator is to store food. You might not know it, but everyone is using databases all the time. When we look up names in the phone book, we're using a kind of database. When searching the Internet, you are also using a database. In addition, when we log in to the network, we need to verify our name and password in the database. Even when we use an ATM, a database is used to verify the PIN code and check your balance.
- A database is, in essence, a set of files or a file system, that is, a repository for storing data. A database stores data in a specific format. Users can add, delete, modify, and query the stored data.
- In daily life, people use languages and their voice to describe things. In a computer, features of these things are broken down and recorded.
- A database is an organized collection of structured information or data, typically stored electronically in a computer system.
- A database can be regarded as an electronic file cabinet, where electronic files are stored. Users can add, truncate, update, and delete data in files.
- A database is a collection of data that is stored together in a certain way. It can be accessed by multiple users, usually has minimal redundancy, and operates independently of applications.

DBMS

- A database management system (DBMS) consists of a collection of interconnected data and a set of programs used to access the data. This dataset is usually called a database. The main purpose of DBMS is to provide a convenient and efficient way to access database information.
- A DBMS acts as an interface between a database and its users or programs, allowing users to retrieve, update, and manage information in an organized and optimized manner. In addition, the DBMS helps monitor and control the database and provides various management operations, such as performance monitoring, optimization, backup, and recovery.
- Typical database management systems include Oracle, Microsoft SQL Server, Access, MySQL, and PostgreSQL.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- A database is usually inseparable from a complete database software program, that is, a DBMS. A DBMS acts as an interface between a database and its users or programs, allowing users to retrieve, update, and manage information in an organized and optimized manner. In addition, the DBMS helps monitor and control the database and provides various management operations, such as performance monitoring, optimization, backup, and recovery.
- Database software is used to create, edit, and maintain database files and records, helping users easily perform operations such as file and record creation, data entry, data editing, update, and report. In addition, database software can handle data storage, backup, reporting, multi-channel access control, and security issues. With the increasing occurrence of data theft today, database security has become essential. Database software is sometimes called a DBMS.
- Common database software includes MySQL, Microsoft Access, Microsoft SQL Server, FileMaker Pro, Oracle Database, and dBase.

DBMS - Functions

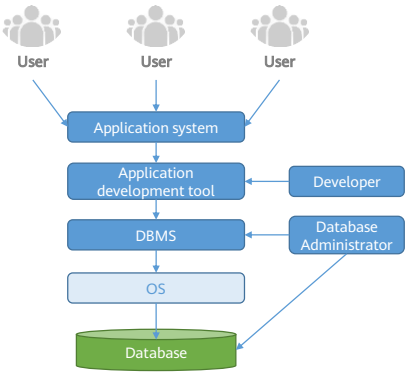
- **Data definition**: The DBMS provides a data definition language (DDL), through which users can easily define data objects in a database.
- **Data manipulation**: The DBMS also provides a data manipulation language (DML), which can be used by users to manipulate data and perform basic operations on the database, such as query, insert, delete, and modify.
- **Database operation management**: The DBMS manages and controls the creation, operation, and maintenance of databases in a unified manner to ensure data security and integrity, concurrent use of data by multiple users, and and to ensure the system recovers after a fault.
- **Interfaces and tools for convenient and effective database access**: Programmers can develop database applications through an interface. Database administrators (DBAs) can use tools to manage databases.
- **Database creation and maintenance**: include the input and conversion of initial data to the database, the dump and restoration of the database, the reorganization of the database, and the performance monitoring and analysis. These functions are usually implemented in software.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **Database operation management**: The DBMS manages and controls the creation, operation, and maintenance of databases in a unified manner to ensure data security and integrity, concurrent use of data by multiple users, and system recovery from faults. For example:
  - Data integrity checks ensure that the data entered by users meets the constraints of the database.
  - Security protection ensures that only authorized users can access data in the database.
  - Concurrency control enables multiple users to access the data in the database at the same time.
  - Fault rectification recovers the database for users when a fault occurs, providing reliable access to the database.

DBS

- A database system (DBS) includes hardware and software. Hardware is there as storage through a medium such as a hard drive. It is also there to support the software, which mainly includes the DBMS, the operating system (OS) that supports the operation of the DBS, and the access technology that supports the application development in multiple languages.
- The DBS is the sum of all the database components. A complete DBS consists of the database, DBMS, application development tool, application system, database administrator, and users.



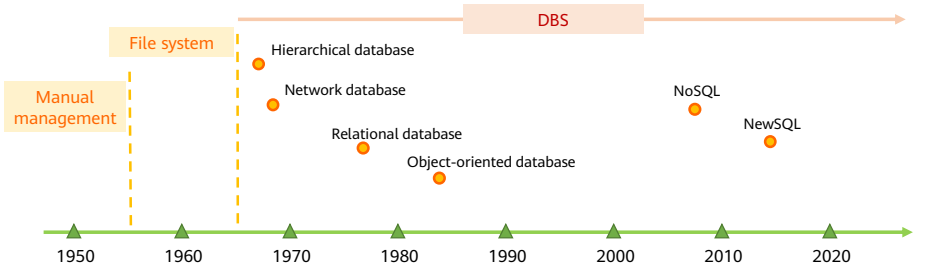
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The DBS consists of the following three parts:
  - Database: stores data.
  - DBMS: manages the database.
  - Database applications: supplement the DBMS, helping improve the processing capability of the DBS.
- A database (DB for short) provides space for storing various data, and can be considered as a container. A database may contain many files, and a DBS usually contains many databases.
- The DBMS is the software used by users to create, manage, and maintain a database. It serves between users and the OS to manage the database in a unified manner. The DBMS defines the data storage structure, provides the mechanism for data operations, and maintains the security, integrity, and reliability of the database. However, it cannot meet the requirements for data management in many cases.
- Database applications can meet higher requirements for data management and make the data management process more intuitive and friendly. Database applications communicate with the DBMS, access and manage data stored in the DBMS, and allow users to insert, modify, and delete data in the database.

Three Stages of Development: Data Management Technologies

Data management is to classify, organize, encode, query, and maintain various forms of data. It mainly goes through three stages: manual management, file system, and DBS, each of which has evolved towards reducing data redundancy, enhancing data independence, and facilitating data operations.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Data storage and management rely on the database. For example, schools need to use databases to store information about students and teachers, and companies need to use databases to store information about companies and employees. When data is stored in a database, the DBMS organizes and manages the data.
  - The DBMS is core to the database. It sits between the OS and the user, allowing them to create, use, and maintain the database.
- [Manual management stage]
- Before computers emerge, people used paper and computing tools (such as abacus and calipers) to record and calculate data, and relied on their own brain power to manage and utilize the data. By the mid 1950s, computers had just begun to show up, and storage data was limited. Only paper tapes, cards, and tapes were available. Computers were limited to application in science and technology, mainly scientific computing. In other words, while still under manual management, data is mainly stored on media such as paper tapes and tapes, or is just manually recorded.
- Manual data management has the following features:
  - Data cannot be stored for a long time.
  - Data query is inconvenient.
  - Data cannot be accessed concurrently, resulting in poor redundancy.
  - Data is not independent.

- [File system stage]
- From the late 1950s to the mid 1960s, direct access storage devices such as disks and magnetic drums became popular and moved to the inside of the computer. The data is stored in files, and managed by the file system. The file system allows access to the data in files based on a file storage path and file name. You can view, modify, add, and delete these files. Compared with that at the manual management stage, data management becomes simple with the help of the file system. You do not need to search for a file. However, the data in these files is not managed in a structured manner, and it is inconvenient to query for the data within.
- The features of data management with a file system are as follows:
    - Data can be stored for a long time.
    - Data is managed by the file system.
    - Poor data redundancy and poor concurrent access.
    - High data independency.
    - Unable to cope with emergencies (for example, files can be deleted by mistake or disks can become faulty)

DBS stage

- In the late 1960s, with the development of network technologies and the improvement of computer software and hardware, database technology emerged. This stage is the so called DBS stage. At the DBS stage, a dedicated database is used to manage data. You can create a database in the DBS, create tables in the database, and store data in these tables. You can query the DBMS for data in a table. Compared with the file system, the DBS structures the data. In a file system, data within a file can be structured, but there are no relationships established between files. Therefore, there is no overarching structure. Although the DBS is often divided into many separate data files, it pays more attention to the relationship between the data in the same database.
- Data management at the DBS stage has the following features:
  - Unified management and control of data.
  - High access concurrency and good redundancy.
  - Data is independent.
  - Fine data granularity.
    - Data granularity defines how detailed data in the database is. A finer granularity indicates more detail, and a coarser granularity indicates less detail.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Comparison of Data Management at Different Stages

- Each stage of data management has its own background and features, and the data management technology is continuously improving in each stage. The following table compares data management at the three stages.

Stage	Manual Management (Mid 1950s)	File System (Late 1950s to Mid 1960s)	DBS (Late 1960s)
Application	Scientific computing	Scientific computing and management	Management of large-scale data and distributed data
Hardware	No direct access storage devices	Tapes, disks, and magnetic drums	Large-capacity disks, erasable CD-ROMs, and tape drives supporting on-demand capacity expansion
Software	No dedicated management software	File system	DBS
Data Processing Method	Batch processing	Online real-time processing and batch processing	Online real-time processing, batch processing, and distributed processing
Data Manager	User/Program	File system	DBMS
Data Application and Extension	Oriented to a certain application, difficult to expand	Oriented to a certain application system, difficult to expand	Oriented to multiple application systems, easy to expand
Data Sharing	No sharing and high redundancy	Poor sharing and high redundancy	Good sharing and low redundancy
Data Independence	Poor data independence	Good physical independence but poor logical independence	High physical independence and good logical independence
Data Structuring	Unstructured data	Structured only within the record, but unstructured on the whole	Unified data model and overall structuring
Data Security	Application protection	File system protection	Comprehensive security protection

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Why Do We Use Databases?

- A database can store data efficiently and orderly, enabling us to manage data more quickly and conveniently:
  - Databases can store a large amount of information, and therefore data, in a structured manner, facilitating effective retrieval and access by users.
  - The database can effectively maintain the consistency and integrity of data information and reduce data redundancy.
  - The database can meet the sharing and security requirements of applications. In most cases, data is stored in databases for security purposes.
  - The database technology can be easily and intelligently analyzed to produce new and useful information.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

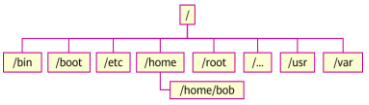
- With the rapid development of Internet technologies, the number of people accessing the Internet worldwide is expected to reach 5 billion by the end of 2020. The increase in the number of users has driven the development of online shopping, microblogging, streaming, and other industries. These users, however, generate a huge amount of network data. How do we store, retrieve, and manage this data securely and effectively? This is where effective data storage, efficient access, convenient sharing, and security control have become vital.
  - The database can store a large amount of data in a structured manner, which facilitates effective retrieval and access by users. The database can store data by category and provide quick queries. For example, when we search for content on Baidu, Baidu uses databases and data classification technology to quickly search the terms for results.
  - The database can effectively maintain the consistency and integrity of data information and reduce data redundancy. In this way, data validity is ensured and data is not lost or damaged. In addition, databases can avoid duplications of data to reduce wasted data.
  - The database can meet the data concurrency and security requirements of applications. In most cases, data is stored in databases for security purposes. For example, the confidentiality of salaries cannot be guaranteed if all employee information and salary data are stored on disk files. If employee information and salary data are stored in a database, only the employee information can be queried for and modified, and the salary information can be viewed only by specified personnel (such as financial

personnel), ensuring data security.

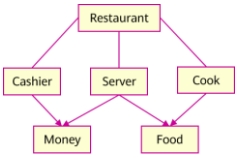
- The database technology can be easily and intelligently analyzed to produce new and useful information. For example, a supermarket might store sales information in the database, so as to determine the goods purchased in the second half of the month based on the monthly sales ranking. Actually, such information can be generated from a query.

Database Models

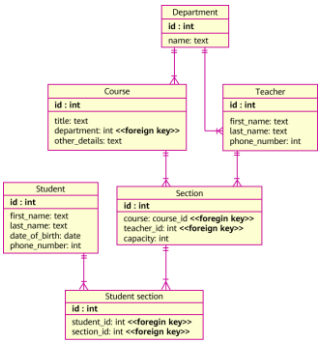
- Since the middle of the 20th century, many interesting database models, such as hierarchical model, network model, and relational model, have emerged. Some database models are no longer used, while others are still playing a significant role.



Hierarchical model



Network model



Relational model

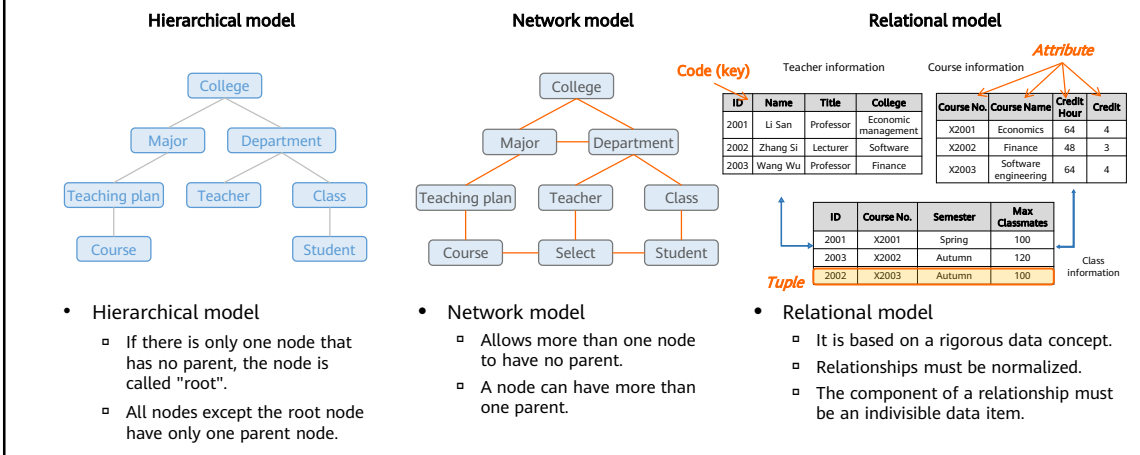
 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- In the hierarchical model, a tree structure is used to indicate the relationship between data records. In the network model, the graph structure is used to indicate the relationship between data records. In the relational model, the two-dimensional table (or relationship) is used to indicate the relationship between data records.
- Hierarchical database
- The hierarchical model came about in 1960s. The hierarchical model is similar to a tree structure. Each record has only one parent node, as shown in the figure. It came about as people tryied to process more complex, improving the performance of some specific methods of reading data. However, as the hierarchical database is based on a tree-shaped model, accessing the hierarchical database is similar to traversing a linked list. So it struggles to read complex data in most situations.
- Although hierarchical databases have little value when applied to modern systems, they are still used in Linux/Windows file systems, DNS, and LDAP systems. Database implementations:
    - Linux/Windows file system
    - DNS
    - LDAP

- Database implementations:
    - MySQL
    - PostgreSQL
    - SQLite

Database Models: Hierarchical, Network, and Relational Models



- Hierarchical model
  - If there is only one node that has no parent, the node is called "root".
  - All nodes except the root node have only one parent node.
- Network model
  - Allows more than one node to have no parent.
  - A node can have more than one parent.
- Relational model
  - It is based on a rigorous data concept.
  - Relationships must be normalized.
  - The component of a relationship must be an indivisible data item.

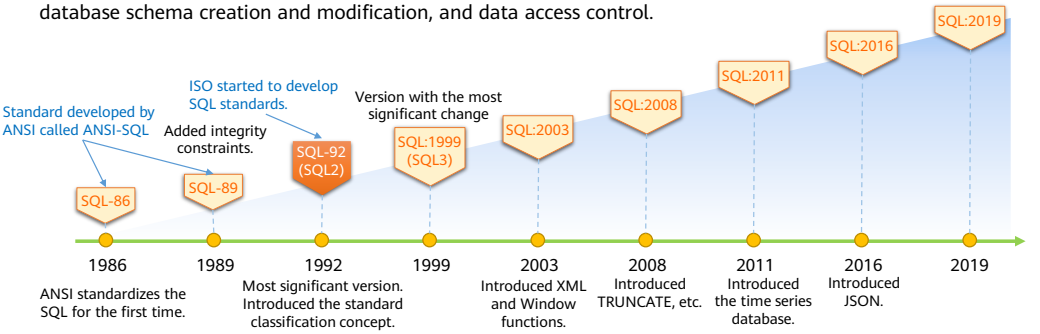
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Hierarchical database
- The hierarchical database was the first successful database system to be developed. It presents the data through the hierarchical structure (tree structure). The hierarchical databases used to be the mainstream database, but with the appearance and popularization of the relational database, it is now seldom used. The typical hierarchical database is the Information Management System (IMS) database, which was developed by IBM.
- Relational database
- The relational database is the most widely used database. It was born in 1969 and has a long history. Like an Excel worksheet, a relational database uses a two-dimensional table consisting of rows and columns to manage data, making it easy to understand and easy to read and query. In addition, SQL is used to perform operations on relational database data.

SQL

- Structured Query Language (SQL) is the standard language for relational databases. SQL is a general-purpose and powerful relational database language. It is a standard interface for accessing relational data and is also the basis for interoperation between different database systems.
- SQL is based on relational algebra and tuple relationship calculations. It integrates data query, data operations, data definition, and data control functions. The scope of SQL includes data insertion, query, update, and deletion, database schema creation and modification, and data access control.



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

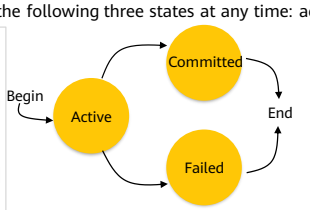
- As a standard language for accessing relational databases, SQL is widespread. It is supported by not only large-scale commercial database products such as Oracle, DB2, Sybase, and SQL Server, but also many open-source database products such as PostgreSQL and MySQL, and even some small products, such as Access. In recent years, the booming NoSQL system once claimed that SQL was no longer needed; however, it had to amend it to "Not Only SQL" later to emphasize its coexistence with SQL.
- 1986: ANSI X3.135-1986, ISO/IEC 9075:1986, SQL-86 is the first time that ANSI has standardized SQL. 1989: ANSI X3.135-1989, ISO/IEC 9075:1989, SQL-89 added integrity constraints. 1992: ANSI X3.135-1992, ISO/IEC 9075:1992, SQL-92 (SQL2) is the most significant version, and the standard classification concept is introduced. 1999: ISO/IEC 9075:1999, SQL:1999 (SQL3) is the version with the most significant change; it changed the definition of standard compliance and added support for object-oriented features, regular expressions, stored procedures, and Java. 2003: ISO/IEC 9075:2003, SQL:2003 introduced XML and Window functions. 2008: ISO/IEC 9075:2008, SQL:2008 introduced TRUNCATE. 2011: ISO/IEC 9075:2011, SQL:2011 introduced the time series database. 2016: ISO/IEC 9075:2016, SQL:2016 introduced JSON.

Transaction

- A transaction is a logical unit of program execution, consisting of a series of operations for accessing or updating data in the system. In computer terminology, a transaction usually refers to a database transaction.
- A database transaction usually contains a sequence of read/write operations on the database. Transactions in a database environment have two main purposes:
  - 1. To provide a method for restoring the database operation sequence from a failure while keeping database consistency even in case of system failure.
  - 2. To provide isolation between programs accessing a database concurrently, so that they do not interfere with one another.
- Transaction status:
  - A transaction is atomic and is regarded as a whole. That is, it can only be in one of the following three states at any time: active, committed, and failed.

One transaction

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00
  WHERE name = 'Alice';
UPDATE branches SET balance = balance - 100.00
  WHERE name = (SELECT branch_name FROM accounts
  WHERE name = 'Alice');
UPDATE accounts SET balance = balance + 100.00
  WHERE name = 'Bob';
UPDATE branches SET balance = balance + 100.00
  WHERE name = (SELECT branch_name FROM accounts
  WHERE name = 'Bob');
COMMIT;
```



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- In a DBMS, a transaction is the basic unit for database recovery and concurrency control. It is a sequence of operations that are either executed or not executed. It is an unsplittable unit of work. Take bank transfer as an example. When money is transferred from an account to another one, the operations of deducting from the source account and adding to the target account must be performed, or not performed at the same time,. Otherwise, the account balance will be incorrect. So, they should be seen as one transaction. In modern databases, transactions can do other things, such as making sure you don't have access to data that has not finished processing by others. The basic idea is the same — transactions are used to make sure that no matter what happens, the data you use will be in a reasonable state. It guarantees money will never be deducted from one account and not deposited into another after the transfer.
- When a transaction is committed to the DBMS, the DBMS needs to ensure that all operations in the transaction were completed successfully and that the results are permanently stored in the database. If some operations in the transaction are not successfully completed, all operations in the transaction need to be rolled back to the status before the transaction is executed. In addition, the transaction cannot affect the operation of the database or other transactions. All transactions run independently.

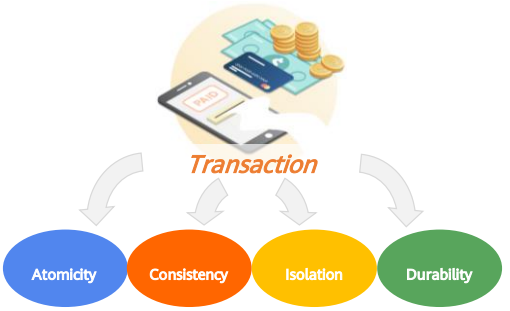
Transaction Status

- A transaction is atomic and is regarded as a whole. That is, it can only be in one of the following three states at any time: active, committed, and failed. In addition, there is an intermediate state of the transaction, which is invisible to external systems. A transaction may be in any of the following states:
  - Active: initial status of a transaction, indicating that the transaction is being executed.
  - Partially committed: after the last statement is executed.
  - Failed: The transaction fails to be executed.
  - Aborted: The transaction is rolled back and the database is restored to the status before the transaction.
  - Committed: The entire transaction is successfully executed.
- After execution, a transaction ends in the aborted or committed.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

ACID Feature of Relational Databases

- *Atomicity*
  - A transaction is the logical working unit of a database. Atomicity means that you guarantee that either all operations in the transaction succeed or none do.
- *Consistency*
  - This ensures that a transaction can only transit from one consistent state to another consistent state.
- *Isolation*
  - This ensures that transactions do not interfere one another. That is, the internal operations and data used in a transaction are isolated from other transactions, and concurrent transactions do not affect each other.
- *Durability*
  - Once a transaction is committed, changes made to data in the database are permanent. Operations or faults occurring after the commit do not affect the result of the transaction.

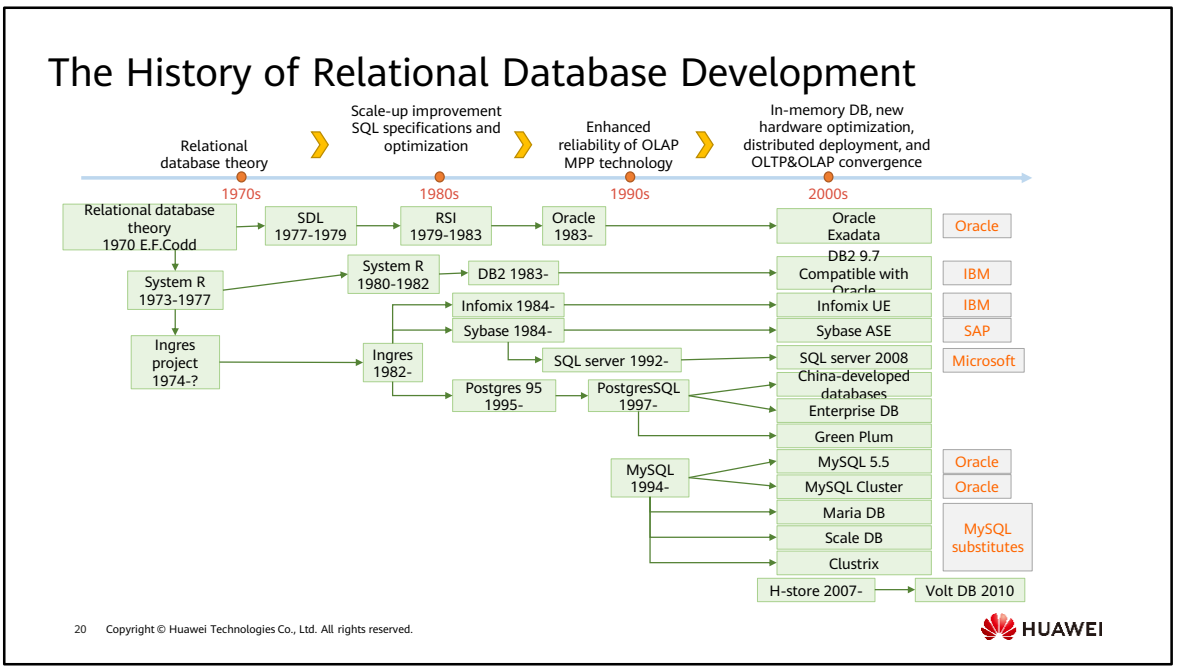


Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- According to strict standards, only transactions that meet the ACID feature are regarded as transactions. However, in the database implementation among major vendors, only a few transactions meet the ACID feature. For example, the NDB Cluster transaction of the MySQL database does not meet the durability and isolation requirements; the default transaction isolation level of the InnoDB is repeatable read, which does not meet the isolation requirements; and the default transaction isolation level of Oracle is read committed, which does not meet the isolation requirement. As such, ACID is not a condition that a transaction must meet, but indicates four dimensions for measuring a transaction.
- Atomicity: A transaction is the logical working unit of a database. Either all operations in the transaction succeed or none do. If an SQL statement in a transaction fails to be executed, the executed statements must be rolled back, and the database returns to its state before the transaction was executed.
  - Taking the previous bank transfer as an example. Subscriber A transfers money to subscriber B. This includes at least two operations: money is deducted from the account of subscriber A, and money is added to the account of subscriber B. The two operations can both be successful or both fail, there is no in-between. This is what is meant by an atomic operation. As shown in the figure, if the transaction fails after T1 is complete but before T2 is complete, the database must roll back T1.
- Isolation: Different from atomicity and durability, isolation focuses on the interaction between different transactions. Isolation means that operations inside a transaction are isolated from other transactions, and transactions that are concurrently executed cannot interfere with each other. Strict isolation corresponds the isolation of **Serializable** transactions. However, **Serializable** is seldom applied due to performance considerations. Isolation aims to ensure that

transactions do not interfere with each other in the case of concurrency.



- The invention of relational model changed the history of database development. The research of relational theory and the great success of relational DBMS further promote the development of relational databases, making the relational model the dominant data model.
- This slide displays the generation time of major database products in a timeline.
- This figure shows IBM System R as the father of relational databases. In 1973, Ingres was born under the lead of Michael Stonebraker and Eugene Wong after absorbing a series of documents from System R. Ingres closed its source and started commercialization. It developed several famous database products, such as Infomix, SyBase, and SQL Server. PostgreSQL was born in the 1980s and inherits a lot of Ingres ideas. In 1995, two Chinese people changed the SQL engine of PostgreSQL, and that's when the PostgreSQL community was set up. After nearly 30 years of continuous development and verification, PostgreSQL has won a good reputation in the database field. Currently, it is recognized as the most advanced open-source database in the world. Especially in recent years, the community has accelerated its iteration and released a major version each year. The latest release is PostgreSQL 11. PostgreSQL 12 is under development and will also be released in the fall of this year.
- The preceding figure also shows the non-stop development of PostgreSQL since its birth, even rivaling that of Oracle.
- Oracle database, developed by Oracle Corporation, is one of the most popular relational databases in the world,
    - Larry Ellison co-founded Oracle Corporation in 1977 with Bob Miner and Ed Oates under the name Software Development Laboratories (SDL). The first

version of the Oracle system was compiled in assembly language.

    - Achievements: 1. Openness; Oracle was compatible with all mainstream platforms at that time, fully meeting various industry standards. 2. High security; 3. High performance, ranking high in the professional database tests on open platforms. 4. In the 1980s and 1990s, Oracle always followed and led the technical trend of relational databases.
    - Oracle EXADATA was released in 2009, seizing the high-end OLAP market.

- Teradata Corporation was formed in 1979. Teradata released its DBC/1012 specialized database computer in 1984. It is the earliest database applying the MPP system for developing large-scale data warehousing applications and provides excellent OLAP performance based on proprietary hardware and software. Also, it is very expensive.
- IBM released DB2 on its Multiple Virtual Storage (MVS) mainframe platform in 1983. Originally, IBM promoted relational databases for mainframe and mid-range servers. In 1995, IBM started to support multiple platforms such as Windows and Unix. DB2 was used to indicate a shift from DB1, which is a hierarchical database.
- Ingres, a relational database research project at the University of California, Berkeley, started in 1974. Ingres code is free of charge. There are many commercial database software, including Sybase, Microsoft SQL Server, and Informix, developed based on Ingres. PostgreSQL is one of the most influential computer research projects in history.
- Informix, the first commercial Ingres product released in 1982, was acquired by IBM in 2000. Then, Informix source code was authorized to General Data Technology Co., LTD, an information technology company in China. This company developed GBase 8t based on the Informix source code.
- MySQL: The internal version 1.0 was released in 1996 and MySQL 3.11.1 was released in October 1996. MySQL can be commercially used for free. However, if used with other commercial products, a special license is required. In April 2020, the old storage engine was renamed MyISAM. In 2001, the InnoDB engine was integrated. In January 2008, MySQL was acquired by Sun Microsystems. In 2009, Sun Microsystems was acquired by Oracle. MySQL is now an Oracle product and is still open-source and free of charge.
- PostgreSQL was released in 1989.
- In 1995, after modifications, the first community version of the SQL engine was officially released. PostgreSQL is designed to be one of the most powerful open-source databases in the world. It has powerful and abundant functions. The Greeplum, Netezza, Amazon Redshift and GaussDB 200 databases are all based on PostgreSQL.

Database Challenges

The databases of large enterprises need to support very complex searches, with users expecting responses almost in real time. As such, database administrators often need to adopt various methods to help enterprises improve performance. Some of the common challenges they face include:

- **Sharp increase in data volume.** The explosive growth of data from sensors, networked devices, and many other sources has kept database administrators busy with data management and organization.
- **Ensuring data security.** Nowadays, data is everywhere posing great leakage risks, and hackers' attack methods are emerging. Therefore, it is more important than ever to ensure data security while ensuring that users can easily access data.
- **Meeting ever-changing requirements.** In today's fast-growing business environment, enterprises need to access data in real time to make the best decisions in a timely manner and seize new opportunities.
- **Managing and maintaining databases and infrastructure.** Database administrators need to continuously monitor problems in the database and perform preventive maintenance, application software upgrades, and patch installations. As databases become increasingly complex and data volumes increase, enterprises need to recruit more personnel to monitor and optimize databases, increasing costs.
- **Breaking the scalability limitation.** In order to survive, modern enterprises must continue to develop, and the corresponding data management must keep pace with the times. However, it is often difficult for database administrators to predict the future data volume of an enterprise, especially if the database is deployed locally.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

NoSQL (1)

- NoSQL, short for Not Only SQL, is a DBMS different from traditional relational databases. NoSQL is used to store ultra-large-scale data. These types of databases do not require fixed storage modes and can be scaled out without extra operations. Based on the structuring method and application scenario, the databases are classified into the following types:
- **Key-value database oriented to high-performance concurrent read/write**
  - Key-value databases feature high concurrent read/write performance. A key-value database is a database that stores data using key-value pairs. It is similar to the map in Java. The entire database can be regarded as a large map, and each key corresponds to a unique value.
- **Document database oriented to massive data access**
  - The main feature of this database is that data can be quickly queried from a large amount of data. Document storage typically uses an internal notation that can be processed directly in the application, primarily in JSON. JSON documents can also be stored as plain text in a key-value store or in a relational DBS.
- **Search engine for content searching**
  - The search engine is a NoSQL DBS that is used to search for data content. It is mainly used for quasi-realtime processing and analysis of massive data and can be used for machine learning and data mining.
- **Distributed database oriented to scalability**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- NoSQL refers to non-relational databases. With the emergence of Internet Web 2.0 websites, traditional relational databases lag far behind in processing ultra-large-scale and highly concurrent SNS website requests. NoSQL databases supportmultiple data types of large-scale datasets, especially from big data applications.
- NoSQL, which means "Not Only SQL", is a revolutionary database movement that gained popularity in 2009. NoSQL advocates the use of non-relational data storage.

NoSQL (2)

- Key-value database for high-performance concurrent read/write
  - Key-value databases feature high concurrent read/write performance. This type of database stores data using key-value pairs. It is similar to the map in Java. The entire database can be regarded as a large map, and each key corresponds to a unique value. Examples: Redis, Amazon DynamoDB, Memcached, Microsoft Azure Cosmos DB, and Hazelcast.
- Document database for massive data access
  - The main feature of this database is that a user can quickly search for what they need among a large amount of data. Document storage typically uses an internal notation that can be processed directly in the application, primarily in JSON. JSON documents can also be stored as plain text in a key-value store or in a relational DBS. Examples: MongoDB, Amazon DynamoDB, Couchbase, Microsoft Azure Cosmos DB, and CouchDB.
- Search engine for content search
  - The search engine is a NoSQL DBS that is used to search for data content. It is mainly used for quasi-realtime processing and analysis of massive data, and can also be used for machine learning and data mining. Examples: Elasticsearch, Splunk, Solr, MarkLogic, and Sphinx.
- Distributed database oriented to scalability
  - Examples: Cassandra, HBase, Microsoft Azure Cosmos DB, Datastax Enterprise, and Accumulo.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

NoSQL – CAP Theory

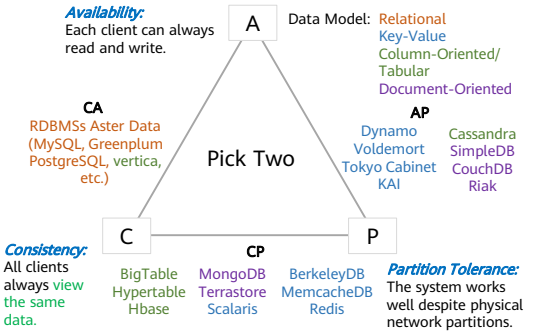
- The basic demand of NoSQL is to support distributed storage. Strict consistency and availability need to be balanced.
- CAP theory: A distributed system cannot meet consistency (C), availability (A), and partition fault tolerance (P) requirements at the same time, and can meet at most two of them. Partition fault tolerance is a basic requirement for a distributed system. Otherwise, the system cannot be called a distributed system. Therefore, a balance needs to be achieved between C and A.

- ✓ *Consistency*

  Consistency indicates that after the update operation is successful and the result is returned to the client, the data of all nodes at the same time is consistent. The notion of consistency is different from that in the ACID feature.
- ✓ *Availability*

  It means that a timely and correct response can be made for each request, but it is not guaranteed that the request result is based on the latest written data.
- ✓ *Partition tolerance*

  Partition fault tolerance means that a distributed system can still provide services that meet consistency and availability requirements when a node or network partition is faulty.



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Comparison between relational and non-relational databases
  - Cost: The NoSQL database is easy to deploy and uses open-source software. It is cheaper than a relational database.
  - Query speed: The NoSQL database stores data in the cache which does not need to be parsed by the SQL layer. The relational database stores data on the hard disk, so its queries are much slower.
  - Data storage format: The NoSQL database stores data in the key, value, document, or image format. Therefore, it can store basic types and various formats, such as objects or sets. Other databases support only basic types.
  - Scalability: The relational database has restrictions on the multi-table query mechanism, such as join. As a result, scaling is difficult. NoSQL is based on key-value pairs, and data is not coupled. Therefore, NoSQL can be easily scaled out.
  - Persistent storage: NoSQL does not use persistent storage. Relational databases are still the only option for persistent storage of massive data.
  - Data consistency: Unlike relational databases, non-relational databases generally emphasize eventual data consistency. Data read from non-relational databases may be in an intermediate state.
- NoSQL does not process transactions.

NewSQL

- NewSQL is defined as the development direction of next-generation databases, which refers to all various new scalable and high-performance databases. It has the massive storage management capability of the NoSQL database and the ACID feature and SQL convenience of the relational database.
- Simply speaking, SQL + NoSQL = NewSQL. Although the internal structure of NewSQL systems varies greatly, all the NewSQL systems have three significant common features. They all support the relational data model, use SQL as the main interface, and meet the features of the distributed database.
- The internal structure of NewSQL systems varies in supporting the relational database transaction feature and the distributed database feature. You can learn it from the architecture, SQL engine, and sharding mode aspects.
  - **New architecture**: Representative databases are Google Spanner, VoltDB, Clustrix and NuoDB which work on distributed nodes in a cluster, data is stored in shards, and SQL query is performed on shards on different nodes.
  - **SQL engine**: Representative databases are TokuDB and MemSQL. This type of databases has a highly optimized SQL engine.
  - **Transparent sharding**: Representative databases are ScaleBase, dbShards, and Scalearc. The system provides a sharding middleware layer. Data is automatically split and executed on multiple nodes.

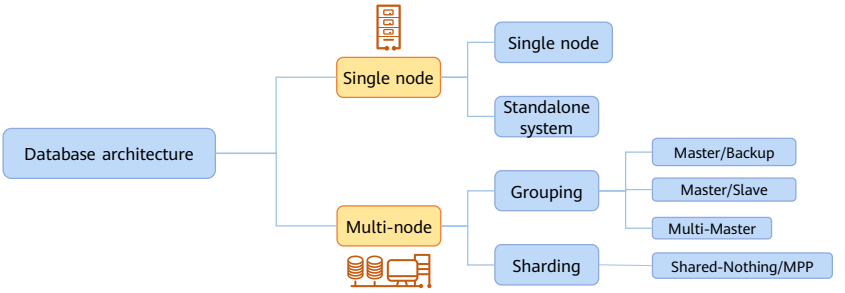
 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- NewSQL is the trend for next-generation databases, which includes a variety of new scalable and high-performance databases. It delivers the massive storage management capability of a NoSQL database along with the ACID feature and SQL convenience of a relational database. In essence, SQL + NoSQL = NewSQL. Although the internal structures of NewSQL systems can vary, all the NewSQL systems share three significant features: they all support the relational data model, use SQL as the main interface, and meet the features of the distributed database.
- Reference: https://labs.sogeti.com/newsql-whats/

Relational Database Architecture

As the service scale increases, the amount of data stored in the database and the service pressure on the database increase. Therefore, the database architecture needs to be changed to provide stable and efficient data services for upper-layer applications.



Evolution 1: Separation of database read and write; ***evolution 2***: Database vertical sharding; ***evolution 3***: Database and table horizontal sharding.

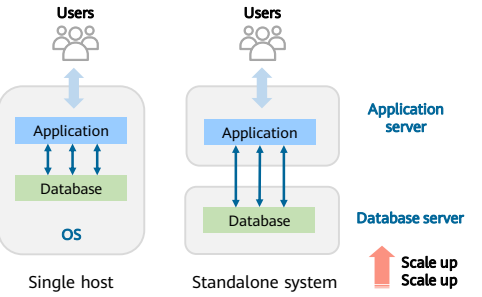
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Common bottlenecks of relational databases
  - Many concurrent read/write operations cause unbearable load pressure on a single database.
  - A large amount of data is stored in a single table, resulting in low retrieval efficiency.
- In this architecture, databases are classified based on the number of hosts. For one host, a single-node architecture is used; for more than one host, a multi-node architecture is used.
- In single-node architecture, the applications and database are deployed on the same host, and the database is deployed on a standalone database server.
- In the multi-node architecture, having more services improves the availability and service capability of the entire database service. There are two types of databases in multi-machine architecture, depending on whether or not data is split into shard.
  - One type is grouping where servers play different roles, such as master/backup, master/slave, and multiple master servers. The other type is sharding. No matter which mode is used, multiple databases have the same structure and store the same data. Synchronization technology is used to replicate data among multiple databases.
  - For sharding architecture, this document mainly describes MPP architecture based on Shared-Nothing.

Single Node

To prevent application services and database services from competing for resources, the single-node database architecture evolves from the single-host mode to the standalone mode, separating applications from data services. For application services, you can increase the number of servers, balance the load, and enhance the concurrent capability of the system.

- ***Advantage***: centralized deployment and easy O&M.
- ***Disadvantages:***
  - Scalability: The single-node database architecture can only be scaled up. That is, the performance can only be improved by adding hardware configurations. However, the hardware resources that can be configured for a single host may reach the upper limit.
  - Single point of failure (SPOF): During scale-up, services need to be stopped. If a hardware fault occurs, the entire service is unavailable and even data is lost.
  - Single-node performance bottleneck



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

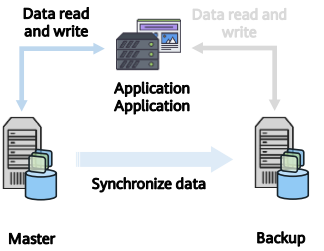
- Standalone deployment is flexible and easy to deploy in R&D, learning, and simulated environments. Early Internet servers with the LAMP (Linux, Apache, MySQL, PHP) architecture are the most typical single-node servers.

Grouping - Master/Backup

Master/Backup architecture:

- The database is deployed on two servers. The server that provides data read and write services is called the master server. The other server uses the data synchronization mechanism to copy the data of the primary server, which is called a backup server. Only one server provides data services at a time.

  - ***Advantages***
    - No extra effort needs to be paid during application development against database faults.
    - Compared with the single-node architecture, this architecture improves data error tolerance.
  - ***Disadvantages***
    - Resources are wasted. The master and backup servers have the same configuration. However, resources in the standby server are basically idle for a long time and cannot be utilized.
    - The performance pressure is still concentrated on a single node, which cannot solve the performance bottleneck problem.
    - When a fault occurs, the switchover between the master and backup servers requires manual intervention or monitoring.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

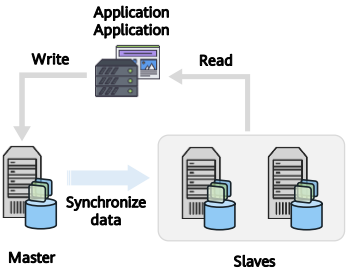
- The master and backup servers are also called master and backup databases. Synchronous and asynchronous replication can be used for data synchronization between them.
- When the master server provides database services, the backup server does not provide data services. When the master server is faulty, the backup server manually or automatically starts to provide data services. In this case, the backup server is promoted to master. To improve data integrity and availability, you can increase the number of backup servers, shifting from master/backup to one-master-multiple-backup mode. This increases security, but also increases costs and wastes resources. Master/backup is only useful to address data availability, because database performance is still subject to the performance of a single server. Increasing the number of servers does not improve database performance.

Grouping - Master/Slave

Master/Slave architecture

- The deployment mode is similar to the master/backup mode. However, the backup server becomes the slave server and provides certain data services. Pressures are balanced by read/write isolation:
  - Write, modify, and delete operations are performed on the write database (master server), and query requests are allocated to the read database (slave server).

- ***Advantages***
  - It improves resource utilization and applies to scenarios where read operations are more frequent than write operations.
  - In high-concurrency read scenarios, load balancing is employed to balance loads among servers.
  - The scalability of slave servers is flexible, and scaling does not affect services.
- ***Disadvantages***
  - There is a delay when data is synchronized to slave databases. As such, applications need to tolerate temporary inconsistency. This mode is not applicable to scenarios that have high consistency requirements.
  - The performance pressure of write operations is still concentrated on the master server.
  - If the master server is faulty, a master/slave switchover is required, where manual intervention requires response time and automatic switchover is complex.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

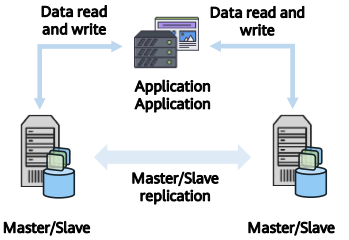
HUAWEI

- The way applications are developed sometimes needs to be adjusted to ensure read/write isolation. Read operations are committed to the master server, and write operations are committed to the slave server.
- In recent years, some database products have developed middleware to support read/write isolation. Database middleware provides distribution function to identify read and write requests, and allocate them to respective databases. Examples of such products include MyCAT and Vitness. In many Internet applications, there are many more read transactions than write transactions, read operations creating performance bottlenecks. In addition, when the write database is faulty, the read database can still provide services externally. That is, the entire database is "read-only". The master/slave architecture can be classified into master/slave and one-master multi-slave. The more slave servers, the more concurrent database read requests can be processed. However, data inconsistency among slave servers also causes many problems. When the master server is faulty, it is difficult to choose the most suitable slave server to replace it and ensure automatic switchover. Some applications do not have high consistency requirements, such as social networks, while others do, such as finance and charging applications.

Grouping - Multi-Master

Two database servers can function as the master and slave servers for each other.

- *Advantages*
  - This improves resource utilization and reduces the risk of SPOF.
- *Disadvantages*
  - Data is written to both servers and must be synchronized bidirectionally. Bidirectional replication also causes latency, and data may be lost in extreme cases.
  - With the increase of database servers, the data synchronization becomes very complex. In actual practice, the two-server cluster mode is commonly used.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

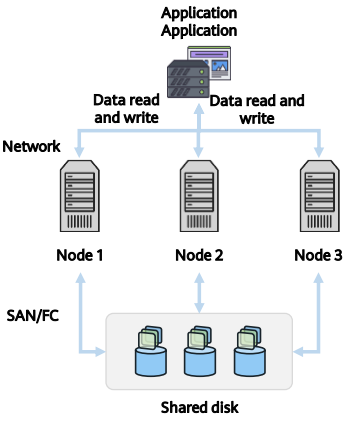
HUAWEI

- The multi-master architecture is also called active-active architecture. In this case, the data synchronization mechanism is more complex, and data delay and loss may occur. Therefore, this architecture is not suitable for scenarios with high consistency requirements.

Shared Disk

Shared disk is a special multi-master architecture. Database servers share data storage, and load balancing is implemented among multiple servers.

- ***Advantages***
  - Multiple compute servers are deployed providing high availability. Scalability is achieved, avoiding the SPOF of the server cluster.
  - Convenient scale-out can improve the parallel processing capability of the entire system.
- ***Disadvantages***
  - Difficult to implement.
  - When the storage interface bandwidth is saturated, adding servers does not improve performance. The storage I/O may become the performance bottleneck of the entire system.

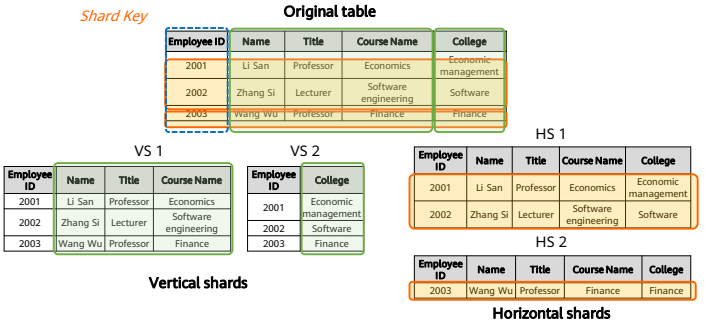


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Shared disk is a special multi-master architecture, which addresses the lack of data consistency caused by data synchronization between the master and slave devices. Currently, there are two successful cases: Oracle Real Application Cluster (RAC) and Microsoft SQL Server Failover Cluster.

Sharding

- Sharding is a database architecture mode based on horizontal partitioning. In this mode, rows in a table are partitioned into different tables (called extents). Each extent has the same schema and columns, but each table has completely different rows. Similarly, the data stored in each extent is unique and independent of the data stored in other extents.
- It may be helpful to understand from the aspect of relationship between horizontal partitioning and vertical partitioning. In a vertical partitioning table, all columns are separated and put into a new, distinct table. Data in each vertical shard (VS) is independent of data in all other shards, and each shard contains different rows and columns.



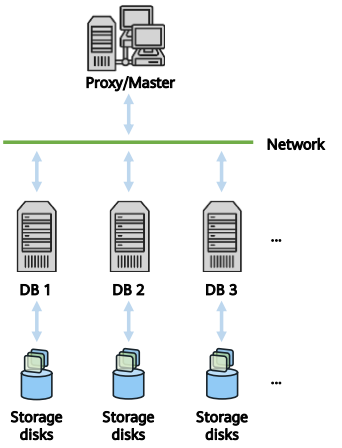
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Sharding is a database architecture based on horizontal partitioning. In this case, rows in a table are partitioned into different tables (called extents). Each extent has the same schema and columns, but each table has completely different rows. Similarly, the data stored in each extent is unique and independent of the data stored in other extents.
- It may be helpful to understand the relationship between horizontal and vertical partitioning. In a vertical partitioning table, all columns are separated and put into a new, distinct table. Data in each vertical shard (VS) is independent of the data in all other shards, and each shard contains different rows and columns.

Shared Nothing

- Each processing unit has its own CPU, memory, and hard disk, and these units do not share resources, similar to the MPP mode. These processing units communicate with each other using protocols, achieving better parallel processing and scalability.
- Nodes are independent of each other and each node processes its own data. The processing results may be summarized and sent to the upper layer, or transferred among different nodes.
- The shared nothing system usually distributes its data to different databases on multiple nodes (different computers process queries from different users) or requires each node to retain its own application data backup by using some coordination protocols. This is usually called database sharding.



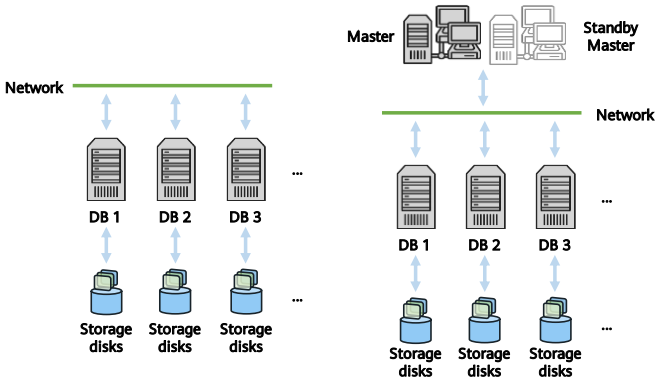
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Shared-Nothing**: Each processing unit has its own CPU, memory, and hard disk, and these units do not share resources, similar to MPP. These processing units communicate with each other using protocols, achieving better parallel processing and scalability. DB2 DPF and Hadoop are typical examples. Nodes are independent of each other and each node processes its own data. The processing results may be summarized and sent to the upper layer, or transferred among different nodes. Sharding is actually a shared nothing architecture. It horizontally partitions a table from the physical storage perspective and allocates them to multiple servers (or instances). Each server can work independently and has a common schema, for example, the MySQL Proxy and Google architectures. You only need to increase the number of servers to improve the processing capability and capacity.
- The shared nothing architecture is a distributed computing architecture. Here, each node is independent and self-sufficient, and there is no single-point competition across the entire system.
- In a pure shared nothing system, you can simply add some cheap computers as system nodes for almost infinite expansion.
- The shared nothing system usually distributes its data to different databases on multiple nodes (different computers process queries from different users) or requires each node to retain its own application data backup by using some coordination protocols. This is usually called database sharding.

Massively Parallel Processing (MPP)

MPP distributes tasks to multiple servers and nodes in parallel. After the calculation on each node is complete, the results of each node are summarized to obtain the final result. It features high performance, high availability, and high scalability. It also provides a cost-effective universal computing platform for hyperscale data management and is widely used to support various data warehouse systems, BI systems, and decision support systems.



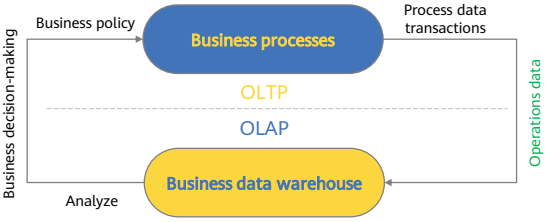
- ✓ Parallel task execution
- ✓ Distributed data storage
- ✓ Distributed computing
- ✓ Scale-out
- ✓ Shared nothing

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The MPP database is a distributed parallel database cluster with a shared nothing architecture. It features high performance, availability, and scalability. It provides a cost-effective universal computing platform for hyperscale data management and is widely used to support various data warehouse, BI, and decision support systems.
- No shared master:
  - 1. All nodes are equivalent.
  - 2. Data can be queried or loaded through any node.
  - 3. There is no performance bottleneck or SPOF risk.

Typical Applications of Relational Databases



Online Transaction Processing (OLTP)

- OLTP is a typical application of traditional relational databases. It is a highly-available online system that processes basic and routine transactions, such as bank transactions, mainly with small-sized transactions and small-sized queries.
- The OLTP system emphasizes on the database memory efficiency. When evaluating an OLTP system, check the number of transactions executed per second and the number of Execute SQL statements. It emphasizes the command rates of memory metrics, bound variables, and concurrent operations.

Online Analytical Processing (OLAP)

- OLAP is a typical application of the data warehouse system. It supports complex analysis, focuses on decision support, and provides query results that are easy to understand. In the OLAP system, the quantity of the executed statements is not an evaluation standard, because if the execution time of a statement is long, the read data is huge.
- The OLAP system focuses on data analysis. The evaluation standards are the throughput (bandwidth) of the disk subsystem, SQL execution duration, disk I/O, and partitioning.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- OLTP is a typical application of traditional relational databases. It is used for basic and routine transactions, such as deposit, withdrawal, and transfers.
  - Features: (1) High throughput: Many short online transactions (insert, update, and delete) are processed quickly. (2) High concurrency and (quasi) real-time response.
  - Typical OLTP scenarios: retail, banking s, train/flight ticket booking, and flash sales.
- OLAP: This concept was first proposed by E.F. Codd in 1993. An OLAP system can query and analyze a large amount of historical data. It takes a long time, there is a large data volume, and transaction processing is complex because of the aggregated operations at different levels.
  - Features: (1) It focuses on complex queries. (2) In data processing, it focuses on analytical data processing and operations, such as data aggregation, sum, grouping, and window calculation. (3) It uses and analyzes data from multiple dimensions.
  - Typical OLAP scenarios: reporting, CRM, financial risk prediction and warning, anti-money laundering, data mart, and data warehouse.

Comparison Between OLTP and OLAP

Item	OLTP	OLAP
Feature	Operation processing	Information processing
Orientation	Transaction	Analysis
User	Clerks, DBAs, and database professionals	Analysis and decision-making personnel
Purpose	Routine operations	Long-term information and decision-making support
Data	Current and latest	Maintained periodically
Summary	Original, highly detailed	Consolidated and integrated
Working unit	Short and simple transactions	Complex query
Access	Read/Write	Mostly read
Focus	Data input	Information output
Operation	Index or hash on the primary keyword	Large-scale scan
Number of access records	Dozens	Millions
DB scale	100 MB to GB	100 GB to TB
Advantages	High performance and availability	High flexibility and end user autonomy
Measurement	Transaction throughput	Query throughput and response time

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Database Overview
2. **Introduction to openGauss**
3. openGauss Technical Specifications
4. Basic Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss

As a leading database at enterprise level, openGauss is developed in collaboration with global partners. This multi-core-oriented open-source relational database provides ultimate performance, full-link service and data security, AI-based tuning, and efficient O&M capabilities. openGauss is released under the Mulan Permissive Software License v2. It deeply integrates Huawei's years of R&D experience in the database field and continuously builds competitive features based on enterprise-level scenario requirements.





High Reliability



Easy O&M



High Performance



High Security



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Multiple storage modes support composite service scenarios.
- The NUMA data structure supports high performance.
- Primary/Standby deployment and CRC support HA.

openGauss Evolution



Incubation for internal use → Joint innovation for productization → Ecosystem building

2001 – 2011

- Enterprise-level in-memory database

2011 – 2019

- Core data warehouse of bank G and commercial use of DWS on HUAWEI CLOUD
- Bank Z's core service system upgrade with the commercial database
- Supporting the commercial use of 30,000 sets of Huawei's more than 40 flagship products, serving over 70 carriers and 2 billion users worldwide

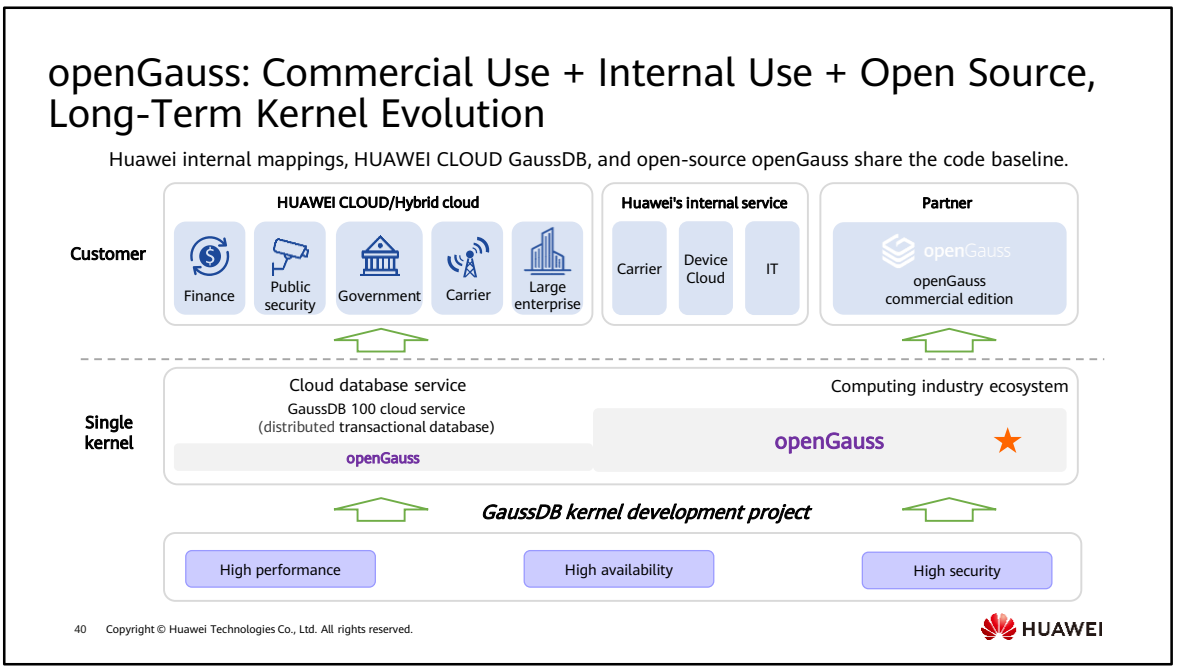
2019 – 2020

- GaussDB global release on May 15th, 2019
- Building an ecosystem with global partners
- Compatible with mainstream industry ecosystems and complete interconnection with industries such as finance
- openGauss goes open source on June 30, 2020.

2021 – Future

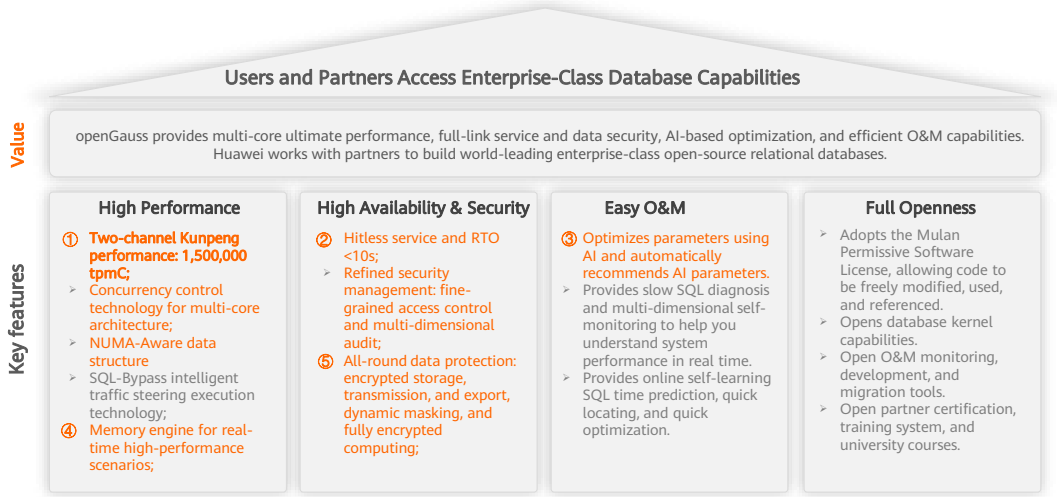
- Sharing enterprise-level data management capabilities
- Leading ecosystem construction
- Promoting the development of database education

Copyright © Huawei Technologies Co., Ltd. All rights reserved.



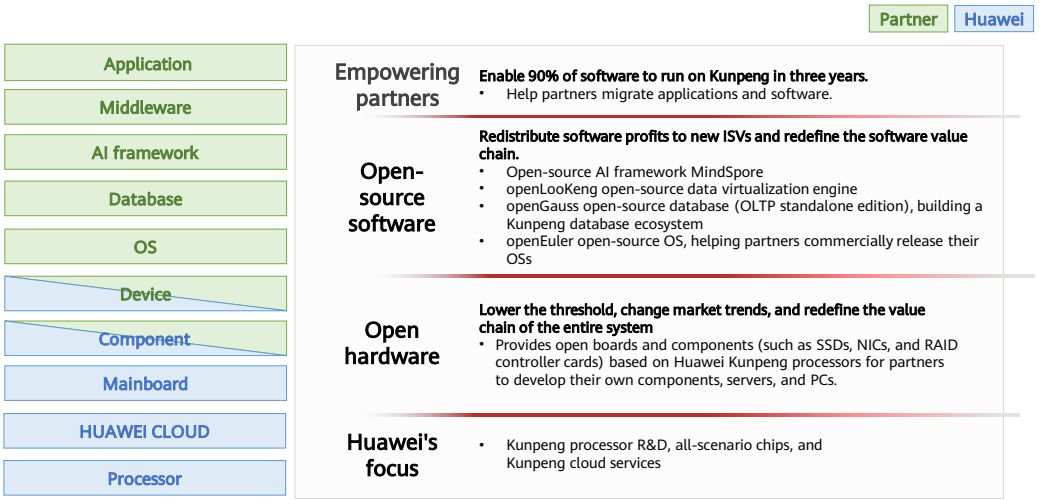
- Applications
- Transactions
  - Applications that require high concurrency, large data volume, and online transaction processing, such as e-commerce, finance, O2O, and telecom CRM/billing, can be deployed in primary/standby mode as required.
- IoT data
  - Many sensors and monitoring devices along with frequent sampling require a lot of device storage as well as concurrent operations and analysis. This is true for IoT scenarios, such as industrial monitoring, remote control, smart city, smart home, and IoV.

openGauss Positioning



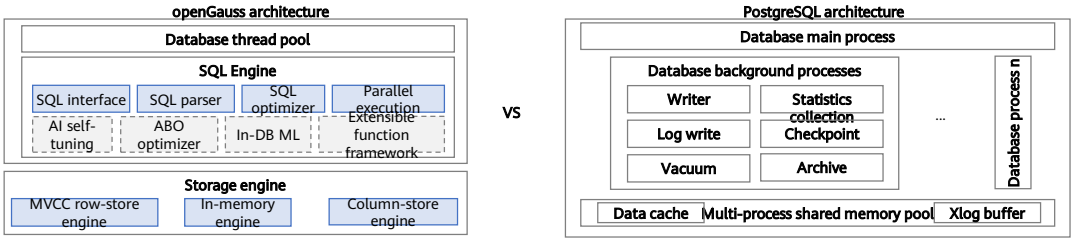
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Huawei Computing Business Strategy



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Comparing openGauss and PG (1)



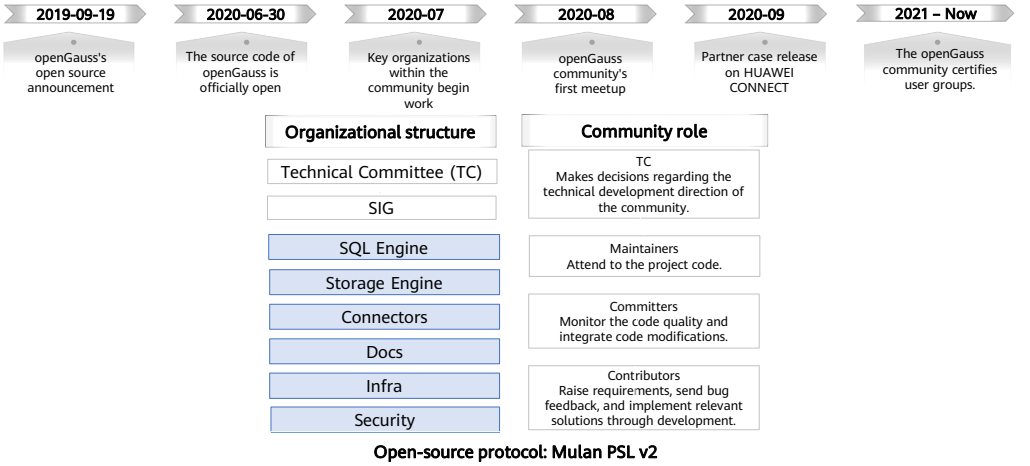
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Comparing openGauss and PG (2)

Key Differentiation Factor		openGauss	PostgreSQL
Transaction processing mechanism	Concurrency control	64-bit transaction ID. CSNs are used to resolve the dynamic snapshot expansion problem. The NUMA-Aware engine is optimized to resolve the "five big locks" problem.	Transaction IDs are wrapped around. The long-term running performance greatly fluctuates due to the ID wraparound period. Five big locks cause bottlenecks in transaction execution efficiency and multi-processor multi-core scalability.
	Logs and checkpoints	Incremental checkpoint, with the performance fluctuation less than 5%	Full checkpoint, with the short-term performance fluctuation more than 15%
	Kunpeng NUMA	NUMA reconstruction, cache-line padding, and native spin-lock	Weak NUMA multi-core capability; a single dual-CPU server < 600,000 tpmC
Data storage and organization	Multi-engine	Row-store, column-store, and in-memory engines; DFV storage under development and in-place update	Row-store only
Query optimizer	Optimizer	Supports CBO to absorb optimization capabilities in large enterprise scenarios.	Supports CBO. The optimization capability in complex scenarios is average.
	SQL parsing	ANSI/ISO standard SQL92, SQL99, SQL2003, and enterprise extension package	ANSI/ISO standard SQL92, SQL99, and SQL2003

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Community: Jointly Building an Ecosystem and Discussing the Development Direction



Open-source protocol: Mulan PSL v2

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

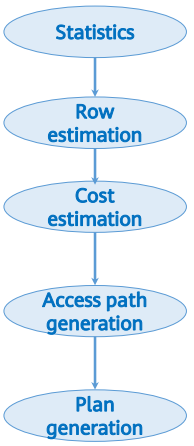
Contents

1. Database Overview
2. Introduction to openGauss
3. **openGauss Technical Specifications**
4. Basic Functions

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

openGauss Query Optimization



Physical optimization, based on table/column-level statistics

Cost estimation, based on base table statistics

Execution cost of the current operator, calculated based on the number of relational rows

Access path, calculated based on the execution costs of several operators

Convert the queried execution path into PlanTree and output it to the executor for query execution.

Physical optimization technologies:

1. Table/Column-level statistics

Describes features of base table data, including unique values and MCV values, which are used to estimate the number of rows.

2. Row estimation

Estimates the size of the baserel, joinrel, and aggregation result sets to prepare for cost estimation.

3. Cost estimation

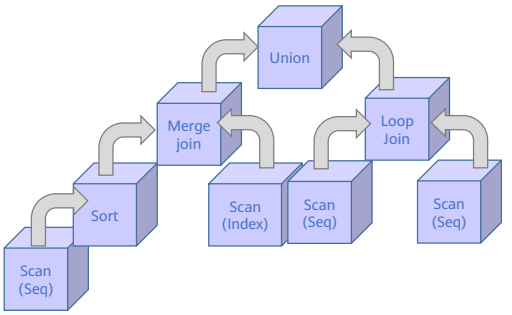
Estimates the execution costs of different operators based on the data volume. The sum of the costs of all operators is the total plan cost.

4. Access path generation

Solves the optimal path algorithm (for example, dynamic programming and genetic algorithm) to deal with the access path search process, with the minimum search space to find the optimal access path.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Execution Engine

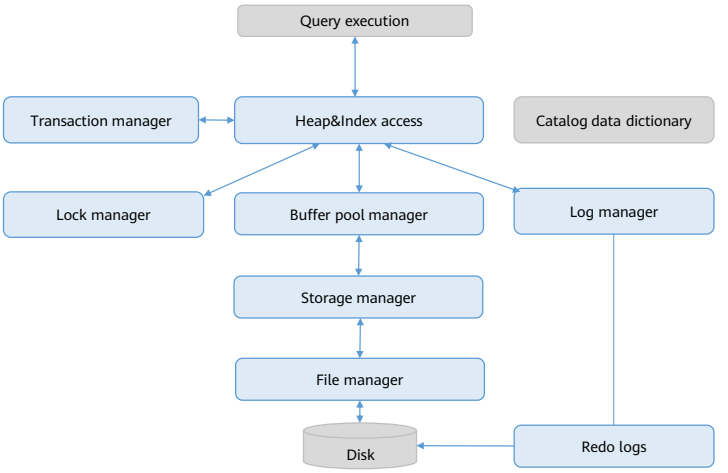


The relational database is used to calculate relation sets, and the execution engine is used as the control logic of the calculation and is implemented based on the relational calculation. The operators can be classified as follows:

1. **Scan plan node**
   The scan node extracts data from the underlying data source, which may come from the file system or the network. Generally, the scan node is located on the leaf node of the execution tree and functions as the input source of execution data. Typical scan nodes include SeqScan, IndexScan, and SubQueryScan.
   Key features: input data, leaf node, and expression filtering
2. **Control plan node**
   Generally, control operators do not map algebra operators, and are introduced by executors to complete some special processes, such as Limit, RecursiveUnion, and Union.
   Key feature: used to control the data process
3. **Materialize plan node**
   These operators generally refer to algorithm requirements. When operator logic processing is performed, lower-layer data needs to be cached. As the amount of data returned by lower-layer operators cannot be predicted in advance, it is necessary to consider scenarios where data cannot be completely stored in the memory, such as Agg and Sort.
   Key feature: Data is returned only after all data is scanned.
4. **Join plan node**
   These operators are used to deal with the most common join operations in the database. They are classified into MergeJoin, SortJoin, and HashJoin based on the processing algorithm and data source.
   Key feature: multiple inputs

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

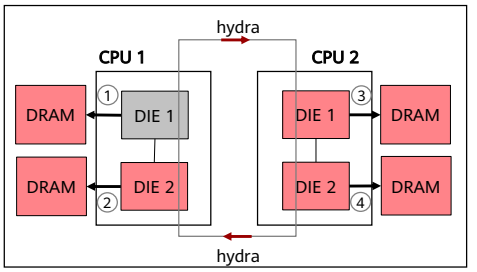
openGauss Storage Engine



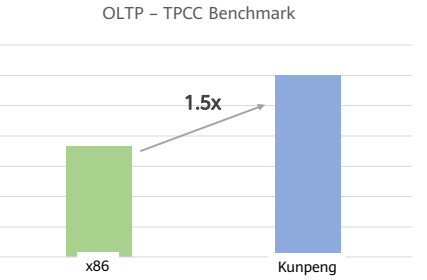
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Optimal Performance of openGauss on Multi-core Servers

Typical Kunpeng multi-core CPU architecture



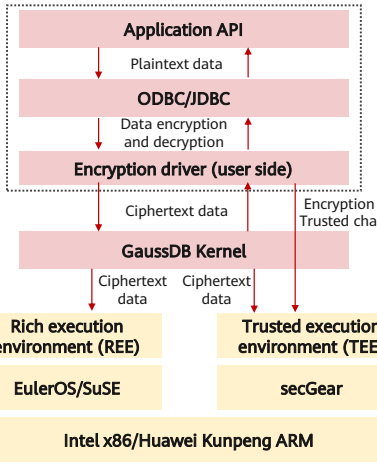
Two Kunpeng chips with 128 cores: 1,500,000 tpmC



Note: When two x86 Intel Xeon 6148 processors are used in the lab environment (configurations are identical except for the processors), the performance of various databases is as follows: Oracle < 1 million tpmC; PG < 700,000 tpmC; MySQL < 500,000 tpmC

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Fully-encrypted Equality Query



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Technical Specifications

Technical Specifications	Maximum Value
Database capacity	Varies depending on the OS and hardware
Size of a single table	32 TB
Size of data in a single row	1 GB
Size of a single field in each record	1 GB
Number of records in each table	2^{48}
Number of columns in a single table	250 to 1600 (varies with field type)
Number of indexes on a single table	Unlimited
Number of columns contained in a composite index	32
Number of constraints in a single table	Unlimited
Number of concurrent connections	10000
Number of partitions in a partitioned table	32768 (range partitioning)/64 (hash or list partitioning)
Size of each partition in a partitioned table	32 TB
Number of records in a single partition of a partitioned table	2^{55}

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Database Overview
2. Introduction to openGauss
3. Technical Specifications
4. **Basic Functions**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

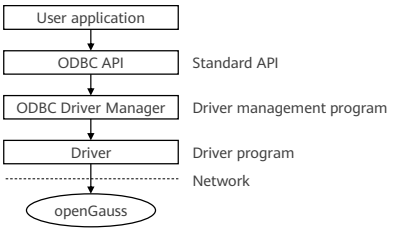
openGauss Supports Standard SQL

- SQL is a standard programming language used to control the access to databases and manage data in databases. SQL:2011 standards are classified into core features and optional features. Most databases do not fully support SQL standards.
- openGauss is a high-performance HA rational database that supports the SQL2003 standard and primary/standby deployment. In addition, openGauss supports most of the core features of SQL:2011 as well as some optional features, providing a unified SQL interface for users.
- With standard SQL, all database vendors use unified SQL interfaces, reducing the costs associated with learning languages and migrating applications.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Supports ODBC-based Development APIs

- Open Database Connectivity (ODBC) is a Microsoft API used to access databases based on the X/OPEN CLI. Applications interact with the database through the APIs provided by ODBC, which enhances their portability, scalability, and maintainability.
  - openGauss supports ODBC 3.5 in the following environments.
  - The ODBC Driver Manager running on UNIX or Linux can be either unixODBC or iODBC. Select unixODBC-2.3.0 here as the component for connecting to the database.
  - Windows has a native ODBC Driver Manager. You can locate **Data Sources (ODBC)** by choosing **Control Panel** > **Administrative Tools**.



ODBC system structure

OS	Platform
CentOS 6.4/6.5/6.6/6.7/6.8/6.9/7.0/7.1/7.2/7.3/7.4	x86_64
CentOS 7.6	ARM64
EulerOS 2.0 SP2/SP3	x86_64
EulerOS 2.0 SP8	ARM64

OS Supported by ODBC

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- openGauss supports ODBC 3.5 in the following environments.
- The ODBC Driver Manager running on UNIX or Linux can be either unixODBC or iODBC. Select unixODBC-2.3.0 here as the component for connecting to the database.
- Windows has a native ODBC Driver Manager. You can locate **Data Sources (ODBC)** by choosing **Control Panel** > **Administrative Tools**.
- Note: The current database ODBC driver is based on an open-source version and may be incompatible with the tinyint, smalldatetime, and nvarchar2 data types.

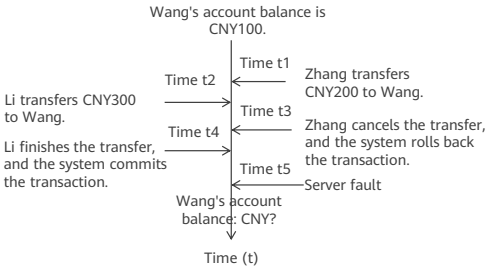
openGauss Supports JDBC-based Development APIs

- Java Database Connectivity (JDBC) is a Java API used to run SQL statements. It provides unified access APIs for different relational databases, based on which applications process data. The openGauss library supports JDBC 4.0 and requires JDK 1.8 for code compiling. It does not support JDBC-ODBC bridging.
  - openGauss supports the JDBC 4.0 standard API.
  - Standard ODBC and JDBC APIs are provided to ensure quick migration of user services to openGauss.
  - A function to connect JDBC to a third-party log framework has been added, enabling JDBC to interconnect with a third-party log framework to meet the log management and control requirements of users.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Database Transaction

- Transaction ***ACID***
  - *Atomicity (A)*: ensures that each transaction is treated as a single "unit", which either succeeds completely, or fails completely.
  - *Consistency (C)*: ensures that a transaction can only bring the database from one valid state to another, maintaining database invariants.
  - *Isolation (I)*: ensures that an unfinished transaction does not expose its result to subsequent other transactions before it is committed.
  - *Durability (D)*: ensures that the result of a committed transaction will not be lost due to faults which occur later.
- Example: Bank transfer (Zhang and Li transfer money to Wang.)

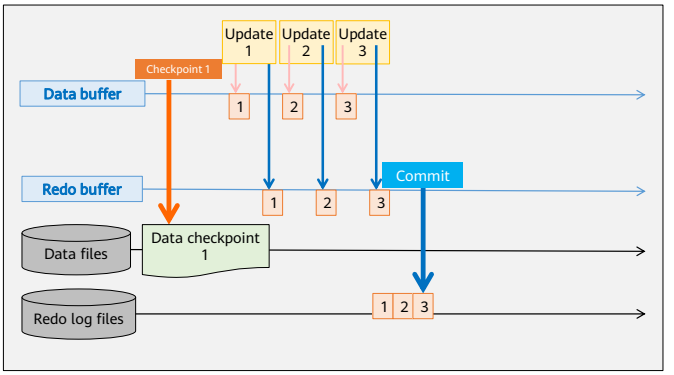


- The ACID features ensure that Wang's account balance is CNY400 after the server restarts.
- Atomicity: After Zhang cancels the transfer, the account balance of Wang is still CNY100.
- Consistency: Li's account balance is CNY300 less, and Wang's is CNY300 more. The sum of the two accounts remains unchanged.
- Isolation: The CNY200 transferred by Zhang is invisible to Li. Otherwise, the result is CNY700.
- Durability: After the server is restored from a fault, Li's transfer is still valid.
- Wang's account balance is CNY400.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Transaction support refers to the system's ability to ensure the atomicity, consistency, isolation, and durability (ACID) features of global transactions. Transaction support and data consistency assurance are the basic functions of most databases and the prerequisites for a database to satisfy transaction-based application requirements.
- Atomicity means that a transaction is composed of an indivisible unit of work. Operations performed in a transaction must all be finished, or have not been performed.
- Consistency means that a transaction is consistent within a system no matter when or how many concurrent transactions are ongoing.
- Isolation means that transactions are isolated for execution, almost as if each one is the only operation performed during the specified period planned by the system. If two transactions are executed within the same period and perform the same function, transaction isolation makes each of them regard itself as the only transaction using the system.
- Durability means that, after a transaction is complete, the changes made by the transaction to the database are permanently stored in the database and will not be rolled back.
- The default transaction isolation level is READ COMMITTED, ensuring no dirty data will be read.
- Transactions are categorized into single-statement transactions and transaction blocks. Their basic APIs are as follows:
  - Start transaction: Start a transaction. Commit: Commit a transaction. Rollback: Roll back a transaction.
- Set transaction: Set the isolation level, read/write mode, and delay mode. For details about the syntax, see the *openGauss Developer Guide*.

openGauss Database Transaction Mechanism



1. Transaction durability is implemented by using the write-ahead log (WAL) algorithm. When a transaction is committed, a redo log is written to disks using the WAL method.

2. Checkpoint: Write the data in the dirty buffer queue to the data file.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Supports Functions and Stored procedures

- Functions and stored procedures are important database objects. They encapsulate SQL statement sets used for certain functions so that the statements can be easily invoked. openGauss supports functions and stored procedures compliant with the SQL standard. These stored procedures are compatible with certain mainstream stored procedure syntax, improving their usability and offering the following advantages:
  - Allows customers to modularize program design and encapsulate SQL statement sets, making them easy to invoke.
  - Caches the compilation results of stored procedures to accelerate SQL statement set execution.
  - Allows system administrators to restrict permission to execute a specific stored procedure and control access to the corresponding type of data. This prevents access from unauthorized users and ensures data security.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- To process SQL statements, the stored procedure process assigns a memory fragment to store context association. Cursors are handles or pointers to context areas. With cursors, stored procedures can control alterations in context areas.
- Six levels of exception information are supported to facilitate the debugging of stored procedures.
- openGauss supports functions and stored procedures in the SQL standard, which enhances the usability of stored procedures.
- For details about how to use the stored procedure, see the *openGauss Developer Guide*.

openGauss Is Compatible with PG APIs

- The basic PostgreSQL publication contains only two client APIs:
  - libpq is included because it is a C language API on which many other client APIs depend.
  - ECPG is included because it relies on the server-side SQL syntax and is therefore very sensitive to changes in PostgreSQL.
- openGauss is compatible with the PostgreSQL clients and standard APIs, and can be seamlessly interconnected with PostgreSQL ecosystem tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Supports SQL Hints

- Plan hints enable you to specify a join order. You can also join, stream, and scan operations, the number of rows in a result, and redistribution skew information to tune an execution plan, improving query performance.
- openGauss supports SQL hints, which can override any execution plan and thus improve SQL query performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-choice) Which of the following is not a component of a database system?
   A. Database:
   B. DBMS
   C. Database application
   D. Database storage medium
2. (Single-choice) Which of the following statements about the ACID feature of database transactions is incorrect?
   A. A indicates atomicity, whereby operations in a transaction are either all successful or all failed.
   B. C refers to consistency, whereby the system status can only be the status before the transaction or after the transaction is successful. No inconsistent intermediate status can occur.
   C. I refers to availability, whereby the database system must provide the highest availability possible for transaction execution in order to ensure as many successfully executed transactions as possible.
   D. D refers to durability, whereby the status of a successful transaction can be maintained, even if the machine is powered off.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - 1. D
  - 2. C

Quiz

3. (Multiple-choice) Which of the following phases are involved in the development of data management technologies?
   A. Manual management
   B. File system
   C. DBS
   D. AI management

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - 3.ABC

Summary

- This chapter describes database development and evolution, database classification and data models, database system composition, and the openGauss database. This document describes the database definition and technology development history, architecture evolution of relational databases, main application scenarios, development history of openGauss database, product positioning, basic indicators, and basic functions.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recommendations

- openGauss help document:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/GettingStarted/GettingStarted.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Acronyms and Abbreviations

Acronym and Abbreviation	Full Spelling
DB	Database
DBMS	Database management system
DDL	Data definition language
DML	Data manipulation language
DBA	Database administrator
DBS	Database system
SQL	Structured query language
MPP	Massive parallel processing
OLTP	On-line transaction processing
OLAP	On-line analytical processing
DWS	Data warehouse service
RTO	Recovery time objective
JDBC	Java Database Connectivity
ODBC	Open Database Connectivity
WAL	Write-ahead log
CLI	Command-line interface
tpmC	transactions per minute C

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.

HUAWEI

02 Database Installation and Deployment



HUAWEI

Foreword

- openGauss is a relational database. It uses a client/server, single-process multi-thread architecture and supports standalone and one-primary multi-standby deployment. When primary/standby deployment is used, the standby node can be read too, and HA and read expansion are supported. This chapter describes how to set up the environment, how to install and deploy a database, and the functions and applications of openGauss-related O&M tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

Upon completion of this course, you will:

- Understand the installation and deployment of the openGauss database.
- Understand openGauss database connection and authentication.
- Understand the functions and operations of common openGauss database tools.
- Understand how to uninstall an openGauss database.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. openGauss Installation
2. Connection and Authentication
3. openGauss Tools
4. Database Uninstallation

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deployment Plan

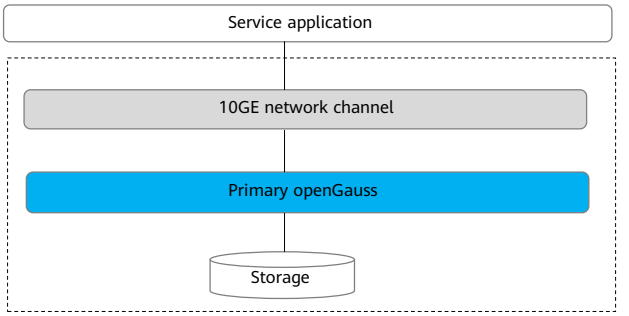
Deployment Mode	Technical Solution	High Availability (HA)	Basic Configuration Requirement	Service Scenario	Deployment Characteristics	Technical Specifications
Standalone	Standalone	HA is not supported.	Single equipment room	Physical machine	No reliability and availability requirements for the system. Suitable for trial use and commissioning scenarios.	RTO and RPO are not controllable. Instance-level disaster recovery (DR) is not supported. The system becomes unavailable when instance failures occur. Lost instance data cannot be recovered.
Primary/ standby	Primary and standby nodes	Instance failures can be withstood.	Single equipment room	Physical machine	There is no network delay between nodes. Instance failures in the database can be withstood. Suitable for scenarios with low reliability requirements.	RPO = 0 Instance fault RTO < 10s AZ-level DR is not supported. It is advised to configure primary and standby nodes for maximum availability.
One primary and multiple standbys	One primary and multiple standbys (Quorum)	Instance failures can be withstood.	Single equipment room	Physical machine	There is no network delay between nodes. Instance failures in the database can be withstood.	RPO = 0 Instance fault RTO < 10s AZ-level DR is not supported. Primary/standby synchronization is recommended. 2 to 4 copies.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Standalone: This configuration includes a single database instance.
- Primary/Standby: The system includes both primary and standby database instances. The primary instance supports both read and write operations, while the standby instance supports only read operations.
- One primary with multiple standbys: The system consists of one primary node and multiple standby nodes.
- Hot/Cold backup:
  - Cold backup: There is only a simple backup set that cannot provide services.
  - Hot backup: Backup databases can provide services for external systems.

Standalone Deployment

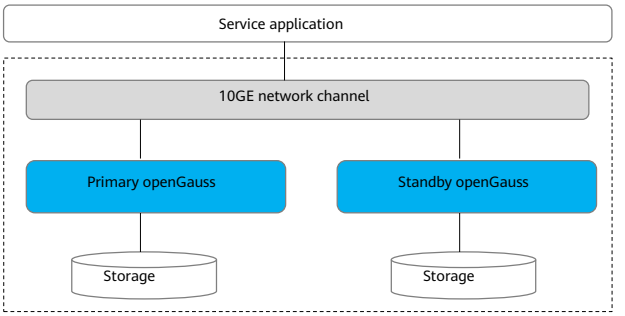
- The standalone deployment does not guarantee reliability or availability. There is only one data copy. Once data is damaged or lost, only physical backups can be used to restore data. Therefore, this deployment mode is primarily intended for trial users and functional testing in sandbox environments. It is not recommended for commercial or production use.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

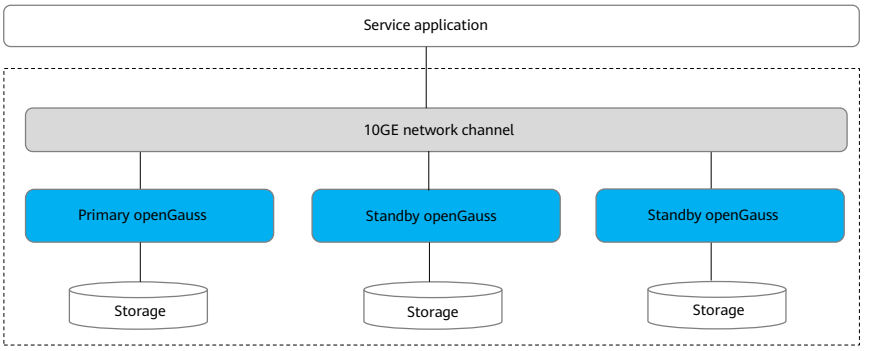
Primary/Standby Deployment

- In the primary/standby deployment, there are two data copies, one for the primary node and the other for the standby node. The standby node receives and plays back the logs.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deployment with One Primary and Multiple Standbys



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

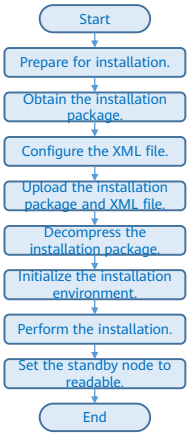
Deployment with One Primary and Multiple Standbys

- Multi-copy deployment offers protection against instance-level failures. It is suitable for scenarios that do not require equipment room–level DR but need to protect against hardware SPOFs.
- Typically, multi-copy deployment involves one primary node and two standby nodes, totaling three copies. This configuration offers 99.99% reliability, which is sufficient for most applications.
  - In primary/standby quorum replication, data is synchronized to at least one standby node to ensure the maximum performance.
  - If any of primary and standby nodes is faulty, services will not be affected.
  - There are three copies of data. If one node is faulty, the system still has two copies of data. Additionally, any standby node can be promoted to primary.
  - The primary and standby nodes cannot be deployed on the same physical machine.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Installation Process

Process	Description
Prepare for installation.	Before the openGauss installation, prepare the software and hardware environment and complete necessary configurations. This document provides the minimum requirements for the openGauss installation.
Obtain the installation package.	You need to download the installation package from the openGauss open-source community and check the package content.
Configure the XML file.	Before installing openGauss, create an XML configuration file. The configuration file contains information about the server where openGauss will be deployed, the installation path, IP address, and port. This file guides how to deploy the openGauss. You need to configure the XML configuration file based on deployment requirements.
Initialize the installation environment.	To initialize the installation environment, upload the installation package and the XML file, decompress the installation package, and use gs_preinstall to prepare the environment.
Perform the installation.	Use gs_install to install the openGauss.
Set the standby node to readable.	This operation is optional, but making the standby node readable improves data consistency.



HUAWEI

- This slide describes the standard installation process for openGauss. There is also a simplified installation process, simpler and faster than that of the enterprise edition. This simplified version is mainly for school or personal test environments.

Obtaining the Installation Package

- Step 1: Download the installation package of the corresponding platform from the openGauss open-source community.
  - Log in to the openGauss open-source community at https://opengauss.org/zh/download.html, select **2.0.0** in the **Version** field, and download the corresponding simplified installation package.
  - Click **Download**.
- Step 2: Check the installation package. Decompress the package and confirm that installation directory is correct and all the files are there.
  - Run the following command in the directory where the installation package is stored:

```
tar -jxf openGauss-x.x.x-openEuler-64bit.tar.bz2
ls -1b
```

  - The **ls** command should display information similar to the following:

```
total 90296
drwx------ 3 root root     4096 Mar 31 21:18 bin
drwx------ 3 root root     4096 Mar 31 21:18 etc
drwx------ 3 root root     4096 Mar 31 21:18 include
drwx------ 4 root root     4096 Mar 31 21:18 jre
drwx------ 5 root root     4096 Mar 31 21:18 lib
-rw------- 1 root root 92427499 Apr  1 09:43 openGauss-2.0.0-openEuler-64bit.tar.bz2
drwx------ 5 root root     4096 Mar 31 21:18 share
drwx------ 2 root root     4096 Mar 31 21:18 simpleInstall
-rw------- 1 root root       32 Mar 31 21:18 version.cfg
```


HUAWEI

- This chapter describes how to prepare and configure the environment for the simplified installation of openGauss. The simplified installation includes a single-node installation, and the installation of one primary node and one standby node.

Hardware Requirements

- The following table lists the minimum hardware requirements for openGauss server. During hardware planning for the actual product, you have to consider how much data will be involved and the database response speed you want to see.

Item	Configuration
Memory	It is recommended that there be at least 32 GB of memory available for function debugging. In performance testing and commercial deployment, it is recommended that there be at least 128 GB of memory for single-instance deployment. Complex queries require much more memory, so there may not be enough memory for high-concurrency scenarios. For high-concurrency scenarios, it is recommended that a large-memory server or load management be used to restrict concurrency on the system.
CPU	At least one 8-core 2.0 GHz CPU should be used for function debugging. For performance testing and commercial deployment, a single 16-core 2.0 GHz CPU is recommended for single-instance deployment. You can configure the CPUs for hyper-threading or non-hyper-threading. For individual developers, the minimum configuration is 2 cores with 4 GB of memory, but the recommended configuration is 4 cores and 8 GB of memory. Currently, openGauss supports only the Kunpeng-powered servers and x86_64-based universal PC servers.
Disk	Hard disks used for installing openGauss must meet the following requirements: • At least 1 GB is used to install the openGauss applications. • Each host requires at least 300 MB for data storage. • More than 70% of the remaining space is used for data storage. You are advised to configure the system disk to RAID 1 and data disk to RAID 5 and plan four groups of RAID 5 data disks for installing openGauss. RAID configurations are not described in this document. You can configure RAID by following instructions in the hardware vendor's manuals. Set **Disk Cache Policy** to **Disabled** to avoid data loss in the event of an unexpected power-off. openGauss supports using an NVMe SSD with the SAS API deployed in RAID mode as the primary storage device of the database.
Network requirements	300 Mbit/s or faster Ethernet. You are advised to bind two NICs for redundancy.

HUAWEI

- It is recommended that servers where the openGauss database is deployed all have the same software and hardware configurations.

Software Environment and Dependency Requirements

Software environment requirements

Software Type	Configuration Description
Operating system	• Arm: openEuler 20.03 LTS, openEuler 22.03 LTS, and KylinOS • x86: openEuler 20.03 LTS, openEuler 22.03 LTS, CentOS 7.6
File system	It is recommended that the number of remaining inodes be greater than 1.5 billion.
Tool	bzip2
Python	• openEuler supports Python 3.7.x. • CentOS supports Python 3.6.x. • KylinOS supports Python 3.7.x.

Software dependency requirements

Software	Recommended Version
libaio-devel	0.3.109-13
readline-devel	7.0-13
expect	--

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying OS Configurations (1)

- **Disable the OS firewall** to ensure that the openGauss can be used properly when the firewall is enabled.
  - Step 1: Change the value of **SELINUX** in the **/etc/selinux/config** file to **disabled**.
    - 1. Use **vim** to open the **config** file. `vim /etc/selinux/config`
    - 2. Change the value of SELINUX to disabled and enter **:wq** to save the change and exit. `SELINUX=disabled`
  - Step 2: Reboot the system. `reboot`
  - Step 3: Check whether the firewall is disabled.

    ```
    systemctl status firewalld
    ```

    - If the firewall status is **active (running)**, the firewall is not disabled. Go to step 4.
    - If the firewall status is **inactive (dead)**, the firewall is disabled. Skip step 4.
  - Step 4: Disable the firewall.

    ```
    systemctl disable firewalld.service
    systemctl stop firewalld.service
    ```

  - Step 5: Repeat step 1 to step 4 on each host.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Currently, EulerOS can only be installed when the firewall is disabled.
- openGauss-related services, protocols, IP addresses, and ports need to be added to the firewall whitelist of each host in openGauss.

Modifying OS Configurations (2)

- **Set the character set parameters.**
  - Set the same character set for all database nodes. You can add **export LANG=***Unicode* to the **/etc/profile** file.

```
vim /etc/profile
export LANG=en_US.UTF-8
```

- **Set the time zone and time.**
  - Ensure that the time zone and time on each database node are consistent.
  - Step 1: Run the following command to check whether the time and time zone of each database node are consistent: If they are not, perform steps 2 to 3.

```
date
```

  - Step 2: Run the following command to copy the **/etc/localtime** file to the **/usr/share/zoneinfo/** directory of each database node:

```
cp /usr/share/zoneinfo/$Locale/$Time zone /etc/localtime
```

  - Step 3: Set the time of each database node to the same time. For example:

```
date -s "Sat Sep 27 16:00:07 CST 2020"
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- *$Locale/$Time zone* indicates the locale and time zone to be set, for example, **Asia/Shanghai**.
- **(Optional) Disable the swap memory.**
  - Disabling swap ensures the access performance of the database and prevents the database buffer memory from being evicted to the disk. If the server doesn't have enough memory, you can enable swap to resolve the issue.
  - To disable swap, run the **swapoff -a** command on each node.

Modifying OS Configurations (3)

- **Disable RemoveIPC.**
  - On each database node, disable RemoveIPC. In CentOS 7.6, RemoveIPC is disabled by default, and you can skip this step.
  - Step 1: Change the value of **RemoveIPC** in the **/etc/systemd/logind.conf** file to **no**.
    - Use **vim** to open the **logind.conf** file.

      ```
      vim  /etc/systemd/logind.conf
      ```
    - Change the value of **RemoveIPC** to **no**.

      ```
      RemoveIPC=no
      ```
  - Step 2: Change the value of **RemoveIPC** in the **/usr/lib/systemd/system/systemd-logind.service** file to **no**.
    - Use **vim** to open the **systemd-logind.service** file.

      ```
      vim /usr/lib/systemd/system/systemd-logind.service
      ```
    - Change the value of **RemoveIPC** to **no**.

      ```
      RemoveIPC=no
      ```
  - Step 3: Reload the configuration parameters.

    ```
    systemctl daemon-reload
    systemctl restart systemd-logind
    ```
  - Step 4: Check whether the modification takes effect.

    ```
    loginctl show-session | grep RemoveIPC
    systemctl show systemd-logind | grep RemoveIPC
    ```
  - Step 5: Repeat steps 1 to 4 on each host.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying OS Configurations (4)

- **Set the NIC MTU.**
  - Set the NIC MTU on each database node to the same value.
  - Step 1: Query the NIC name of the server: `ifconfig`
  - In the example shown here, the server IP address is **10.244.53.173**, and the NIC name for the server is **eth0**.

```
[root@pekpopgsc100181 ~]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.244.53.173  netmask 255.255.254.0  broadcast 10.244.53.255
        inet6 fe80::f816:3eff:fef4:e0c3  prefixlen 64  scopeid 0x20<link>
        ether fa:16:3e:f4:e0:c3  txqueuelen 1000  (Ethernet)
        RX packets 99430  bytes 86571697 (82.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 72881  bytes 6419720 (6.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.31.8.93  netmask 255.255.240.0  broadcast 172.31.15.255
        inet6 fe80::f816:3eff:felc:7244  prefixlen 64  scopeid 0x20<link>
        ether fa:16:3e:1c:72:44  txqueuelen 1000  (Ethernet)
        RX packets 9408  bytes 2123616 (2.0 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 38  bytes 5793 (5.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

  - Step 2: Set the NIC MTU on each database node to the same value. For x86, the recommended MTU value is **1500**. For ARM, the recommended MTU value is **8192**.

    ifconfig *NIC name mtu value*

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Enabling Remote Login for **root**

- During openGauss installation, the **root** account is required for remote login. This section describes how to enable remote login for **root**.
- Step 1: Modify the **PermitRootLogin** configuration to enable remote login as **root**.
  - Open the **sshd_config** file.
  - Modify permissions for **root** using either of the following methods:
    - Comment out **PermitRootLogin no**.
    - Change the value of **PermitRootLogin** to **yes**.
    - Run the **:wq** command to save the modification and exit.

```
vim /etc/ssh/sshd_config
#PermitRootLogin no
PermitRootLogin yes
```

- Step 2: Modify the **Banner** configuration to delete the welcome information displayed when you connect to the system. The welcome information affects remote operations during the installation.
  - Edit the **sshd_config** file.
  - Comment out the line where **Banner** is.
  - Run **:wq** to save the modification and exit.

```
vim /etc/ssh/sshd_config
#Banner XXXX
```

- Step 3: Restart sshd to make the settings take effect:

```
systemctl restart sshd.service
```

- Step 4: Log in to the system as user **root** again.

```
ssh xxx.xxx.xxx.xxx
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Note: *xxx.xxx.xxx.xxx* is the IP address for installing the openGauss environment.

Installation User and User Group

- To minimize the account permissions during the installation and ensure system security of openGauss after the installation is complete, the installation script automatically creates a database installation user and this user will be used as the administrator for subsequent operations and maintenance of the openGauss.

User/Group Name	Type	Planning Suggestions
dbgrp	OS	Plan a separate user group, for example, **dbgrp**. The database installation user will belong to this group. This user group is specified by the **-G** parameter during installation environment initialization. If this user group does not exist, the installation script automatically creates it. Alternatively, you can create a user group before the installation by running **groupadd** dbgrp. User permissions are checked when the gs_preinstall script is executed. The gs_preinstall script automatically grants access and execution permissions for the installation directory and data directory to all users in this user group.
omm	OS	You are advised to plan the user **omm** for openGauss operation and maintenance. The user **omm** is the OS user specified by the **-U** parameter during initialization of the installation environment. If this user already exists, delete it or change the initial user. For security purposes, the group for this user is **dbgrp**.

- During openGauss installation, when gs_install is executed, a database user **omm**, with the same name as the installation user is created. This user has the highest operation permissions on the database. The initial password of this user is specified by the user.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Installing openGauss – Installing openGauss on a Single Node (1)

- **Prerequisites**
  - A user group and a common user have been created.
  - All the server OSs and networks are functioning properly.
  - Common users must have the read, write, and execute permissions on the database package decompression directory and installation directory, and the installation directory must be empty.
  - Common users have the execution permission for the downloaded openGauss package.
  - Before the installation, check whether the specified openGauss port is occupied. If the port is occupied, change the port number or stop the process that uses the port.
- **Procedure**
  - Step 1: Log in to the host where the openGauss package is installed as a common user and decompress the openGauss package to the installation directory.

    ```
    tar -jxf openGauss-x.x.x-openEuler-64bit.tar.bz2 -C /opt/software/openGauss
    ```
  - Step 2: Assuming the decompressed package is stored in **/opt/software/openGauss**, go to the **simpleInstall** directory.

    ```
    cd /opt/software/openGauss/simpleInstall
    ```
  - Step 3: Run the **install.sh** script to install openGauss.

    ```
    sh install.sh  -w xxxx
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Notes:
  - **-w**: initializes the database password (specified by **gs_initdb**). This parameter is mandatory for security purposes.
  - **-p**: specifies the openGauss port. If the port is not specified, the default value **5432** is used.
  - **-h|--help**: displays usage instructions.
  - After the installation, the database name is **sgnode**.
  - The database installation directory is **/opt/software/openGauss/data/single_node**, in which **/opt/software/openGauss** is where the installation packages are extracted, and **data/single_node** is the newly created database node directory.

Installing openGauss – Installing openGauss on a Single Node (2)

- **Procedure**
  - Step 4: After the installation is complete, use **ps** and **gs_ctl** to check whether the process is normal.

    Running **ps** command should display information similar to the following:

```
>> ps ux | grep gaussdb
omm      24209 11.9  1.0 1852000 355816 pts/0  Sl   01:54   0:33 /opt/software/openGauss/bin/gaussdb -D
/opt/software/openGauss/single_node
omm      20377  0.0  0.0 119880  1216 pts/0    S+   15:37   0:00 grep --color=auto gaussdb
```

    Running **gs_ctl** should display information similar to the following:

```
>> gs_ctl query -D /opt/software/openGauss/data/single_node
gs_ctl query ,datadir is /opt/software/openGauss/data/single_node
HA state:
    local_role                : Normal
    static_connections        : 0
    db_state                  : Normal
    detail_information        : Normal

Senders info:
    No information


 Receiver info:
No information
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Notes:
  - **-w**: initializes the database password (specified by **gs_initdb**). This parameter is mandatory for security purposes.
  - **-p**: specifies the openGauss port. If the port is not specified, the default value **5432** is used.
  - **-h|--help**: displays usage instructions.
  - After the installation, the database name is **sgnode**.
  - The database installation directory is **/opt/software/openGauss/data/single_node**, in which **/opt/software/openGauss** is where the installation packages are extracted, and **data/single_node** is the newly created database node directory.

Starting and Stopping openGauss

- Starting openGauss
  - Step 1: Log in to the primary database node as **omm**.
  - Step 2: Start openGauss:

```
gs_om -t start
```

Start openGauss:

```
>> gs_om -t start
Starting cluster.
=========================================
=========================================
Successfully started.
```

- Stopping openGauss
  - Step 1: Log in to the primary database node as **omm**.
  - Step 2: Stop openGauss:

```
gs_om -t stop
```

Stop openGauss:

```
>> gs_om -t stop
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
```

21 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- An HA cluster must be started in HA mode. If the cluster is started in standalone mode, you need to restore the HA relationship by running the **gs_ctl build** command. For details about how to use the **gs_ctl** tool, see the *Tool Reference*.
- For details about how to start and stop nodes and availability zones (AZs), see "Server Tools > gs_om" in the *Tool Reference*.
- If starting or stopping openGauss fails, troubleshoot the problem based on log details. If the startup fails due to a timeout, you can run the following command to change the startup timeout interval, which is 300s by default:
  - gs_om -t start --time-out=300

Contents

1. openGauss Installation
2. **Connection and Authentication**
3. openGauss Tools
4. Database Uninstallation

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Setting the Client Authentication Policy

- The current default values for parameters in the openGauss configuration file (**pg_hba.conf**) are all for standalone deployment. Applications can set parameters as needed by invoking **gs_guc**. For more details, see *Developer Guide*. Set the client authentication policy and send semaphores to the database process.

```
gs_guc [ set | reload ] [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -h "HOSTTYPE DATABASE
USERNAME IPADDR-WITH-IPMASK AUTHMEHOD authentication-options"
```

- **set**

Modifies only parameters in the configuration file.

- **reload**

Modifies the parameters in the configuration file and sends semaphores to the database process for reloading the configuration file.

- **-N**

Specifies the name of the host to be set.
Value: the name of the existing host
When this parameter is set to **ALL**, all the hosts in openGauss are configured.

- **-I INSTANCE-NAME**

Specifies the name of the instance to be configured.
Value: the name of the existing instance
When this parameter is set to **ALL**, all the instances in the host are to be set.

- **-D**

Specifies the openGauss instance directory where the command is run. When the **encrypt** command is used, this parameter indicates the directory where the generated password file is stored.

- **-h host-auth-policy**

Specifies the client authentication policy added to the **pg_hba.conf** configuration file. Values supported:

  - HOSTTYPE DATABASE USERNAME IPADDR IPMASK [authentication-options]
  - HOSTTYPE DATABASE USERNAME IPADDR-WITH-IPMASK [authentication-options]
  - HOSTTYPE DATABASE USERNAME HOSTNAME [authentication-options]

The **HOSTTYPE** parameter is mandatory and can be set to any of the following values:

  - Local
  - Host
  - Hostssl
  - hostnossl

local is a Unix domain socket. **host** is a common or SSL-encrypted TCP/IP socket. **hostssl** is an SSL-encrypted TCP/IP socket. **hostnossl** is a TCP/IP-only socket. The **authentication-options** parameter is optional and can be set to any of the following values:

  - trust
  - reject
  - md5
  - sha256
  - cert
  - gss

For details about the parameters, see their description in the **pg_hba.conf** configuration file.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Note**: **authentication-options** supports the following options:
  - **trust**: no authentication. Remote hosts are not allowed to access openGauss if **trust** is configured.
  - **reject**: access rejected
  - **md5**, **sha256**, and **cert**: client certificate authentication
  - **gss**: kerberos authentication
- **gs_guc set -N** *all* **-I** *all* **-h** *"host replication testuser 10.252.95.191/32 sha256"*
  Total instances: 2. Failed instances: 0. Success to perform gs_guc!

gsql Client Connection – Confirming the Connection Information

- The client tool connects to the database through the primary database node. Before connecting to the database, you need the IP address and port for the server where the primary database node is located.
- Step 1: Log in to the primary database node as user **omm**.
- Step 2: Run the **gs_om -t status--detail** command to query instances in openGauss.

```
gs_om -t status --detail
[ DBnode State ]
node        node_ip         instance                            state
--------------------------------------------------------------------------------
1  plat1 192.168.0.11  5001 /srv/BigData/gaussdb/data1/dbnode Normal
```

  - In the command output shown here, **192.168.10.11** is the IP address of the server where the primary database node instance is deployed. The data directory of the primary database node is **/srv/gaussdb/data1/dbnode**.
- Step 3: Check the port for the primary database node. Check the port in the **postgresql.conf** file in the data directory of the primary database node obtained in step 2. For example:

```
cat /srv/BigData/gaussdb/data1/dbnode/postgresql.conf | grep port
port = 8000    # (change requires restart)
#comm_sctp_port = 1024    # Assigned by installation (change requires restart)
# supported by the operating system:
      # e.g. 'localhost=10.145.130.2 localport=12211 remotehost=10.145.130.3 remoteport=12212, localhost=10.145.133.2 localport=12213
remotehost=10.145.133.3 remoteport=12214'
      # e.g. 'localhost=10.145.130.2 localport=12311 remotehost=10.145.130.4 remoteport=12312, localhost=10.145.133.2 localport=12313
remotehost=10.145.133.4 remoteport=12314'
      #   %r = remote host and port
alarm_report_interval = 10
support_extended_features=true
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **8000**: the port of the primary database node.

Connecting to a Database Locally Using gsql (1)

- gsql is a CLI database connection tool provided by openGauss. gsql provides both basic and more advanced database functions to facilitate user operations. By default, if a client idle after connecting to a database, the client automatically disconnects from the database after a period specified by **session_timeout**. To disable session timeouts, set **session_timeout** to **0**.
- Step 1: Log in to the primary database node as **omm**.
- Step 2: Connect to the database. After the database is installed, the **postgres** database is generated by default. The first time you connect, you can connect to this database.

  Run the following command to connect to the database:

  ```
  gsql -d postgres -p 8000
  ```

  postgres is the name of the database to be connected, and **8000** is the port of the primary database node. Replace the values as required.

  If information similar to the following is displayed, the connection was successful:

  ```
  gsql ((openGauss 2.0.0 build 78689da9) compiled at 2021-03-31 21:03:52 commit 0 last mr  )
  Non-SSL connection (SSL connection is recommended when requiring high-security)
  Type "help" for help.

  postgres=#
  ```

  If you log in and connect to the database as administrator **omm**, **DBNAME=#** is displayed. If you log in and connect to the database as a common user, **DBNAME=>** is displayed. **Non-SSL connection** indicates that the database connection is not using SSL. For improved security, you are advised to connect to the database using SSL.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This section describes how to use gsql to connect to a database. For details about its usage, see the *Tool Reference*.

Connecting to a Database Locally Using gsql (2)

- Step 3: Change the password after your first login. (You do not need to change the password in openGauss 2.0.0 and later versions.) The initial password is set manually during openGauss database installation. For details, see "Installing the openGauss > Executing Installation" in the *Installation Guide*. You need to change the password during initial login. You can use the following command:

```
postgres=# ALTER ROLE omm IDENTIFIED BY 'newpassword' REPLACE 'XuanYuan@2012';
```

- Step 4: Exit the database.

```
postgres=# \q
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Overview of System Catalogs and System Views

- System catalogs store structured metadata of openGauss. They are the source of information used by openGauss to control system operations and are a core component of the database system.
- System views provide ways to query the system catalogs and internal database status.
- System catalogs and system views are visible to either system administrators or all users.
- You can delete and re-create system catalogs, add columns to them, and insert and update values in them, but doing so may make system information inconsistent and cause system faults. Generally, users should not modify system catalogs or system views, or rename their schemas. They are automatically maintained by the system.
- **Notice**: Do not add, delete, or modify system catalogs because doing so will result in exceptions or even openGauss unavailability.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. openGauss Installation
2. Connection and Authentication
3. **openGauss Tools**
4. Database Uninstallation

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Tools

Category	Tool	Description
Client tools	gsql	gsql is a CLI database connection tool provided by openGauss. Users can use gsql to connect to a server and perform operations and maintenance on the server. In addition to basic functions for performing operations on a database, gsql provides several other more advanced features.
Server tools	gs_check	gs_check has been enhanced, incorporating various check tools, such as gs_check and gs_checkos. It helps users check all sorts of items, such as openGauss runtime, the OS, network, and database environments, and can perform comprehensive environmental checks before major operations in openGauss. gs_check helps ensure smoother operations.
	gs_checkos	gs_checkos checks OS version information, control parameters, and disk configurations, and configures control parameters, I/O parameters, network parameters, and THP services.
	gs_collector	If there is a problem with openGauss, gs_collector collects OS, log, and configuration file information to make troubleshooting easier.
	gs_dump	gs_dump exports database information. It can export complete, consistent data of database objects (such as databases, schemas, tables, and views) without affecting normal access for database users.
	gs_dumpall	gs_dumpall exports database information. It can export complete and consistent data of the database without affecting normal access for database users.
	gs_guc	gs_guc sets parameters in the openGauss configuration file (**postgresql.conf** or **pg_hba.conf**). The default parameter values in the configuration file are for standalone deployment, but you can modify parameter values as needed by invoking gs_guc.
	gs_om	openGauss provides gs_om to help you maintain openGauss, including starting openGauss, stopping openGauss, querying openGauss status, generating static configuration files, updating dynamic configuration files, replacing SSL certificates, starting and stopping Kerberos authentication, and displaying help and version information.
	gs_restore	gs_restore, provided by openGauss, is used to import data that was exported using **gs_dump**. It can also be used to import files exported by **gs_dump**.
	gs_ssh	gs_ssh, provided by openGauss, helps users run the same command on multiple nodes in openGauss.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- openGauss provides client and server applications (tools) to help users better maintain openGauss. The preceding table lists all the tools provided by openGauss. The tools are stored in *$GPHOME*/**script** and *$GAUSSHOME*/**bin**, where the database servers are installed.

Client Tools – gsql

- After a database is deployed, you need certain tools to connect to the database for operations and commissioning. openGauss provides some convenient tools to help you connect to your database. You can use these tools to easily connect to a database and perform various operations.
- gsql is a CLI database connection tool provided by openGauss. Users can use gsql to connect to a server and perform operations and maintenance on the server. In addition to basic functions for performing operations on a database, gsql also provides several more advanced features.
- Basic functions:
  - **Connect to the database.** For details, see section "Connection and Authentication."
  - **Run SQL statements.** Interactively entered SQL statements and specified SQL statements in a file can be run.
  - **Run meta-commands.** Meta-commands help the administrator view database object information, query cache information, format SQL output, and connect to a new database.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Connect to a database**: For details, see "Working with Databases > Connecting to a Database > Using gsql to Connect to a Database > Remotely Connecting to a Database" in the *Developer Guide*.
- If the gsql client is used to connect to a database, the connection timeout interval will be 5 minutes. If the database is unable to establish a connection and authenticate the identity of the client within this period, gsql will time out and exit.

Serveaxr Tools

- During the use of openGauss, installation, upgrade, and health management are all involved. To facilitate openGauss maintenance, openGauss provides a set of management tools.
  - gs_check
  - gs_checkos
  - gs_checkperf
  - gs_clean
  - gs_collector
  - gs_dump
  - gs_dumpall
  - gs_guc
  - gs_om
  - gs_restore
  - gs_ssh

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_check (1)

- gs_check has been improved, incorporating various check tools, such as gs_check and gs_checkos. These tools allow you to check openGauss runtime; the OS, network, and database running environments, and they let you perform comprehensive environmental checks before major operations in openGauss, ensuring smoother database operations.
- Notes:
  - The **-i** or **-e** arguments must be used. **-i** specifies a single item to be checked, and **-e** specifies an inspection scenario where multiple items will be checked.
  - If **-i** is not set to a **root** item or no such items are contained in the checklist of the scenario specified by **-e**, you do not need to enter the name or password of user **root**.
  - You can run **--skip-root-items** to skip **root** items.
  - If the MTU values are inconsistent, the check may be slow or check process may become unresponsive. If the inspection tool displays a message, change the MTU values of the nodes to be the same and then repeat the inspection.
  - If the switch does not support the configured MTU value, process response failures may be caused by communication problems even if the MTU values are the same. In this case, you need to adjust the MTU based on the switch.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_check (2)

- Syntax:
  - Perform a single-item check:

```
gs_check -i ITEM [...] [-U USER] [-L] [-l LOGFILE] [-o OUTPUTDIR] [--skip-root-items][--set][--routing]
```

  - Perform a scenario-specific check:

```
gs_check -e SCENE_NAME [-U USER] [-L] [-l LOGFILE] [-o OUTPUTDIR] [--skip-root-items] [--time-out=SECS][--set][--routing][--skip-items]
```

  - Display help information:

```
gs_check -? | --help
```

  - Display version information.

```
gs_check -V | --version
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- -U
  - Specifies the name of the user running openGauss.
  - Value range: name of the user running the cluster
- -L
  - Locally performs the check.
- -i
  - Specifies a check item.
  - Its format is **-i Check***XX*. For details about check items, see the openGauss status checklist.
- -e
  - Specifies scenario-specific check items. Default scenarios include **inspect** (routine inspection), **upgrade** (pre-upgrade inspection), **binary_upgrade** (local pre-upgrade inspection), **health** (health check inspection), and **install** (installation). You can also define scenarios as required.
- -l
  - Specifies the log file path. The path must end in ".log".
- -o
  - Specifies where the check results will be saved.

Server Tools – gs_checkos

- gs_checkos helps you check the OS version, various control parameters and disk configurations, and it lets you configure control parameters, I/O parameters, network parameters, and THP services.
- Prerequisites:
  - The hardware and network are working properly.
  - Among the hosts, the mutual trust relationships for **root** are normal.
  - Only **root** is authorized to run the **gs_checkos** command.
- Syntax:
  - Check OS information. `gs_checkos -i ITEM [-f HOSTFILE] [-h HOSTNAME] [-X XMLFILE] [--detail] [-o OUTPUT] [-l LOGFILE]`
  - Display the help information. `gs_checkos -? | --help`
  - Display the version information. `gs_checkos -V | --version`

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- -I
  - Specifies the number of a check item. The format is -i A, -i B1, -i A1 -i A2, or -i A1,A2.
  - Value range: A1...A14 and B1...B8
  - Note: Items from A1 to A14 indicate that OS parameters need to be checked but not to be set. Items from B1 to B8 indicate that system parameters need to be set to expected values. Items A and items B cannot be simultaneously set.
- -f
  - Specifies the host name list file.
  - Note: The **-f** and **-h** parameters cannot be used together.
  - Value range: a host name list
- -h
  - Specifies the name of a host to be checked. You can specify multiple hosts and separate the host names with commas (,).
  - Value range: openGauss host name. If no host is specified, the current host is checked.
- -X
  - Specifies the openGauss XML configuration file.

Server Tools – gs_checkperf (1)

- openGauss provides the gs_checkperf tool for you to routinely check the openGauss-level performance (for instance, host CPU usage, GaussDB CPU usage, and I/O usage), host-level performance (CPU, memory, and I/O usage), session-/process-level performance (CPU, memory, and I/O usage), and SSD performance (write and read performance). This helps you track the load on openGauss and fine-tune database performance accordingly.
- Prerequisites:
  - openGauss is running properly and is not in read-only mode.
  - Services on the database are running normally.
- Note:
  - The monitoring information of gs_checkperf comes from tables in PMK mode. If no ANALYZE operation is performed on such tables, gs_checkperf execution may fail. Typical error information is as follows:

```
LOG: Statistics in some tables or columns(pmk.pmk_snapshot.snapshot_id) are not collected.
HINT: Do analyze for them in order to generate optimized plan.
```

  - To fix this error, log in to a primary database node, connect to the **postgres** database, and run the following SQL statements:

```
analyze pmk.pmk_configuration;
analyze pmk.pmk_meta_data;
analyze pmk.pmk_snapshot;
analyze pmk.pmk_snapshot_dbnode_stat;
analyze pmk.pmk_snapshot_datanode_stat;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_checkperf (2)

- Syntax:
  - Check the SSD performance (as **root**).

```
gs_checkperf -U USER [-o OUTPUT] -i SSD [-l LOGFILE]
```

  - Check the openGauss performance (as the openGauss installation user).

```
gs_checkperf [-U USER] [-o OUTPUT] [-i PMK] [--detail] [-l LOGFILE]
```

  - Display the help information.

```
gs_checkperf -? | --help
```

  - Display the version information.

```
gs_checkperf -V | --version
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- -h
  - Specifies the name of the host whose information is to be collected.
  - Value range: host names. If this parameter is not specified, information about all hosts is collected by default.
- -f
  - Specifies the host name list file. The file is a plain text file.
  - Note: The **-f** and **-h** parameters cannot be used together.
  - Value range: a host name list
- -o
  - Compresses the logs into a single package and then saves it to a specified folder.
  - If this parameter is not specified, the check results are saved as a package into the directory specified by **tmpMppdbPath** in the configuration file.
  - If **tmpMppdbPath** is not configured in the configuration file, the check results are saved to **/tmp/***username***_mppdb/** by default.
- -l
  - Specifies a log file and its save path.

Server Tools – gs_checkperf (3)

Category	Performance Parameter	Description
openGauss level	Host CPU usage	CPU usage of the host
	GaussDB CPU usage	CPU usage of the GaussDB
	Shared memory hit rate	Hit rate of the shared memory
	In-memory sort ratio	Ratio of completed sorts in memory
	I/O usage	Number and time of file reads and writes
	Disk usage	Number of file writes, average write time, and maximum write time
	Transaction statistics	Number of current SQL executions and sessions
Host level	CPU usage	Host CPU usage, including CPU busy time and CPU idle time
	Memory usage	Host memory usage, including total physical memory and used memory
	I/O usage	Number and time of file reads and writes
Session/Process level	CPU usage	Session CPU usage, including CPU busy time and CPU idle time
	Memory usage	Session memory usage, including total physical memory and used memory
	I/O usage	Number of shared buffer hits in a session
SSD performance (Only **root** can view it.)	Write performance	The **dd** command (**flag=direct bs=8M count=2560**) is used to write data into an SSD every 10 seconds.
	Read performance	The **dd** command (**flag=direct bs=8M count=2560**) is used to read data from an SSD every 7s.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Performance check items

Server Tools – gs_clean

- If a database breaks down unexpectedly, there may be residual temporary tables left over. gs_clean can clear out these tables.
- Syntax: Connect to the primary DN and clear out any residual temporary tables from transactions on the primary DN.

```
gs_clean [OPTION ...] [USERNAME]
```

- Parameter description: The values of **OPTION** are as follows:
  - -a, --all
    Clears all residual temporary tables in available databases.
  - -h, --host=HOSTADDRESS
    Specifies the IP address of the host where the target DN is located.
    Value range: IP address of the host where the target DN is located
    Default value: **localhost**
  - -p, --port=PORT
    Specifies the port number of the primary DN.
    Value range: port number of the target DN
    Default value: **5432** If the environment variable *PGPORT* of the OS has been set, the value of *PGPORT* is used by default.
  - -q, --quiet
    Prints only error information in quiet mode.
  - -r, --rollback
    Rolls back all abnormal transactions.
  - -t, --timeout=SECS
    Specifies the timeout mechanism.
    Default value: **5**, in seconds.
  - -U, --username=USERNAME
    Specifies the name of the user for connecting to the database.
    Value range: a valid username in the database
    Default value: *OS username*
  - -v, --verbose
    Outputs more detailed restoration information.
  - -V, --version
    Outputs version information about this tool.
  - -w, --no-password
    Specifies that no password authentication is required.
  - -W, --password=PASSWORD
    Specifies that password authentication is required.
    Value range: a valid string. The password must meet complexity requirements.
  - -e, --exclusive
    Only deletes temporary tables.
  - -j, --jobs
    Specifies the number of concurrent tasks for clearing out two-phase residual files.
    Value range: 1–10
    The default value is **3**.
  - -?, --help
    Displays the help information.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_collector

- If openGauss becomes faulty, gs_collector helps with troubleshooting by collecting logs, configuration files, and information about the OS. You can use the **-C** parameter to specify the information to be collected.
- Prerequisites: An OS tool (for example, gstack) that gs_collector depends on has been installed. If it is not installed, an error message is displayed, and corresponding collection item is skipped.
- Syntax:
  - Collect logs (as a non-**root** user).

    ```
    gs_collector --begin-time= "BEGINTIME" --end-time= "ENDTIME" [-h HOSTNAME | -f HOSTFILE] [--keyword=KEYWORD] [--speed-limit=SPEED] [-o OUTPUT] [-l LOGFILE] [-C CONFIGFILE]
    ```

  - Display the help information.

    ```
    gs_collector -? | --help
    ```

  - Display the version information.

    ```
    gs_collector -V | --version
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- -h
  - Specifies the name of the host whose information is to be collected.
  - Value range: host names. If this parameter is not specified, information about all hosts is collected by default.
- -f
  - Specifies the host name list file. The file is a plain text file.
  - Note: The **-f** and **-h** parameters cannot be used together.
  - Value range: a host name list
- -o
  - Saves collected logs as a package into a specified folder.
  - If this parameter is not specified, the check results are saved as a package into the directory specified by **tmpMppdbPath** in the configuration file.
  - If **tmpMppdbPath** is not configured in the configuration file, the check results are saved as a package into the **/tmp/***username*_**mppdb/** directory by default.
- -l
  - Specifies a log file and its save path.

Server Tools – gs_dump (1)

- gs_dump, provided by openGauss, is used to export database information. You can export a database or its objects (such as schemas, tables, and views). The database can be the default **postgres** database or a user-specified database.
- gs_dump is executed by the OS user **omm**.
- When gs_dump is used to export data, other users can still access (read and write) the openGauss database.
- gs_dump can export complete and consistent data. For example, if gs_dump has started exporting database A at T1, data of the database at T1 will be exported, but any modifications to the database after T1 will not be exported.
- gs_dump can export database information as a plain-text SQL script or as an archive file.
  - Plain-text SQL script: It contains the SQL statements required to restore the database. You can use gsql to execute the SQL script. With only minor modifications, the SQL script can rebuild a database on other hosts or database products.
  - Archive file: It contains data required to restore the database. It can be a .tar file, a directory, or a custom format. For details, see the table on the next slide. The export result must be used with gs_restore to restore the database. The system allows users to select or even to sort the content to be imported.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_dump (2)

- gs_dump can create four export file formats, which are specified by the **-F** or **--format** option. The following table lists the file formats.

Format	Value of -F	Description	Suggestion	Import Tool
Plain-text	p	A plain-text script file containing SQL statements and commands. The commands can be executed on gsql, a command line terminal, to recreate database objects and load table data.	You are advised to use plain-text export files for small databases.	Before using gsql to restore database objects, you can use a text editor to edit the file as required.
Custom	c	A binary file that enables the restoration of all or just selected database objects from an exported file.	You are advised to use custom archive files for medium or large databases.	You can use gs_restore to import database objects from an exported custom archive file.
Directory	d	A directory containing directory files and the data files of tables and BLOBs.	-	
.tar	t	A .tar archive file that allows for the restoration of all or selected database objects from an exported file. It cannot be further compressed and has an 8-GB limitation on the size of a single table.	-	

- Syntax:

```
gs_dump [OPTION]... [DBNAME]
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Note: To reduce the size of an exported file, you can use gs_dump to compress it to a plain-text file or custom file format. By default, plain-text files are not compressed when being generated. When a custom archive file is generated, a medium level of compression is applied by default. Archived exported files cannot be compressed using gs_dump. Data objects in plain-text file format cannot be imported using gsql if the plain-text file was compressed when being exported.
- Note:
  - Do not modify any exported file or content, or restoration may fail.
  - To ensure the data consistency and integrity, gs_dump acquires a shared lock on the table to be dumped. If another transaction has a shared lock on the table, gs_dump waits until this lock is released and then locks the table for the data dump. If the table cannot be locked within a certain amount of time, the dump will fail. You can change how long it waits before timing out by configuring the **--lock-wait-timeout** parameter.
- The description is as follows:
  - **gs_dump** -p *port_number* postgres -f dump1.sql or export *PGDATABASE*=postgres; **gs_dump** -p *port_number* -f dump1.sql
- Do not add a short or long option before **dbname**. **dbname** specifies the database to connect to. Do not use the **-d** option before that parameter.

Server Tools – gs_dumpall (1)

- gs_dumpall, provided by openGauss, is used to export all openGauss database information, including data of the default database **postgres**, user-defined databases, and common global objects of all openGauss databases.
- gs_dumpall is executed by the OS user **omm**.
- When gs_dumpall is used to export data, other users can still access (read and write) databases.
- gs_dumpall can export complete and consistent data. For example, if gs_dumpall is enabled to export openGauss database at T1, the openGauss data status at the time point T1 is exported, but modifications to openGauss after T1 will not be exported.
- gs_dumpall exports all databases in two parts:
  - gs_dumpall exports all global objects, including information about database users and groups, tablespaces, and attributes (for example, global access permissions).
  - gs_dumpall calls gs_dump to export SQL scripts from each openGauss database, which contain all the SQL statements required to restore databases.
- The exported files are both plain-text SQL scripts. Use gsql to execute them to restore openGauss databases.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_dumpall (2)

- Note:
  - Do not modify any exported file or content, or restoration may fail.
  - To ensure the data consistency and integrity, gs_dumpall acquires a shared lock on the table to be dumped. If another transaction has acquired a shared lock on the table, gs_dumpall waits until this lock is released and then locks the table for dumping. If the table cannot be locked in the specified time, the dump will fail. You can customize the timeout interval to wait for lock releasing by specifying the **--lock-wait-timeout** parameter.
  - During an export, gs_dumpall reads all the tables in a database, so you need to connect to the database as an openGauss administrator to export a complete file. When you use gsql to execute SQL scripts, administrator permissions are also required to add users and user groups, and create databases.
- Syntax:

```
gs_dumpall [OPTION]...
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Common parameters:
- -f, --filename=FILENAME
  - Writes the output to the file specified. If this option is omitted, the standard output is used.
- -v, --verbose
  - Enables verbose output. This will cause gs_dumpall to output detailed object comments and start/stop times to the dump file, and progress messages to standard error.
- -V, --version
  - Prints the gs_dumpall version and exits.
- --lock-wait-timeout=TIMEOUT
  - Will prevent gs_dumpall from waiting to obtain shared table locks at the beginning of the dump. If a table cannot be locked within the configured timeout period, the task will fail. The timeout interval can be specified in any of the formats accepted by **SET statement_timeout**.
- -?, --help
  - Displays gs_dumpall help for the command line parameters and exits.

Server Tools – gs_guc (1)

- The current default values for parameters in the openGauss configuration file (**postgresql.conf** or **pg_hba.conf**) are all for standalone deployment. Applications can set parameters as needed by invoking **gs_guc**. gs_guc is executed by OS user **omm**.
  - gs_guc-current.log
    - This file records the logs generated by gs_guc.
    - Default directory: *$GAUSSLOG*/**bin/gs_guc**
  - gs_guc-*Year-Month-Day*_HHMMSS.log
    - A backup file is generated based on the current time when the size of the **gs_guc-current.log** file reaches 16 MB.
  - server.key.cipher, server.key.rand
    - When you encrypt a user password using the **gs_guc encrypt** command and the **-M** option is set to **server**, the **server.key.cipher** and **server.key.rand** files are generated. **server.key.cipher** stores the encrypted password. **server.key.rand** stores the encryption factor.
  - client.key.cipher, client.key.rand
    - When you encrypt a user password using the **gs_guc encrypt** command and the **-M** option is set to **client**, the **client.key.cipher** and **client.key.rand** files are generated. **client.key.cipher** stores the encrypted password. **client.key.rand** stores the encryption factor.
  - datasource.key.cipher, datasource.key.rand
    - When you encrypt a user password using the **gs_guc encrypt** command and the **-M** option is set to **source**, the **datasource.key.cipher** and **datasource.key.rand** files are generated. **datasource.key.cipher** stores the encrypted password. **datasource.key.rand** stores the encryption factor.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **client.key.cipher** and **client.key.rand** indicate the ciphertext file and encryption factor files generated when the **-U** option is not specified. If you specify a username using the **-U** option, a file whose name starts with the username is generated. Taking **-U test** as an example, the names of the files generated would be **test.key.cipher** and **test.key.rand**.
- **datasource.key.cipher** and **datasource.key.rand** are key files invoked during data source creation. gs_guc has the read permission when the files are generated. Before using the two files, place them in the *$GAUSSHOME*/**bin** directory on each node and ensure that they are readable.

Server Tools – gs_guc (2)

- Syntax:
  - Check parameters in the configuration file.

```
gs_guc check [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c "parameter"
```

  - Modify parameters in the configuration file.

```
gs_guc set [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c "parameter = value"
```

  - Reset parameters to their default values.

```
gs_guc [ set | reload ] [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c "parameter"
```

  - Modify parameters in the configuration file and send semaphores to the **postgresql.conf** configuration file.

```
gs_guc reload [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c parameter=value
```

  - Modify the client authentication policy and send semaphores to the **pg_hba.conf** configuration file.

```
gs_guc [ set | reload ] [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -h "HOSTTYPE DATABASE USERNAME IPADDR-WITH-IPMASK AUTHMEHOD authentication-options"
```

  - Comment out a configured client authentication policy and send semaphores to **pg_hba.conf**.

```
gs_guc [ set | reload ] [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -h "HOSTTYPE DATABASE USERNAME IPADDR-WITH-IPMASK AUTHMEHOD"
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Note: **authentication-options** supports the following options:
  - **trust**: no authentication. Remote hosts are not allowed to access openGauss in trust mode.
  - **reject**: access rejected
  - **md5**: MD5 authentication, which is not supported by default.
  - **sha256**: SHA-256 authentication (recommended).
  - **cert**: client certificate authentication.
  - **gss**: kerberos authentication.
- If you need to comment out a configured authentication policy, use **-h "HOSTTYPE DATABASE USERNAME IPADDR-WITH-IPMASK AUTHMEHOD"**, without specifying **authentication-options**.

Server Tools – gs_om (1)

- openGauss provides the gs_om tool to help maintain the database, including starting and stopping openGauss, querying openGauss status, querying static configurations, generating static configuration files, querying the openGauss status details, generating the dynamic configuration file, replacing the SSL certificate, and displaying the help information and version number. gs_om is executed by OS user **omm**.
  - Start openGauss.

    ```
    gs_om -t start [-h HOSTNAME] [-D dataDir] [--time-out=SECS] [--security-mode=MODE] [-l LOGFILE]
    ```

  - Stopp openGauss.

    ```
    gs_om -t stop [-h HOSTNAME] [-D dataDir]  [--time-out=SECS] [-m MODE] [-l LOGFILE]
    ```

  - Restart openGauss.

    ```
    gs_om -t restart [-h HOSTNAME] [-D dataDir] [--time-out=SECS] [--security-mode=MODE] [-l LOGFILE] [-m MODE]
    ```

  - Query the openGauss status.

    ```
    gs_om -t status [-h HOSTNAME] [-o OUTPUT] [--detail] [--all] [-l LOGFILE]
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Server Tools – gs_om (2)

- Generate a static configuration file.

```
gs_om -t generateconf -X XMLFILE [--distribute] [-l LOGFILE]
```

- Generate a dynamic configuration file. Perform this operation after a failover or switchover from the standby node to the primary node.

```
gs_om -t refreshconf
```

- Display the static configurations.

```
gs_om -t view [-o OUTPUT]
```

- Query openGauss status details.

```
gs_om -t query [-o OUTPUT]
```

- Replace the SSL certificate.

```
gs_om -t cert --cert-file=CERTFILE [-l LOGFILE]
gs_om -t cert --rollback
```

- Enable or disable the Kerberos authentication in the database.

```
gs_om -t kerberos -m [install|uninstall] -U USER [-l LOGFILE] [--krb-client|--krb-server]
```

- Display the help information.

```
gs_om -? | --help
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Server Tools – gs_restore

- gs_restore, provided by openGauss, is used to import data or files that were exported using gs_dump.
- gs_restore is executed by OS user **omm**.
- It has the following functions:
  - Import data to a database.
    - If a database is specified, data is imported to the database. For parallel import, the database password is required.
  - Create a script for database recreation.
    - If a database is not specified for storing the imported data, a script containing the SQL statement needed to recreate the database is created and written to a file or standard output. This script output is equivalent to the plain text output used by gs_dump.
- Syntax:

```
gs_restore [OPTION]... FILE
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Notes:

- **FILE** does not have a short or long option. It is used to specify the location for the archive files.
- Either the **dbname** or **-l** option is required. Users cannot use both **dbname** and **-l** parameters at the same time.
- gs_restore imports data incrementally by default. To prevent data exceptions caused by multiple import operations, you are advised to use the **-c** option during the import. Before recreating database objects, delete any database objects that already exist in the database to be restored.
- There is no option to prevent logs from being printed. To hide logs, you can redirect them to a log file. If a large amount of table data needs to be restored, the table data will be restored in batches. As a result, there will be multiple logs indicating that the table data was imported.

Server Tools – gs_ssh

- gs_ssh, provided by openGauss, helps users run the same command on multiple nodes in openGauss.
- Note:
  - gs_ssh can run only commands that the current database user has permissions to run.
  - The commands executed by gs_ssh do not affect the current session. For example, the **cd** or **source** command affects only the process environment. It does not affect the current session environment.
- Prerequisites:
  - The trust relationships between the hosts are normal.
  - openGauss has been installed and deployed properly.
  - The command to be run can be found by the **which** command, and the current user has the required execution permissions.
  - gs_ssh is executed by OS user **omm**.
- Syntax:
  - Run commands synchronously. `gs_ssh -c cmd`
  - Display the help information. `gs_ssh -? | --help`
  - Display the version information `gs_ssh -V | --version`

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Parameter Description

- -c
  - Specifies the command name of Linux shell on each openGauss host.
- -?, --help
  - Displays the help information.
- -V, --version
  - Displays the version information.

Contents

1. openGauss Installation
2. Connection and Authentication
3. openGauss Tools
4. **Database Uninstallation**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Uninstalling the openGauss (1)

- openGauss provides an uninstallation script.
- The process of uninstalling openGauss includes uninstalling openGauss and deleting the environment of the openGauss server.
- Procedure:
  - Step 1: Log in to the primary database node as **omm**.
  - Step 2: Use gs_uninstall to uninstall openGauss.

    ```
    gs_uninstall --delete-data
    ```

    Alternatively, locally uninstall openGauss node by node.

    ```
    gs_uninstall --delete-data -L
    ```
- Troubleshooting
  - If the uninstallation fails, troubleshoot the issue by following the log information provided in the *$GAUSSLOG*/om/gs_uninstall-YYYY-MM-DD_HHMMSS.log file.

Execute the gs_uninstall script to uninstall the openGauss.

```
gs_uninstall --delete-data
Checking uninstallation.
Successfully checked uninstallation.
Stopping the cluster.
Successfully stopped the cluster.
Successfully deleted instances.
Uninstalling application.
Successfully uninstalled application.
Uninstallation succeeded.
```

Execute the gs_uninstall script to perform single-node uninstallation.

```
gs_uninstall --delete-data
Checking uninstallation.
Successfully checked uninstallation.
Stopping the cluster.
Successfully stopped the cluster.
Successfully deleted instances.
Uninstalling application.
Successfully uninstalled application.
Uninstallation succeeded.
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The process of uninstalling openGauss includes uninstalling openGauss and clearing out the environment of the openGauss server.

Uninstalling openGauss (2)

- **One-click environment cleanup**: After openGauss is uninstalled, if you do plan on deploying openGauss again, you can run the gs_postuninstall script to clean up the environment information from the openGauss server. The gs_postuninstall script is used to clean up the environment settings made by the gs_preinstall script.
- **Prerequisites**:
  - openGauss has already been uninstalled.
  - Mutual trust has been established between different **root** users.
  - Only **root** is authorized to run the **gs_postuninstall** command.
- **Procedure**:
  - Step 1: Log in to the openGauss server as **root**.
  - Step 2: Check whether mutual trust has been established between different **root** users. If it has not, establish mutual trust manually. Run **ssh** *{hostname}* to check if the mutual trust relationships have been successfully established. Then, enter **exit**.

```
plat1:~ # ssh plat2
Last login: Tue Jan  5 10:28:18 2016 from plat1
Huawei's internal systems must only be used for conducting Huawei's business or for purposes authorized by
Huawei management.Use is subject to audit at any time by Huawei management.
plat2:~ # exit
logout
Connection to plat2 closed.
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-choice) Which of the following is not an openGauss server tool?
   A. gs_check
   B. gs_guc
   C. gsql
   D. gs_collector
2. (Single-choice) When the gs_checkperf tool is used to check system performance, which of the following can be viewed only by user **root**?
   A. openGauss level
   B. Host level
   C. Session/Process level
   D. SSD performance

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - 1. C
  - 2. D

Quiz

3. (Multiple-choice) Which of the following are the prerequisites for installing openGauss on a single node?

   A. User groups and common users have been created, and the operating systems and networks of all servers are running properly.

   B. Common users must have the read, write, and execute permissions on the database package decompression directory and installation directory, and the installation directory must be empty.

   C. A common user has the execution permission on the downloaded openGauss package.

   D. Before the installation, check whether the specified openGauss port is occupied. If the port is occupied, change the port number or stop the process that uses the port.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - 3.ABCD

Summary

- This chapter describes how to install, deploy, connect, authenticate, and uninstall the openGauss database, as well as the functions and usage of common tools provided by openGauss.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recommendations

- openGauss help document:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/GettingStarted/GettingStarted.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

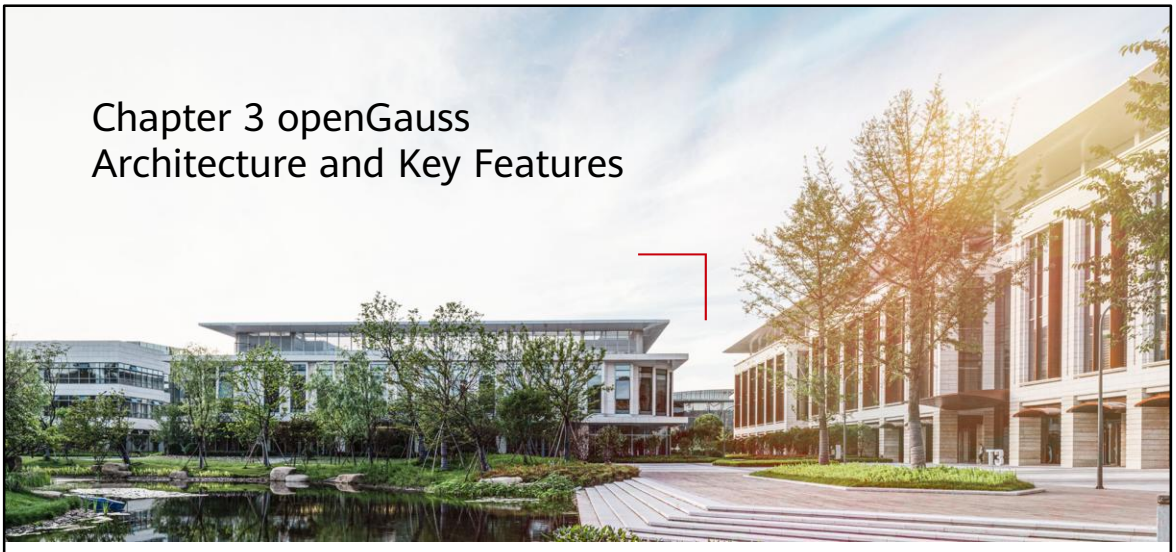
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.

HUAWEI

Chapter 3 openGauss Architecture and Key Features



HUAWEI

Foreword

- This chapter describes the system architecture, logical architecture, and resource pooling feature of the openGauss database. It also delves into the database's high performance, availability, security, ease of maintenance, and AI functionality.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completion of this course, you will be able to:
  - Familiarize yourself with the physical and logical architectures of openGauss.
  - Understand the resource pooling feature of openGauss.
  - Understand the key features of openGauss databases.

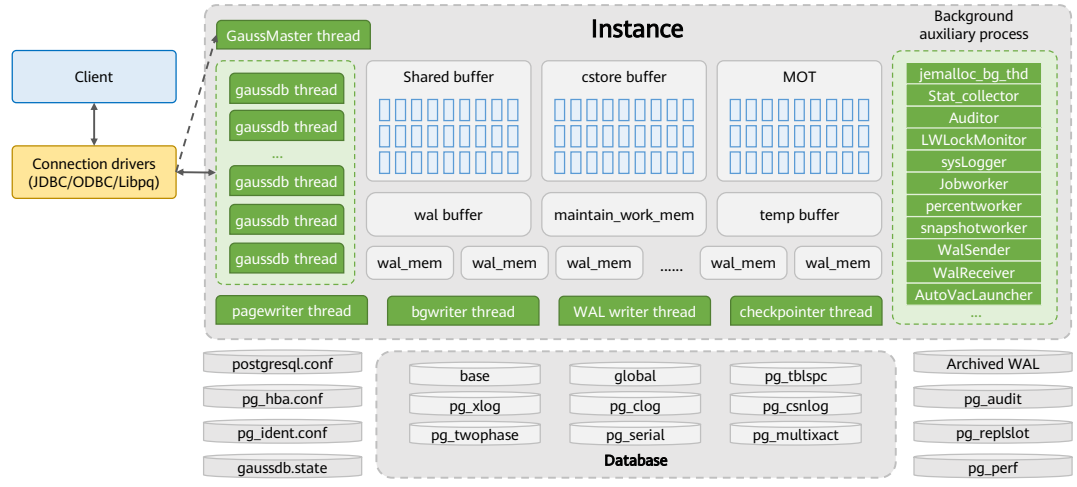
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. System Architecture
2. Resource Pooling
3. Key Features

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Architecture



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Run **ps -ef | grep omm** to view the openGauss main process, and then run **ps -T -p** *Main process ID* to view its threads.

Memory Structure (1)

- **Shared buffer**: shared memory buffer of the database server. Database read and write operations are performed in memory. Before processing, data must be loaded from disks to the database cache in memory. The memory acts as a bridge between disks and the CPU to accelerate I/O access.
- **cstore buffer**: buffer used by column stores. Column-store tables rarely use shared buffer. In this case, users should reduce **shared_buffers** and increase **cstore_buffers**.
- **MOT**: memory-optimized table. All data and indexes are stored in memory. MOT features high performance (low query and transaction latency), high scalability (high throughput and concurrency), and cost savings (high resource utilization).

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- MOT fully supports ACID to ensure reliable data handling in memory. The transaction isolation level can be RC or RR. Syntax for creating a table: create FOREIGN table test(x int);

Memory Structure (2)

- **wal buffer**: the shared memory for WALs before they are written to disks.
- **maintain_work_mem**: local memory used for maintenance operations such as VACUUM, index creation, and table modification.
- **work_mem**: used for query operations such as sorting or hash tables.
- **temp buffer**: a temporary buffer for database sessions to access temporary tables.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

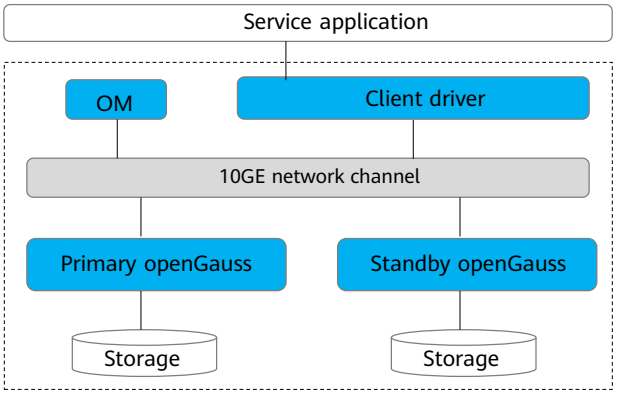
Main Threads

- **GaussMaster**: openGauss management thread, which is also called postmaster. It manages the database like starting, stopping, and message forwarding.
- **pagewriter**: flushes dirty page data from memory to disks.
- **bgwriter**: copies dirty page data to the double-writer space and flushes the data to disks. Then, the dirty pages are forwarded to the bgwriter subthread to flush the data to disks.
- **walwriter**: updates the WAL page data in the memory to the WAL file to ensure all committed transactions are permanently stored.
- **checkpoint**: triggered periodically. Each time it is triggered, all dirty pages are flushed to disks.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Both pagewriter and bgwriter are data page writer threads of openGauss.
- The pagewriter copies dirty page data to the double-writer space and flushes the data to disks. Then, the dirty pages are forwarded to the bgwriter subthread to flush the data to disks. This prevents data page corruption by enabling full recovery of data pages from the double-writer space.
- The bgwriter flushes dirty page data from the shared buffer to disks. Database threads do not need to wait for write operations during user queries because the bgwriter handles the actual writing process. It also mitigates performance deterioration caused by checkpoints.

openGauss Primary/Standby Logical Architecture



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

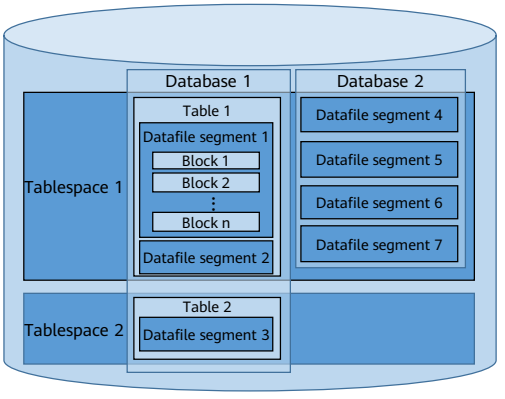
Logical Architecture Description

Software	Description
OM	Operation Manager. It provides APIs and tools for managing and maintaining the database.
Client driver	The client driver receives access requests from applications and returns the execution results. It communicates with openGauss instances, delivers SQL commands from applications, and receives openGauss execution results.
openGauss (primary/standby)	Data nodes (DNs) store service data, execute data queries, and return execution results to clients. openGauss supports one primary and multiple standby instances. You are advised to deploy primary and standby instances on different physical nodes.
Storage	Local storage resource for data persistence.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Logical Architecture

- DNs in openGauss store data on disks. The following explains the logical objects on each DN and their relationships.
- Database logical architecture:
  - Tablespace: Directories storing the actual files of the databases. Multiple tablespaces can coexist, and each of them can contain files belonging to different databases.
  - Database: manages various data objects and is isolated from each other. Objects managed by a database can be distributed to multiple tablespaces.
  - Datafile segment: A data file that stores data of only one table. A table containing more than 1 GB of data is stored in multiple data files.
  - Table: One table belongs to only one database and one tablespace. The data files of each table must be in the same tablespace.
  - Block: Basic unit of database management. Its default size is 8 KB.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Directory Structure (1)

- By default, database files are stored in the data directory created during database initialization (command: **initdb**). The data directory contains various types of directories and files, such as data files, parameter files, control files, database run logs, and WALs.
- Data directory structure:

Directory/File	Parent Directory	Purpose
bin	-	Saves binary files of the database.
lib	-	Saves library files of the database.
share	-	Saves common files required for the database to run, such as the configuration file template.
data (database node/primary database node)	-	Stores database node data. For the primary node, the directory name is **data_dn***xxx*. For the standby node, the directory name is **data_dnS***xxx*. *xxx* indicates the database node number.
base	Instance data directory	Contains subdirectories of each database.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Parent directory: **/opt/gaussdb/app** or **opt/gaussdb/app-***xxxxx*. Instance data directory: **/gaussdb/data/***Database node name*
- During its installation, the database sets permissions for its files, including files (such as log files) generated during the running process. File permissions are set as follows:
  - The permission for program directories in the database is set to **0750**.
  - The permission for data file directories in the database is set to **0700**. During deployment, openGauss sets the **tmpMppdbPath** parameter in the XML configuration file to specify the directory for storing **.s.PGSQL.*** files. If the directory is not specified, the **/tmp/***$USER*_**mppdb** directory is created. The directory and file permission is set to **0700**.
  - The permission for database data files, audit logs, and related program-generated files is set to **0600**. The permission for run logs is no higher than **0640** by default.
  - Common OS users are not allowed to modify or delete database files and log files.

Database Directory Structure (2)

- Data directory structure (continued):

Directory/File	Parent Directory	Purpose
global	Instance data directory	Contains the subdirectories of the cluster range table.
pg_audit	Instance data directory (configurable)	Stores database audit logs.
pg_log	Instance data directory (configurable)	Stores the run logs of database node instances.
pg_xlog	Instance data directory	Stores write-ahead logs.
postgresql.conf	Instance data directory	Parameter files
pg_hba.conf	Instance data directory	Client authentication control file
postmaster.opts	Instance data directory	Records the command line parameters used during server startup.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Directory Structure (3)

- Data directory structure (continued):

Directory/File	Parent Directory	Purpose
gs_initdb	bin	Database initialization tool
gs_dump	bin	Exports database information.
gs_ctl	bin	Database service control tool
gs_guc	bin	Sets application parameters.
gsql	bin	Database connection tool running in the CLI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

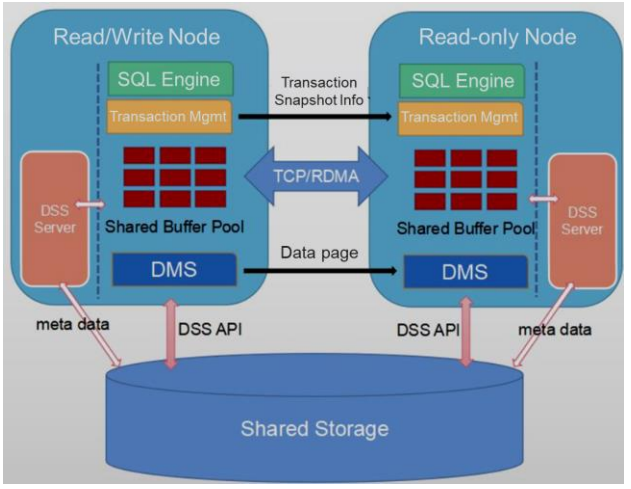
1. System Architecture
2. **Resource Pooling**
3. Key Features

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Resource Pooling Architecture

- Resource pooling innovates HA deployment by allowing primary and standby nodes to share storage resources. Compared to traditional HA setups, it cuts the storage needs and costs, while enabling real-time consistent reads for the standby node.



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- openGauss resource pooling is a new cluster architecture that uses DMS and DSS components to share underlying storage and memory across nodes in a cluster. This saves storage resources and allows one-write-many-reads and real-time consistent reads across nodes.
- The read/write node and read-only node share the same underlying storage.
- The read/write node and read-only node use the DMS component to share hot data pages in the shared buffer pool through the TCP or RDMA protocol.
- The read/write node and read-only node access persistent data in the underlying shared storage through DSS APIs and DSS servers.

Resource Pooling Architecture (1)

- UltraPath has been installed on the disk array, and the disk array is available.
- Resource pooling depends on two components:
  - Distributed Storage Service (DSS)
    - DSS includes DSS API and DSS Server. DSS Server is an independent process that directly manages raw devices of disk arrays and provides capabilities similar to distributed file systems externally. DSS API is a dynamic library integrated in the database. Using the shared memory and client API dynamic library, DSS enables the database to create, delete, expand, shrink, read, and write files.
  - Distributed Memory Service (DMS)
    - DMS is a dynamic library integrated into the database. It transmits page data over TCP/RDMA networks and pools memory between primary and standby nodes. This ensures real-time consistent reads on the standby node.
- Page exchange between primary and standby nodes is accelerated over RDMA, which depends on the CX5 NIC and OCK RDMA dynamic library.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Resource Pooling Architecture (2)

- Storage pooling uses DSS to enable the primary and standby nodes to share the same storage. Compared to traditional database setups, resource pooling classifies directories into three types: directories exclusively used by each instance, directories exclusively used but shared among instances, and directories shared by all instances. Shared directories must be stored on the disk array, while non-shared directories must be stored on local disks. Additionally, to set up a database on the standby node, you only need to create the node's own directories, without the need to create directories shared by all instances. GUC parameters are added for resource pooling, and system catalog storage is switched from page mode to segment-page mode.
- Memory pooling uses DMS to implement real-time page exchange between primary and standby nodes and ensure real-time consistent data reads on the standby node. For instance, a transaction committed on the primary node can be read on the standby node immediately. There is no delayed read (the transaction isolation level is read committed).
- Memory pooling can utilize OCK RDMA to reduce the latency of page exchanges between the DMS primary and standby nodes. Compared to that over TCP, the latency of consistent reads on the standby node over OCK RDMA is reduced by at least 20%.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Resource Pooling Highlights

- The primary and standby nodes share a copy of data, reducing storage capacity compared to traditional HA.
- Log replication is not required between the primary and standby nodes. Instead, page exchange is implemented between the primary and standby nodes to support real-time consistent reads on the standby node.
- By default, the primary and standby nodes exchange pages in real time over the TCP network. To reduce the page exchange latency, you can use the OCK RDMA dynamic library to boost the real-time consistent read performance of the standby node.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Application Scenarios and Restrictions

- The storage device must be a disk array, such as OceanStor Dorado 5000. The LUN of the disk array must support the SCSI3 PR protocol, including PERSISTENT RESERVE OUT (PR OUT), PERSISTENT RESERVE IN (PR IN), and INQUIRY, to implement cluster IO FENCE. In addition, the COMPARE AND WRITE (CAW) protocol of SCSI3 must be supported to implement shared disk locks.
- The HA deployment mode implemented by resource pooling supports a maximum of one primary node and seven standby nodes.
- Because resource pooling uses functions similar to distributed file systems to implement the real-time read consistency on standby nodes, changes to file metadata should be limited. To ensure performance, this feature supports only segment-page tables.
- The primary and standby nodes must be deployed on the same disk array. DR and hybrid deployment of the primary and standby nodes (like on different disk arrays) are not supported.
- Page exchange between the primary and standby nodes can be accelerated by RDMA, which depends on the CX5 NIC and OCK RDMA dynamic library.
- Currently, standby node rebuild, node replacement, and node restoration are not supported.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Traditional and resource-pooling databases cannot change between each other.
- Ceph is not supported.

Feature Limitations

- In resource pooling mode, the gs_xlogdump_xid, gs_xlogdump_lsn, gs_xlogdump_tablepath, gs_xlogdump_parsepage_tablepath, gs_verify_and_tryrepair_page, gs_repair_page, and gs_repair_file functions are not available.
- In resource pooling mode, the T_CreatePublicationStmt, T_AlterPublicationStmt, T_CreateSubscriptionStmt, T_AlterSubscriptionStmt, and T_DropSubscriptionStmt subscription functions are not available.
- Global temporary tables are not supported in resource pooling mode.
- In the event of failover in resource pooling mode, you must wait until the failover is complete before you can add the original primary node to the cluster again.
- In the event of switchover in resource pooling mode, if a failure occurs during the switchover (for example, cluster nodes are restarting), the original primary node must be restarted.
- Hash indexes are not supported in resource pooling mode.
- Cascaded standby nodes are not supported in resource pooling mode.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Resource Pooling Performance Optimization

- Feature overview
  - This feature includes the following sub-features:
    - Resource pooling optimizes the visibility logic on the standby node. The commit sequence number (CSN) corresponding to the transaction XID obtained from the primary node is cached locally to reduce network overhead and message interaction.
    - Resource pooling optimizes how the primary node retrieves the oldest Xmin from the cluster by locally caching this value. This allows the standby node to access snapshots in real-time. Additionally, the standby node periodically sends its local oldest Xmin to the primary node, reducing broadcast overhead.
    - Resource pooling optimizes how standby nodes fetch snapshots. The primary node broadcasts the latest snapshot to the standby nodes, which then access this snapshot locally. This reduces network overhead and minimizes message interactions.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This feature is available since openGauss 5.1.0 and applies only to the resource pooling architecture.
- Benefits
  - These optimizations of the resource pooling architecture boost standby node performance in read-only scenarios. In a typical sysbench test with one read-write primary node and one read-only standby node, the standby node performance increases by 50% to 80%.

Resource Pooling Performance Optimization - Feature Description

- This feature includes the following sub-features:
  - In the original logic, the standby node determines the visibility of a tuple as follows: if the transaction associated with the tuple's XID is not marked as committed, the standby node retrieves the real-time CSN for that XID from the primary node to assess visibility. When a large number of pages are queried on the standby node, the extended query process and frequent interactions can impact execution efficiency on the primary and standby nodes. This feature stores transaction commit status information in the local L2 cache of the standby node. When assessing the tuple visibility, the standby node first checks the local cache for the relevant information. If it is not available locally, the node retrieves it from the primary node and subsequently updates the local cache.
  - In the original logic, each time the primary node generates a new snapshot, it triggers a broadcast message to gather the oldest Xmin from all standby nodes in the cluster. This process updates the cluster's oldest Xmin for use in vacuuming and heap tuple pruning operations. This feature records a snapshot's Xmin in the primary node's memory when the standby node receives the snapshot from the primary node. Additionally, the standby node periodically sends its local oldest Xmin to the primary node via a background thread. The primary node also periodically clears invalid Xmin through a background thread.
  - In the original logic, each standby node must fetch the latest snapshot from the primary node in real time for every read operation. This frequent interaction can be burdensome when read services on the standby node are heavy. This feature enables the primary node to broadcast its latest snapshot to all standby nodes each time it creates one. Each standby node caches the latest snapshot locally and preferentially reads the local snapshot. This reduces interactions during heavy read services on the standby node. However, to reduce the impact of broadcasting on the primary node when the number of standby nodes is great, this feature is disabled by default.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Feature Enhancement
  - This feature enhances the resource pooling architecture by improving visibility assessment on the standby node, optimizing the retrieval of the oldest Xmin from the primary node, and streamlining snapshot fetch on the standby node.

Transparent Write Forwarding on the Standby Node Within a Transaction

- Feature overview
  - This feature enhances transaction write forwarding in traditional primary/standby mode based on the resource pooling architecture. In the traditional architecture, each transaction committed on the standby node will be forwarded to the primary node. This feature enables the standby node to locally execute the read requests in transactions and forward the write requests to the primary node for execution.
- Benefits
  - In the resource pooling architecture, a cluster supports concurrent data writes across multiple nodes. This enhances the linear performance of the entire cluster, especially when there are minimal concurrent data conflicts between primary and standby nodes.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This feature is available since openGauss 5.1.0 and applies only to the resource pooling architecture.
- Feature Description
  - This feature depends on the write-on-standby feature. In the resource pooling architecture, after the write-on-standby feature is enabled, the database automatically forwards all write SQL statements in explicit transactions (that is, SQL statements enclosed by BEGIN and END) on the standby node to the primary node for execution. Read statements in the transactions are still executed on the standby node.

DSS Enhancements

- Feature overview
  - DSS supports thread pools and configurable DSS access capabilities and resource allocation.
  - DSS can generate black box logs if the DSS Server breaks down.
  - DSS supports the NoF/NoF+ API.
- Benefits
  - DSS supports thread pools, allowing on-demand utilization, scaling, and configuration of server CPU and memory resources.
  - DSS uses black box logs to locate DSS Server faults, which enhances the DFX capability.
  - DSS supports the NoF/NoF+ API, which expands the underlying storage forms and types supported by DSS.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This feature is available since openGauss 5.1.0 and applies only to the resource pooling architecture.
- Feature Description
  - DSS supports on-demand configuration of concurrent database connections. By using a thread pool, DSS manages resource usage such as memory and CPU, thereby improving server memory and CPU utilization. This configuration is static and does not take effect dynamically.
  - During a system crash, process and thread context information related to heaps, stacks, and registers is dumped to the black box logs, helping R&D engineers to locate and trace DSS Server faults. The black box log function is enabled by default and can be disabled by setting the _LOG_LEVEL=0 parameter. The setting takes effect after the node service restarts.
  - DSS supports the NoF/NoF+ API. The NoF/NoF+ network can be used to connect to centralized enterprise storage that supports the NoF protocol, improving the I/O performance of the cluster.

Resource Pooling Operator Offloading

- Feature overview
  - Shared storage offers excellent elasticity and reliability, but performs slower than local host storage. This is caused by delays from network I/Os and distributed storage, particularly when many queries overload the buffer pools. In such scenarios, a large volume of data must be transferred from storage nodes to compute nodes, with only a small fraction being relevant. This results in substantial network I/O waste and degraded performance. By offloading operators, data filtering is executed on storage nodes to remove unnecessary data, reducing network communications and improving end-to-end performance.
- Benefits
  - In the resource pooling scenario, performance is impacted when a large volume of queries retrieve unnecessary data over the network for analysis and processing. For users primarily engaged in query operations, data preprocessing can be performed on storage nodes to filter out unnecessary data, reducing the volume of data transmitted over the network and improving query performance.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This feature is available since openGauss 5.1.0.
- Feature Description
  - The openGauss resource pooling feature enables offloading of operators such as Agg, SeqScan, Filter, and Projection to storage nodes. The filtered, projected, or aggregated results are then transmitted to compute nodes for further processing. This approach, known as near-data processing (NDP), allows compute nodes to perform fewer calculations and generate final results using upper-layer operators. Services remain unaffected by the operator offloading process. This reduces data I/Os transmitted over the network and improves query performance by utilizing the computing power of storage nodes.

Resource Pooling Dual-Cluster DR

- Feature overview
  - This feature synchronizes Xlogs between the primary and standby clusters to ensure Xlog consistency, enhance the DR capability, reduce storage occupation, and ensure proper node switchover within or between the primary and standby clusters. This feature has two modes:
    - Network-based streaming replication: resource pooling network dual-cluster deployment
    - Synchronous replication based on OceanStor Dorado storage: resource pooling OceanStor Dorado dual-cluster deployment
- Benefits
  - This feature ensures data security and service availability in the event of disasters, and provides multiple modes to meet diverse requirements.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Dual-cluster DR based on streaming replication is available since openGauss 6.0.0-RC1.
- Dual-cluster DR based on OceanStor Dorado synchronous replication is available since openGauss 5.1.0.
- Feature Description
  - Resource pooling dual-cluster DR synchronizes Xlogs from the primary cluster to the standby cluster in multiple modes. The standby cluster replays the Xlogs from the primary cluster to ensure data consistency. This enables the standby cluster to take over services from the primary cluster in a timely manner in the event of a fault.
  - Dual-cluster DR based on streaming replication uses the Xlog physical replication capability of the openGauss cluster to flexibly configure the replication relationship between the primary and standby clusters. It is applicable to intra-city or remote DR scenarios.
  - OceanStor Dorado-based dual-cluster DR uses synchronous replication of the storage system. After Xlogs are written to the primary cluster, the standby cluster can read the same Xlogs when the link is normal. This mode is applicable to intra-city dual-center DR scenarios with a distance of 100 km, offering high-speed data transfer. In typical service scenarios, if one cluster fails entirely, this feature ensures 0 RPO and an RTO < 30s.

On-demand Replay for Ultimate RTO

- Feature overview
  - In the resource pooling architecture, this feature enables the primary database node to quickly recover from a restart.
  - In the resource pooling architecture, this feature accelerates failover of standby nodes, reducing the RTO.
  - This feature evolves from the existing ultimate RTO feature.
- Benefits
  - In the resource pooling architecture, the standby node does not replay logs of the primary node. If the primary node runs into a fault, the standby node restores data of the primary node from the last checkpoint, which takes a long time. In this process, the database is unavailable, which severely affects the system availability.
  - After the on-demand replay feature is enabled, the downtime of the database after a primary node fault is reduced, achieving the ultimate RTO and enhancing overall database availability.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This feature is available since openGauss 5.1.0.
- Feature Description
  - With on-demand replay for achieving ultimate RTO, only the essential content is rebuilt for disaster recovery. During this process, services are immediately available without requiring full replay, thereby minimizing the RTO. Once services are available, logs are replayed in the background as needed to ensure that users can access the most recent data, matching the state before the fault occurred.

Resource Pooling Multi-Node Parallelism

- Feature overview
  - Shared-Everything Parallel Query (SPQ) is a multi-node parallel query framework in openGauss. It is designed for resource pooling setups where a cluster has one write-read node and multiple read nodes. The existing openGauss cluster restricts queries to just one read node at a time, limiting overall query speed. SPQ uses a Shared-Everything architecture where all nodes in a cluster share resources. The optimizer generates a multi-node execution plan, which is then distributed to each node. The query node aggregates data from each node to enable parallel queries by all read nodes, maximizing the cluster's OLAP performance. This provides strong TP and AP capabilities for resource pooling.
- Benefits
  - Large-scale data queries on a read node only utilize that specific node's computing power, leaving other nodes' resources unused. Multi-node parallelism can fully utilize the computing power of all nodes, greatly improving the query efficiency.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- This feature is available since openGauss 6.0.0-RC1.
- Feature Description
  - The one-write-many-reads architecture of openGauss resource pooling isolates OLTP and OLAP services by building virtual Node-Groups. Any node can initiate a SQL query request. The system selects either multi-node or single-node queries depending on cost and predefined rules. This process is transparent to upper-layer applications. To address these needs, SPQ enhances the original openGauss design with the following features:
    - Connects to the open-source ORCA optimizer to generate SPQ multi-node parallel execution plans. The adapted components are loaded as dynamic libraries to the openGauss kernel.
    - Adds basic scan and compute operators for multi-node plans, implementing data exchange and distribution between nodes.
    - Adds the adaptive scan mechanism to eliminate data skew.
    - Supports DML and DDL enhancement.

Contents

1. System Architecture
2. Resource Pooling
3. **Key Features**
   - High Performance
   - High Availability
   - High Security
   - Easy Maintenance
   - AI Capabilities

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Optimizers

- Optimizers are built in databases and are used to generate the optimal execution plans during SQL parsing.
  - Cost-based optimizer (CBO)
  - AI-based optimizer (ABO)
    - As the third-generation intelligent optimizer of openGauss, it uses AI technologies to enhance the generation efficiency and execution performance of query plans.
    - Application scenarios: finance, communications, and other fields requiring high-concurrency processing, as it ensures a failover time of less than 10s and supports an architecture with one primary node and multiple standby nodes.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- The CBO is introduced since openGauss 1.0.0.
- The ABO is introduced since openGauss 3.1.0.

ABO - Main Feature and Implementation

- Main feature and implementation
  - Intelligent cardinality estimation
    - The ABO employs a Bayesian network model to characterize the joint distribution of multi-column data, significantly enhancing the accuracy of cardinality estimation for multi-column equality queries. To enable this function, perform the following steps:
      - Set GUC parameters: Set **enable_ai_stats** to **on** and **multi_stats_type** to **'BAYESNET'** or **'ALL'**.
      - Run the ANALYZE command to create a statistics model and run the ALTER TABLE command to delete unnecessary models.
  - Performance optimization effect
    - Cardinality estimation helps the optimizer accurately select execution plans and operators, improving the overall database throughput. Combined with the multi-core architecture (for example, 2-socket Kunpeng processors for 1.5 million tpmC) and NUMA-based optimization of openGauss, the ABO further enhances the performance in high-concurrency scenarios.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

ABO - Intelligent Cardinality Estimation

- Introduction
  - This feature uses lightweight algorithms in the database to model multi-column data distributions and applies to multi-column equality queries. In scenarios with skewed data distributions and strong column correlations, it delivers precise cost references to the optimizer, improving plan generation accuracy and query execution efficiency.
- Benefits
  - Users can create an intelligent statistics model to improve the accuracy of multi-column statistics and the query optimization performance.
- Description
  - Intelligent cardinality estimation models data distributions of samples in the database and then compresses and stores the model in the database. The optimizer triggers intelligent estimation during execution plan generation to obtain more accurate cost estimation and generate a better plan.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

ABO - Adaptive Plan Selection

- Introduction
  - This feature triggers plan selection based on the selectivity of base table conditions and provides multi-plan cache management and adaptive selection for queries using partial indexes and offsets. It can deliver multi-fold improvements in query throughput in typical scenarios.
- Benefits
  - Users can maintain multiple cache plans to adapt to different query parameters, improving query execution performance.
- Description
  - Adaptive plan selection applies to scenarios where a general cache plan is used for plan execution. It explores cache plans using range linear expansion, and selects plans using range coverage matching. It resolves performance issues caused by the traditional single cache plan that cannot change with query condition parameters, and avoids frequent query optimizer calls.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CBO (1)

- A CBO consists of three key components:
  - Query converter
    - Rewrites certain SQL statements into equivalent ones with a lower cost and determines whether the conversion is worth performing.
  - Cost estimator
    - Determines the overall cost of a given execution plan.
  - Execution plan generator
    - Explores various SQL execution plans by trying different access paths, join methods, and join orders.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CBO (2)

- The CBO estimates execution costs based on statistics. By using the CBO, the database calculates the number of tuples and the execution cost for each execution step under each execution plan based on the number of table tuples, column width, NULL record ratio, and characteristic values (such as distinct and MCV values), as well as specific cost calculation methods. The database then selects the execution plan that takes the lowest costs for the overall execution or for the return of the first tuple.
- When using the CBO, ensure that sufficient statistics are collected for tables and related indexes. You are advised to periodically analyze tables and indexes that are frequently added, deleted, or modified.
- Only when the database has sufficient statistical data to reflect the actual situation can it make the right choice.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- MCV value: **most_common_vals** is a column in a system view, indicating the most common value.

Statistics

- What are statistics?
  - The CBO needs to select the data query mode based on the data in the table or index. Because the table contains a large amount of data, the data volume and data distribution in the statistics table cannot be collected in real time during each query. Therefore, data needs to be periodically analyzed, and the data distribution of the table and index needs to be saved to the data dictionary for the optimizer to use. This is the statistics.
- openGauss generates the optimal execution plans based on cost estimation. The CBO needs to estimate the number of data rows and the cost based on global statistics collected using ANALYZE, including **relpages** and **reltuples** in the **pg_class** table; and **stadistinct**, **stanullfrac**, **stanumbersN**, **stavaluesN**, and **histogram_bounds** in the **pg_statistic** table.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- In a database, statistics are the source data for the planner to generate execution plans. Missing or outdated statistics often lead to poor execution plans, causing performance issues.

When to Collect Statistics

- After executing batch insertions and deletions, you are advised to run the ANALYZE statement on the table or the entire database to update statistics.
- For an intermediate table generated during the execution of a batch processing script or stored procedure, you also need to run the ANALYZE statement.
- If there are multiple interrelated columns in a table and the conditions or grouping operations based on these columns are involved in the query, collect statistics about these columns so that the query optimizer can more accurately estimate the number of rows and generate a more effective execution plan.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The ANALYZE statement collects statistics on tables in the database. The statistics are stored in the system catalog **PG_STATISTIC**. The query optimizer uses these statistics to generate the most efficient execution plan.
- By default, 30,000 rows of statistics are sampled. (The default value of the GUC parameter **default_statistics_target** is **100**.) If the total number of rows in a table exceeds 1,600,000, you are advised to set **default_statistics_target** to **-2**, indicating that 2% of the statistics are collected.

How to Collect Statistics

- The ANALYZE statement collects statistics on tables in the database. The statistics are stored in the system catalog **PG_STATISTIC**. The query optimizer uses these statistics to generate the most efficient execution plan.
- Procedure
  - Update statistics on a single table.

    ```
    ANALYZE tablename;
    ```
  - Update statistics on the entire database.

    ```
    ANALYZE;
    ```
  - Collect statistics on the **col_1** and **col_2** columns of the **tablename** table.

    ```
    ANALYZE tablename (col_1, col_2);
    ```
  - Add the declaration of statistics on the **col_1** and **col_2** columns in the **tablename** table.

    ```
    ALTER TABLE tablename ADD STATISTICS (col_1, col_2);
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- After the statistics are declared for multiple columns by running the **ALTER TABLE tablename ADD STATISTICS** statement, the system collects the statistics about these columns the next time ANALYZE is performed on the table or the entire database.

Statistics Information View

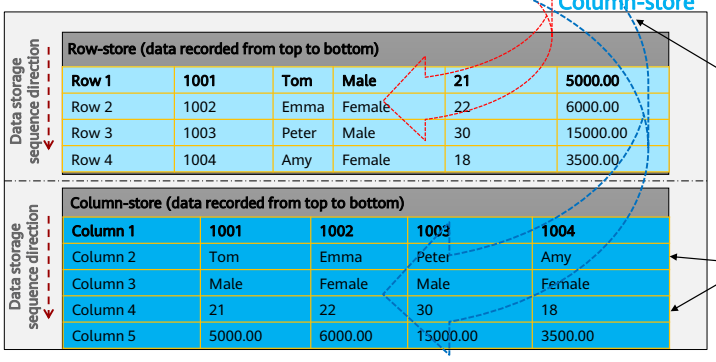
- The system catalog **PG_STATISTIC** stores statistics on tables and index columns in the database. It is accessible only to system administrators. Users can access the table or the corresponding view only after being authorized.

Name	Data Type	Description
starelid	oid	Table or index to which the described column belongs.
starelkind	char	Object type.
staattnum	smallint	Number of the described column in the table, starting from 1.
stainherit	Boolean	Determines whether to collect statistics on objects that have inheritance relationships.
stanullfrac	real	Ratio of NULL records to all records in the column.
stawidth	integer	Average storage width of non-NULL records, in bytes.
stadistinct	real	Number of distinct, non-NULL data values in the column for all data nodes (DNs).
stakindN	smallint	Code number stating that the type of statistics is stored in slot *N* of the **pg_statistic** row.
staopN	oid	Operator used to generate the statistics stored in slot *N*.
stanumbers	real[]	Column data values of the appropriate type for slot *N*. The value is NULL if the slot type does not store any data values.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Row-Store and Column-Store

Employee ID	Name	Gender	Age	Salary
1001	Tom	Male	21	5000.00
1002	Emma	Female	22	6000.00
1003	Peter	Male	30	15000.00
1004	Amy	Female	18	3500.00



- There are two types of tables by storage mode:
  - Row-store table
  - Column-store table

During the write process, records are split, and data in different columns is written to different storage areas. Multiple write processes increase the I/O and reduce efficiency.

For column-store table queries (for example, **select name,age from employee;**), only a small amount of storage corresponding to the required columns needs to be scanned, so there is little I/O overhead.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Comparison Between Row-Store and Column-Store

- Row-store table
  - Table data is stored in rows, and all attributes in each row are stored together.
  - Row-store tables are used to query all attributes of a row.
  - INSERT and UPDATE operations are efficient.
  - Row-store tables are used by default.
- Column-store table
  - Table data is stored in columns, and multiple records in each column are stored together.
  - Column-store tables feature efficient projection, are applied to querying massive data, and can reduce the volume of accessed disk data.
  - INSERT and UPDATE operations are slightly inefficient.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Hybrid Row-Column Store

- Use the keyword **ORIENTATION** to specify the storage mode.
  - **ROW**
  - **COLUMN**
- Use the keyword **COMPRESSION** to specify the compression level.
  - A higher compression level indicates a higher compression ratio.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Partitioning (1)

- In openGauss, data is partitioned horizontally on a node using a specified policy. This operation splits table data into multiple partitions that do not overlap.
- In common scenarios, a partitioned table has the following advantages over a common table:
  - High query efficiency: You can specify partitions when querying partitioned tables.
  - High availability: If a partition in a partitioned table is faulty, data in other partitions is still available.
  - Easy maintenance: If a partition in a partitioned table is faulty, only data in this partition needs to be repaired.
  - Balanced I/O: Partitions can be mapped to different disks to balance I/O and improve the overall system performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Partitioning (2)

- openGauss supports range partitioned tables, list partitioned tables, and hash partitioned tables.
  - Range partitioned table: Data is mapped to each partition based on the range determined by the partition key specified when the partitioned table is created. This partitioning mode is most commonly used.
    - Range partitioning divides the records to be inserted into a table into multiple ranges (non-overlapping) based on one or multiple columns of the table, and creates a partition for each range to store the corresponding data. To use this function, add **PARTITION** parameters when running the CREATE TABLE statement.
  - List partitioned table: Data is mapped to each partition based on the key values contained in each partition. The key values contained in a partition are specified when the partition is created.
    - List partitioning divides the key values in the records to be inserted into a table into multiple lists (non-overlapping) based on a column of the table, and then creates a partition for each list to store the corresponding data.
  - Hash partitioned table: Data is mapped to each partition using the hash algorithm, and each partition stores records with the same hash value.
    - Hash partitioning uses the internal hash algorithms to divide records to be inserted into a table into partitions based on a column of the table.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Partitioning (3)

- If you specify the **PARTITION** parameter when running the CREATE TABLE statement, data in the table will be partitioned. You can modify partition keys as needed during table creation to make the query result stored in the same or least partitions (called partition pruning), obtaining consecutive I/O to improve the query performance.
- In actual use, time is often used as a filter for query objects, so you can select the time column as the partition key. The key value range can be adjusted based on the total data volume or the amount of data queried at a time.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Managing Partitioned Tables

Operation	Command
Create a partitioned table.	CREATE TABLE
Modify the row movement attributes of the partitioned table.	ALTER TABLE { ENABLE | DISABLE } ROW MOVEMENT
Add a partition.	ALTER TABLE ADD PARTITION
Delete a partition.	ALTER TABLE DROP PARTITION
Rename a partition.	ALTER TABLE RENAME PARTITION
Clear a partition.	ALTER TABLE TRUNCATE PARTITION
Query a partition.	SELECT
Create an index for a partitioned table.	CREATE INDEX
Rename an index partition.	ALTER INDEX RENAME PARTITION

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Basic Operations on Partitioned Tables (1)

- Creation
  - Create a range partitioned table **student1**. The table has three partitions and their partition keys are integers. The value ranges of the three partitions are: **score1** < 60, 60 ≤ **score2** < 80, and 80 ≤ **score3** < 90.

```
CREATE TABLE student1
(
        name varchar(30),
        score int,
        birthday timestamp
)
PARTITION BY RANGE(score)
(
        PARTITION score1 VALUES LESS THAN(60),
        PARTITION score2 VALUES LESS THAN(80),
        PARTITION score3 VALUES LESS THAN(90)
);
```

- Deletion
  - Delete the partition **score1**.

```
ALTER TABLE student1 DROP PARTITION score1;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Basic Operations on Partitioned Tables (2)

- Addition
  - Add a partition **score4**, and the value range is 90 ≤ **score4** ≤ 100.

```
ALTER TABLE student1 ADD PARTITION score4 VALUES LESS THAN (101);
```

- Renaming
  - Rename the partition **score2** as **score2_new**.

```
ALTER TABLE student1 RENAME PARTITION score2 TO score2_new;
```

- Query
  - Query the partition **score2_new**.

```
SELECT * FROM student1 PARTITION (score2_new);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Basic Operations on Partitioned Tables (3)

- Partitioned table index
  - Create a partitioned table index **student1_index1** without specifying index partitions.

```
CREATE INDEX student1_index1 ON student1 (score) LOCAL;
```

  - Create a partitioned table index **student1_index2** with the partition names specified.

```
CREATE INDEX student1_index2 ON student1 (score) LOCAL
(
        PARTITION score1_index,
        PARTITION score2_index TABLESPACE ts2,
        PARTITION score3_index TABLESPACE ts3
) TABLESPACE ts0;
```

  - Rename an index partition.

```
ALTER INDEX student1_index2 RENAME PARTITION score1_index TO score1_index_new;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Partition Design Summary

- To partition the existing tables, you are advised to create indexes after data migration is complete.
- To fully utilize the query characteristics of partitioned tables, use the columns used during partitioning as filters.
- After partitions are created, the universally unique identifiers (UUIDs) of the partitions can be the same.
- Querying data using partition keys is efficient.
- The partition columns cannot be empty. For example, the case filing date or case closing date cannot be used as the partition column.
- You can back up each partition separately. However, if you want to back up all of the partitions, you can back up only the entire schema.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

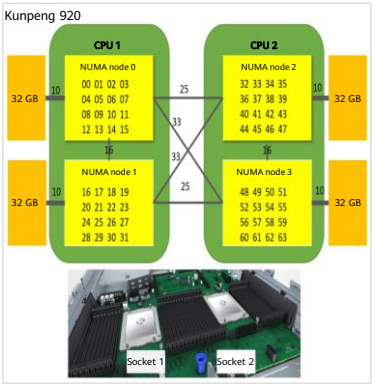
Kunpeng NUMA Architecture Optimization

- openGauss uses a multi-core Kunpeng NUMA architecture that has been optimized to reduce cross-core memory access latency and maximize multi-core Kunpeng computing capabilities. The key technologies include redo log batch insertion, NUMA distribution of hotspot data, and Clog partitions, greatly improving the performance of the online transaction processing (OLTP) system.
- openGauss uses an LSE instruction set based on ARMv8.1 for efficient atomic operations, effectively improving the CPU usage, multi-thread synchronization performance, and Xlog write performance.
- Leveraging the larger L3 cache line provided by the Kunpeng chips, openGauss optimizes hotspot data access. This enhancement improves the cache hit ratio, reduces the overhead of cache consistency maintenance, and significantly boosts overall data access performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- The CPU and memory were scarce resources for early computer systems. Therefore, the Unified Memory Architecture (UMA) was used when the SMP architecture was developed, where all CPUs access the memory through the bus controller of the northbridge.

Kunpeng NUMA CPU Structure



NUMA distance: physical distance between the processor and memory block of a NUMA node. You can use the numactl tool to query the CPU access distance.

Node/Node Distance	0	1	2	3
0	10	16	32	33
1	16	10	25	32
2	32	25	10	16
3	33	32	16	10

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Non-uniform memory access (NUMA): The memory access time depends on the memory location relative to the processor. Under NUMA, a processor accesses its local memory faster than accessing non-local memory.
- The sockets in the figure are the CPU slots. Each socket houses one physical CPU. The number of cores in yellow ranges from 00 to 63.
- The access distance of 0-0 and 1-1 is 10, and that of 0-1 is 16. This indicator is linear. That is, the latency of 0-1 remote access is 1.6 times that of local access.
- You can run the **numactl –H** command to view the access example information.

NUMA Arm Atomic Instruction

- The Compare and Swap (CAS) operation is used to ensure the consistency of data written during a multi-thread operation.
- Uninterrupted data exchange operations are implemented for multi-thread programming to avoid data inconsistencies caused by execution sequence uncertainty and interrupt unpredictability when a piece of data is modified while an operation is in progress.

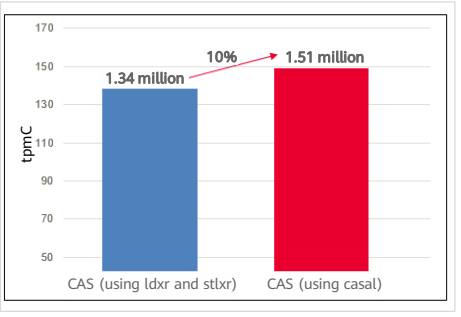
CAS implementation using ldxr and stlxr instructions

```
20        os_compare_and_swap_ulint(&old,old,new);
   0x00000000004005bc <+24>:    ldr     w0, [x29,#24]
   0x00000000004005c0 <+28>:    mov     w2, w0
   0x00000000004005c4 <+32>:    ldr     w1, [x29,#28]
   0x00000000004005c8 <+36>:    add     x0, x29, #0x18
   0x00000000004005cc <+40>:    ldxr    w3, [x0]
   0x00000000004005d0 <+44>:    cmp     w3, w2
   0x00000000004005d4 <+48>:    b.ne    0x4005e0 <main+60>
   0x00000000004005d8 <+52>:    stlxr   w4, w1, [x0]
   0x00000000004005dc <+56>:    cbnz    w4, 0x4005cc <main+40>
   0x00000000004005e0 <+60>:    dmb     ish
```

CAS implementation using casal instruction

```
//  int *ptr=100;
  os_compare_and_swap_ulint(&old,old,new);
  4005bc:       b9401ba0        ldr     w0, [x29,#24]
  4005c0:       2a0003e2        mov     w2, w0
  4005c4:       b9401fa1        ldr     w1, [x29,#28]
  4005c8:       910063a0        add     x0, x29, #0x18
  4005cc:       2a0203e3        mov     w3, w2
  4005d0:       88e3fc01        casal   w3, w1, [x0]
  4005d4:       6b02007f        cmp     w3, w2
```

More efficient execution: 4 instructions 1 instruction

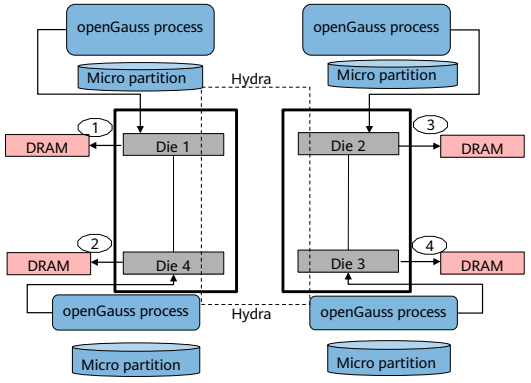


 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The CAS operation is a CPU-level atomic instruction as well as an optimistic lock. It is used to identify thread conflicts. Once a conflict is detected, the current operation is retried until no conflict occurs.
- The Transaction Processing Performance Council (TPC) is a non-profit organization established by dozens of member companies. Its headquarters is located in the United States. The TPC is responsible for formulating standards, performance, and price measurement of business application benchmark programs, and managing the release of test results.
- The tpmC value is widely used to measure the transaction processing capability of a computer system in and outside China. tpm is short for transactions per minute, and C refers to the C benchmark in TPC.

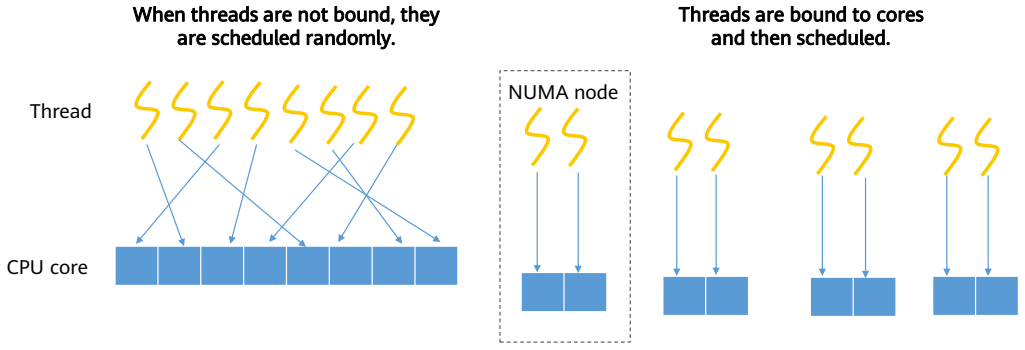
NUMA Kernel Data Structure



- Threads are bound to cores to prevent thread migration between cores.
- The NUMA-based data structure reconstruction reduces cross-core access.
- Data is partitioned to reduce thread access conflicts.
- The algorithms have been adjusted to reduce single-point bottlenecks.
- Arm atomic instructions are used to reduce computational overhead.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Enabling NUMA for Binding Threads to Cores



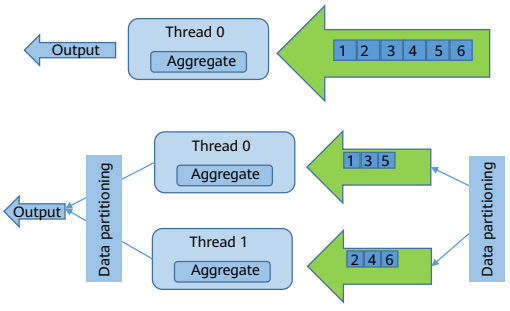
1. Network interrupts and background service threads are bound to cores separately.
2. Service threads are bound to different cores.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Without core binding, threads enter the thread queue and wait for time slices; threads compete for resources with each other; and the operating system needs to determine the physical CPU on which threads run based on task scheduling.
- Core binding reduces time spent on thread switching between different CPU cores and mitigates inter-thread resource contention.

SMP Technology (1)



SMP parallel execution

- The symmetric multi-processing (SMP) technology of openGauss uses the computer's multi-core CPU architecture to implement multi-thread parallel computing, capitalizing on CPU resources to improve query performance.
- SMP parallel execution refers to the parallel execution of tasks at the **thread** level. Theoretically, there can be as many subtasks executed concurrently as there are cores on the physical server.
- SMP parallel threads operate within the same process, and **data can be exchanged directly through the memory**, without occupying network connections and bandwidth. This reduces the impact of network factors that restrict the performance improvement of the massively parallel processing (MPP) system.
- As the parallel subtasks **do not need to be attached to another background worker thread** after being started, the utilization of system computing resources can be increased.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Complex queries may take a long time. In a system with low concurrency support, this can be a problem. SMP is used to implement operator-level parallel execution, which can effectively speed up queries, improving query performance and resource utilization. The overall implementation of the SMP technology is as follows: For query operators that can be executed in parallel, data is sliced, multiple worker threads are started for computation, and then the results are summarized and returned to the frontend. A data interaction operator named Stream is added to the SMP architecture to implement data interaction between multiple worker threads, ensuring the query accuracy and integrity.

SMP Technology (2)

- The database requires not only MPP for horizontal scale-out, but also SMP for better scale-up on a single node.
- SMP can improve the database performance in two ways:
  - Significantly reduces the time needed to execute a single query.
  - Improves the system throughput for a given period of time and improves system resource utilization.
- SMP parallel technology achieves efficient utilization of system resources through multi-threading and parallel execution of subtasks. The SMP multi-threading lightweight execution model can address the limitations of MPP architecture deployment.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Observe the current system load situation. If resources are sufficient (the resource usage is smaller than 50%), perform the next step. Otherwise, exit this system.
- Set **query_dop** to **1** (default value), use EXPLAIN to generate an execution plan, and check whether the plan complies with the SMP application scenarios. If it does, go to the next step.
- Set **query_dop** to **0** (adaptive). The system dynamically selects the optimal parallelism degree between [1, 8] for each query based on the resource usage and query plans to maximize the query performance.
- Other factors affecting SMP performance:
  - Severe data skew deteriorates SMP performance.
  - When the system concurrency is high, the SMP increases resource usage and contention, causing performance deterioration.

Contents

1. System Architecture
2. Resource Pooling
3. **Key Features**
   - High Performance
   - High Availability
   - High Security
   - Easy Maintenance
   - AI Capabilities

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

High-Availability Architecture

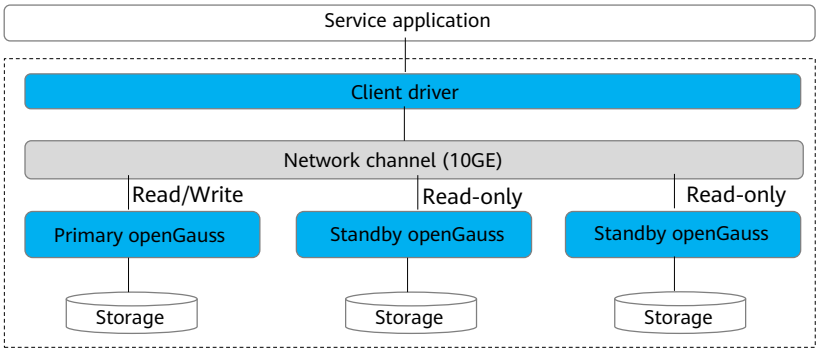
- To ensure that a fault can be recovered, there needs to be multiple copies of the data available. Multiple copies are configured for the primary and standby nodes, and logs are used for data synchronization. In this way, with openGauss, no data is lost if a node fails or if the system restarts after a stop. openGauss meets ACID requirements.
- There are two types of primary/standby deployment modes:
  - One primary node and one standby node
  - One primary node and multiple standby nodes
- To swap their roles, you can perform a switchover between the primary and standby nodes. In addition, if the primary node experiences a fault, the standby node can be promoted to primary through failover.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Atomicity, consistency, isolation, and durability (ACID)
- openGauss 2.0.0 and later versions support a maximum of one primary node and eight standby nodes.
- Switchover: A non-faulty role switchover is performed manually as planned. It is mainly used for database version upgrade and does not cause data loss.
- Failover: If the primary database is faulty, the standby database must be promoted to primary, which may cause data loss.

Architecture with One Primary and Multiple Standby Nodes

- This mode provides more robust DR and is suitable for OLTP systems with higher availability for transaction processing.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Primary/Standby Switchover (1)

- Scenario
  - The database administrator may need to manually perform a primary/standby switchover on an openGauss database node that is currently running. For example, you may need to restore the original primary/standby roles after a primary/standby database node failover, or to manually perform a primary/standby switchover due to a hardware fault. A cascaded standby node cannot be directly upgraded to the primary node. You need to first perform a switchover or failover to change the cascaded standby node to a standby node, and then to a primary node.
- Procedure
  - Log in to any database node as the OS user **omm** and run the following command to check the primary/standby status:

    ```
    gs_om -t status --detail
    ```
  - Log in to the standby node to be switched to the primary node as the OS user **omm** and run the following command:

    ```
    gs_ctl switchover -D /home/omm/cluster/dn1/
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **/home/omm/cluster/dn1/** is the data directory of the standby database node.
- The primary/standby switchover is a maintenance operation. Ensure that the openGauss status is normal and all services are complete before performing the switchover.
- When ultimate RTO is enabled, cascaded standby nodes are not supported. The standby node cannot be connected when the ultimate RTO is enabled. As a result, data cannot be synchronized with the cascaded standby node.

Primary/Standby Switchover (2)

- If the primary node is faulty, run the following command on the standby node:

```
gs_ctl failover -D /home/omm/cluster/dn1/
```

- After the switchover or failover is successful, run the following command to record information about the current primary and standby nodes:

```
gs_om -t refreshconf
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

High Availability - Primary and Standby Nodes

- Multi-copy deployment offers protection against instance-level failures. This is applicable to scenarios where equipment room-level DR is not required but some hardware faults need to be prevented.
- Generally, multi-copy deployment means one primary node and two standby nodes, so there are three copies in total. This deployment mode provides 99.99% reliability, which is good enough for most applications.
  - In primary/standby quorum replication, data is synchronized to at least one standby node to ensure the maximum performance.
  - If any node (primary or standby) is faulty, services will not be affected.
  - There are three copies of the data. If one node fails, the system still has two copies of the data in reserve, and any standby node can be promoted to primary.
  - The primary and standby nodes cannot be deployed on the same physical machine.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Ultimate RTO (1)

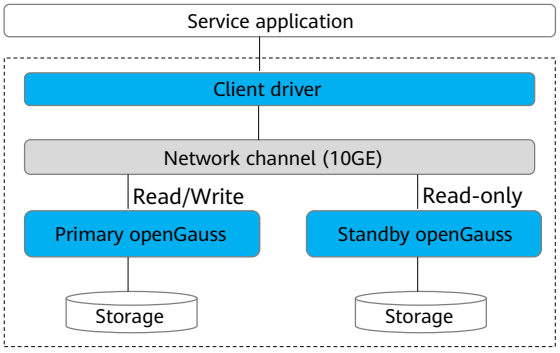
- Concepts
  - Recovery time objective (RTO)
    - Application-oriented
    - Amount of application downtime that can be tolerated
  - Recovery point objective (RPO)
    - Data-oriented
    - Amount of lost data that can be tolerated, as measured by time

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- For example, if a service needs to be recovered within half a day after a disaster occurs, the RTO is 12 hours.
  - Common technologies for improving RTO include tape recovery, manual transfer, and remote switchover of application systems.
- RPO
  - Common technologies for improving RPO include tape backup, periodic data replication, asynchronous data replication, and synchronous data replication.

Ultimate RTO (2)



1. RPO is the longest acceptable period for which data might be lost.
   Amount of lost data after the service system is recovered. This indicator reflects the data integrity.

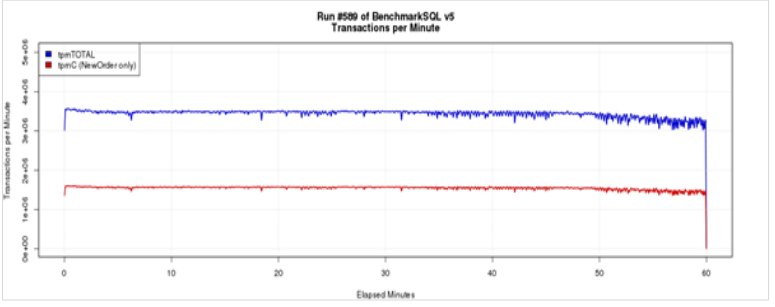
2. RTO is the maximum amount of acceptable downtime for the entire system after a disaster occurs.
   Duration from the time when a service is interrupted to the time when the service is restored. This indicator reflects the timeliness of service recovery.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- When the service load is heavy, the replay speed of the standby node cannot catch up with that of the primary node. After the system runs for a long time, logs are accumulated on the standby node. If the primary node is faulty, data restoration takes a long time, and the database is unavailable, which severely affects system availability.
- The ultimate RTO can be enabled to reduce the data recovery time after a primary node fault occurs and improve availability.
- Ultimate RTO: **recovery_parse_workers** indicates the number of Xlog threads parsed during parallel recovery. **recovery_parallelism** indicates the actual number of Xlog threads during parallel recovery. When **recovery_parse_workers** is greater than 1, ultimate RTO recovery is implemented. Only when both **recovery_parse_workers** and **recovery_parallelism** are set to a value greater than 1 can parallel recovery be implemented.

Ultimate RTO (3)

- Incremental checkpoint technology: more efficient, more frequent checkpoints and less downtime for a lower RTO



66 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This feature enables the primary database node to quickly recover from a restart.
- This feature implements log synchronization between primary and standby nodes to accelerate replay on the standby node.
- Once the ultimate RTO function is enabled, multi-level pipelines are established to enhance the concurrency and speed up Xlog replay.

Logical Replication

- Introduction
  - openGauss provides the logical decoding function to reversely parse physical logs into logical logs. Logical replication tools such as DRS convert logical logs to SQL statements and replay the SQL statements in the peer database. In this way, data can be synchronized between heterogeneous databases. Currently, unidirectional and bidirectional logical replication between the openGauss database and the MySQL or Oracle database is supported.
- Benefits
  - Logical replication is applicable to real-time database data migration, dual-database active-active systems, and rolling upgrades.
- Description
  - DNs reversely parse physical logs to logical logs. Logical replication tools such as DRS extract logical logs from DNs, convert the logs to SQL statements, and replay the SQL statements in the peer database (MySQL). Logical replication tools also extract logical logs from the MySQL database, reversely parse the logs to SQL statements, and replay the SQL statements in openGauss. In this way, data can be synchronized between heterogeneous databases.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- openGauss 1.0.0 logical decoding supports the extraction of logs from full and incremental logs.
- openGauss 1.1.0 supports logical decoding on a standby node.
- openGauss 6.0.0-RC1 logical replication supports many DDL operations.
- Constraint: Column-store replication is not supported.

Contents

1. System Architecture
2. Resource Pooling
3. **Key Features**
   - High Performance
   - High Availability
   - High Security
   - Easy Maintenance
   - AI Capabilities

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

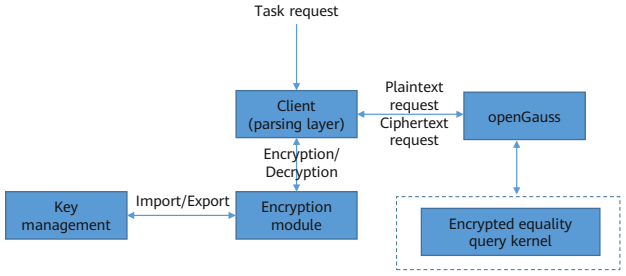
Equality Query in a Fully-Encrypted Database

- An encrypted database aims to protect privacy throughout the data lifecycle. In this way, data is always in ciphertext during transmission, computing, and storage regardless of the service scenario and environment.
- After the data owner encrypts data on the client and sends it to the server, an attacker cannot obtain valuable information even if the attacker steals user data by exploiting system vulnerabilities. In this way, data privacy is protected.
- The entire service data flow is in ciphertext during data processing, so the following can be implemented by using a fully-encrypted database:
  - Protects data privacy and security throughout the lifecycle on the cloud. Attackers cannot obtain information from the database server regardless of the data status.
  - Helps cloud service providers earn the trust of users. Users, including service administrators and O&M administrators in enterprise service scenarios and application developers in consumer cloud services, can keep the encryption keys themselves so that even users with high permissions cannot access unencrypted data.
  - Enables cloud databases to better comply with personal privacy protection laws and regulations.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Fully-Encrypted Equality Query Process

- Three-layer key management for root key, master key, and column encryption key.
- The client encrypts and decrypts data, and the server processes the encrypted data.
- Fields that do not need to be encrypted are still processed in plaintext.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The whole process is transparent to customer applications:
- (1) Generate a client master key. You need to deploy the keytool on the local host and use the keytool commands to manage keys, such as creating or deleting keys.
- (2) Record the CMK and CEK information and create an encrypted table in the system. After the key information generated in step 1 is available, design a dedicated key creation syntax for the database, including the new client master key (CMK) and column encryption key (CEK). The CMK stored in the key tool is accessed through the kernel API and recorded in the system. The CEK is encrypted by the CMK on the client and then stored on the database server. Data is encrypted by the CEK on the client and then transmitted to the server. The table creation syntax is modified to specify the encryption type of columns.
- (3) Conduct encrypted equality query. The client parses and identifies column attributes. If no encrypted columns are involved, the query result is returned to the server directly. If encrypted columns are involved, the columns are encrypted using the corresponding CEK and encryption algorithms and parameters and then sent to the server. After data encryption, the storage type on the server changes. Queries on the server must be executed considering this new storage type. Consequently, the optimizer and executor need to be adjusted to accommodate this change.

- **enc_type**: Determines whether to use deterministic encryption or randomized encryption.

Equality Query in a Fully-Encrypted Database (1)

- Encrypted equality query supports the following data types:

Data	Type	Description
Integer	tinyint/tinyint(n)	Tiny integer, which is the same as int1.
	smallint	Small integer, which is the same as int2.
	int4	Common integer.
	binary_integer	Oracle compatibility type. Generally, the value is an integer.
	bigint/bigint(n)	Big integer, which is the same as int8.
Numeric	numeric(p,s)	A number with the precision **p**.
	number	Oracle compatibility type, which is the same as numeric(p,s).
Floating point	float4	Single-precision floating point number.
	float8	Double-precision floating point number.
	double precision	Double-precision floating point number.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Equality Query in a Fully-Encrypted Database (2)

Data	Type	Description
Character	char/char(n)	Fixed-length character string. If the length is insufficient, add spaces. The default precision is **1**.
	varchar(n)	Variable-length character string, where **n** indicates the maximum number of bytes.
	text	Text type.
	varchar2(n)	Oracle compatibility type, which is the same as varchar(n).
	clob	Character large object.
Binary	bytea	Varying-length binary string
	blob	Binary large object.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Dynamic Data Masking (1)

- Data privacy is a required database security capability. Unauthorized access to user data needs to be restricted and data security needs to be ensured.
- The dynamic data masking mechanism is a technology that protects privacy data by customizing masking policies. It can effectively prevent unauthorized users from accessing sensitive information while retaining original data.
- When an administrator specifies an object to be masked and sets up a custom data masking policy, the data is masked according to the user's identity and the masking policy if the queried database resources are linked to a masking policy.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Benefits: Data privacy is a required database security capability. Unauthorized access to user data need to be restricted and data security needs to be ensured. The dynamic data masking mechanism can protect the privacy of specified database resources by configuring masking policies. In addition, the masking policy configuration is flexible and can implement targeted privacy protection in specific user scenarios.

Dynamic Data Masking (2)

- Dynamic data masking includes custom masking policies based on resource labels. You can select masking modes based on the scenario requirements or create custom masking policies for specific users. The SQL syntax for creating a masking policy is as follows:

```
CREATE RESOURCE LABEL label_for_creditcard ADD COLUMN(user1.table1.creditcard);
CREATE RESOURCE LABEL label_for_name ADD COLUMN(user1.table1.name);
CREATE MASKING POLICY msk_creditcard creditcardmasking ON LABEL(label_for_creditcard);
CREATE MASKING POLICY msk_name randommasking ON LABEL(label_for_name) FILTER ON IP(local), ROLES(dev);
```

- **label_for_creditcard** and **msk_name** are the resource labels for masking. Each label is allocated to two column objects. **creditcardmasking** and **randommasking** are preset masking functions. **msk_creditcard** specifies that the masking policy **creditcardmasking** will be applied when any user accesses resources with **label_for_creditcard**, regardless of the access source. **msk_name** specifies that the masking policy **randommasking** will be applied when local user **dev** accesses resources with **label_for_name**. If FILTER is not specified, the setting takes effect for all users. Otherwise, the setting takes effect only for specified users.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This slide describes the syntax of dynamic data masking.
  - Add resource tags for **table1.creditcard** and **table1.name**.
  - Then, use the preset functions **creditcardmasking** and **randommasking** to mask the data.

Dynamic Data Masking (3)

- The following table shows the preconfigured masking functions:

Masking Function	Example
creditcardmasking	'4880-9898-4545-2525' will be displayed as 'xxxx-xxxx-xxxx-2525'. This function masks digits except the last four digits.
basicemailmasking	'abcd@mymail.com' will be displayed as 'xxxx@mymail.com'. This function masks all text before the @.
fullemailmasking	'abcd@gmail.com' will be masked as 'xxxx@xxxxx.com'. This function masks text before the first dot (.) (except @).
alldigitsmasking	'alex123alex' will be displayed as 'alex000alex'. This function masks only the digits.
shufflemasking	'hello world' will be displayed as something like 'hlwoeor dl'. This weak masking function is implemented through character dislocation. It is not advised to use this function to mask strings with strong semantics.
randommasking	'hello world' will be displayed as something like 'ad5f5ghdf5'. This function randomly masks text by character.
maskall	'4880-9898-4545-2525' will be displayed as 'xxxxxxxxxxxxxxxxxx'.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- This slide describes the preset masking function and provides an example. It helps trainees better understand the preset functions.

Dynamic Data Masking – Feature Constraints

- The dynamic data masking policy must be created by a user with the POLADMIN or SYSADMIN attribute, or by the initial user. Common users do not have the permission to access the security policy system catalog and system view.
- Dynamic data masking takes effect only on data tables for which masking policies are configured. Audit logs are not within the effective scope of the masking policies.
- In a masking policy, only one masking mode can be specified for a resource label.
- Multiple masking policies cannot be used to anonymize the same resource label, except when FILTER is used to specify user scenarios where the policies take effect and there is no intersection between user scenarios of different masking policies that contain the same resource label. In this case, you can identify the policy that a resource label is anonymized by based on the user scenario.
- It is recommended that **APP** in FILTER be set to applications in the same trusted domain. Since a client may be forged, a security mechanism must be formed on the client when **APP** is used, to reduce misuse risks. Generally, It is not advised to use it. If it is used, pay attention to the risk of client spoofing.
- For INSERT or MERGE INTO operations with the query clause, if the source table contains masked columns, the inserted or updated result in the preceding two operations is the masked value and cannot be restored.
- When the built-in security policy is enabled, the execution of the ALTER TABLE EXCHANGE PARTITION operation will fail if the source table contains the masked column.
- If a dynamic data masking policy is configured for a table, grant the trigger permission of the table to other users with caution to prevent other users from using the trigger to bypass the masking policy.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- A maximum of 98 dynamic data masking policies can be created.
- Only the preceding seven preset masking policies can be used.
- Only data with the resource labels containing the COLUMN attribute can be masked.
- Data masking can be performed only on columns of basic tables.
- Only the data queried using SELECT can be masked.

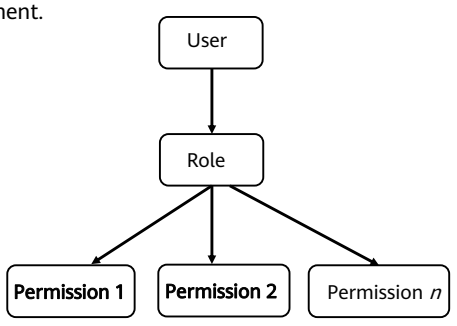
Access Control

- Access control manages users' access permissions to database, including system permissions and object permissions.
- Role-based access control is supported. In this mode, specific permissions are assigned to distinct roles and then these roles are assigned to users. Login access control is managed through user ID and authentication. Object access control is managed by verifying the user's permissions on the objects. Users are assigned with the minimum permissions required to reduce the risk of database misuse.
- Access control with separation of duties is supported. Database roles include system administrator, security administrator, and audit administrator. The security administrator creates and manages users, the system administrator grants and revokes user permissions, and the audit administrator audits all user behaviors.
- By default, the role-based access control model is used. Access control with separation of duties can be enabled via parameter settings.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Role-based Access Control (1)

- RBAC
  - Role-based access control (RBAC) grants permissions to roles so that users can obtain the permissions of these roles by becoming appropriate roles.
  - RBAC greatly simplifies permission management.
- RBAC model
  - Assign appropriate permissions to roles.
  - Associate users with roles.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Role-based Access Control (2)

- RBAC authorization defines relationships among who, what, and how.
  - Who: owner or subject of the permission (for example, a user).
  - What: object (table or function) to which the permission applies.
  - How: specific permission (positive authorization or negative authorization).
- Relationships between users, roles, and permissions in the RBAC model are as follows.
  - A user can correspond to multiple roles.
  - A role can correspond to multiple users.
  - A role can have multiple permissions.
  - A permission type can be assigned to multiple roles.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Role-based Access Control (3)

- Other access control models
  - Access control lists (ACL)
  - Attribute-based access control (ABAC)
  - Policy-based access control (PBAC)
- RBAC features and advantages
  - Indirect relationships
  - Differentiated responsibilities
  - Efficient permission management
  - Principle of least privilege, differentiated responsibilities, and data abstraction

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Row-level Security (1)

- The row-level security feature enables database access control to be accurate to each row of data tables.
- When different users perform the same SQL query operation, the read results may be different.
- Users can create a row-level security policy for a data table. The policy defines an expression that takes effect only for specific database users and SQL operations.
  - When a database user accesses the data table, if a SQL statement meets the specified row-level security policy of the data table, the expressions that meet the specified condition will be combined by using **AND** or **OR** based on the attribute type (PERMISSIVE or RESTRICTIVE) and applied to the execution plan in the query optimization phase.
- Row-level security is used to control the visibility of row-level data in tables. By predefining filters for data tables, the expressions that meet the specified condition can be applied to execution plans in the query optimization phase, which will affect the final execution result.
- Currently, the SQL statements that can be affected include SELECT, UPDATE, and DELETE.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Row-level Security (2)

- Enable row-level security.

```
ALTER TABLE tablename ENABLE ROW LEVEL SECURITY;
```

- Create a row-level security policy to specify that the current user can view only their own data.

```
CREATE ROW LEVEL SECURITY POLICY tablename_rls ON tablename USING(role = CURRENT_USER);
```

  - Note: **tablename** indicates the name of the created table, and **tablename_rls** indicates the name of the created row-level security policy.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Row-level Security – Feature Constraints

- Row-level security policies can be applied only to SELECT, UPDATE, and DELETE operations and cannot be applied to INSERT and MERGE operations.
- Row-level security policies can be defined for row-store tables, row-store partitioned tables, column-store tables, column-store partitioned tables, replication tables, unlogged tables, and hash tables. Row-level security policies cannot be defined for foreign tables or temporary tables.
- Row-level security policies cannot be defined for views.
- A maximum of 100 row-level security policies can be defined for a table.
- Initial users and system administrators are not affected by row-level security policies.
- If a row-level security policy is configured for a table, grant the trigger permission of the table to other users with caution to prevent other users from using the trigger to bypass the policy.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Audit

- The audit log mechanism tracks user operations on the database, including start/stop, connection, and DDL, DML, and DCL operations. This mechanism strengthens the database's ability to trace unauthorized operations and gather evidence.
- Users can configure parameters to specify which statements or operations should be logged.
- Audit logs record the event time, type, execution result, username, database, connection information, and database object, instance name, port number, and details. Users can query logs based on start time and end time, as well as filter logs by specific fields.
- Database audit administrators can use the audit logs to reproduce a series of events that cause faults in the database and identify unauthorized users, unauthorized operations, and the time when these operations are performed.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Enhanced features:
  - In addition to the traditional audit function, version 5.0.0 supports user-level audit. The GUC parameter **full_audit_users** is added to configure the full audit user list. Audit logs are recorded for all auditable operations performed by users in the list. The GUC parameter **no_audit_client** is added to configure the list of clients that do not need to be audited. The GUC parameter **audit_system_function_exec** is added to enable or disable the audit of system functions.
  - In version 6.0.0, integrity verification can be performed on audit logs. The database tool audit log function is added to record key O&M operations such as backup and restoration.

Unified Audit Mechanism

- In addition to the preceding data audit functions, the unified audit mechanism is also supported. Security audits are made more efficient with the use of customized audit policies. Administrators can define specific audit objects and operations. When a task associated with an audit policy is performed, the corresponding audit behavior is triggered, and a log entry is recorded. Customized audit policies can cover common user management activities, as well as DDL and DML operations, meeting routine audit requirements.
- Benefits
  - Audit is indispensable for routine security management. When a traditional audit mechanism is used to audit an operation, such as SELECT, a large number of audit logs are generated, increasing the I/O of the entire system and affecting the system performance and audit efficiency of administrators. The unified audit mechanism allows users to customize the policy for generating audit logs. For example, only the behavior of database account **A** querying a table is audited. Customized audit greatly reduces the number of generated audit logs, ensuring audit behaviors and reducing the impact on system performance. In addition, customized audit policies can improve the audit efficiency of administrators.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Unified Auditing

- Unified auditing allows administrators to configure audit policies for database resources or resource labels to simplify management, generate audit logs, reduce redundancy, and improve management efficiency.
- Administrators can customize audit policies to focus on specific user scenarios, behaviors, or database resources. Once the unified auditing feature is enabled, the system matches user identity information (such as access IP address, client tool, and username) with the configured audit policies. Based on the resource label and the operation type (DML or DDL), the system performs the audit accordingly.
- The goal of unified auditing is to shift from traditional auditing to a more focused approach, excluding irrelevant behaviors from the audit. This not only simplifies management but also strengthens the security of the audit data generated by the database.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Creating a Unified Audit Policy

- Only the poladmin, sysadmin, or initial user can create an audit policy. In addition, users need to enable the security policy, that is, set the GUC parameter **enable_security_policy** to **on** to make the policy take effect.

```
CREATE AUDIT POLICY [ IF NOT EXISTS ] policy_name { { privilege_audit_clause |
access_audit_clause } [ filter_group_clause ] [ ENABLE | DISABLE ] };
• privilege_audit_clause:
PRIVILEGES { DDL | ALL } [ ON LABEL ( resource_label_name [, ... ] ) ]
• access_audit_clause:
ACCESS { DML | ALL } [ ON LABEL ( resource_label_name [, ... ] ) ]
• filter_group_clause:
FILTER ON { ( FILTER_TYPE ( filter_value [, ... ] ) ) [, ... ] }
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **Resource_label_name**: specifies the resource label name.
- **Filter_type**: specifies the types of information to be filtered by the policy, including **IP**, **APP**, and **ROLES**.
- **Filter_value**: specifies the detailed information to be filtered.
- GUC parameters: globally unified configuration parameters

Example of Creating a Unified Audit Policy (1)

- Step 1: Create the **dev_audit** and **bob_audit** users.

```
CREATE USER dev_audit PASSWORD 'dev@1234';
CREATE USER bob_audit password 'bob@1234';
```

- Step 2: Create the **tb_for_audit** table.

```
CREATE TABLE tb_for_audit(col1 text, col2 text, col3 text);
```

- Step 3: Create a resource label.

```
CREATE RESOURCE LABEL adt_lb0 add TABLE(tb_for_audit);
```

- Step 4: Perform the CREATE operation on the database to create an audit policy.

```
CREATE AUDIT POLICY adt2 PRIVILEGES SELECT;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example of Creating a Unified Audit Policy (2)

- Step 5: Create an audit policy to audit only the CREATE operation performed on the **adt_lb0** resource by users **dev_audit** and **bob_audit**.

```
CREATE AUDIT POLICY adt3 PRIVILEGES CREATE ON LABEL(adt_lb0) FILTER ON
ROLES(dev_audit, bob_audit);
```

- Create an audit policy to audit only the SELECT, INSERT, and DELETE operations performed on the **adt_lb0** resource by users **dev_audit** and **bob_audit** using client tools psql and gsql on the servers whose IP addresses are **10.20.30.40** and **127.0.0.0/24**.

```
CREATE AUDIT POLICY adt4 ACCESS SELECT ON LABEL(adt_lb0), INSERT ON LABEL(adt_lb0),DELETE
FILTER ON ROLES(dev_audit, bob_audit), APP(psql, gsql), IP('10.20.30.40', '127.0.0.0/24');
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying and Deleting a Unified Audit Policy

- Only the poladmin, sysadmin, or initial user can modify or delete the unified audit policy.

```
ALTER AUDIT POLICY [ IF EXISTS ] policy_name { ADD | REMOVE } { [
privilege_audit_clause ] [ access_audit_clause ] };
ALTER AUDIT POLICY [ IF EXISTS ] policy_name MODIFY ( filter_group_clause );
ALTER AUDIT POLICY [ IF EXISTS ] policy_name DROP FILTER;
ALTER AUDIT POLICY [ IF EXISTS ] policy_name COMMENTS policy_comments;
ALTER AUDIT POLICY [ IF EXISTS ] policy_name { ENABLE | DISABLE };
```

```
DROP AUDIT POLICY [IF EXISTS] policy_name;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

System Catalogs Related to Unified Auditing

- **GS_AUDITING_POLICY** records the main information about the audit. Each record corresponds to a design policy.
- **GS_AUDITING_POLICY_ACCESS** records the audit information for DML database operations.
- **GS_AUDITING_POLICY_FILTERS** records the filter policies of the audit. Each record corresponds to a design policy.
- **GS_AUDITING_POLICY_PRIVILEGES** records the DDL database operations for the audit. Each record corresponds to a design policy.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Only the system administrator or security policy administrator can access these system catalogs.

Unified Auditing – Feature Constraints

- The unified audit policy must be created by a user with the POLADMIN or SYSADMIN attribute, or by the initial user. Common users do not have the permission to access the security policy system catalog and system view.
- The unified audit policy syntax applies to either DDL or DML operations. A unified audit policy cannot contain both DDL and DML operations. Currently, a maximum of 98 unified audit policies can be set.
- Unified auditing monitors the SQL statements executed by users on the clients, but does not record the internal SQL statements of databases.
- In the same audit policy, the same resource label can be bound to different audit behaviors, and the same behavior can be bound to different resource labels. The ALL operation type includes all operations supported by DDL or DML.
- A resource label can be associated with different unified audit policies. Unified auditing outputs audit information in sequence based on the policies matched by SQL statements.
- Audit logs of unified audit policies are recorded separately. Currently, no visualized query APIs are provided. Audit logs depend on the OS service rsyslog and are archived through the service configuration.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- In cloud service scenarios, logs need to be stored in the OBS. In hybrid cloud scenarios, you can deploy Elasticsearch to collect CN logs and perform visualized processing.
- It is recommended that **APP** in FILTER be set to applications in the same trusted domain. Since a client may be forged, a security mechanism must be formed on the client when **APP** is used to reduce misuse risks. Generally, you are not advised to use it. If **APP** is used, pay attention to the risk of client spoofing.

Transparent Data Encryption

- Introduction
  - Transparent data encryption (TDE) encrypts data when the database writes data to storage media and automatically decrypts the data when reading the data from the storage media. This prevents attackers from reading data in the data file without database authentication, preventing static data leakage. This function is almost transparent to the application layer. Users can determine whether to enable the TDE function as required.
- Description
  - A three-layer key structure is used to implement the key management mechanism, including the root key (RK), cluster master key (CMK), and data encryption key (DEK). CMKs are encrypted and protected by RKs, and DEKs are encrypted and protected by CMKs. DEKs are used to encrypt and decrypt user data. Each table has a corresponding DEK.
  - Table-level encryption is supported. When creating a table, users can specify whether to encrypt the table and the cryptographic algorithm to be used. The cryptographic algorithm can be **AES_128_CTR** or **SM4_CTR**, which cannot be changed once specified. If an encrypted table is created, the database automatically applies for a DEK for the table and saves details like the cryptographic algorithm, key ciphertext, and corresponding CMK ID in the **reloptions** column of the **pg_class** system catalog in **keyword=value** format.
  - Users can switch the encryption status of an encrypted table, that is, from an encrypted table to a non-encrypted table or from a non-encrypted table to an encrypted table. If the encryption function is not enabled when a table is created, the table cannot be switched to an encrypted table.
  - For encrypted tables, DEK rotation is supported. After the key rotation, the data encrypted using the old key is decrypted using the old key, and the newly written data is encrypted using the new key. The cryptographic algorithm is not changed during key rotation.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Benefits
  - To prevent attackers from reading data files without authentication, you can use the TDE function to encrypt data files in the database. This ensures that users can read decrypted data only after starting and connecting to the database properly.

Transparent Data Encryption – Feature Constraints

- The current version interconnects with Huawei Cloud KMS to support table-level key storage and row-store table encryption. The specifications are as follows:
  - Encryption of a row-store table stored as a heap is supported.
  - Column-store encryption, materialized view encryption, and Ustore-based encryption are not supported.
  - Indexes, sequences, Xlogs, MOTs, and system catalogs cannot be encrypted.
  - Users can specify an encryption algorithm when creating a table. Once specified, the encryption algorithm cannot be changed. If **enable_tde** is set to **on** but the encryption algorithm **encrypt_algo** is not specified when a table is created, the **AES-128-CTR** encryption algorithm is used by default.
  - If the encryption function is not enabled or the encryption algorithm is not specified when a table is created, the table cannot be switched to an encrypted table.
  - For a table that has been assigned an encryption key, switching between the encrypted and unencrypted table states does not change the key or encryption algorithm.
  - Data key rotation is supported only when the table encryption function is enabled.
  - Cross-region primary/standby synchronization of multiple copies in a single cluster is not supported. Cross-region scale-out of a single cluster is not supported. Cross-region backup and restoration, cluster DR, and data migration are not supported.
  - In hybrid cloud scenarios, if the Huawei Cloud KMS and management plane functions are used, TDE is supported. For other KMS services, TDE is not supported if no compatible API is available.
  - The query performance of encrypted tables is lower than that of non-encrypted tables. If high performance is required, exercise caution when enabling the encryption function.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Dependency
  - The key management service is provided by the external KMS. The current version can interconnect with Huawei Cloud KMS.

Contents

1. System Architecture
2. Resource Pooling
3. **Key Features**
   - High Performance
   - High Availability
   - High Security
   - Easy Maintenance
   - AI Capabilities

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Workload Diagnosis Report (WDR) Overview

- The WDR generates a performance report for the period of time between two snapshots. The report is used to diagnose database kernel performance faults.
- Benefits
  - WDR is the main method for diagnosing long-term performance problems. Multiple aspects of system performance are analyzed and compared to snapshot baselines. These analyses inform DBAs of the system load and the performance of each component, so they can identify performance bottlenecks.
  - Snapshots are also an important data source for subsequent performance problem self-diagnosis and self-optimization suggestions.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- You can use **generate_wdr_report(...)** to generate a performance report based on two performance snapshots.

Workload Diagnosis Report (WDR)

- WDR depends on the following two components:
  - Snapshot: The performance snapshot can be configured to collect a certain amount of performance data from the kernel at a specified interval and store the data in the user tablespace. Any snapshot can be used as a performance baseline for comparison with other snapshots.
  - WDR reporter: This tool analyzes the overall system performance based on two snapshots, calculates the changes of more specific performance indicators between two points in time, and generates performance data by two levels: SUMMARY and DETAIL.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Feature Constraints
  - The WDR snapshot collects performance data of different databases. If there are a large number of databases or tables in the database instance, it takes a long time to create a WDR snapshot.
  - If WDR snapshot is performed when a large number of DDL statements are executed, WDR snapshot may fail.
  - When the database is dropped, WDR snapshot may fail.

WDR Generation

- Prerequisites
  - The WDR snapshot is enabled (that is, **enable_wdr_snapshot** is set to **on**), and the number of snapshots is greater than or equal to **2**.
- Procedure
  - Run the following command to query the generated snapshot and obtain **snapshot_id**:

    ```
    select * from snapshot.snapshot;
    ```

  - (Optional) Run the following command to manually create a snapshot. If only one snapshot exists in the database or users want to view the monitoring data of the database in the current period, manually create a snapshot. This command is only available to the sysadmin user.

    ```
    select create_wdr_snapshot();
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- The value of **enable_wdr_snapshot** is set **off** by default.
- **wdr_snapshot_retention_days**: number of days for storing database monitoring snapshot data in the system. The default value is **8**.
- **wdr_snapshot_interval**: specifies the interval at which the backend thread Snapshot automatically performs snapshot operations on the database monitoring data. The default value is 1 hour.

Example of WDR Generation

- Query the snapshots that have been generated.

```
postgres=# select * from snapshot.snapshot;
 snapshot_id |           start_ts            |            end_ts
-------------+-------------------------------+-------------------------------
           1 | 2020-09-07 10:20:36.763244+08 | 2020-09-07 10:20:42.166511+08
           2 | 2020-09-07 10:21:13.416352+08 | 2020-09-07 10:21:19.470911+08
```

- Write the queried information to a performance report.

```
select generate_wdr_report(begin_snap_id Oid, end_snap_id Oid, varchar report_type, varchar report_scope, int
node_name );
```

- Generate the formatted performance report **wdrTestNode.html**.

```
postgres=# \a \t \o /home/om/wdrTestNode.html
Output format is unaligned.
Showing only tuples.
```

- Close the performance report.

```
postgres=# \o \a \t
Output format is aligned.
Tuples only is off.
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

The parameters in the preceding commands are described as follows:

- **\a**: switches to the unaligned mode.
- **\t**: switches the information and row count footer of the output column name.
- **\o**: specifies that all the query results are sent to the server file.
- Server file path: specifies the path for storing the generated performance report file. The user must have the read and write permissions on the path.
- - **begin_snap_id**: specifies the ID of a snapshot when a query starts.
  - **end_snap_id**: specifies the ID of a snapshot when a query ends.
  - **report_type**: specifies the type of the report to be generated.
  - **report_scope**: specifies the scope of the report to be generated. The value can be **cluster** or **node**.
  - **node_name**: When **report_scope** is set to **node**, set this parameter to the name of the corresponding node.

Summarized Diagnosis Report

Diagnosis Type	Description
Database Stat	Evaluates the load and I/O status of the current database. Load and I/O are the most important characteristics of a TP system. Includes the number of sessions connected to the database, number of committed and rolled back transactions, number of read disk blocks, number of disk blocks found in the cache, number of rows returned, captured, inserted, updated, and deleted through database query, number of conflicts and deadlocks, usage of temporary files, and I/O read/write time.
Load Profile	Evaluates the current system load from the time, I/O, transaction, and SQL dimensions. Includes the job running elapsed time, CPU time, transaction log volume, logical and physical read volume, read/write I/O times and size, login and logout times, SQL, transaction execution volume, and SQL P85 and P90 response time.
Instance Efficiency Percentages	Evaluates the cache efficiency of the current system. Includes the database cache hit ratio.
Events	Evaluates the performance of key system kernel resources and key events. Includes the number of times that the key events of the database kernel have occurred and the wait times.
Wait Classes	Evaluates the performance of key events in the system. Includes the release of the data kernel in the main types of wait events, such as STATUS, LWLOCK_EVENT, LOCK_EVENT, and IO_EVENT.
CPU	Includes time release of the CPU in user, kernel, wait I/O, or idle modes.
I/O Profile	Includes the number of database I/O times, database I/O data volume, number of redo I/O times, and redo I/O volume.
Memory Statistics	Includes the maximum process memory, used process memory, maximum shared memory, and used shared memory.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Detailed Diagnosis Report

Diagnosis Type	Description
Time Model	Evaluates the performance of the current system based on how much time various items take. Includes time consumed by the system in each phase: the kernel time, CPU time, execution time, parsing time, compilation time, query rewriting time, plan generation time, network time, and I/O time.
SQL Statistics	Diagnoses SQL statement performance problems. Includes a range of different normalized SQL performance indicators: elapsed time, CPU time, rows returned, tuple reads, executions, physical reads, and logical reads. These indicators can be classified into execution time, number of execution times, row activity, and cache I/O.
Wait Events	Diagnoses performance of key system resources and key time in detail. Includes the performance of all key events in a period of time: the number of events and the time consumed.
Cache I/O Stats	Diagnoses the performance of user tables and indexes. Includes read and write operations on all user tables and indexes, and the cache hit ratio.
Utility status	Diagnoses the performance of backend jobs. Contains the performance of backend operations such as page operations and replication.
Object stats	Diagnoses the performance of database objects. Includes user tables, tables on indexes, index scan activities, the number of valid rows, table maintenance statuses, and INSERT, UPDATE, and DELETE activities.
Configuration settings	Determines whether a configuration has changed. Includes snapshots of all current configuration parameters.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

One-Click Diagnosis Information Collection (1)

- Multiple suites are provided to capture, collect, and analyze diagnosis data, enabling fault diagnosis and accelerating the diagnostic process. Necessary database logs, cluster management logs, and stack information can be extracted from the production environment based on the requirements of development and fault locating personnel. Fault locating personnel demarcate and locate faults based on this information.
- The one-click collection tool obtains different information from the production environment depending on the actual faults, improving the fault locating and demarcation efficiency. The user can modify the configuration file based on the information to be collected. For example, the user can collect:
  - OS information by running Linux commands
  - Database information by querying system catalogs or views
  - Run logs of the database system and logs related to cluster management
  - Database system configuration information

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

One-Click Diagnosis Information Collection (2)

- Core files generated by database-related processes
- Stack information about database-related processes
- Trace information generated by the database processes
- Redo log files (xlogs) generated by the database
- Planned reproduction information

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Slow SQL Diagnosis

- This feature provides detailed information required for slow SQL diagnosis. The user can diagnose performance problems of specific slow SQL statements offline without reproducing the problem. The table-based and function-based APIs help users collect statistics on slow SQL indicators and connect to third-party platforms.
- Slow SQL diagnosis provides information necessary for diagnosing slow SQL statements, helping developers backtrack SQL statements whose execution time exceeded the configured threshold, and diagnose SQL performance bottlenecks.
- Slow SQL diagnosis records information about all jobs whose execution time exceeds the threshold **log_min_duration_statement**.
- Slow SQL provides table-based and function-based query APIs. The user can query the execution plan, start time, end time, query statement, row activity, kernel time, CPU time, execution time, parsing time, compilation time, query rewriting time, plan generation time, network time, I/O time, network overhead, and lock overhead. All information is masked.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This feature is available since openGauss 3.1.0.
- On the standby node, slow SQL statements provide a dedicated function as the query API. The standby node cannot write performance diagnosis data to the **statement_history table**. Therefore, the standby node uses a new method to record data and queries the data through the function API. The information queried by users in the API is the same as that in the **statement_history** table on the primary node.

Slow SQL Diagnosis – Feature Enhancement

- Optimized slow SQL indicators, security (data masking), execution plans, and query interfaces.

```
Run the following command to check the execution information about the SQL statements in the database instance:
gsql> select * from dbe_perf.get_global_full_sql_by_timestamp(start_timestamp, end_timestamp);
Example:
postgres=# select * from DBE_PERF.get_global_full_sql_by_timestamp('2020-12-01 09:25:22', '2020-12-31 23:54:41');
-[ RECORD 1 ]--------+-----------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------
node_name            | coordinator1
db_name              | postgres
schema_name          | "$user",public
origin_node          | 1938253334
user_name            | user_dj
application_name     | gsql
client_addr          |
client_port          | -1
unique_query_id      | 3671179229
debug_query_id       | 72339069014839210
query                | select name, setting from pg_settings where name in (?)
start_time           | 2020-12-19 16:19:51.216818+08
finish_time          | 2020-12-19 16:19:51.224513+08
...
query_plan           | Coordinator Name: coordinator1
                     | Function Scan on pg_show_all_settings a  (cost=0.00..12.50 rows=5 width=64)
                     |   Filter: (name = '***'::text)
...
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The information in bold indicates that the data in the query conditions is masked when the user views the execution plan of the SQL statement.

Slow SQL Diagnosis – Feature Constraints

- The SQL tracing information is based on the normal execution logic. The tracing information may be inaccurate if SQL statements fail to be executed.
- Restarting a node may cause data loss on the node.
- If the user exits a session immediately after SQL statements are executed, the session data that is not updated to the system catalog may be lost.
- The number of SQL statements to be collected is specified by a GUC parameter. If the number of SQL statements exceeds the threshold, new SQL statement execution information will not be collected.
- The maximum number of bytes of lock event details collected by a single SQL statement is specified by a GUC parameter. If the number of bytes exceeds the threshold, new lock event details will not be collected.
- The SQL statement information is updated in asynchronous mode. Therefore, after a query statement is executed, the related view function result may be slightly delayed.
- When **track_stmt_parameter** is set to **off**, the maximum length of the **query** field is determined by the value of **by track_activity_query_size**.
- Certain indicator information (such as row activity, cache I/O, and time distribution) depends on the **dbe_perf.statement** view. If the number of records in the view exceeds the preset size (depending on the GUC parameter: **instr_unique_sql_count**), related indicators may not be collected.
- The **details** column in the functions and views related to the **statement_history** table and standby node function dbe_perf.standby_statement_history is in binary format. To parse the detailed information, use the pg_catalog.statement_detail_decode(details, 'plaintext', true) function.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The **statement_history** table can be queried only in the **postgres** database. The data in other databases is empty.
- To query the dbe_perf.standby_statement_history function on the standby node, the user needs to switch to the **postgres** database. If the user queries the dbe_perf.standby_statement_history function in other databases, a message is displayed indicating that the function is unavailable.
- The standby node uses **the track_stmt_standby_chain_size** parameter to limit the memory and disk space occupied by recorded data.
- The content of the **statement_history** table and standby node function dbe_perf.standby_statement_history is controlled by **track_stmt_stat_level**. The default value is **'OFF,L0'**. The first part of the parameter indicates the full SQL statement, and the second part indicates the slow SQL statement. Slow SQL statements are recorded only when the SQL statement execution time exceeds the value of **log_min_duration_statement**.

Session Performance Diagnosis

- This feature diagnoses performance of all active sessions in the system. As real-time collection of indicators from all active sessions significantly impacts user load, session snapshots are used to sample statistical indicators instead. The statistics of the active sessions from the samples reflect the basic information, status, and resources of active sessions from the dimensions of client information, execution start time, execution end time, SQL text, wait events, and current database objects. This information can help users diagnose which sessions consume more CPU and memory resources, which database objects are hot objects, and which SQL statements consume more key event resources in the system. In this way, users can locate faulty sessions, SQL statements, and database designs.
- Session sampling data can be divided into two levels:
  - The first level is real-time information stored in the memory. The active session information in the latest several minutes is displayed, which has the highest precision.
  - The second level is the persistent historical information stored in disk files. It displays the historical active session information in a long period of time and is sampled from the memory data. This level is suitable for long-run statistics and analysis.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Session Diagnosis Benefits

- Check the latest events that consume the most resources of user sessions.
- Check the wait events that occupy the most resource-consuming SQL statements.
- Check the wait events that occupy the most resource-consuming sessions.
- Check information about the most resource-consuming users.
- Check the wait relationship between blocked sessions.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common Statements for Session Performance Diagnosis (1)

- Check the blocking relationship between sessions.

```
select sessionid, block_sessionid from pg_thread_wait_status;
```

- Sample information about blocked sessions.

```
select sessionid, block_sessionid from DBE_PERF.local_active_session;
```

- Display the final blocked session.

```
select sessionid, block_sessionid, final_block_sessionid from DBE_PERF.local_active_session;
```

- Check the wait event that consumes the most resources.

```
SELECT s.type, s.event, t.count
FROM dbe_perf.wait_events s, (
SELECT event, COUNT(*)
FROM dbe_perf.local_active_session
WHERE sample_time > now() - 5 / (24 * 60)
GROUP BY event)t WHERE s.event = t.event ORDER BY count DESC;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common Statements for Session Performance Diagnosis (2)

- Check the events that occupy the most resource-consuming sessions.

```
SELECT sessionid, start_time, event, count
FROM (
SELECT sessionid, start_time, event, COUNT(*)
FROM dbe_perf.local_active_session
WHERE sample_time > now() - 5 / (24 * 60)
GROUP BY sessionid, start_time, event) as t ORDER BY SUM(t.count) OVER (PARTITION BY t.  sessionid,
start_time)DESC, t.event;
```

- Check the events that occupy the most resource-consuming SQL statements.

```
SELECT query_id, event, count
FROM (
SELECT query_id, event, COUNT(*)
FROM dbe_perf.local_active_session
WHERE sample_time > now() - 5 / (24 * 60)
GROUP BY query_id, event) t ORDER BY SUM(t.count) OVER (PARTITION BY t.query_id ) DESC, t.event DESC;
```

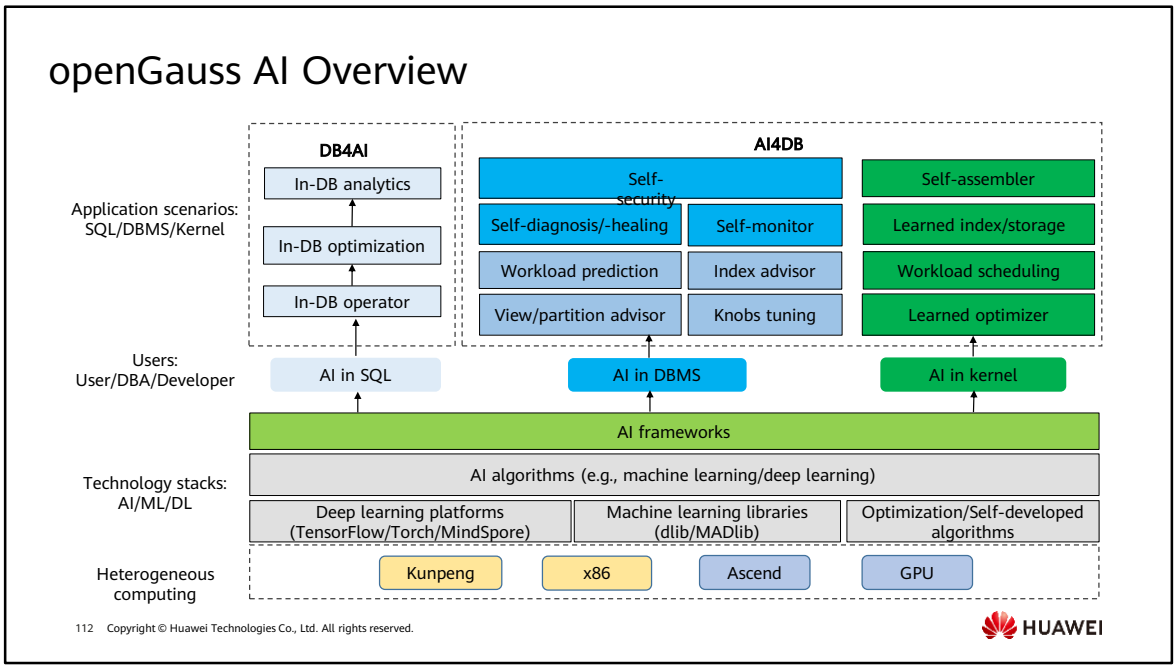

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. System Architecture
2. Resource Pooling
3. **Key Features**
   - High Performance
   - High Availability
   - High Security
   - Easy Maintenance
   - AI Capabilities

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI



- The AI capabilities, known as DBMind, is a submodule independent from other database components. It consists of two parts: AI4DB and DB4AI.
- AI4DB leverages AI technologies to enhance database execution performance and enable autonomous, maintenance-free operation. It includes self-tuning, self-diagnosis, self-security, self-O&M, and self-healing.
- DB4AI streamlines the E2E process from databases to AI applications and unifies the AI technology stack to achieve out-of-the-box availability, high performance, and cost-efficiency. For instance, the user can utilize SQL-like statements to access functions like recommendation systems, image retrieval, and time series forecasting, thereby maximizing GaussDB's strengths, such as high parallelism and column store.

X-Tuner Overview

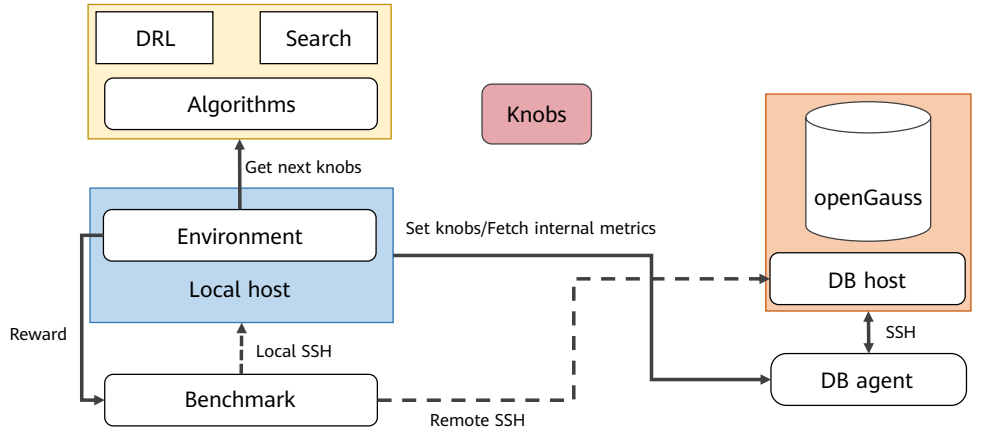
- X-Tuner is a parameter tuning tool integrated into the database. It uses AI technologies such as deep reinforcement learning and global search algorithm to obtain the optimal database parameter settings without manual intervention.
- The tuning program is a tool independent from the database kernel. This tool is not necessarily deployed within the database environment. It can be deployed independently and run without the database installation environment. The usernames and passwords for the database and instances are required to control the database benchmark performance testing. Before starting X-Tuner, ensure that the interaction in the test environment is normal, the benchmark test script can be run properly, and the database can be connected properly.
- This tool can quickly provide the parameter adjustment configuration of the current load in any scenario, reducing the database administrator's manual intervention, improving the O&M effect, and meeting customer expectations.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- First, let's look at AI4DB. It includes X-Tuner, SQLdiag, Index-advisor, and Anomaly-detection.
- Next, let's look at DB4AI. It includes Predictor and DeepSQL.

X-Tuner Structure



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The entire system comprises the following components:
  - DB (short for database): The DB agent abstracts database instances to obtain the internal database status information and current database parameters, and set database parameters. The user can log in to the database environment through SSH.
  - On the algorithm side, the algorithm packages used for tuning include global search algorithms (such as Bayesian optimization and particle swarm optimization) and deep reinforcement learning (such as DDPG).
  - Main logic module of X-Tuner: encapsulated by the environment module. Each step is a tuning iteration. The entire tuning process is iterated through multiple steps.
  - Benchmark: a user-specified benchmark testing script, which is used to run benchmark jobs. The benchmark testing result reflects the performance of the database system.

X-Tuner Running Modes

- X-Tuner runs in three modes:
  - **recommend**: Logs in to the database using the specified username, obtains the feature information about the running workload, and generates a parameter recommendation report based on the feature information. It identifies inappropriate parameter settings and potential risks in the current database, and then reports behavior and characteristics of current workloads, along with recommended parameter settings. **In this mode, operations are completed in seconds and the database does not need to be restarted. In other modes, the database may need to be restarted repeatedly.**
  - **train**: Modifies parameters and executes the benchmark based on the benchmark information provided by users. The reinforcement learning model is trained through repeated iterations so that the user can load the model in **tune** mode for tuning.
  - **tune**: Uses an optimization algorithm to tune database parameters. Currently, two types of algorithms are supported: deep reinforcement learning and global search algorithm (global optimization algorithm). The deep reinforcement learning mode requires **train** mode to generate a tuned model after training. In contrast, when a global search algorithm is used, no pre-training of the model is required, and tuning can be performed directly through search.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- To use deep reinforcement learning algorithms in **tune** mode, the user must prepare a trained model and the parameters used for training the model must be the same as those in the parameter list (including **max** and **min**) for tuning.

Installing and Running X-Tuner (1)

- There are two ways to run X-Tuner. One is to run the X-Tuner directly from the source code. The other is to install X-Tuner on the system using Python setup tools, and then run the **gs_xtuner** command to call the X-Tuner tool.
- Method 1: Run the X-Tuner directly from the source code.
  - Switch to the **xtuner** source code directory. To run it from the openGauss community, use the following source code path: **openGauss-server/src/gausskernel/dbmind/tools/xtuner**. For an installed database system, the source code path is ***$GAUSSHOME*/bin/dbmind/xtuner**.
  - The user can view the **requirements.txt** file in the current directory. Use the pip package management tool to install the

    ```
    pip install -r requirements.txt
    ```
  - After the installation is successful, add the environment variable **PYTHONPATH**, and then run **main.py**. To view the help information, run the following commands:

    ```
    cd tuner # Switch to the directory where the main.py entry file is located.
    export PYTHONPATH='..' # Add the upper-level directory to the search path.
    python main.py --help # View help information. The methods of using other functions are similar.
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Installing and Running X-Tuner (2)

- Method 2: Install the X-Tuner in the system.
  - The user can use the **setup.py** file to install the X-Tuner to the system and then run the **gs_xtuner** command. Switch to the root directory of **xtuner**. The path is **openGauss-server/src/gausskernel/dbmind/tools/xtuner**. For an installed database system, the source code path is ***$GAUSSHOME*/bin/dbmind/xtuner**.
  - Run the following command to install the tool in the Python environment using Python setup tools:

    ```
    python setup.py install
    ```

  - If the **bin** directory of Python is added to the **PATH** environment variable, the **gs_xtuner** command can be directly called anywhere.
  - To obtain the help information, run the following command:

    ```
    gs_xtuner --help
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

SQLdiag Overview

- SQLdiag is an SQL statement execution time prediction tool. Using a template, it predicts the execution time of SQL statements based on the statement logic similarity and historical execution records without obtaining the SQL statement execution plan.
- Existing prediction technologies are mainly based on evaluations of the execution plans, which are applicable only in OLAP scenarios where the execution plans can be obtained. They are not very useful for quick simple OLTP or HTAP queries. SQLdiag, in contrast, examines the database SQL statement history and analyzes the performance of these historical statements to predict the performance of new, unknown services. The execution duration of SQL statements in the database does not differ greatly in a short period of time. SQLdiag can detect the statement result set similar to the executed SQL statements from historical data and predict the execution duration of SQL statements based on the SQL vectorization technology and template-based method.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

SQLdiag Advantages and Usage Conditions

- The SQLdiag tool has the following advantages:
  - Execution plans of SQL statements do not need to be collected, so there is no impact on database performance.
  - SQLdiag is widely used, unlike many other algorithms with limitations in the industry. For example, some of them are applicable only to OLTP or OLAP scenarios.
  - The tool is robust and easy to understand. The users can train their own prediction models by performing simple operations.
- The prerequisites for using the SQLdiag tool are as follows:
  - The historical logs and the format of the workload to be predicted need to meet certain requirements. The user can use the GUC parameter of the database to enable the collection or use the monitoring tool to collect logs.
  - To ensure accurate prediction, the logs of historical statements provided by users should be as comprehensive and representative as possible.
  - The Python environment should be configured as required.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Robustness refers to the capability of a computer system to process errors during execution and to continue to run properly when exceptions such as input and calculation errors occur.

SQLdiag Usage

- The SQLdiag file structure is as follows:

```
Sqldiag
├── data --Test dataset
├── src --Source code file
├── README.md --Description document
└── main.py --Program entry
```

- To collect SQL statements, use the GUC parameters **log_statement** and **log_statement_stats** to enable log collection. The parameters are as follows:
  - log_statement = all
  - log_statement_stats = on
- Procedure
  - Historical logs are provided for model training.
  - Perform training and prediction.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- After this parameter is enabled, a certain amount of system resources may be occupied but the usage is generally low. In continuous high-concurrency scenarios, this may cause a performance loss less than 5%. If the database concurrency is low, the performance loss can be ignored. After the parameter is enabled, the execution statement and its cost are recorded in the database log file.

SQLdiag Usage Example

- Use the provided training data for training.

```
python main.py -m train -f data/train.csv
```

- Use the provided test data for prediction.

```
python main.py -m predict -f data/predict.csv
```

- Analyze the results.

```
status: prediction status
data:
    time: SQL statement execution time
    point: spatial point coordinate of an SQL statement
    cluster: SQL type ID
    background: template-based model summary
    stmts: SQL statement sample of the corresponding type
    center: central point coordinate of the SQL statement of the corresponding type
    points: spatial point coordinate of the sample SQL statement
    avg_time: average execution time of the corresponding type
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Index-advisor: Index Recommendation

- An index in a database functions similarly to a catalog in a library; it points to the data value stored in a specific column of a table, which accelerates the query process for that table.
- The Index-advisor has three functions, including single-query index recommendation, virtual index recommendation, and workload-level index recommendation.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Single-Query Index Recommendation

- The single-query index recommendation feature allows users to perform operations directly within the database. This function generates index recommendations based on the semantic information of a query statement and the database's statistical data, providing recommended indexes for individual queries entered by the user.
- This function involves the following API.

Function	Parameter	Function
gs_index_advise	SQL statement string	Generates a recommendation index for a single query statement.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This function supports only a single SELECT statement and does not support other types of SQL statements.
- Column-store tables, segment-paged tables, common views, materialized views, global temporary tables, and encrypted databases are not supported.

Single-Query Index Recommendation Example (1)

- Use the **gs_index_advise** function to obtain the recommended index generated for the query. The recommended index result consists of two parts: table name and column name.
- Example 1:

```
postgres=> select * from gs_index_advise('SELECT c_discount from bmsql_customer where c_w_id = 10');
table | column
----------------+----------
bmsql_customer | (c_w_id)
(1 row)
```

  - The preceding information indicates that an index should be created on the **c_w_id** column of the **bmsql_customer** table. Run the following SQL statement to create an index:

```
CREATE INDEX idx on bmsql_customer(c_w_id);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Single-Query Index Recommendation Example (2)

- Example 2:

```
postgres=# select * from gs_index_advise('select name, age, sex from t1 where age >= 18 and age < 35 and sex = ''f'';');
table | column
-------+------------
t1 | (age, sex)
(1 row)
```

  - The preceding statement indicates that a composite index **(age, sex)** needs to be created in the **t1** table. Run the following command to create a composite index:

```
CREATE INDEX idx1 on t1(age, sex);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Virtual Index Recommendation

- The virtual index function allows users to directly perform operations in the database. The virtual index function simulates the creation of a real index, avoiding the time and space overhead required for creating a real index. Based on the virtual index, users can evaluate the impact of the index on the specified query statement cost through the optimizer.
- To enable this feature, set the GUC parameter **enable_hypo_index**. The default value is **off**.
- This function involves the following APIs.

Function	Description	Function Description
hypopg_create_index	Character string of the statement for creating an index	Creates a virtual index.
hypopg_display_index	None	Displays information about all created virtual indexes.
hypopg_drop_index	Index OID	Deletes a specified virtual index.
hypopg_reset_index	None	Clears all virtual indexes.
hypopg_estimate_size	Index OID	Estimates the space required for creating a specified index.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Virtual Index Example (1)

- Call the **hypopg_create_index** function to create a virtual index. For example:

```
postgres=> select * from hypopg_create_index('create index on bmsql_customer(c_w_id)');
indexrelid | indexname
 ------------+---------------------------------------
329726 | <329726>btree_bmsql_customer_c_w_id
(1 row)
```

- Enable the GUC parameter **enable_hypo_index**. This parameter controls whether the database optimizer considers the created virtual index when executing the EXPLAIN statement. By executing EXPLAIN on a specific query statement, the user can evaluate whether the index can improve the execution efficiency of the query statement based on the execution plan provided by the optimizer. For

```
postgres=> set enable_hypo_index = on;
SET
```

- Before enabling the GUC parameter, run EXPLAIN and the query statement.

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                                 QUERY PLAN
-----------------------------------------------------------------------------
Seq Scan on bmsql_customer  (cost=0.00..52963.06 rows=31224 width=4)
Filter: (c_w_id = 10)
(2 rows)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Virtual Index Example (2)

- After enabling the GUC parameter, run EXPLAIN and the query statement.

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                                                                 QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------
[Bypass]
Index Scan using <329726>btree_bmsql_customer_c_w_id on bmsql_customer (cost=0.00..39678.69 rows=31224 width=4)
Index Cond: (c_w_id = 10)
(3 rows)
```

- By comparing the two execution plans, the user can find that the index may reduce the execution cost of the specified query statement. Then, the user can consider creating a real index.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Virtual Index – Feature Constraints

- Running EXPLAIN or ANALYZE does not involve the virtual index function.
- The created virtual index is at the database instance level and can be shared among sessions. After a session is closed, the virtual index still exists. However, the virtual index will be cleared after the database is restarted.
- Currently, this function does not support ordinary views, materialized views, or column-store tables.
- When the virtual index function is enabled and the EXPLAIN statement is executed, the execution plan can be generated after the virtual index is created. In addition, the indexscan and indexonlyscan hints support virtual indexes.
- The settings of the session-level virtual index in each session do not affect each other. After the session is closed, the session-level virtual index is cleared.
- Different from real indexes, operations related to virtual indexes cannot be rolled back.
- Functions related to virtual indexes cannot be remotely called by dblink.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Workload-level Index Recommendation

- For workload-level indexes, the user can run scripts outside the database to use this function. This function uses the workload of multiple DML statements as the input to generate a batch of indexes that can optimize the overall workload execution performance.
- Prerequisites:
  - The database is functioning normally, and the client can be connected properly.
  - The gsql tool has been installed by the current user, and the tool path has been added to the **PATH** environment variable.
  - An environment running Python 3.6 or later versions is available.
- Procedure:
  - Prepare a file that contains multiple DML statements as the input workload. Each statement in the file occupies a line. The user can obtain historical service statements from the offline logs of the database.
  - Run the Python script **index_advisor_workload.py**:

```
python index_advisor_workload.py [p PORT] [d DATABASE] [f FILE] [--h HOST] [-U USERNAME] [-W PASSWORD] [--max_index_num MAX_INDEX_NUM] [--multi_iter_mode]
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The parameters are described as follows.
  - **PORT**: Port of the connected database
  - **DATABASE**: Name of the connected database.
  - **FILE**: File path that contains the workload statements.
  - **HOST** (optional): ID of the host that connects to the database.
  - **USERNAME** (optional): Username for connecting to a database.
  - **PASSWORD** (optional): Password for connecting to a database.
  - **MAX_INDEX_NUM** (optional): Maximum number of recommended indexes.
  - **multi_iter_mode** (optional): Algorithm mode. The user can switch the algorithm mode by setting this parameter.

Workload-level Index Recommendation Example

- Run the **index_advisor_workload.py** script to store multiple DML statements to **tpcc_log.txt**.

```
python index_advisor_workload.py 6001 postgres tpcc_log.txt --max_index_num 10 --multi_iter_mode
```

- The recommendation result is a set of indexes, which are displayed on the screen as multiple CREATE INDEX statements.

```
create index ind0 on bmsql_stock(s_i_id,s_w_id);
create index ind1 on bmsql_customer(c_w_id,c_id,c_d_id);
create index ind2 on bmsql_order_line(ol_w_id,ol_o_id,ol_d_id);
create index ind3 on bmsql_item(i_id);
create index ind4 on bmsql_oorder(o_w_id,o_id,o_d_id);
create index ind5 on bmsql_new_order(no_w_id,no_d_id,no_o_id);
create index ind6 on bmsql_customer(c_w_id,c_d_id,c_last,c_first);
create index ind7 on bmsql_new_order(no_w_id);
create index ind8 on bmsql_oorder(o_w_id,o_c_id,o_d_id);
create index ind9 on bmsql_district(d_w_id);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DeepSQL Overview (1)

- The database DeepSQL feature implements the DB4AI function, that is, the AI algorithm is implemented in the database to better support quick analysis and computing of big data. openGauss provides a complete set of SQL-based machine learning, data mining, and statistics algorithms. Users can directly use SQL statements to perform machine learning.
- DeepSQL can abstract the end-to-end R&D process from data to models. With the bottom-layer engine and automated optimization, technical personnel with basic SQL knowledge can complete most machine learning model training and prediction tasks. The entire analysis and processing run on the database engine. Users can directly analyze and process data in the database without transferring data between the database and other platforms. The need to move data between multiple environments is eliminated.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Benefits: Users can directly use AI algorithms in the database to avoid extra costs caused by massive data migration. In addition, models can be centrally managed by the database, which is easy to use.

DeepSQL Overview (2)

- DeepSQL is an enhancement to openGauss DB4AI. DeepSQL encapsulates common machine learning algorithms into SQL statements and supports more than 60 general algorithms, including regression algorithms (such as linear regression, logistic regression, and random forest), classification algorithms (such as KNN), and clustering algorithms (such as K-means). In addition to common machine learning algorithms, graph-related algorithms are also included, such as algorithms about the shortest path and graph diameter. Also, it supports data processing (such as PCA), sparse vectors, common statistical algorithms (such as covariance and Pearson coefficient calculation), training set and test set segmentation, and cross validation.
- The DeepSQL environment consists of two parts: compiling the database and installing the algorithm library.
- Prerequisites:
  - An environment with Python 2.7.12 or later versions is available.
  - The database supports the PL/Python stored procedure.
  - The user has the administrator permission to install the algorithm library.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Native DB4AI Engine (1)

- The current version of openGauss supports the native DB4AI capability. By introducing native AI operators, openGauss simplifies the operation process and fully utilizes the capabilities of the database optimizer and executor to obtain the high-performance model training capability in the database. With a simpler model training and prediction process and higher performance, developers can focus on model tuning and data analysis in a shorter period of time, avoiding fragmented technology stacks and redundant code implementation.
- Core features:
  - One-stop modeling: The native AI computing engine in the database prevents repeated data transfer between multiple platforms and simplifies the development process.
  - High performance and ease of use: The database optimizer and executor capabilities are used to implement efficient model training and prediction through native AI operators, lowering the learning threshold.
  - Unified technology stack: SQL-like syntax (such as CREATE MODEL and PREDICT BY) is supported, which is close to natural language and reduces the dependency on script languages such as Python.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Technologies
  - AI operator integration: Native AI operators are added to complete AI computing in the database, avoiding performance loss caused by data transfer.
  - Model management: Model training, inference, and management are provided. Scalar and vector converged query is supported, improving the question answering precision of LLMs.
  - Resource pooling architecture: The DataPod resource pooling technology is used to enhance the fault self-healing capability and ensure the high availability of AI tasks.

Native DB4AI Engine (2)

- Introduction
  - DB4AI uses database capabilities to drive AI tasks and implement data storage and technology stack isomorphism. By integrating AI algorithms into the database, openGauss supports the native AI computing engine, model management, AI operators, and native AI execution planning, providing users with inclusive AI technologies. Different from the traditional AI modeling process, DB4AI one-stop modeling eliminates repeated data flowing among different platforms, simplifies the development process, and plans the optimal execution path through the database, so that developers can focus on the tuning of specific services and models. It outcompetes similar products in ease-of-use and performance.
- Benefits
  - With this feature, the user does not need to manually compile AI model code. Instead, the user can use out-of-the-box SQL statements to train and predict machine learning models, reducing the learning and usage costs.
  - Extra overhead that is caused by fragmented data storage and repeated data transfer can be avoided.
  - A higher execution efficiency can be achieved. With this feature, the AI model training efficiency is enhanced. Compared with manual model training, the performance is improved by several times.
  - Stricter security protection prevents data leakage during AI model training.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Multiple-choice question) Which of the following are the openGauss deployment modes?
   A. Single-node deployment
   B. Deployment with one primary and one standby
   C. Deployment with one primary and multiple standbys
   D. Deployment with dual primary nodes

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 1.ABC

Summary

- This chapter describes the physical and logical architectures of the openGauss database, as well as its typical networking and deployment schemes. In addition, this chapter describes the key features of openGauss database in terms of high performance, high availability, high security, easy maintenance, and AI capabilities.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

More Information

- openGauss documentation:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/ReleaseNotes/Releasenotes.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Abbreviations

- MOT: memory-optimized table
- WAL: write-ahead log
- OM: operation manager
- RBO: rule-based optimizer
- CBO: cost-based optimizer
- MCV: most_common_vals
- NUMA: non-uniform memory access
- SMP: symmetric multi-processing
- MPP: massively parallel processing
- WDR: workload diagnosis report

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.



04 Database and Object Management



HUAWEI

Foreword

- openGauss is an open-source relational database management system released under the Mulan Permissive Software License v2. The openGauss kernel is derived from PostgreSQL. It deeply integrates Huawei's years of experience in the database field and continuously builds competitive features based on enterprise-level scenario requirements.
- This chapter describes the openGauss logical structure, storage engine selection, tablespace creation and management, user and role management, system catalogs and views, data import and export, and high-risk operations.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- On completion of this course, you will be able to:
  - Understand the openGauss logical structure.
  - Master how to select different storage engines.
  - Master how to create and manage tablespaces.
  - Master how to manage users and roles.
  - Understand common system catalogs and views.
  - Master how to import and export data.

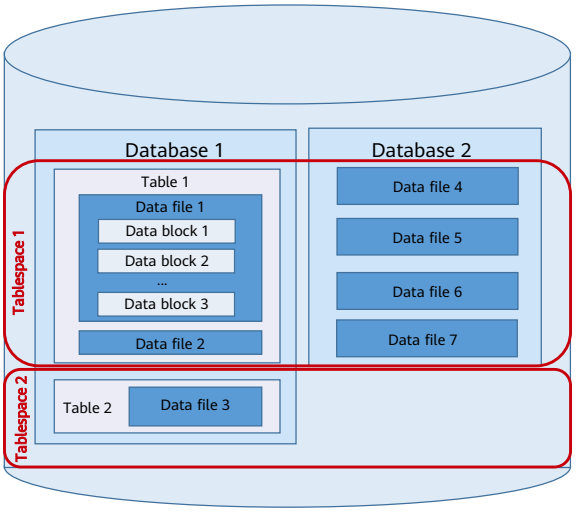
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **Logical Structure**
2. Storage Engine
3. Tablespace
4. User Roles
5. System Catalogs and System Views
6. Data Import and Export

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

openGauss Logical Architecture



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Related Concepts (1)

- A tablespace is a directory that can contain multiple physical files of databases. Each tablespace can correspond to multiple databases.
- Databases manage various data objects and operate independently of one another. Objects managed by the database can be distributed across multiple tablespaces.
- Datafile segment: Each datafile segment stores the data for only one table. A table containing more than 1 GB of data is stored in multiple datafile segments.
- Each table belongs to only one database and one tablespace. The datafile segments of each table must be in the same tablespace.
- A block is the basic unit for database management, with a default size of 8 KB.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Row: A row (tuple or record) consists of a set of related data, such as a piece of data subscribed to by a user.
- Column: Each column is referred to as a field. The value in each field represents a data type. For example, a table has three columns: **Name**, **City**, and **Country**, and each row contains a corresponding name, city, and country.

Related Concepts (2)

- Schema
  - A collection of database objects, including logical structures such as tables, views, sequences, stored procedures, synonyms, indexes, and database links.
- Transaction
  - A logical unit of execution in a database management system (DBMS), consisting of a finite sequence of database operations. In a relational database, a transaction can consist of a single SQL statement, multiple SQL statements, or an entire program. In addition, a transaction is the basic unit for recovery and concurrency control. It must have the ACID properties.
    - Atomicity: All operations within a transaction must either complete successfully or not at all, as a transaction is an indivisible unit of work.
    - Consistency: A transaction must transition the database from one consistent state to another. Consistency is closely related to atomicity.
    - Isolation: The transaction's execution must not be interfered with by other transactions. It means that operations and data used in a transaction are isolated from those in other concurrent transactions. Concurrent transactions are independent of each other.
    - Durability or Permanence: Once a transaction is committed, the changes to the database are permanent. Subsequent operations or failures should not affect them.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Creating a Database

- After installing the database, a default database named **postgres** is created. You can create your own database.
  - Create a database.

    ```
    CREATE DATABASE database_name;
    ```

  - View a database.
    - Run **\l** to view the existing databases.
    - Run **\c +** *Database name* to access the existing database.
  - Modify a database.

    ```
    ALTER DATABASE database_name RENAME TO new_name;
    ```

  - Delete a database.

    ```
    DROP DATABASE new_name ;
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Creating a Table

- Create a table.
  - Create an empty table in the current database. The table is owned by the user who executes the command. The same table can be stored in different databases. You can use the **CREATE TABLE** statement to create a table.
- Example
  - Run the following commands to create a table named **customer_t1**. This table has four columns: **c_customer_sk**, **c_customer_id**, **c_first_name**, and **c_last_name**. Their respective data types are integer, char(5), char(6), and char(8).

```
openGauss=# CREATE TABLE customer_t1
(
    c_customer_sk             integer,
    c_customer_id             char(5),
    c_first_name              char(6),
    c_last_name               char(8),
    Amount                    integer
);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

INSERT INTO Statement

- INSERT INTO statement
  - A new table contains no data. You need to insert data to the table before using it. You can execute the **INSERT INTO** statement to insert data into the table. This statement can be used to insert a row of data or multiple rows of data at the same time.
- Syntax

```
INSERT INTO table_name [ ( column_name [, ...] ) ]
    { DEFAULT VALUES
    | VALUES {( { expression | DEFAULT } [, ...] ) }[, ...] };
```

  - **table_name**: Specifies the name of the target table for data insertion. Value range: names of existing tables.
  - **column_name**: Specifies the name of the column in the target table. Value range: existing column names.
  - **expression**: Specifies the expression or value to be assigned to the corresponding column.
  - **DEFAULT**: Specifies the default value for a column. The value is **NULL** if no default value has been specified for it.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

UPDATE Statement

- UPDATE statement
  - Updating refers to the process of modifying stored data. You can target a single row, a selection, or the entire table. The **UPDATE** statement allows you to change specific fields while leaving others untouched. By using the **SET** clause, you specify which columns to update for rows that meet the **WHERE** clause conditions. Any columns not listed in the statement will keep their current values.
- Syntax

```
UPDATE table_name
SET column_name = { expression | DEFAULT }
[WHERE condition ];
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DELETE Statement

- **DELETE** statement
  - The **DELETE** statement can be used to remove rows that meet the conditions of the **WHERE** clause from a specified table. If the **WHERE** clause is absent, all rows in the table will be deleted, leaving only the table structure intact.
- Syntax

```
DELETE FROM table_name
    [WHERE condition];
```

  - **table_name**: Specifies the name (optionally schema-qualified) of the target table. Value range: names of existing tables.
  - **condition**: Specifies an expression that returns a Boolean value, determining which rows will be deleted.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Viewing Objects

- gsql provides several advanced features to facilitate user operations. The common usage is as follows:
  - Query the command's help information.

    ```
    \h [NAME]
    ```

  - Switch databases.

    ```
    \c dbname
    ```

  - Query all databases.

    ```
    \l
    ```

  - Query all tables in the current database.

    ```
    \dt
    ```

  - View the table structure.

    ```
    \d tablename
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

SELECT Statement

- **SELECT** statement
  - The **SELECT** statement can be executed to retrieve data from a table or view. The result is stored in a result table, which is called a result set.
  - It serves as an overlaid filter for a database table, the **SELECT** statement filters required data from the table using SQL keywords.
- Syntax

```
SELECT
{ * | [column, ...] }
[ FROM from_item [, ...] ];
```

  - The **SELECT** list specifies the column names from the queried table. It can include specific columns or all columns, with the wildcard (*) indicating all columns.
  - The **FROM** clause specifies one or more source tables for the **SELECT** statement.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Alias

- Alias
  - SQL can be used to rename a table or a column. The new name serves as an alias for the table or column. Aliases are used to enhance the readability of table names or column names. In SQL, **AS** is used to create an alias.
- Syntax
  - Column alias syntax

```
SELECT
{ * | [column [ AS ] output_name, ...] }
[ FROM from_item [, ...] ]
[ WHERE condition ];
```

  - Table alias syntax

```
SELECT column1, column2....
FROM table_name AS output_name
WHERE [condition];
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Index

- An index is a pointer to data in a table. The index in a database is very similar to the index directory in a book.
- Indexes are mainly used to improve database query performance, though their improper use can lead to performance degradation. It is advised to create indexes on:
  - Frequently queried columns
  - Join conditions: For queries involving joined columns, it is advised to create a composite index on those columns. For example, for **select * from t1 join t2 on t1.a=t2.a and t1.b=t2.b**, you can create a composite index on columns **a** and **b** in table **t1**.
  - Columns with filter criteria, especially scope criteria, in a **WHERE** clause
  - Columns frequently used after **ORDER BY**, **GROUP BY**, and **DISTINCT**

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Index - Syntax

- A single-column index is an index created based on a single column of a table.

```
CREATE INDEX [ [schema_name.]index_name ] ON table_name (column_name);
```

- A composite index is an index created based on multiple columns of a table.

```
CREATE INDEX [ [schema_name.]index_name ] ON table_name (column1_name,column2_name,...);
```

- A unique index ensures that duplicate values cannot be inserted into the column.

```
CREATE UNIQUE INDEX [ [schema_name.]index_name ] ON table_name (column_name);
```

- A local index is created on a subset of a table, defined by a condition expression.

```
CREATE INDEX [ [schema_name.]index_name ] ON table_name (expression);
```

- A partial index includes entries for only a portion of a table, typically the part that is more beneficial for indexing compared to the rest of the table.

```
CREATE INDEX [ [schema_name.]index_name ] ON table_name (column_name) [ WHERE predicate ]
```

- Delete an index.

```
DROP INDEX index_name;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

View

- The view is a virtual table rather than a base table. Only the view definition is stored in the database, not the view data itself. The actual data is stored in the base table. If data in the base table changes, the data queried from the view changes accordingly. In this sense, a view is like a window through which users can know their interested data and data changes in the database.
- Syntax
  - Create a view.

    ```
    CREATE [ TEMP | TEMPORARY ] VIEW view_name [ ( column_name [, ...] ) ]
        AS query;
    ```

  - Delete a view.

    ```
    DROP VIEW  view_name ;
    ```

  - **TEMP | TEMPORARY**: Create a temporary view.
  - **query**: Specifies a **SELECT** or **VALUES** statement to provide the columns and rows for the view.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Schema

- A SCHEMA is a logical container that allows multiple users to share a single database without mutual interference. It organizes database objects into manageable logical groups and enables the seamless addition of third-party applications without causing naming conflicts.
- Each database contains one or more schemas. Each schema includes tables and other object types. When a database is initially created, it has a public schema by default, and all users have the usage permission on the schema. Only the system administrator and initial users can create common functions, aggregate functions, stored procedures, and synonym objects in the public schema. Only the initial users can create operators in the public schema. Other users cannot create the preceding five types of objects even if they are granted the create permission. You can organize database objects into schemas. A schema is similar to an OS directory but cannot be nested.
- The same database object name can be used in different schemas of the same database without causing conflicts. For example, both **a_schema** and **b_schema** may include a table named **mytable**. Users with the necessary permissions can access objects across multiple schemas within the same database.
- When you execute the **CREATE USER** statement to create a user, the system creates a schema with the same name as the user in the database where the statement is executed.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Schema - Syntax

- Create a schema.

```
CREATE SCHEMA schema_name
    [ AUTHORIZATION user_name ] ;
```

- Modify a schema.
  - Rename a schema.

```
ALTER SCHEMA schema_name
    RENAME TO new_name;
```

  - Change the owner of a schema.

```
ALTER SCHEMA schema_name
    OWNER TO new_owner;
```

- Delete a schema and its associated objects.

```
DROP SCHEMA  schema_name;
```

- Schemas beginning with **pg_temp** or **pg_toast_temp** are for internal use. Do not drop them. Otherwise, unexpected consequences may be incurred.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Transactions

- A transaction is a user-defined sequence of database operations, which form an integral unit of work. In openGauss, you can start, set, commit, and roll back transactions. openGauss supports the **READ COMMITTED** and **REPEATABLE READ** transaction isolation levels.
- **READ COMMITTED** (default): At this isolation level, a transaction can access only committed data.
  - In effect, a **SELECT** query sees a snapshot of the database as of the moment the query begins. However, it can see the effects of previous updates executed within its own transaction, even if those updates have not yet been committed. Note that two consecutive **SELECT** statements within the same transaction might access different database snapshots. This can occur because other transactions may commit changes while the first **SELECT** statement is being executed.
  - At the **READ COMMITTED** level, each statement starts with a new snapshot containing all transactions committed by the execution time. Consequently, within a transaction, a statement can access the results of other committed transactions. This level ensures that one statement can access consistent views in a database.
  - This level of transaction isolation satisfies the needs of numerous applications and offers both speed and ease of use. However, applications performing complicated queries and updates may require data that is more consistent than this level can provide.
- **REPEATABLE READ**: At this isolation level, a transaction can read only the data that was committed before it began. It cannot read uncommitted data or changes made by other concurrent transactions. **READ COMMITTED** differs from this level because a transaction reads the snapshot taken at the start of the transaction, rather than at the beginning of each individual query within the transaction. Consequently, the **SELECT** statements within a transaction always retrieve the same data and do not see any changes committed by other transactions after the initial snapshot was taken. Applications at this level must be capable of retrying transactions due to potential serialization failures.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Logical Structure
2. **Storage Engine**
3. Tablespace
4. User Roles
5. System Catalogs and System Views
6. Data Import and Export

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Storage Models

- openGauss supports the hybrid row-column store. Each storage mode applies to specific scenarios. Select an appropriate mode when creating a table. Generally, openGauss is used for OLTP databases. By default, the row store is used. The column store is used only in OLAP scenarios where complex queries in large data volume are performed.
- openGauss uses a Memory-Optimized Table (MOT) to provide a higher-performance load for transactional work. MOT has significant advantages in terms of high performance (query and transaction latency), high scalability (throughput and concurrency), and even cost (high resource utilization) in some cases.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Row-Store Table

- Tables are stored to disk partitions by row and are called row-store tables. The following shows a row-store table on a hard disk, with content being stored by row.

Row ID	Time/ Date	Matenal	Customer Name	Quantity
1	845	2	3	1
2	851	5	2	2
3	872	4	4	1
4	878	1	5	2
5	888	2	3	3
6	895	3	4	1
7	901	4	1	1

1	845	2	3	1	2	851	5	2	2	3	872	4	4	1	4	878	1	5	2

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Row-Store Table Example

- Row-store tables are created by default. Data is continuously stored by row. Therefore, this storage model applies to scenarios where data needs to be updated frequently.
- Example:

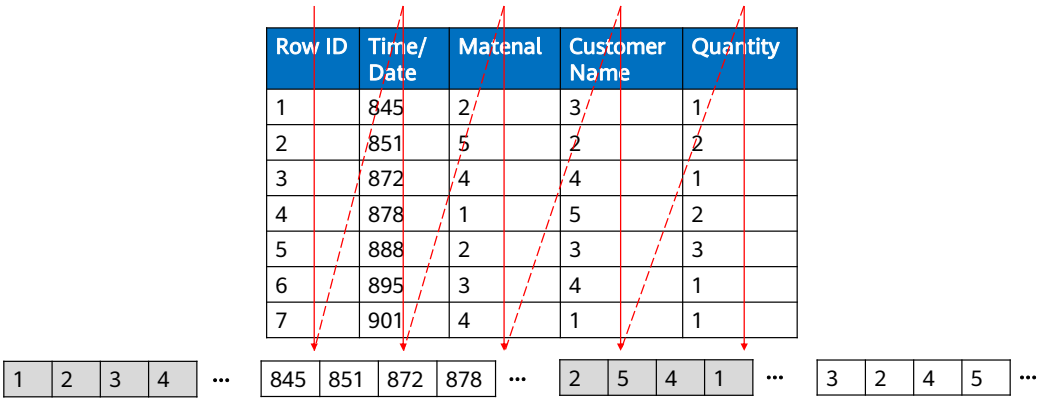
```
postgres=# CREATE TABLE customer_t1
(
  state_ID    CHAR(2),
  state_NAME VARCHAR2(40),
  area_ID     NUMBER
);

-- Delete the table.
postgres=# DROP TABLE customer_t1;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Column-Store Table

- Tables are stored to disk partitions by column. These tables are column-store tables. The following shows a column-store table on a hard disk, with the content being stored by column.



Row ID	Time/ Date	Matenal	Customer Name	Quantity
1	845	2	3	1
2	851	5	2	2
3	872	4	4	1
4	878	1	5	2
5	888	2	3	3
6	895	3	4	1
7	901	4	1	1

1	2	3	4	...	845	851	872	878	...	2	5	4	1	...	3	2	4	5	...

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Column-Store Table Example

- Column-store tables need to be declared upon creation. In a column-store table, data is stored by column, with data in each column being stored continuously. The I/O of a single-column query is small and occupies less storage space than that of a row-store table. This storage model applies to scenarios where data is inserted in batches, is less frequently updated, and is queried for statistical analysis. However, column-store tables are not suitable for point queries.
- Example:

```
postgres=# CREATE TABLE customer_t2
(
  state_ID   CHAR(2),
  state_NAME VARCHAR2(40),
  area_ID    NUMBER
)
WITH (ORIENTATION = COLUMN);

-- Delete the table.
postgres=# DROP TABLE customer_t2;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Comparison Between Row-Store and Column-Store Tables

Storage Model	Advantage	Disadvantage	Scenario
Row store	Data is stored together. INSERT and UPDATE operations are efficient.	All the columns of a record are read after the **SELECT** statement is executed even if only certain columns are required.	Point queries (simple index-based queries that only return a few records). Scenarios requiring frequent addition, deletion, and modification.
Column store	Only the columns involved in a query are read. Projections are efficient. Any column can serve as an index.	The selected columns need to be reconstructed after the **SELECT** statement is executed. INSERT and UPDATE are complex.	Statistical analysis queries (requiring a large number of association and grouping operations). Ad hoc queries (using uncertain query conditions and unable to utilize indexes to scan row-store tables).

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Storage Model Selection

- Update frequency
  - If data is frequently updated, use a row-store table.
- Data insert frequency
  - If data is frequently inserted and the data volume is small each time, use a row-store table. If a large amount of data is inserted at a time, use a column-store table.
- Number of columns
  - If many columns are required, use a column-store table.
- Number of columns to be queried
  - If few columns in a table are required (less than 50% of the total number of columns) in each query, use a column-store table.
- Compression ratio
  - A column-store table has a higher compression rate than a row-store table, but a high compression rate consumes more CPU resources.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

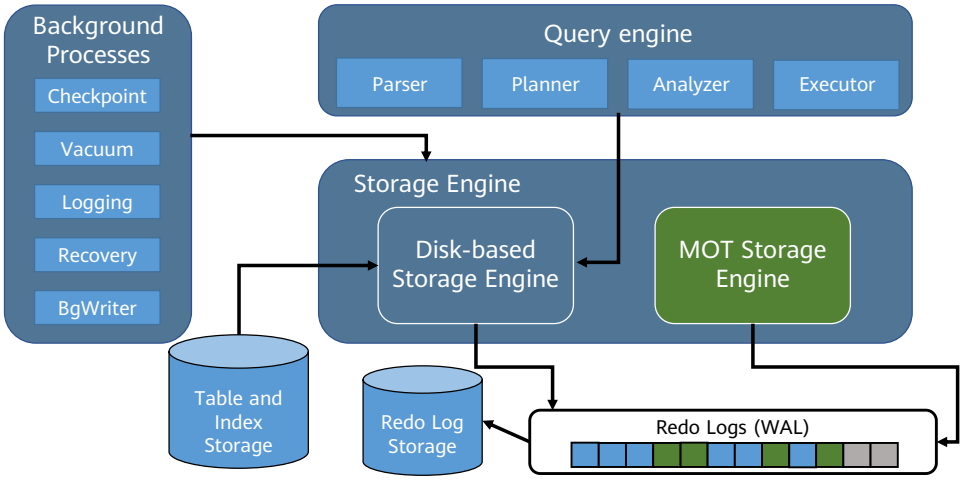
MOT

- openGauss uses the MOT storage engine, which is a transactional row storage and is optimized for multi-core and large-memory servers. MOT is the most advanced production-level feature of openGauss databases. It provides higher performance for transactional workloads. Enterprises can use MOT in mission-critical and performance-sensitive online transaction processing (OLTP) to achieve high performance, high throughput, predictable low latency, and high utilization of multi-core servers. MOT and disk-based common tables are created side by side. The effective design of MOT implements almost complete SQL coverage and supports a complete set of database functions, such as stored procedures and user-defined functions.
- MOT fully supports ACID features and includes strict persistence and high availability support.
- MOT provides faster data access and more efficient transaction execution through data and indexes completely stored in memory, non-uniform memory access-aware (NUMA-aware) design, algorithms that eliminate locks and lock contention, and query native compilation.
- The lock-free design and highly optimized implementation of MOT enables it to achieve excellent near-linear throughput expansion on multi-core servers.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- MOT is particularly suited to running on modern servers with multi-channel and multi-core processors, such as Huawei TaiShan servers based on ARM/ Kunpeng processors and Dell or similar x86 servers.

openGauss Structure



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- The openGauss database MOT storage engine component (in green) manages MOTs and transactions. The blue part indicates the existing openGauss components.

MOT Features and Benefits

- MOT has significant advantages in terms of high performance (query and transaction latency), high scalability (throughput and concurrency), and even cost (high resource utilization) in some cases.
  - Low latency: provides fast query and transaction response time.
  - High throughput: supports peak and continuous high user concurrency.
  - High resource utilization: fully utilizes hardware.
- Applications that use MOT can reach 2.5 to 4 times the throughput compared to applications that do not use MOT. For example, after the TPC-C benchmark test is performed on the Huawei TaiShan servers based on ARM/Kunpeng processors, MOT provides 2.5x throughput gains on 2-socket servers and 4.8 million tpmC on 4-socket 256-core Arm servers.
- In the TPC-C benchmark test, you can observe that MOT provides lower latency and reduces the transaction speed by 3 to 5.5 times.
- The high load and high contention situation is a recognized problem for all industry-leading databases, and MOT can make extremely high use of server resources in this situation. After MOT is used, the resource utilization of 4-socket servers reaches 99%.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

MOT Usage

- The memory configuration must meet the following requirements: (max_mot_global_memory + max_mot_local_memory) + 2 GB < max_process_memory
  - Global memory is long-term memory that is shared by all cores and is used primarily to store all table data and indexes.
  - Local memory is short-term memory that is used primarily by sessions for handling transactions and store data changes in primate to transaction memory until the commit phase.
- To enable a specific user to create and access MOT (DDL, DML, SELECT), execute the following statement only once:

```
GRANT USAGE ON FOREIGN SERVER mot_server TO <user>;
```

- Create an MOT.

```
create FOREIGN table test(x int) [server mot_server];
```

- Delete an MOT.

```
drop FOREIGN table test;
```

- Create an index for an MOT.

```
create index  text_index1 on test(x) ;
```

- Convert a disk table to MOT: Use the gs_dump tool to dump the disk table data to the physical file of the disk, and then use the gs_restore tool to load or restore the data from the disk file to the database table.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- MOT is integrated with the openGauss database through the foreign data wrapper (FDW) mechanism. Therefore, user permissions need to be authorized.
- The example creates a memory table named **test** which has an integer column named **x**.
- When creating or deleting an index, you do not need to specify the FOREIGN keyword before the MOT name because it is used only in the commands for creating and deleting tables.

Contents

1. Logical Structure
2. Storage Engine
3. **Tablespace**
4. User Roles
5. System Catalogs and System Views
6. Data Import and Export

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tablespace

- The administrator can use tablespaces to control the layout of disks where a database is installed. The following are the advantages:
  - If the disk partition or volume allocated to the initialized database is full and the space cannot be logically increased, you can create and use tablespaces in other partitions until the space is reconfigured.
  - Tablespaces allow the administrator to distribute data based on the schema of database objects, improving system performance.
    - A frequently used index can be placed in a disk with stable performance and high computing speed.
    - A table that stores archived data and is rarely used or has low performance requirements can be placed in a disk with a slow computing speed.
  - You can use tablespaces to control the disk space occupied by data in a database. If the usage of a disk where a tablespace resides reaches 90%, the database switches to read-only mode. It switches back to read/write mode when the disk usage becomes less than 90%.
  - A tablespace corresponds to a file system directory, and you must have the read and write permissions on an empty directory.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tablespace Example (1)

- Create a tablespace.

```
postgres=# CREATE TABLESPACE fastspace RELATIVE LOCATION 'tablespace/tablespace_1';
```

  - **fastspace** indicates the created tablespace.
  - **/tablespace/tablespace_1** is the data directory relative to the database node. It is an empty directory in which you have the read and write permissions.

- A database system administrator can run the following command to grant the permission of accessing the **fastspace** tablespace to user **jack**:

```
postgres=# GRANT CREATE ON TABLESPACE fastspace TO jack;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tablespace Example (2)

- Create an object in a tablespace.
  - If you have the CREATE permission on the tablespace, you can create database objects in the tablespace.
    - Method 1: Create a table in the specified tablespace.

```
postgres=# CREATE TABLE foo(i int) TABLESPACE fastspace;
```

    - Method 2: Set the default tablespace and then create a table.

```
postgres=# SET default_tablespace = 'fastspace';
postgres=# CREATE TABLE foo2(i int);
```

- Check the tablespace.
  - Method 1: Check the **pg_tablespace** system catalog.
  - Method 2: Run the gsql meta-command **\db** to check.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Tablespace Example (3)

- Check the tablespace usage.
  - Query the current usage of the tablespace.

```
postgres=#  SELECT PG_TABLESPACE_SIZE('example');
```

  - Calculate the tablespace usage using the following formula: Tablespace usage = PG_TABLESPACE_SIZE/Size of the disk where the tablespace directory is located.
- Modify the tablespace.
  - Run the following command to rename tablespace **fastspace** to **fspace**:

```
postgres=# ALTER TABLESPACE fastspace RENAME TO fspace;
```

- Delete the tablespace.

```
postgres=# DROP TABLESPACE fspace;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Only the tablespace owner or system administrator can delete a tablespace.

Contents

1. Logical Structure
2. Storage Engine
3. Tablespace
4. **User Roles**
5. System Catalogs and System Views
6. Data Import and Export

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

User

- User
  - A user is an individual who uses the database service.
  - A user has responsibilities and is often affiliated to an organization or department.
- Database users can:
  - Use a tool to connect to the database.
  - Access database objects.
  - Execute SQL statements.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Role

- Role
  - A role is a set of users. Roles with different permissions are divided based on the responsibilities in the database system. A role is a carrier of permission sets.
  - A role represents the behavioral constraint of a database user or a group of data users.
- openGauss provides an implicitly defined group **PUBLIC** that contains all roles. By default, all new users and roles have the permissions of **PUBLIC**.
- To revoke permissions of **PUBLIC** from a user or role, or re-grant these permissions to them, add the **PUBLIC** keyword in the **REVOKE** or **GRANT** statement.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Users and Roles

- A user is an entity, and a role is a behavior.
- A user is assigned one or more roles.
- A role is a set of permissions. It should not have permission to log in to the database and execute SQL statements.
- User permission management is simplified to role permission management.
- In openGauss, users and roles use the same operation and maintenance methods.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

User and Role Operations (1)

- For creation operations, view the **CREATE USER** syntax. (**CREATE ROLE** is the same as **CREATE USER**.)

```
postgres=# \help create user
Command:     CREATE USER
Description: define a new database role
Syntax:
CREATE USER user_name [ [ WITH ] option [ ... ] ] [ ENCRYPTED | UNENCRYPTED ] { PASSWORD | IDENTIFIED
BY } { 'password' | DISABLE };

where option can be:
{SYSADMIN | NOSYSADMIN}
    | {AUDITADMIN | NOAUDITADMIN} | {CREATEDB | NOCREATEDB} | {USEFT | NOUSEFT}
    | {CREATEROLE | NOCREATEROLE} | {INHERIT | NOINHERIT} | {LOGIN | NOLOGIN}
    | {REPLICATION | NOREPLICATION} | {INDEPENDENT | NOINDEPENDENT} | {VCADMIN | NOVCADMIN}
    | CONNECTION LIMIT connlimit | VALID BEGIN 'timestamp' | VALID UNTIL 'timestamp'
    | RESOURCE POOL 'respool' | USER GROUP 'groupuser' | PERM SPACE 'spacelimit'
    | NODE GROUP logic_cluster_name | IN ROLE role_name [, ...] | IN GROUP role_name [, ...]
    | ROLE role_name [, ...] | ADMIN role_name [, ...] | USER role_name [, ...]
    | SYSID uid | DEFAULT TABLESPACE tablespace_name | PROFILE DEFAULT
    | PROFILE profile_name | PGUSER
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

User and Role Operations (2)

- For modification operations, view the **ALTER USER** syntax.

```
postgres=# \help alter user
Command:     ALTER USER
Description: change a database role
Syntax:
ALTER USER user_name [ [ WITH ] option [ ... ] ];
ALTER USER user_name
    RENAME TO new_name;
ALTER USER user_name [ IN DATABASE database_name ]
    SET configuration_parameter {{ TO | = } { value | DEFAULT }|FROM CURRENT};
ALTER USER user_name
    [ IN DATABASE database_name ] RESET {configuration_parameter|ALL};

where option can be:
{CREATEDB | NOCREATEDB}
    | {CREATEROLE | NOCREATEROLE} | {INHERIT | NOINHERIT} | {AUDITADMIN | NOAUDITADMIN}
    | {SYSADMIN | NOSYSADMIN} | {USEFT | NOUSEFT} | {LOGIN | NOLOGIN} | {REPLICATION | NOREPLICATION}
    | {INDEPENDENT | NOINDEPENDENT} | {VCADMIN | NOVCADMIN} | CONNECTION LIMIT connlimit
    | [ ENCRYPTED | UNENCRYPTED ] PASSWORD { 'password' | DISABLE }
    | [ ENCRYPTED | UNENCRYPTED ] IDENTIFIED BY { 'password' [ REPLACE 'old_password' ] | DISABLE }
    | VALID BEGIN 'timestamp' | VALID UNTIL 'timestamp' | RESOURCE POOL 'respool'
    | USER GROUP 'groupuser'  | PERM SPACE 'spacelimit' | NODE GROUP logic_cluster_name
    | ACCOUNT { LOCK | UNLOCK } | PGUSER
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

User and Role Operations (3)

- Query
  - The openGauss user and role attributes are maintained in the **pg_authid** system catalog. You can also query the user and role information in the **pg_user** and **pg_roles** views.
  - Run the **\d** command to view the **pg_authid** details.
  - Run the **select** command to query information in **pg_authid**.

```
postgres=# \d pg_authid
postgres=# select * from pg_authid;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This slide describes the role definition.

User and Role Operations (4)

- Delete
  - View the **DROP USER** syntax.
  - If a user has database objects, **RESTRICT** is used by default and the user cannot be deleted. In this case, the **CASCADE** option is required.

```
postgres=# \help drop user
Command:     DROP USER
Description: remove a database role
Syntax:
DROP USER [ IF EXISTS ] user_name [, ...] [ CASCADE | RESTRICT ];

postgres=# \help drop role
Command:     DROP ROLE
Description: remove a database role
Syntax:
DROP ROLE [ IF EXISTS ] role_name [, ...];
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Set User Account Validity Period

- When creating a user, you need to specify the validity period of the user, including the start time and end time.
- To enable a user not within the validity period to use an account, set a new validity period.
  - Run the following command to create a user and specify the start time and end time of the account's validity period:

    ```
    CREATE USER joe WITH PASSWORD 'Bigdata@123' VALID BEGIN '2015-10-10 08:00:00' VALID UNTIL '2016-10-10 08:00:00';
    ```

  - If the user is not within the specified validity period, run the following command to set the start time and end time of a new validity period:

    ```
    ALTER USER joe WITH VALID BEGIN '2016-11-10 08:00:00' VALID UNTIL '2017-11-10 08:00:00';
    ```

  - Note: If **VALID BEGIN** is not specified in the **CREATE ROLE** or **ALTER ROLE** statement, the start time of the validity period is not limited. If **VALID UNTIL** is not specified, the end time of the validity period is not limited. If both of the parameters are not specified, the user is always valid.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Manually Locking and Unlocking Accounts

- Run the following commands to manually lock and unlock the **user_read** user.
- To manually lock the account:

```
ALTER USER user_read ACCOUNT LOCK;
```

  - If the following information is displayed, the account has been successfully locked:

```
ALTER ROLE
```

- To manually unlock the account:

```
ALTER USER user_read ACCOUNT UNLOCK;
```

  - If the following information is displayed, the account has been successfully unlocked:

```
ALTER ROLE
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Private User

- Background
  - If multiple service departments use different database user accounts to perform service operations and a database maintenance department at the same level uses administrator user accounts to perform maintenance operations, service departments require that database administrators, without specific authorization, can manage (**DROP**, **ALTER**, and **TRUNCATE**) but cannot access (**INSERT**, **DELETE**, **UPDATE**, **SELECT**, and **COPY**) the data. That is, the control permissions of database administrators for tables need to be isolated from their access permissions to improve the data security for normal users.
  - In separation-of-duties mode, a database administrator does not have permission to access the tables in schemas of other users. In this case, database administrators have neither management permissions nor access permissions, which does not meet the requirements of the service departments mentioned above. Therefore, openGauss makes use of private users to solve the problem. That is, private users with the **INDEPENDENT** attribute in non-separation-of-duties mode would have to be created.
- Definition:

```
CREATE USER user_independent WITH INDEPENDENT IDENTIFIED BY "1234@abc";
```

  - Database administrators can manage (**DROP**, **ALTER**, and **TRUNCATE**) objects of private users but cannot access (**INSERT**, **DELETE**, **SELECT**, **UPDATE**, **COPY**, **GRANT**, **REVOKE**, and **ALTER OWNER**) the objects without authorization.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Separation of control and access permissions

Contents

1. Logical Structure
2. Storage Engine
3. Tablespace
4. User Roles
5. **System Catalogs and System Views**
6. Data Import and Export

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Overview of System Catalogs and System Views

- System catalogs store the structured metadata of openGauss. They are the source of information used by openGauss to control system running and are a core component of the database system.
- System views provide ways to query the system catalogs and internal database status.
- System catalogs and system views are visible to either system administrators or all users. Some system catalogs and views have marked the need of administrator permissions, so they are accessible only to administrators.
- You can delete and re-create system catalogs, add columns to them, and insert and update values in them, but doing so may make system information inconsistent and cause system faults. Generally, users should avoid modifying system catalogs or system views, or renaming their schemas. They are automatically maintained by the system.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Do not add, delete, or modify system catalogs because doing so will result in exceptions or even openGauss unavailability.

Common System Catalogs (1)

System Catalog Name	Description
GS_WLM_USER_RESOURCE_HISTORY	Stores information about resources used by users and is valid only on CNs. Each record in this system catalog indicates resource usage of a user at a point in time, including the memory, number of CPU cores, storage space, temporary space, operator flushing space, logical I/O traffic, number of logical I/O operations, and logical I/O rate. The memory, CPU, and I/O monitoring items record only resource usage of complex jobs. For I/O monitoring items, this parameter is valid only when **enable_logical_io_statistics** is set to **on**. The function of saving user monitoring data is enabled only when **enable_user_metric_persistent** is set to **on**.
PG_AM	Stores information about index access methods. There is one row for each index access method supported by the system.
PG_ATTRIBUTE	Stores information about table columns.
PG_AUTHID	Stores information about database authorization identifiers (roles). The concept of users is contained in that of roles. A user is actually a role whose **rolcanlogin** has been set. Any role, whether its **rolcanlogin** is set or not, can use other roles as members. For a cluster, only one **pg_authid** exists which is not available for every database. This system catalog is accessible only to system administrators.
PG_CLASS	Stores information about database objects and their relationships.
PG_CONSTRAINT	Stores check, primary key, unique, and foreign key constraints on tables.
PG_DEPEND	Records dependencies among database objects. This information allows **DROP** commands to find which other objects must be dropped by **DROP CASCADE** or prevent dropping in the **DROP RESTRICT** case.
PG_DATABASE	Stores information about the available databases.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common System Catalogs (2)

System Catalog Name	Description
PG_FOREIGN_TABLE	Stores auxiliary information about foreign tables.
PG_INDEX	Stores part of the information about indexes. The rest is mostly stored in **PG_CLASS**.
PG_JOB	Stores detailed information about jobs created by users. Dedicated threads poll the **PG_JOB** table and trigger jobs based on scheduled job execution time, and update job status in the **PG_JOB** table. This system catalog belongs to the Shared Relation category. All job records are visible to all databases.
PG_LARGEOBJECT	Stores data making up large objects. A large object is identified by an OID assigned when it is created. Each large object is broken into segments or "pages" small enough to be conveniently stored as rows in **pg_largeobject**. The amount of data per page is defined as **LOBLKSIZE**. This system catalog is accessible only to system administrators.
PG_LARGEOBJECT_METADATA	Stores metadata associated with large objects. The actual large object data is stored in **PG_LARGEOBJECT**.
PG_NAMESPACE	Stores namespaces, that is, schema-related information.
PG_PARTITION	Stores all partitioned tables, table partitions, toast tables on table partitions, and index partitions in the database. Partitioned index information is not stored in the system catalog **PG_PARTITION**.
PG_PROC	Stores information about functions or procedures.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common System Views (1)

System View Name	Description
GS_AUDITING	Displays all audit information about database-related operations. Only users with system administrator or security policy administrator permission can access this view.
GS_FILE_STAT	Records statistics about data file I/Os to indicate I/O performance and detect performance problems such as abnormal I/O operations.
GS_CLUSTER_RESOURCE_INFO	Displays a DN resource summary. This view can be queried only when **enable_dynamic_workload** is set to **on** and the view cannot be executed on DNs. Only users with sysadmin permission can query this view.
GS_LABELS	Displays all configured resource labels. Only users with system administrator or security policy administrator permission can access this view.
GS_MASKING	Displays all configured dynamic masking policies. Only users with system administrator or security policy administrator permission can access this view.
GS_MATVIEWS	Provides information about each materialized view in the database.
GS_SQL_COUNT	Displays statistics about five types of running statements (**SELECT**, **INSERT**, **UPDATE**, **DELETE**, and **MERGE INTO**) on the current node of the database.
GS_SESSION_TIME	Collects statistics about the running time of session threads and time consumed in each execution phase.
GS_SESSION_MEMORY	Collects statistics about memory usage at the session level, including all the memory allocated to Postgres and Stream threads on DNs for jobs currently executed by users.
GS_WLM_PLAN_OPERATOR_HISTORY	Displays the operator-level records of the execution plan after the job is executed on the primary node of the current user's database.
PG_LOCKS	Stores information about locks held by opened transactions.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common System Views (2)

System View Name	Description
PG_ROLES	Provides information about database roles. Initialization users and users with the sysadmin or createrole attribute can view information about all roles. Other users can view only their own information.
PG_RULES	Provides access to query useful information about rewrite rules.
PG_TOTAL_USER_RESOURCE_INFO_OID	Displays resource usage of all users. Only administrators can query this view. This view is valid only if **use_workload_manager** is set to **on**.
PG_SETTINGS	Displays information about parameters of the running database.
PG_STATS	Provides access to the single-column statistics stored in the **pg_statistic** table.
PG_STAT_ACTIVITY	Displays information about the current user's queries.
PG_STAT_ALL_TABLES	Contains one row for each table in the current database (including TOAST tables), showing statistics about access to that specific table.
PG_TABLES	Provides useful information about access to each table in the database.
PG_TOTAL_USER_RESOURCE_INFO	Displays resource usage of all users. Only administrators can query this view. This view is valid only if **use_workload_manager** is set to **on**.
PG_USER	Provides information about database users. By default, only the initial user and users with the sysadmin attribute can view the information. Other users can view the information only after being granted permission.
PG_VIEWS	Provides useful information about access to each view in the database.
PLAN_TABLE	Displays plan information collected by **EXPLAIN PLAN**. Plan information is in a session-level lifecycle. After a session exits, the data will be deleted. Data is isolated between sessions and between users.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Reserved System Catalogs and System Views

- Reserved System Catalog
  - GS_WLM_USER_RESOURCE_HISTORY
- Reserved System Views
  - GS_WLM_OPERATOR_HISTORY
  - GS_WLM_OPERATOR_STATISTICS
  - GS_WLM_SESSION_HISTORY
  - GS_WLM_SESSION_INFO_ALL
  - GS_WLM_SESSION_STATISTICS
  - PG_SESSION_IOSTAT
  - PG_SESSION_WLMSTAT
  - PG_WLM_STATISTICS

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Contents

1. Logical Structure
2. Storage Engine
3. Tablespace
4. User Roles
5. System Catalogs and System Views
6. **Data Import and Export**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Data Import and Export - INSERT

- Run the INSERT statement to add data into the openGauss database in either of the following ways:
  - Use the client tool provided by the openGauss database to add data to the openGauss database.
  - Connect to the database using the JDBC or ODBC driver and run the **INSERT** statement to add data into the openGauss database.
- You can add, modify, and delete database transactions for the openGauss database. INSERT is the simplest way to add data. It is applicable to scenarios with small data volume and low concurrency.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Import and Export – COPY FROM STDIN (1)

- Use either of the following methods to add data to openGauss using the **COPY FROM STDIN** statement:
  - Add data to openGauss by typing. COPY copies data between tables and files. COPY FROM copies data from a file to a table. COPY TO copies data from a table to a file.
    - COPY FROM and COPY TO apply to low concurrency and local data import and export in small amount.
  - Import data from a file or a database to openGauss through the CopyManager API of the JDBC driver. You can use any parameters in the COPY syntax.
    - CopyManager is an API class provided by the JDBC driver in openGauss. It is used to import data to the openGauss database in batches.
    - The CopyManager class is located in **org.postgresql.copy** package and inherits from the java.lang.Object class.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- COPY applies only to tables and not to views. You must have permission to insert data into a table.
- CopyManager construction mode:
  - public CopyManager(BaseConnection connection)
  - throws SQLException

Data Import and Export – COPY FROM STDIN (2)

- COPY applies to the following scenarios:
  - Import text data as small tables.
  - Export small tables.
  - Export a query result set.
- The usage is as follows:
  - Import TEXT data.

```
copy t1 from  '/data/input/t1.txt' delimiter '^';
```

  - Export table data.

```
copy t1 to '/data/input/t1_output.txt' delimiter '^';
```

  - Export a query result set.

```
copy (select * from t1 where a2=1) to '/data/input/t1_output.txt' delimiter '^';
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Data Import and Export – gsql

- The gsql tool of openGauss provides the **\copy** meta-command to import and export data.
- **\copy** meta-command syntax

```
\copy { table [ ( column_list ) ] |
( query ) } { from | to } { filename |
stdin | stdout | pstdin | pstdout }
[ with ] [ binary ] [ delimiter
[ as ] 'character' ] [ null [ as ] 'string' ]
[ csv [ header ] [ quote [ as ]
'character' ] [ escape [ as ] 'character' ]
[ force quote column_list | * ] [ force
not null column_list ] ]
```

- Notes:
  - You can run this command to import or export data after logging in to a database on any gsql client. Different from the COPY statement in SQL, this command performs read/write operations on local files rather than files on database servers. The accessibility and permissions of local files are restricted to local users.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **table**: table name (optionally schema-qualified). Value range: an existing table name.
- **column_list**: list of optional columns to be copied. Value range: any columns in the table. If no column list is specified, all columns of the table will be copied.
- **query**: specifies that the results are to be copied. Valid value: a **SELECT** or **VALUES** command in parentheses.
- **filename**: absolute path of the file. To run the **COPY** command, the user must have the write permission for this path.
- **stdin\stdout**: declares that the input is from the standard input/output.
- **pstdin\pstout**: declares that the input is the standard input/output of gsql.
- **binary**: specifies that data is stored and read in binary mode instead of text mode. In binary mode, you cannot declare **DELIMITER**, **NULL**, or **CSV**. After **binary** is specified, CSV, FIXED, and TEXT cannot be specified through **option** or **copy_option**.
- **delimiter [as]'character':** specifies the column delimiter of data in a data file. Default values:
  - A tab character in text format
  - A comma in CSV format
  - No delimiter in FIXED format

Data Import and Export – gs_dump

- openGauss provides **gs_dump** and **gs_dumpall** to export required database objects and related information. You can use a tool to import the exported data to a target database for database migration.
- **gs_dump** exports a single database or its objects. **gs_dumpall** exports all databases or global objects in openGauss.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

gs_dump Parameters

Parameter	Description
-U	Username for database connection.
-W	User password for database connection.
-f	Folder to store exported files.
-p	TCP port or local Unix-domain socket file name extension on which the server is listening for connections.
dbname	Name of the database to be exported.
-F	Format of the file to be exported. The values of **-F** are as follows: **p**: plaintext **c**: custom **d**: directory **t**: .tar
-n	Names of schemas to be exported. Data of the specified schemas will also be exported. Single schema: Enter **-n** *schemaname*. Multiple schemas: Enter **-n** *schemaname* for each schema.
-t	Table (or view, sequence, foreign table) to be exported. You can specify multiple tables by listing them or using wildcard characters. Single table: Enter **-t** *schema.table*. Multiple tables: Enter **-t** *schema.table* for each table.
-T	A list of tables, views, sequences, or foreign tables cannot be dumped. You can use multiple **-T** options or wildcard characters to specify tables.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- If a username for connecting to the database is not specified, the initial system administrator name created during installation is used by default.
- When **-t** and **-T** are input, the object will be stored in **-t** list not **-T** table object.

Exporting a Database

- You can use gs_dump to export data and all object definitions of a database from openGauss. You can specify the information to export as follows:
  - Export full information of a database, including its data and all object definitions. You can use the exported information to create a duplicate database containing the same data as the current one.
  - Export all object definitions of a database, including the definitions of the database, functions, schemas, tables, indexes, and stored procedures. You can use the exported object definitions to quickly create a database that is the same as the current one, except the new database will not have data.
  - Export data of a database.
- Example: Use gs_dump to export the **postgres** database.

```
gs_dump -U jack -f /home/omm/backup/postgres_backup.tar -p 8000 postgres -F t
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Exporting a Schema

- You can use gs_dump to export data and all object definitions of a schema from openGauss.
- You can export one or more specified schemas as needed. You can specify the information to export as follows:
  - Export full information of a schema, including its data and object definitions.
  - Export data of a schema, excluding its object definitions.
  - Export the object definitions of a schema, including the definitions of tables, stored procedures, and indexes.
- Example: Use gs_dump to export the **hr** and **public** schemas at the same time.

```
gs_dump -U jack -f /home/omm/backup/MPPDB_schema_backup -p 8000 human_resource -n hr -n
public -F d
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Exporting a Table

- You can use gs_dump to export data and definition of a table-level object from openGauss. Views, sequences, and foreign tables are special tables.
- You can export one or more specified tables as needed. You can specify the information to export as follows:
  - Export full information of a table, including its data and definition.
  - Export data of a table.
  - Export the definition of a table.
- Example: Use gs_dump to export the **hr.staffs** and **hr.employments** tables.

```
gs_dump -U jack -f /home/omm/backup/MPPDB_table_backup -p 8000 human_resource -t hr.staffs -t
hr.employments -F d
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Exporting all Databases

- You can use gs_dumpall to export full information of all databases in a cluster from openGauss, including information about each database and global objects in the cluster. You can specify the information to export as follows:
  - Export full information of all databases, including information about each database and global objects (such as roles and tablespaces) to openGauss. You can use the exported information to create a host environment containing the same databases, global objects, and data as the current one.
  - Export data of all databases, excluding all object definitions and global objects.
  - Export all object definitions of all databases, including the definitions of tablespaces, databases, functions, schemas, tables, indexes, and stored procedures. You can use the exported object definitions to quickly create a host environment that is the same as the current one, containing the same databases and tablespaces but no data.
- Example: Use gs_dumpall to export all database information at a time.

```
gs_dumpall -U omm -f /home/omm/backup/MPPDB_backup.sql -p 8000
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Export By a User Without Required Permissions

- For gs_dump and gs_dumpall, use **-U** to specify the user that performs the export. If the specified user does not have the required permissions, data cannot be exported. In this case, you can set the **--role** parameter in the export command to specify the role that has the permission. Then, gs_dump or gs_dumpall uses the **--role** parameter to specify a role to export data.
- Example: Use gs_dump to export data of the **human_resource** database.
  - Assume that user **jack** does not have the permission to export data of the **human_resource** database and the role **role1** has this permission. To export data of the **human_resource** database, you can set **--role** to **role1** in the export command.

```
gs_dump -U jack -f /home/omm/backup/MPPDB_backup.tar -p 8000 human_resource --role role1 --
rolepassword  abc@1234 -F t
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Import and Export – gs_restore

- gs_restore is an import tool provided by the openGauss database. You can use this tool to import the files exported by gs_dump to a database. gs_restore can import the files in .tar, custom, or directory format.
- gs_restore can:
  - Import data to a database.
    - If a database is specified, data is imported to the database. For parallel import, the password for connecting to the database is required.
  - Import data to a script.
    - If no database is specified, a script containing the SQL statement to recreate the database is created and added to a file or standard output. This script output is equivalent to the plain text output of gs_dump.
- You can specify and sort the data to be imported.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- gs_restore incrementally imports data by default. To prevent data exceptions caused by multiple import operations, you are advised to use the **-c** and **-e** parameters during the import. **-c** indicates that the database objects that already exist in the database to be restored are cleared (deleted) before the database objects are re-created. **-e** indicates that if an error occurs when the SQL statement is sent to the database, the system exits. By default, the system continues to send the SQL statement and displays a series of error information after the SQL statement is imported.

gs_restore Parameters

Parameter	Description
-U	Username for database connection.
-W	User password for database connection.
-p	TCP port or local Unix-domain socket file name extension on which the server is listening for connections.
-d	Name of a database to which data will be imported.
-e	Exits if an error occurs when you send the SQL statement to the database. Error messages are displayed after the import process is complete.
-c	Cleans existing objects from the target database before the import.
-s	Imports only object definitions in schemas and does not import data. Sequence values will also not be imported.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

gs_restore Example

- Run **gs_restore** to import all object definitions from the exported file of the whole **postgres** database to the **backupdb** database.

```
gs_restore -U jack /home/omm/backup/MPPDB_backup.tar -p 8000 -d backupdb -s -e -c
```

- Run **gs_restore** to import data and all object definitions of the **postgres** database from the **MPPDB_backup.dmp** file (custom format).

```
gs_restore backup/MPPDB_backup.dmp -p 8000 -d backupdb
```

- Run **gs_restore** to import all object definitions of the **postgres** database to the **backupdb** database. Before the import, complete definitions and data exist in Postgres. After the import, all object definitions exist in the **backupdb** database and no data exists in the table.

```
gs_restore /home/omm/backup/MPPDB_backup.tar -p 8000 -d backupdb -s -e -c
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

ANALYZE

- The execution plan generator needs to use table statistics to generate the most effective query execution plan to improve query performance. After data is imported, you are advised to run the ANALYZE statement to update table statistics. The statistics are stored in the system catalog **PG_STATISTIC**.
- ANALYZE supports row-store and column-store tables. ANALYZE can also collect statistics about specified columns of a local table.
- ANALYZE syntax (the **product_info** table is used as an example)

```
postgres=# ANALYZE product_info;
ANALYZE
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

VACUUM

- If a large number of rows were updated or deleted during import, run **VACUUM FULL** before **ANALYZE**. A large number of updates and deletions generate huge disk page fragments, which reduces query efficiency. VACUUM FULL can restore disk page fragments and return them to the OS.
- VACUUM FULL syntax (the **product_info** table is used as an example)

```
postgres=# VACUUM FULL product_info;
VACUUM
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (True or False) Users and roles have the same syntax in the openGauss database. Therefore, they are fully equivalent.
   A. True
   B. False
2. (Single-answer question) Which of the following commands can be used to export partial data from the student table?
   A. gs_dump
   B. gs_dumpall
   C. copy *Conditional statement*
   D. gs_backup

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - 1. B
  - 2. C

Quiz

3. (Multiple-answer question) Which of the following statements about row-store tables and column-store tables are true?
   A. Row-store tables are used in scenarios where INSERT and UPDATE operations are frequently performed.
   B. Row-store tables are applicable to point queries, that is, simple index-based queries that return a small number of records.
   C. Column-store tables are more efficient in compression and projection.
   D. Row-store tables are used in scenarios where there are a large number of queries for statistical analysis.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Answers:
  - 3. ABC

Summary

- On completion of this course, you will be able to:
  - Understand the openGauss logical structure.
  - Master how to select different storage engines.
  - Master how to create and manage tablespaces.
  - Master how to manage users and roles.
  - Understand common system catalogs and views.
  - Master how to import and export data.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Recommendations

- openGauss help document:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/GettingStarted/GettingStarted.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Acronyms and Abbreviations

- MOT: memory-optimized table
- ACID: atomicity, consistency, isolation, and durability.
- NUMA: non-uniform memory access
- WAL: write-ahead logging
- TPC-C: A specification for online transaction processing (OLTP).

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

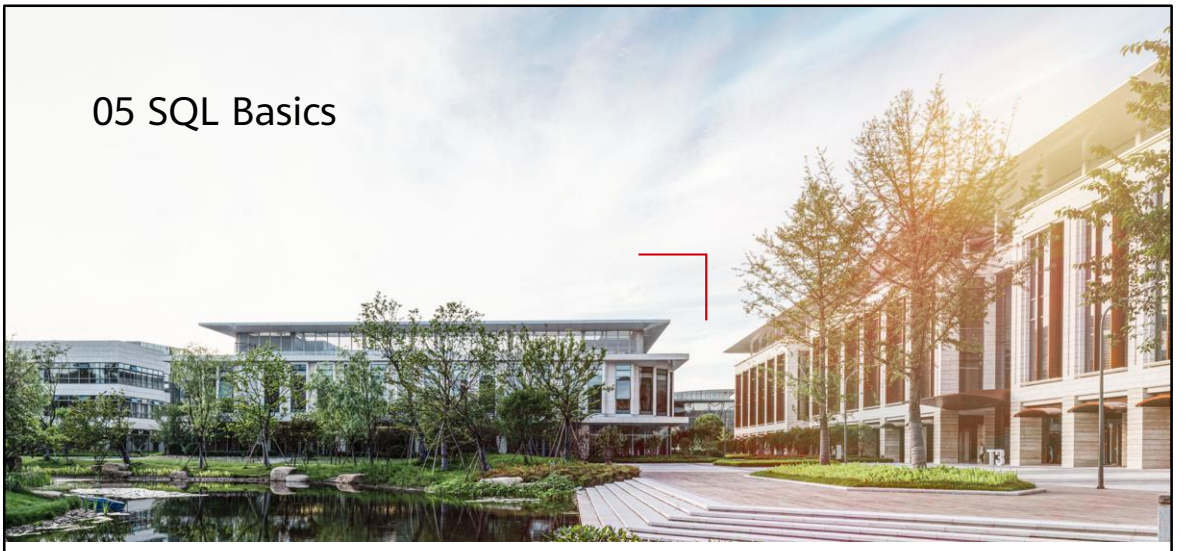
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.

HUAWEI

05 SQL Basics



HUAWEI

Foreword

- openGauss is an open-source relational database management system. It supports SQL standards and is compatible with the SQL syntax as well as the capabilities of mainstream commercial databases.
- This chapter describes the SQL syntax, SQL syntax classification, common system functions, and operators, aiming to help beginners master the basic knowledge and operations of SQL.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completion of this course, you will be able to:
  - Define SQL and classify SQL statements
  - List common data types and know how to use them
  - Understand the types and usage of common functions and operators

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **SQL Syntax Quick Start**
   - Introduction to SQL Statements
     - SQL Statement Classification
     - Data Type
2. SQL Syntax Classification
3. Operators and Common Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Introduction to SQL Statements

- Wikipedia:
  - Structured Query Language (SQL) is a domain-specific language used in programming. It is designed for managing data held in a relational database management system, or for stream processing in a relational data stream management system.
  - SQL is based on relational algebra and tuple relational calculus, including a data definition language (DDL) and data manipulation language (DML). The scope of SQL includes data insertion, query, update, and deletion, database schema creation and modification, and data access control.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- openGauss supports SQL-92, SQL-99, SQL-2003, and SQL-2011 standards, GBK and UTF-8 character sets, SQL standard and analytic functions, and stored procedures.
- SQL is a standard computer language used to control access to and manage data in databases.
- SQL provides different statements to enable you to:
  - Query data.
  - Insert, update, and delete rows.
  - Create, replace, modify, and delete objects.
  - Control access to a database and its objects.
  - Maintain the consistency and integrity of a database.
- SQL consists of commands and functions that are used to manage databases and database objects. SQL can also forcibly implement rules for data types, expressions, and texts. Therefore, in addition to SQL syntax, this chapter also describes data types, expressions, functions, and operators.

Contents

1. **SQL Syntax Quick Start**
   - Introduction to SQL Statements
   - SQL Statement Classification
   - Data Type
2. SQL Syntax Classification
3. Operators and Common Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

SQL Statement Classification

- DDL
  - Defines or modifies objects in a database, such as tables, indexes, views, databases, sequences, users, roles, tablespaces, and stored procedures.
- DML
  - Performs operations on data in database tables, such as inserting, updating, and deleting data.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Common DDL statements:
  - Database definition: CREATE DATABASE and ALTER DATABASE
  - Tablespace definition: CREATE TABLESPACE, ALTER TABLESPACE, and DROP TABLESPACE
  - Table definition: CREATE TABLE, ALTER TABLE, DROP TABLE, and TRUNCATE TABLE
  - Index definition: CREATE INDEX, ALTER INDEX, and DROP INDEX
  - Role definition: CREATE ROLE and DROP ROLE
  - User definition: CREATE USER, ALTER USER, and DROP USER
  - View definition: CREATE VIEW and DROP VIEW
  - Function definition: CREATE FUNCTION, ALTER FUNCTION, and DROP FUNCTION
  - Procedural language definition: CREATE LANGUAGE, ALTER LANGUAGE, and DROP LANGUAGE
  - Operator definition: CREATE OPERATOR, ALTER OPERATOR, and DROP OPERATOR

SQL Statement Classification

- Data Control Language (DCL)
  - Sets or changes database transactions, performs authorization operations (such as granting permissions to users or roles, revoking permissions, creating roles, and deleting roles), locks tables (supporting shared and exclusive locks), and stops services.
- Data Query Language (DQL)
  - Queries data in a database, for example, querying data and combining the result sets of multiple SELECT statements.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- DML:
  - Data operation: INSERT, UPDATE, and DELETE
  - Import and export: LOAD and DUMP
  - Others: EXPLAIN PLAN
- DCL:
  - Transaction management: COMMIT and ROLLBACK
  - Authorization operation: GRANT and REVOKE
  - Table locking: LOCK TABLE
  - Shutdown: SHUTDOWN
- DQL:
  - Data query: SELECT
  - Combination of the result sets of multiple SELECT statements

Contents

1. SQL Syntax Quick Start
   - Introduction to SQL Statements
   - SQL Statement Classification
   - Data Type
2. SQL Syntax Classification
3. Operators and Common Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Types

- A data type is a basic data attribute. Occupied storage space and allowed operations vary according to data types.
- Data in a database is stored in tables, and data types are defined for each column in the table. When storing data, you must comply with the attributes of these data types. Otherwise, errors may occur. The data types are as follows:
  - Common data types
    - Numeric, character, and date
  - Uncommon data types
    - Binary and Boolean

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Note: Occupied storage space and allowed operations vary according to data types. Data in a database is stored in tables, and data types are defined for each column in the table. When storing data, you must comply with the attributes of these data types. Otherwise, errors may occur.

Numeric Types

Data Type	Description	Storage Space
Integers		
TINYINT	Tiny integer, also called INT1	One byte
SMALLINT	Small integer, also called INT2	Two bytes
INTEGER	Typical choice for integers, also called INT4	Four bytes
BINARY_INTEGER	Alias of the common integer type INTEGER	Four bytes
BIGINT	Big integer, also called INT8	Eight bytes
Arbitrary precision numbers		
NUMERIC[(p[,s])], DECIMAL[(p[,s])]	The value range of **p** (precision) is [1,1000], and the value range of **s** (scale) is [0,*p*]. **p** indicates the total digits, and **s** indicates the decimal digits.	Users specify precision. Two bytes are occupied for every four decimals of precision. An extra eight-byte overhead is added for numbers of this type.
NUMBER[(p[,s])]	Alias of the NUMERIC type.	Users specify precision. Two bytes are occupied for every four decimals of precision. An extra eight-byte overhead is added for numbers of this type.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Numeric data types in openGauss include integer, floating point, and high precision types.
- Notes:
  - TINYINT, SMALLINT, INTEGER, and BIGINT store whole numbers (those without fractional components) of various ranges. Saving a number with a decimal in any of the data types will result in errors.
  - The common type is INTEGER, because it provides the best balance between range, storage size, and performance. Generally, SMALLINT is used only if you are sure that the value range is within the SMALLINT value range. INTEGER storage is much faster. BIGINT is used only when the range of INTEGER is not large enough.
- Compared with INTEGER, the arbitrary precision type requires more storage space, and has lower storage efficiency, computing efficiency, and compression ratio effect. INTEGER is usually used when numeric types are defined. The arbitrary precision type can be selected only when the value exceeds the maximum range that can be represented by an integer.
- When NUMERIC/DECIMAL is used for defining a column, you are advised to specify the precision (p) and scale (s) for the column.

Example

- Create a table.

```
postgres=# CREATE TABLE type_t1
(
    a TINYINT,
    b INTEGER,
    c BIGINT,
    d DECIMAL(10,4)
);
```

  - Add data.

```
postgres=# INSERT INTO type_t1 VALUES(100, 1000, 10000, 123456.1223);
```

  - View data.

```
postgres=# SELECT * FROM type_t1;
```

  - Delete the table.

```
postgres=# DROP TABLE type_t1;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- When creating a table, specify the data type for the columns in the table.

Numeric Types

Data Type	Description	Storage Space
Serial integers		
SMALLSERIAL	Two-byte serial integer	Two bytes
SERIAL	Four-byte serial integer	Four bytes
BIGSERIAL	Eight-byte serial integer	Eight bytes
Floating-point numbers		
REAL, FLOAT4	Single-precision floating-point number, which is not very precise	Four bytes
DOUBLE RECISION, FLOAT8	Double-precision floating-point number, which is not very precise	Eight bytes
FLOAT[(p)]	Floating-point number, which is not very precise. The value range of **p** (precision) is [1,53]. **p** is the precision, indicating the total decimal digits.	Four or eight bytes
BINARY_DOUBLE	Alias of DOUBLE PRECISION	Eight bytes
DEC[(p[,s])]	The value range of **p** (precision) is [1,1000], and the value range of **s** (scale) is [0,*p*]. **p** indicates the total digits, and **s** indicates the decimal digits.	Users specify precision. Two bytes are occupied for every four decimals of precision. An extra eight-byte overhead is added for numbers of this type.
INTEGER[(p[,s])]	The value range of **p** (precision) is [1,1000], and the value range of **s** (scale) is [0,*p*].	Users specify precision. Two bytes are occupied for every four decimals of precision. An extra eight-byte overhead is added for numbers of this type.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- SMALLSERIAL, SERIAL, and BIGSERIAL are not real types. They are concepts used to set a unique identifier for a table. You can create an integer column and arrange its default value to be read from a sequencer. A NOT NULL constraint is used to ensure **NULL** is not inserted.
- Generally, UNIQUE or PRIMARY KEY constraints are added to prevent a duplicate value from being inserted by mistake. However, this is not automatic. The sequencer is set to belong to a column. In this case, deleting the column or table, also deletes the sequencer.
- Currently, a **SERIAL** column can be specified only when you create a table and cannot be inserted in an existing table. In addition, **SERIAL** columns cannot be created in temporary tables. Since SERIAL is not a data type, columns cannot be converted to this type either.

Example

- Create a table.

```
postgres=# CREATE TABLE type_t2
(
    a SMALLSERIAL,
    b SERIAL,
    c BIGSERIAL,
    d FLOAT4,
    e DECIMAL(10,4)
);
```

  - Add data.

```
postgres=# INSERT INTO type_t2 VALUES(default, default, default, 10.365456, 123.123654);
postgres=# INSERT INTO type_t2 VALUES(default, default, default, 10.365456, 123.123654);
```

  - View data.

```
postgres=# SELECT * FROM type_t2;
```

  - Delete the table.

```
postgres=# DROP TABLE type_t2;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- When creating a table, specify the data type for the columns in the table.

Character Types

Data Type	Description	Storage Space
Character type		
CHAR(n) CHARACTER(n) NCHAR(n)	Character string with fixed length. Empty characters are filled in with blank spaces. **n** indicates the string length. If it is not specified, the default precision of **1** is used.	Maximum: 10 MB
VARCHAR(n) CHARACTER VARYING(n)	Character string with variable length. **n** indicates the string length.	Maximum: 10 MB
VARCHAR2(n)	Character string with variable length. It is the alias of the VARCHAR(n) type. **n** indicates the string length.	Maximum: 10 MB
NVARCHAR2(n)	Character string with variable length. **n** indicates the string length.	Maximum: 10 MB
TEXT	Character string with variable length.	Maximum: 1 GB minus 1 byte
CLOB	Big text object. It is the alias of the TEXT type.	Maximum: 1 GB minus 1 byte

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Digits and letters refer to the characters in the basic ASCII table, excluding the characters in the extended ASCII table.
- In the basic ASCII table, the number of bytes = string length = the number of characters, because each character is one byte. In Unicode, the number of bytes/2 = string length = the number of characters, because each character is two bytes.
- If the length of the stored content is specified, for example, when storing gender data using only male or female, NCHAR(1) is used to save storage space.
- If the length of the stored content is not fixed, the variable-length string type VARCHAR can be used to save storage space, because space padding is not required.
- CLOB is used to store long data content, such as detailed product descriptions.

Example

- Create a table.

```
postgres=# CREATE TABLE type_t3
(
    a CHARACTER(4),
    b VARCHAR(5),
);
```

  - Add data.

```
postgres=# INSERT INTO type_t3 VALUES('ok', 'good');
```

  - View data.

```
postgres=# SELECT * FROM type_t3;
```

  - Delete the table.

```
postgres=# DROP TABLE type_t3;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Date Types

Data Type	Description	Storage Space
Date/Time types		
DATE	Date and time.	Four bytes
TIME [(p)] [WITHOUT TIME ZONE]	Time within one day. **p** indicates the precision after the decimal point. The value ranges from 0 to 6.	Eight bytes
TIME [(p)] [WITH TIME ZONE]	Time within one day (with time zone). **p** indicates the precision after the decimal point. The value ranges from 0 to 6.	12 bytes
TIMESTAMP[(p)] [WITHOUT TIME ZONE]	Date and time. **p** indicates the precision after the decimal point. The value ranges from 0 to 6.	Eight bytes
TIMESTAMP[(p)][WITH TIME ZONE]	Date and time (with time zone). TIMESTAMP is also called TIMESTAMPTZ. **p** indicates the precision after the decimal point. The value ranges from 0 to 6.	Eight bytes
SMALLDATETIME	Date and time (without time zone). The precision is minute. A duration between 30s and 60s is rounded to 1 minute.	Eight bytes
INTERVAL DAY (l) TO SECOND (p)	Time interval (*X* days *X* hours *X* minutes *X* seconds). **l** indicates the precision of days. The value ranges from 0 to 6. **p** indicates the precision of seconds. The value ranges from 0 to 6. The digit 0 at the end of a decimal number is not displayed.	16 bytes
INTERVAL [FIELDS] [(p)]	Time interval. The value of **FIELDS** can be **YEAR**, **MONTH**, **DAY**, **HOUR**, **MINUTE**, **SECOND**, or **DAY TO HOUR**. **p** indicates the precision of seconds. The value ranges from 0 to 6. **p** takes effect only when the value of **FIELDS** is **SECOND**, **DAY TO SECOND**, **HOUR TO SECOND**, or **MINUTE TO SECOND**.	12 bytes

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- INTERVAL [FIELDS] [(p)]: Specifies the time interval.
  - The value of **FIELDS** can be **YEAR**, **MONTH**, **DAY**, **HOUR**, **MINUTE**, **SECOND**, **DAY TO HOUR**, **DAY TO MINUTE**, **DAY TO SECOND**, **HOUR TO MINUTE**, **HOUR TO SECOND**, or **MINUTE TO SECOND**.
  - **p** indicates the precision of seconds. The value ranges from 0 to 6. **p** takes effect only when the value of **FIELDS** is **SECOND**, **DAY TO SECOND**, **HOUR TO SECOND**, or **MINUTE TO SECOND**. The digit 0 at the end of a decimal number is not displayed.
- reltime: Specifies the relative time interval. The format is:
  - *X* years *X* months *X* days *XX:XX:XX*
  - The Julian calendar is used. It specifies that a year has 365.25 days and a month has 30 days. The relative time interval needs to be calculated based on the input value. The output format is POSTGRES.
- abstime: Specifies date and time. The format is:
  - YYYY-MM-DD hh:mm:ss+timezone

Example

- Create a table.

```
postgres=# CREATE TABLE type_t4
(
  a TIME WITHOUT TIME ZONE,
  b TIME WITH TIME ZONE,
  c TIMESTAMP WITHOUT TIME ZONE,
  d TIMESTAMP WITH TIME ZONE,
  e SMALLDATETIME
);
```

  - Add data.

```
postgres=# INSERT INTO type_t4 VALUES('21:21:21','21:21:21 pst','2010-12-12','2013-12-11 pst','2003-04-12
04:05:06');
```

  - View data.

```
postgres=# SELECT * FROM type_t4;
   a          b              c                         d                          e
---------- -------------- ------------------------- ---------------------------- ---------------------
21:21:21 21:21:21-08   2010-12-12 00:00:00  2013-12-11 16:00:00+08  2003-04-12 04:05:00
```

  - Delete the table.

```
postgres=# DROP TABLE type_t4;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- When length is specified, strings exceeding this length are truncated.

Uncommon Data Types

- Binary

Data Type	Description	Occupied Space
BYTEA	Variable-length binary string.	Four bytes plus the actual binary string. The maximum size is 1073733621 bytes (1 GB minus 8203 bytes).
RAW	Variable-length hexadecimal string.	Four bytes plus the actual hexadecimal string. The maximum size is 1 GB minus 8203 bytes.
BYTEAWITHOUTORDERWITHEQUALCOL	Variable-length binary character string (new type for encryption). If deterministic encryption is specified in the encrypted column, the column type is BYTEAWITHOUTORDERWITHEQUALCO.	Four bytes plus the actual binary string. The maximum size is 1073741771 bytes (1 GB minus 53 bytes).
BLOB	Stores binary data of variable-length objects. It is the large object data of the raw type.	The maximum size is 1 GB minus 8203 bytes.

- Boolean

Data Type	Description	Value Range	Occupied Space
BOOLEAN	Stores Boolean data.	TRUE,FALSE	One byte

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The preceding slides describe common data types. Next, we will look at uncommon data types.
- A binary string is a byte sequence. Binary strings are used to store what programmers regard as raw byte data while strings are used to store texts.
- Boolean type data can be converted to INT and BIGINT types, because Boolean can be regarded as digit 0 or 1. Therefore, a Boolean value can be converted to integer 0 or 1, and an integer can be converted into a Boolean value. Integer 0 corresponds to the Boolean value **FALSE** and other non-zero integers correspond to the Boolean value **TRUE**.
    - The value range of *n* is [0,4], which indicates the precision of the year. The default value is **2**.
    - The value range of *n1* is [0,7], which indicates the precision of the day. The default value is **2**.
    - The value range of *n2* is [0,6], which indicates the precision after the second. The default value is **6**.

Example

- Create a table.

```
postgres=# CREATE TABLE type_t5
(
    a BLOB,
    b RAW,
    c BYTEA,
    e BOOLEAN
);
```

  - Add data.

```
postgres=# INSERT INTO type_t5 VALUES(empty_blob(), HEXTORAW('DEADBEEF'),E'\\xDEADBEEF' , TRUE);
```

  - View data.

```
postgres=# SELECT * FROM type_t5;
  a                b              c             d
---------  -------------  ---------------  ------------
           DEADBEEF    \xdeadbeef         t
```

  - Delete the table.

```
postgres=# DROP TABLE type_t5;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (True or false) BIGINT occupies four bytes.
   A. True
   B. False
2. (True or false) CLOB is used to store large object data of the TEXT type.
   A. True
   B. False

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - B, it occupies eight bytes.
  - A

Contents

1. SQL Syntax Quick Start
2. **SQL Syntax Classification**
   - Data Query
   - Data Update
   - Data Definition
3. Operators and Common Functions

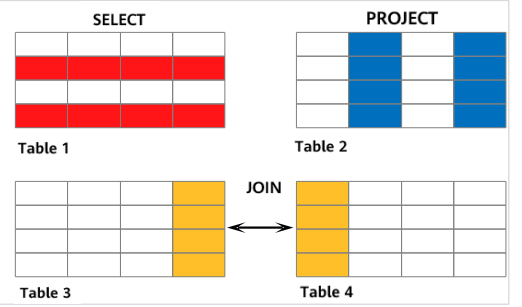
 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Query Overview

- Data in a database is stored in tables. Generally, these tables consist of tuples and columns. Columns form the structure of a table and rows form the data of a table.
- The SELECT statement can be used to query required information from a database table, but does not change the said data. All it does is to extract and display the data.

- Key elements of the SELECT statement
  - **SELECT**: Queries the tuples that meet the conditions and filters out those that do not.
  - **PROJECT**: Queries the columns that meet the conditions and filters out those that do not.
  - **JOIN**: Queries data from multiple tables. Joins are the core of a relational database. Data is stored in different tables, and complete information is obtained through queries using **JOIN**.



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- DQL is the most basic application of a database. The DQL syntax is complex, including simple query, conditional query, join query, subquery, set operation, as well as data grouping, sorting, and restriction. This section describes DQL types and how to use them in real-life scenarios.

Simple Query

- Syntax:

```
SELECT [/*+ plan_hint */] [ ALL | DISTINCT [ ON ( expression [, ...] ) ] ]
{ * | {expression [ [ AS ] output_name ]} [, ...] }
[ FROM FROM_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ ORDER BY {expression [ [ ASC | DESC | USING operator ] | nlssort_expression_clause ] [ NULLS { FIRST |
LAST } ]} [, ...] ]
[ LIMIT { [offset,] count | ALL } ]
[ OFFSET start [ ROW | ROWS ] ]
[ , ... ]
```

- Usage:
  - The expression between the **SELECT** keyword and the **FROM** clause is called a **SELECT** item. **SELECT** items are used to specify the columns to be queried, and the **FROM** clause specifies the table where the columns are located.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- To query all columns, use an asterisk (*) after **SELECT**. To query certain columns, specify the column names after **SELECT** and separate the names with commas (,).

Example

- Use an asterisk (*) after **SELECT** to query all columns in the **bank_card** table.

```
SELECT * FROM bank_card;
b_number                    | b_type      | b_c_id
--------------------------------+-----------------+--------------------
6222021302020000001         | Credit Card | 1
6222021302020000002         | Credit Card | 3
6222021302020000003         | Credit Card | 5
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Run **gsql –d postgres –p 5432** to connect to the database.

Selecting a Query Column

- When you select a query column, the column name can be expressed in the following ways:
  - Manually enter column names and separate them with commas (,).

    ```
    SELECT a, b, f1, f2 FROM t1, t2;
    ```

    The columns **a** and **b** are columns in table **t1**, and **f1** and **f2** are columns in table **t2**.
  - Use calculated columns.

    ```
    SELECT a+b FROM t1;
    ```
  - If two or more tables have the same column names, you are advised to specify both table and column names. Query results can be returned without specifying column names but will require extra work. Example:

    ```
    SELECT t1.f1, t2.f1 FROM t1, t2;
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example

- View the card number and card type in the **bank_card** table.

```
postgres=# SELECT b_number,b_type FROM bank_card;
b_number                    |         b_type
--------------------------------+--------------------
6222021302020000001        | Credit Card
6222021302020000002        | Credit Card
6222021302020000003        | Credit Card
```

- View the ID, name, email address, and bank card number of customer 1.

```
postgres=# SELECT a.c_id,a.c_name, a.c_mail, b.b_number FROM client a, bank_card b where
a.c_id= 1 and b.b_c_id = 1;

c_id         | c_name     |     c_mail                    |     b_number
------------+---------------+-------------------------------+-----------------------------
1            |   Zhang Yi   |   zhangyi@Huawei.com  |  6222021302020000001
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Alias

- You can use the **AS some_name** clause to specify an alias for a table or column. Generally, an alias is created to improve the readability of a table or column name.
- Syntax
  - The SQL aliases of columns and tables follow the corresponding column and table names. You can choose whether to add the keyword **AS** in the middle.
- Example: Use an alias.

```
postgres=# SELECT b_c_id AS CardID, b_type CardType FROM bank_card;
cardid     |        cardtype
------------+---------------------------------
1            |    Credit Card
3            |    Credit Card
5            |    Savings Card
```

```
postgres=# SELECT a.c_id CID ,a.c_name Name, a.c_mail Email, b.b_number "Card Number" FROM client a,
bank_card b where a.c_id= 1 and b.b_c_id = 1;

cid          | name       |   email                        |   Card Number
------------+------------+--------------------------------+---------------------------
1            | Zhang Yi   |  zhangyi@Huawei.com      | 6222021302020000001
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Aliases can also be enclosed in double quotation marks ("").

Conditional Query

- In a SELECT statement, you can specify conditions for a more accurate query. Conditions are specified by using expressions and operators, and the return value is **TRUE**, **FALSE**, or **UNKNOWN**. Query conditions are used in the **WHERE** and **HAVING** clauses.
- Syntax:
  - **condition** clause

```
select_statement { predicate } [ { AND | OR } condition ] [ , ... n ]
```

  - **predicate** clause

```
{ expression { = | <> | != | > | >= | < | <= } { ALL | ANY } expression | ( SELECT)
 | string_expression [ NOT ] LIKE string_expression
 | expression [ NOT ] BETWEEN expression AND expression
 | expression IS [ NOT ] NULL
 | expression [ NOT ] IN ( SELECT| expression [ , ... n ] )
 | [ NOT ] EXISTS ( SELECT)
}
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Conditional Query

- Query conditions are defined by using expressions and operators. Common conditions are defined as follows:
  - Comparison operators (such as **>**, **<**, **>=**, **<=**, **!=**, **<>**, and **=**), specify the comparison query conditions. In the expression, single quotation marks (') are optional for the numeric type but mandatory for character and date types.
  - Test operators specify query ranges. To specify multiple conditions, use the **AND** logical operator to connect these conditions. To specify one of multiple conditions, use the **OR** logical operator to connect these conditions.
- Example: Use a comparison operator to query the information about a credit card.

```
postgres=# SELECT * FROM bank_card WHERE b_type='Credit Card';
b_number           |    b_type          |  b_c_id
-------------------+--------------------+----------
6222021302020000001 | Credit Card        |  1
6222021302020000002 | Credit Card        |  3
6222021302020000003 | Credit Card        |  5
6222021302020000004 | Credit Card        |  7
6222021302020000005 | Credit Card        |  9
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Conditional Query

- Logical Operators
  - Common logical operators include AND, OR, and NOT. The operation result can be **TRUE**, **FALSE**, or **NULL** (which means unknown). Their priorities are NOT > AND > OR.

a	b	a AND b	a OR b	NOT a
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL

- Example: Query information about a credit card whose customer ID is 1 from the **bank_card** table.

```
postgres=# SELECT* FROM bank_card where b_c_id= 1and b_type='Credit Card';
b_number                          |    b_type             |   b_c_id
-----------------------------------+-----------------------+----------------
6222021302020000001          | Credit Card           |   1
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Join Query

- Generally, data to be queried is distributed in multiple tables. A query across tables or views is called a join query. In most cases, a join query is executed between a table and its child tables.
- Syntax

```
SELECT [ , ... ] FROM table_reference
        [LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER] | INNER]
        JOIN table_reference
        [ON { predicate } [ { AND | OR } condition ]  [ , ... n ]]
```

  - **table_reference** clause

```
{  table_name [ [AS] alias ]
  | view_name [ [AS] alias]
  | ( SELECTquery ) [ [AS] alias ]
}
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- A Cartesian product is also called a direct product. For sets *X* and *Y*, the Cartesian product of the two sets is presented as *X* x *Y*. Therefore, if *X* has *I* tuples and *N* attributes and *Y* has *J* tuples and *M* attributes, their Cartesian product will have *I* x *J* tuples and *N* + *M* attributes. The two relations may have attributes with the same name.
- Example: Perform the Cartesian product operation on tables A and B. When you perform a join query for tables A (4 records) and B (5 records) without specifying conditions, 20 (4 x 5) records are obtained. SELECT * FROM A, B;
- The **table_reference** clause can contain ordinary tables, views, and subqueries.

Join Query

- If multiple tables are included in the **FROM** clause for a query, the database executes a join query.
  - The **SELECT** items of the query can be any columns in these tables.

    ```
    SELECT table1.column, table2.column FROM table1, table2;
    ```

  - Most join queries contain at least one query condition, which can be either in the **FROM** or **WHERE** clause.

    ```
    SELECT table1.column, table2.column FROM table1 JOIN table2 ON(table1.column1 =
    table2.column2);
    SELECT table1.column, table2.column FROM table1, table2 WHERE table1.column1 =
    table2.column2;
    ```

- In the **WHERE** clause, you can use the operator (+) to convert a table join to an outer join. However, this method is not recommended because it is not the standard SQL syntax.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Inner Join

- **INNER JOIN**: The syntax keyword is **INNER JOIN**, where **INNER** can be omitted. If an inner join is used, the join execution sequence will follow the sequence of tables in the statement.
- Example: Query the customer ID, bank card number, and bank card type. Use the **c_id** column of the **client** and **bank_card** tables to perform the query operation.

```
postgres=# SELECT c.c_id, b.b_number, b.b_type FROM client c JOIN bank_card b ON (b.b_c_id =
c.c_id);
c_id       |          b_numer                  |       b_type
-----------+-----------------------------------+------------------------------
1          | 6222021302020000001       |  Credit Card
3          | 6222021302020000002       |  Credit Card
5          | 6222021302020000003       |  Credit Card
7          | 6222021302020000004       |  Credit Card
9          | 6222021302020000005       |  Credit Card
10         | 6222021302020000006       |  Credit Card
12         | 6222021302020000007       |  Credit Card
14         | 6222021302020000008       |  Credit Card
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Outer Join

- In inner joins, the two data sources specified are equal. However, in outer joins, one data source is used as the base for the other source to match according to conditions.
- An inner join returns all data records in both tables that meet join conditions. An outer join returns both records that meet join conditions and those that do not.
- Outer joins include left, right, and full outer joins.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- LEFT [OUTER] JOIN
  - Returns all rows from the Cartesian product of two tables, which meet the join conditions, along with the rows in the left table that do not match the rows from the right table by join conditions; **NULL** values are inserted to complete the table. Note that the system only uses **JOIN** clause's conditions to decide which rows have matches. Outer conditions are applied later.
- RIGHT [OUTER] JOIN
  - Returns all the joined rows, plus one row for each unmatched right-hand row (extended with **NULL** on the left).
  - The notation is used for convenience, since you could convert it to a **LEFT OUTER JOIN** by switching the left and right inputs.
- FULL [OUTER] JOIN
  - Returns all the joined rows, plus one row for each unmatched left-hand row (extended with **NULL** on the right) and one row for each unmatched right-hand row (extended with **NULL** on the left).

Left Outer Join

- Also called a left join, indicating that the query is driven by the left table.
- All data from the left table is combined with data from the right table which satisfies the specified join conditions. Columns of the right table that do not match the conditions are set to **NULL**.
- Example:

```
postgres=# SELECT e.staff_id, e.higest_degree, t.score FROM
education e LEFT JOIN training t ON (e.staff_id = t.staff_id);

STAFF_ID     |HIGEST_DEGREE | SCORE
--------------+--------------------+------------
11             | master             |   92
12             | doctor             |   95
13             | scholar
```

education

Staff_id	Higest_degree
11	master
12	doctor
13	scholar

training

Staff_id	score
10	90
11	92
12	95

Staff_id	Higest_degree	score
11	master	92
12	doctor	95
13	scholar	

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

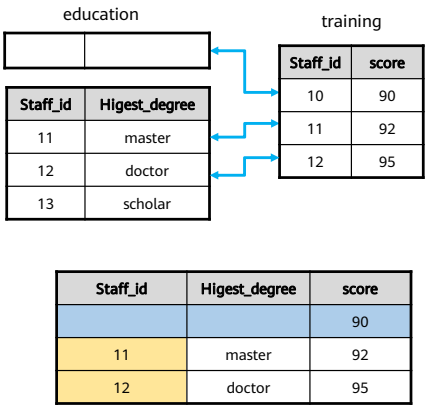
- LEFT [OUTER] JOIN
  - Returns all rows from the Cartesian product of two tables, which meet the join conditions, along with the rows in the left table that do not match the rows from the right table by join conditions; **NULL** values are inserted to complete the table. Note that the system only uses **JOIN** clause's conditions to decide which rows have matches. Outer conditions are applied later.

Right Outer Join

- Also called a right join, indicating that the query is driven by the right table.
- All data from the right table is combined with data from the left table which satisfies the specified join conditions. Columns of the left table that do not match the conditions are set to **NULL**.
- Example:

```
postgres=# SELECT e.staff_id, e.higest_degree, t.score FROM
education e RIGHT JOIN training t ON (e.staff_id = t.staff_id);

STAFF_ID     HIGEST_DEGREE SCORE
------------ ------------- ------------
                           90
  11         master        92
  12         doctor        95
```



Staff_id	Higest_degree	score
		90
11	master	92
12	doctor	95

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

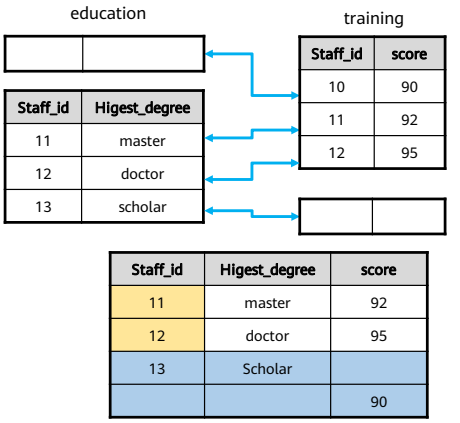
- RIGHT [OUTER] JOIN
  - Returns all the joined rows, plus one row for each unmatched right-hand row (extended with **NULL** on the left).
  - The notation is used for convenience, since you could convert it to a **LEFT OUTER JOIN** by switching the left and right inputs.

Full Outer Join

- Also called a full join; the query returns all records (whether or not they satisfy the conditions) from either the left or right table. The other side that does not meet join conditions is filled with **NULL**.
- Example:

```
postgres=# SELECT e.staff_id, e.higest_degree, t.score FROM
education e FULL JOIN training t ON (e.staff_id = t.staff_id);

STAFF_ID    HIGEST_DEGREE SCORE
----------- ------------------- ------------
   11          master              92
   12          doctor              95
   13          scholar
                                   90
```



37 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- FULL [OUTER] JOIN returns all the joined rows, plus one row for each unmatched left-hand row (extended with **NULL** on the right) and one row for each unmatched right-hand row (extended with **NULL** on the left).

Data Sorting

- **ORDER BY** clause
  - The **ORDER BY** clause is used to sort rows returned by a query by a specified column. If there is no **ORDER BY**, the same query may retrieve data organized in different orders.
- Syntax

```
ORDER BY { column_name | number | expression } [ ASC | DESC ][ NULLS FIRST | NULLS LAST ]
[ , ... ]
```

- Usage
  - By default, the **ORDER BY** clause sorts records in ascending order. For descending order, use the keyword **DESC**.
  - The **NULLS FIRST | NULLS LAST** keyword specifies the position of **NULL** values in the **ORDER BY** column. **FIRST** indicates that **NULL** values are placed before non-**NULL** values and **LAST** indicates that **NULL** values are placed after non-**NULL** values. If this keyword is not specified, **NULLS FIRST** is the default for **ASC** and **NULLS LAST** is the default for **DESC**.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- column_name | number | expression
- **ORDER BY** can be followed by a column name, number, or expression.
  - Number: **ORDER BY 1** indicates that query results are sorted by the first field in the query selection column.
  - Expression: **ORDER BY (column1+column2)** indicates that sorting is based on the sum of column 1 and column 2.

Example

- Query the names, amounts, and target people of insurances with insurance numbers higher than 2 in descending order.

```
postgres=# SELECT i_name,i_amount,i_person FROM insurance WHERE i_id>2 ORDER BY i_amount
DESC;
i_name                              |    i_amount          |   i_person
------------------------------------+----------------------+------------------
Accident Insurance                  |     5000             |  all
Medical Insurance                   |     2000             |  all
Loss of Property Insurance          |     1500             |  Middle-aged
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Limit

- The data limit function involves two independent clauses: **LIMIT** and **OFFSET**.
  - The **LIMIT** clause allows users to limit the rows returned by a query.

    ```
    LIMIT { count | ALL }
    ```

  - The **OFFSET** clause sets the position from which the return begins.

    ```
    OFFSET start
    ```

- Usage
  - **start**: Specifies the number of rows to be skipped before returning rows.
  - **count**: Specifies the maximum number of rows to be returned.
  - When both **start** and **count** are specified, the *start* rows are skipped before the *count* rows to be returned are calculated.
  - **LIMIT 5,20** is equivalent to **LIMIT 20 OFFSET 5** and **OFFSET 5 LIMIT 20**.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **LIMIT 5,20**, **LIMIT 20 OFFSET 5**, and **OFFSET 5 LIMIT 20** indicate that 20 rows of records are output after 5 rows are skipped.

Example

- Query the insurance information in the following table. Add **LIMIT 2 OFFSET 1** to skip the first row and query a total of two rows.

```
postgres=# SELECT i_name, i_id, i_amount, i_person FROM insurance LIMIT 2 OFFSET 1;

i_name                       |        i_id         | i_amount   |  i_person
-----------------------------+---------------------+------------+---------------
Life Insurance               |          2          |  3000      | Seniors
Accident Insurance           |          3          |  5000      |  All
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-choice) Which of the following logical expressions is used to search for records whose position is an engineer and salary is greater than 6000?
   A. Position = 'Engineer' OR Salary > 6000
   B. Position = Engineer AND Salary > 6000
   C. Position = Engineer OR Salary > 6000
   D. Position = 'Engineer' AND Salary > 6000
2. (Single-choice) In a **WHERE** clause, the expression "age between 20 and 30" is equivalent to.
   A. age >= 20 and age <= 30
   B. age >= 20 or age <=30
   C. age > 20 and age < 30
   D. age > 20 and age < 30

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - D
  - A

Quiz

3. (Multiple-choice) Assume that you want to query student names, ages, and scores from the **student** table, and want the results sorted in descending order by age. If there are students of the same age, they are sorted in ascending order by score instead. Which of the following SQL statements can be used?

   A. SELECT name, age, score FROM student order by age desc, score;

   B. SELECT name, age, score FROM student order by age, score asc;

   C. SELECT name, age, score FROM student order by 2 desc, 3 asc;

   D. SELECT name, age, score FROM student order by 1 desc, 2;

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - AC

Contents

1. SQL Syntax Quick Start
2. **SQL Syntax Classification**
   - Data Query
   - Data Update
   - Data Definition
3. Operators and Common Functions

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Inserting Data

- Description
  - **INSERT** inserts data into a table.
- Precautions
  - You must have the **INSERT** permission for a table in order to insert into it.
  - You must have the **SELECT** permission for the table in order to use the **RETURNING** clause.
  - If **ON DUPLICATE KEY UPDATE** is used, you must have the **SELECT** and **UPDATE** permissions for the table and the **SELECT** permission for the unique constraint (primary key or unique index).
  - If you use the **query** clause to insert rows from a query, you need to have the **SELECT** permission for any table or column used in the query.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Precautions
  - To run this statement, you must have the **INSERT** permission for the table or the **INSERT ANY TABLE** system permission. Common users are not allowed to insert objects of user **SYS**.
  - If **INSERT...SELECT** is used, the number of columns in *select_list* must be the same as that of columns to be inserted.
  - The data size of a single record must be less than 64,000 bytes.

Inserting Data

- Syntax
  - There are three types of the **INSERT** statements.
  - Insert values. Create a row of records, and insert them into a table.

```
INSERT [hint_info] [IGNORE] [ INTO ] [ schema_name. ]table_name [ ( column_name [ , ... ] ) ]
VALUES ( expression [ , ... ] )
```

  - Insert a query. Create one or more rows of records based on the result set returned by **SELECT** and insert the records into a table.

```
INSERT [IGNORE] [ INTO ] [ schema_name. ]table_name [table_alais][ ( column_name [ , ... ] )
]select_clause
```

  - Insert records into a table. If primary key conflicts are reported, values in the specified column are updated.

```
INSERT [ INTO ] [ schema_name. ]table_name [ ( column_name [ , ... ] ) ]  VALUES ( expression [ ,
... ] ) ON DUPLICATE KEY UPDATE {column_name = expression} [ , ... ]
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Parameter Description
- IGNORE
  - Ignores records that cause the error of repeated keywords. This parameter cannot be used together with **ON DUPLICATE KEY UPDATE**.
- table_name
  - Specifies the name of the table into which data is inserted.
- column_name
  - Specifies the name of the column into which data is inserted.
  - If data is to be inserted into all columns in a table, column names can be omitted in the **INSERT** statement.
  - Value range: an existing column name
- expression
  - Specifies the value or expression of the column into which data is to be inserted.
- select_clause
  - Specifies the **SELECT** clause that generates the records to be inserted. For details, see **SELECT** parameters in this document.
- select_list
  - Specifies the column to be queried.

Example

- Insert data into the **bank_card** table.
  - Create a table named **bank_card**.

```
postgres=# CREATE TABLE bank_card(b_number NCHAR(30) PRIMARY KEY, b_type NCHAR(20),
b_c_id INT NOT NULL);
```

  - Insert a value. Insert a record into the **bank_card** table.

```
postgres=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000001',
'Credit Card',1);
```

  - Insert all data from the **bank_card** table into the **bank_card1** table using a subquery.

```
postgres=# INSERT INTO bank_card1 SELECT * FROM bank_card;
```

  - Insert a record and update a value for columns in which this primary key is a duplicate.

```
INSERT INTO bank_card VALUES ('6222021302020000001', 'Credit Card',1) ON DUPLICATE KEY
UPDATE b_type = 'Savings Card';
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Updating Data

- Description
  - **UPDATE** updates row values in a table.
- Precautions:
  - You must have the **UPDATE** permission for tables to be updated.
  - You must have the **SELECT** permission for all tables involved in the expressions or conditions.
  - The **RETURNING** clause is currently not supported for column-store tables.
  - Column-store tables cannot be updated if the results are uncertain. If you update one row of data with multiple rows of data in a column-store table, an error will be reported.
  - Memory that holds old records is not recycled after a column-store table is updated. You need to clean it by executing **VACUUM FULL table_name**.
  - **UPDATE** is currently not supported for column-store replication tables.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Updating Data

- Syntax

```
UPDATE table_reference SET { [col_name = expression] [ , ... ] | (col_name[,...]) = (SELECT
expression[,...]) } [ WHERE condition ]
```

  - **table_reference** clause

```
{   [ schema_name. ] table_name
  | join_table
}
```

  - **join_table** clause

```
table_reference [LEFT [OUTER] | RIGHT [OUTER] | INNER ] JOIN table_reference ON
conditional_expr
```

  (col_name[,...]) = (expression[,...]) can be used only when the **join_table** clause is used.

  - Example: Update the record whose customer ID is 1 in the **bank_card** table and change the value of **b_type** to **Credit Card**.

```
postgres=# UPDATE bank_card SET b_type = 'Credit Card' WHERE b_c_id=1;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- table_reference
  - Specifies the table or table set to be updated. Value range: an existing table or table set
- table_name
  - Specifies the name of the table to be updated. Value range: an existing table name
- col_name
  - Specifies the name of the column to be modified. Value range: an existing column name
- expression
  - Specifies the value or expression assigned to a column.
- condition
  - Specifies an expression that returns a value of the Boolean type. Only rows for which the expression returns **true** will be updated.
- join_table
  - Specifies a set of tables used for join queries.
  - INNER JOIN returns records that have matched values in both tables.
  - LEFT [OUTER] JOIN returns all records from the left table and the matched records from the right table. If no match is found, null values are returned.
  - RIGHT [OUTER] JOIN returns all records from the right table and the matched records from the left table. If no match is found, null values are returned.

Deleting Data

- Description
  - **DELETE** deletes records from a table.
- Precautions
  - You must have the **DELETE** permission for the table.
  - You must have the **SELECT** permission for any table in the **USING** clause or whose values are read in **condition**.
  - The **RETURNING** clause is currently not supported for column-store tables.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deleting Data

- **Syntax**

```
DELETE FROM [ schema_name. ]table_name
 [ WHERE condition ]
 [ ORDER BY  { column_name  [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ] } [ , ... ] ]
 [ LIMIT [ start, ] count
    | LIMIT count OFFSET start
    | OFFSET start[ LIMIT count ] ]
```

Delete the records from a table if they match the records in another table.

```
DELETE table_ref_list FROM join_table
```

Or

```
DELETE FROM table_ref_list USING join_table
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **table_ref_list** clause:
  - [schema_name.]table_name
- **join_table** clause:
  - table_reference [LEFT [OUTER] | RIGHT [OUTER] | INNER] JOIN table_reference ON conditional_expr
- **table_reference** clause:

  { [schema_name.]table_name [[AS] alias]

  | [schema_name.]view_name [[AS] alias]

  | (SELECTquery) [[AS] alias]

  | join_table

  }
- Parameter Description
  - [schema_name.]table_name
    - Specifies the name of the table from which data is to be deleted.
  - condition
    - Specifies the condition for deleting data.

Example

- Delete the records, whose **b_c_id** is **10** and whose bank card type is credit card, from the **bank_card** table.
  - Delete the **bank_card** table.

```
postgres=# DROP TABLE IF EXISTS bank_card;
```

  - Create a table named **bank_card**.

```
postgres=# CREATE TABLE bank_card( b_number NCHAR(30) PRIMARY KEY, b_type NCHAR(20),b_c_id INT NOT NULL);
```

  - Insert a record into the **bank_card** table.

```
postgres=# INSERT INTO bank_card(b_number, b_type, b_c_id) VALUES ('6222021302020000021','Savings Card', 30);
postgres=# INSERT INTO bank_card(b_number, b_type, b_c_id) VALUES ('6222021302020000022','Savings Card', 31);
postgres=# INSERT INTO bank_card(b_number, b_type, b_c_id) VALUES ('6222021302020000023','Savings Card', 32);
```

  - Delete the records, whose **b_type** is **Credit Card** and **b_c_id** is **10**, from the **bank_card** table.

```
postgres=# DELETE FROM bank_card WHERE b_type='Credit Card' AND b_c_id=10;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-choice) Which of the following SQL commands can be used to increase the value of the **AGE** column in the **STAFFS** table by five years?
   A. UPDATE SET AGE WITH AGE+5
   B. UPDATE AGE WITH AGE+5
   C. UPDATE STAFFS SET AGE = AGE+5
   D. UPDATE STAFFS AGE WITH AGE+5
2. (Single-choice) Which of the following four groups of SQL commands belong to the DML commands?
   A. CREATE, DROP, UPDATE
   B. INSERT, UPDATE, DELETE
   C. INSERT, DROP, ALTER
   D. UPDATE, DELETE, ALTER

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - C
  - B

Quiz

3. (Single-choice) Which of the following SQL statements will delete records for all students whose class ID (specified by **cid**) is 6 from the **student** table?
   A. delete FROM student where cid = 6;
   B. delete * FROM student where cid = 6;
   C. delete FROM student on cid = 6;
   D. delete * FROM student on cid = 6;

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answer:
  - A

Contents

1. SQL Syntax Quick Start
2. **SQL Syntax Classification**
   - Data Query
   - Data Update
   - Data Definition
3. Operators and Common Functions

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

DDL Classification

- Database objects are part of what makes up a database. They include tables, indexes, views, stored procedures, default values, rules, triggers, users, and functions.
- Three DDL statements are used to define, modify, and remove objects in a database:
  - **CREATE** is used to create database objects.
  - **ALTER** is used to modify the attributes of database objects.
  - **DROP** is used to delete database objects.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Database·objects·are·the·parts·that·make·up·a·database.·Database·objects·include·tables,·indexes,·views,·stored·procedures,·default·values,·rules,·triggers,·users,·and·functions.
- Table
  - A table is a special data structure in a database and is used to store data objects and relationships between data objects. A table consists of rows and columns.
- Index
  - An index is a sequence of values in one or more columns in a database table. It is a data structure that improves the speed of data access to specific information in a database table.
- View
  - A view is a virtual table exported from one or more base tables, and is used to control user access to data.
- Stored procedure
  - A stored procedure is a set of SQL statements used for specific functions. Generally, it is used for report statistics and data migration.
- Default value
  - A default value is to assign a preset value to a column or column data item that does not have a specific value when the column is created or data is inserted into a table.

Defining a Database

- A database is the warehouse for organizing, storing, and managing data. Defining a database includes creating a database, modifying database attributes, and deleting a database.

Function	SQL Statement
Creating a database	CREATE DATABASE
Modifying database attributes	ALTER DATABASE
Deleting a database	DROP DATABASE

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Rule
  - A rule restricts the data in a database table. It defines the columns of a table.
- Trigger
  - A trigger acts as a special stored procedure that is triggered by a specified event. It is generally used for auditing and backing up data.
- Function
  - Functions encapsulate service logic to implement specific functions. After the functions are executed, corresponding results are returned.

Creating a Database

- Syntax

```
CREATE DATABASE database_name
   [ [ WITH ] { [ OWNER [=] user_name ] |
   [ TEMPLATE [=] template ] |
   [ ENCODING [=] encoding ] |
   [ LC_COLLATE [=] lc_collate ] |
   [ LC_CTYPE [=] lc_ctype ] |
   [ DBCOMPATIBILITY [=] compatibilty_type ] |
   [ TABLESPACE [=] tablespace_name ] |
   [ CONNECTION LIMIT [=] connlimit ]}[...] ];
```

- Note
  - Only system administrators or users with the **CREATEDB** permission can create a database.
  - **CREATE DATABASE** cannot be executed within a transaction block.
  - If an error message similar to "could not initialize database directory" is displayed during database creation, permissions or disk space for the file system directory may be insufficient.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Parameter Description
  - database_name
    - Specifies the database name.
  - OWNER [=] user_name
    - Specifies the owner of the new database. By default, the owner of the database is the current user.
  - TEMPLATE [=] template
    - Specifies a template name, that is, the template from which the database is created. openGauss creates a database by copying data from a template database. openGauss has two default template databases **template0** and **template1** and a default user database **postgres**.
  - ENCODING [=] encoding
    - Specifies the encoding format used by the new database. The value can be a string (for example, **SQL_ASCII**) or an integer. By default, the encoding format of the template database is used. The encoding formats of the template databases **template0** and **template1** depend on the OS. The encoding format of **template1** cannot be changed. If you need to change the encoding format when creating a database, use **template0**. Common values : **GBK**, **UTF8**, and **Latin1**

Example

- Create a database user named **jim**.

```
postgres=# CREATE USER jim PASSWORD 'Bigdata@123';
postgres=# CREATE USER jim01 PASSWORD 'Bigdata@123';
```

- Create a tablespace.

```
postgres=# CREATE TABLESPACE tablespace01 RELATIVE LOCATION 'tablespace01/tablespace01';
```

- Create database **music** using **template0** and specify user **jim** as its owner.

```
postgres=# CREATE DATABASE music OWNER jim TEMPLATE template0;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- A partitioned table is a logical table that is divided into several physical partitions for storage based on a specific plan.

Modifying a Database

- **ALTER DATABASE** modifies a database, including its name, owner, connection limitation, and object isolation.
- Syntax

```
ALTER DATABASE database_name
    [ [ WITH ] [ CONNECTION LIMIT connlimit ]|
      [ RENAME TO new_name ]|
      [ OWNER TO new_owner ]|
      [ SET TABALSPACE new_tablespace]|
      [...] ];
```

- Note
  - To modify the name of a database, you must have the **CREATEDB** permission.
  - To modify the owner of a database, you must be the database owner or system administrator and a member of the new owner role, with the **CREATEDB** permission.
  - To modify the default tablespace of a database, you must have the **CREATE** permission to create a tablespace. This statement physically migrates tables and indexes in a default tablespace to a new tablespace. Note that tables and indexes outside the default tablespace are not affected.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example

- Change the maximum number of connections to database **music**.

```
postgres=# ALTER DATABASE music WITH CONNECTION LIMIT 30;
```

- Rename the database.

```
postgres=# ALTER DATABASE music RENAME TO music01;
```

- Change the default tablespace of the database.

```
postgres=# ALTER DATABASE music01 SET TABLESPACE tablespace01;
```

- Change the database owner.

```
postgres=# ALTER DATABASE music01 OWNER TO jim01;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deleting a Database

- Syntax

```
DROP DATABASE [ IF EXISTS ] database_name;
```

- Example:
  - Delete database **music**.

```
DROP DATABASE music01;
```

  - Delete a user.

```
DROP USER jim;
DROP USER jim01;
```

  - Delete a tablespace.

```
DROP TABLESPACE tablespace01;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- IF EXISTS
  - Reports a notice instead of an error if a specified database does not exist.
- database_name
  - Specifies the name of the database to be deleted.
- Note
  - Only the owner of a database or the system administrator has the **DROP DATABASE** permission.
  - The preinstalled **POSTGRES**, **TEMPLATE0**, and **TEMPLATE1** databases are protected and therefore cannot be deleted.
  - The database cannot be deleted while users are connected to a database. To check the current database connections, open the **v$session** view.
  - **DROP DATABASE** cannot be executed within a transaction block.

Defining a Schema

- A schema is a set of database objects and is used to control the access to the database objects.

Function	SQL Statement
Creating a schema	CREATE SCHEMA
Modifying schema attributes	ALTER SCHEMA
Deleting a schema	DROP SCHEMA

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Creating a Schema

- Syntax
  - Create a schema based on a specified name.

```
CREATE SCHEMA schema_name
[ AUTHORIZATION user_name ] [ schema_element [ ... ] ];
```

  - Create a schema based on a username.

```
CREATE SCHEMA AUTHORIZATION user_name [ schema_element [ ... ] ];
```

- Note
  - Only a user with the **CREATE** permission for the current database can perform this operation.
  - The owner of an object created by the system administrator in a schema with the same name as a common user is that common user, not the system administrator.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Named objects are accessed either by "qualifying" their names with the schema name as a prefix, or by setting a search path that includes the desired schema. When creating named objects, you can also use the schema name as a prefix.
- Optionally, **CREATE SCHEMA** can include sub-commands to create objects within the new schema. The sub-commands are treated essentially the same as separate commands issued after creating the schema. If the **AUTHORIZATION** clause is used, all the created objects are owned by this user.
- Parameter Description
  - AUTHORIZATION user_name
    - Specifies the owner of a schema. If **schema_name** is not specified, the value of **user_name** will be used as the schema name. In this case, the value of **user_name** can only be a role name.
  - schema_element
    - Specifies an SQL statement defining an object to be created within the schema that is created by sub-commands. Currently, only the **CREATE TABLE**, **CREATE VIEW**, **CREATE INDEX**, **CREATE PARTITION**, and **GRANT** clauses are supported.
    - Created objects are owned by the user specified by **AUTHORIZATION**.
- If objects in the schema on the current search path have the same name, specify the schemas for different objects. You can run **SHOW SEARCH_PATH** to check the schemas on the current search path.

Example

- Create a role named **role1**.

```
postgres=# CREATE ROLE role1 IDENTIFIED BY 'Bigdata@123';
```

- Create a schema named **role1** for the **role1** role. The owner of the **films** and **winners** tables is **role1**, as they are created by using sub-commands under the **role1** schema.

```
postgres=# CREATE SCHEMA AUTHORIZATION role1
CREATE TABLE films (title text, release date, awards text[])
CREATE VIEW winners AS
SELECT title, release FROM films WHERE awards IS NOT NULL;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying a Schema

- Modifying schema attributes
  - Rename the schema.

    ```
    postgres=# ALTER SCHEMA role1 RENAME TO role1_new;
    ```

  - Create a user named **jack**.

    ```
    postgres=# CREATE USER jack PASSWORD 'Bigdata@123';
    ```

  - Change the owner of the schema.

    ```
    postgres=# ALTER SCHEMA role1_new OWNER TO jack;
    ```

- Note
  - Only the owner of a schema or the system administrator has the permission to run the **ALTER SCHEMA** statement.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deleting a Schema

- Deleting a schema from a database
- Syntax

```
DROP SCHEMA [ IF EXISTS ] schema_name [, ...] [ CASCADE | RESTRICT ];
```

- Example: Delete user **jack** and schema **role1_new**.

```
postgres=# DROP SCHEMA role1_new;
postgres=# DROP USER jack;
```

- Note
  - Only the owner of a schema or the system administrator has the **DROP SCHEMA** permission.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- IF EXISTS
  - Reports a notice instead of an error if a specified schema does not exist.
- schema_name
  - Specifies the schema name.
- CASCADE | RESTRICT
  - **CASCADE**: Automatically deletes all the objects contained in the schema.
  - **RESTRICT**: Refuses to delete the schema if the schema contains objects. This is the default action.
- Schemas beginning with **pg_temp** or **pg_toast_temp** are for internal use. Do not delete them. You may incur unexpected consequences.

Defining a Table

- A table is a special data structure in a database and is used to store data objects and relationships between data objects. The following table lists the involved SQL statements.

Function	SQL Statement
Creating a table	CREATE TABLE
Modifying table attributes	ALTER TABLE
Deleting a table	DROP TABLE
Deleting all data from a table	TRUNCATE TABLE

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Creating a Table

- Syntax

```
CREATE [ [ GLOBAL | LOCAL ] [ TEMPORARY | TEMP ] | UNLOGGED ] TABLE [ IF NOT EXISTS ]
table_name ({ column_name data_type [ compress_mode ] [ COLLATE collation ] [
column_constraint [ ... ] ]
| table_constraint
| LIKE source_table [ like_option [...] ] } [, ... ])
[ WITH ( {storage_parameter = value} [, ... ] ) ]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ COMPRESS | NOCOMPRESS ]
[ TABLESPACE tablespace_name ];
```

- Note
  - Column-store tables support only **PARTIAL CLUSTER KEY** table-level constraints, but do not support primary and foreign key table-level constraints.
  - Only the **NULL**, **NOT NULL**, and **DEFAULT** constant values can be used as column-store table constraints.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **column_constraint** is set to **[CONSTRAINT constraint_name] { NOT NULL | NULL | CHECK (expression) | DEFAULT default_expr | UNIQUE index_parameters | ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = column_encryption_key, ENCRYPTION_TYPE = encryption_type_value) | PRIMARY KEY index_parameters | REFERENCES reftable [(refcolumn)] [MATCH FULL | MATCH PARTIAL | MATCH SIMPLE] [ON DELETE action] [ON UPDATE action] } [DEFERRABLE | NOT DEFERRABLE | INITIALLY DEFERRED | INITIALLY IMMEDIATE]**.
- **compress_mode** of a column is set to **{ DELTA | PREFIX | DICTIONARY | NUMSTR | NOCOMPRESS }**.
- **table_constraint** is set to **[CONSTRAINT constraint_name] { CHECK (expression) | UNIQUE (column_name [, ...]) index_parameters | PRIMARY KEY (column_name [, ...]) index_parameters | FOREIGN KEY (column_name [, ...]) REFERENCES reftable [(refcolumn [, ...])] [MATCH FULL | MATCH PARTIAL | MATCH SIMPLE] [ON DELETE action] [ON UPDATE action] | PARTIAL CLUSTER KEY (column_name [, ...]) } [DEFERRABLE | NOT DEFERRABLE | INITIALLY DEFERRED | INITIALLY IMMEDIATE]**.
- **like_option** is set to **{ INCLUDING | EXCLUDING } { DEFAULTS | CONSTRAINTS | INDEXES | STORAGE | COMMENTS | PARTITION | RELOPTIONS | ALL }**.

Example

- Create a table named **bank_card**.

```
postgres=# CREATE TABLE bank_card1( b_number NCHAR(30) PRIMARY KEY, b_type
NCHAR(20),b_c_id INT NOT NULL);
```

- Create a temporary table.

```
postgres=# CREATE TEMPORARY TABLE bank_card2(b_number NCHAR(30) PRIMARY KEY, b_type
NCHAR(20),b_c_id INT NOT NULL);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying Table Attributes

- **ALTER TABLE** changes the definition of a table by changing, adding, or deleting columns and constraints. With **ALTER TABLE**, you can:
  - Add, delete, modify, or rename columns.
  - Add or delete constraints.
  - Enable or disable constraints.
  - Change the name of a partition.
  - Modify partition tablespaces.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying Table Attributes

- Syntax

```
ALTER TABLE [ schema_name. ]table_name
{ alter_table_properties
| column_clauses
| references_clause
| constraint_clauses
| partition_clauses
| enable_disable_clause
| set_interval_clause
}
```

- Note
  - To run this statement, you must have the **ALTER ANY TABLE** system permission. Common users cannot modify objects of system users.
  - When you add a column to a table, ensure that there is no record in the table.
  - When you modify a column, ensure that the all the values in this column are **NULL**.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example

- Add the **full_masks** column to the **Bank_card** table.

```
postgres=# ALTER TABLE bank_card ADD full_masks INTEGER;
```

- Change the data type of a column.

```
postgres=# ALTER TABLE bank_card MODIFY full_masks NVARCHAR2(20);
```

- Add a constraint.

```
postgres=# ALTER TABLE bank_card ADD CONSTRAINT ck_bank_card CHECK(b_c_id>0);
postgres=# ALTER TABLE bank_card ADD CONSTRAINT uk_bank_card UNIQUE(full_masks);
```

- Delete the **full_masks** column.

```
postgres=# ALTER TABLE bank_card DROP full_masks;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **UNIQUE** indicates a unique constraint. The column value of the unique constraint column cannot be duplicate, but can be **NULL**.
- **CHECK** creates a **CHECK** constraint by using any logical (Boolean) expression that returns **TRUE** or **FALSE** based on logical operators. For example, **CHECK (salary BETWEEN 1000 AND 5000)** indicates that the value in the **salary** column must range from 1000 to 5000.

Deleting a Table

- Syntax

```
DROP TABLE [ IF EXISTS ] { [schema.]table_name } [, ...] [ CASCADE | RESTRICT ];
```

- Example

```
postgres=# DROP TABLE IF EXISTS training;
```

- Note
  - **DROP TABLE** forcibly deletes a specified table and the indexes depending on the table. After the table is deleted, no functions or stored procedures that need to use this table can be executed.
  - Deleting a partitioned table also deletes all partitions in the table.
  - Only the table owner or a user with the **DROP** permission can run **DROP TABLE**. The system administrator has this permission by default.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Parameter Description
  - CASCADE | RESTRICT
    - **CASCADE**: Automatically deletes all objects (such as views) that depend on the table.
    - **RESTRICT**: Refuses to delete the table if any objects depend on it. This is the default value.
  - **IF EXISTS**: Indicates whether the table exists. If it does, it is deleted. If it does not, no error is reported during the deletion.
  - A foreign key is used to associate with other tables. A foreign key is the primary key of another table. Foreign keys can be duplicate or empty.

Defining a Partitioned Table

- openGauss supports range partitioning, list partitioning, and hash partitioning.
- A partitioned table has the following advantages over an ordinary table:
  - High query performance: Users can specify partitions when querying a partitioned table, improving query efficiency.
  - High availability: If a partition in a partitioned table is faulty, data in the other partitions is still available.
  - Easy maintenance: If a partition in a partitioned table is faulty, only this partition needs to be repaired.
  - Balanced I/O: Partitions can be mapped to different disks to balance I/O and improve the overall system performance.

Function	SQL Statement
Creating a partitioned table	CREATE TABLE PARTITION
Modifying partitioned table attributes	ALTER TABLE PARTITION
Deleting a partitioned table	DROP TABLE

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- openGauss supports range partitioning, list partitioning, and hash partitioning.
  - Range partitioning: Data within a certain range is mapped to each partition. The range is determined by the partition key specified when the partitioned table is created. This partitioning mode is most commonly used. The partition key is usually a date. For example, sales data is partitioned by month.
  - List partitioning: Key values contained in the data are stored in different partitions, and the data is mapped to each partition in sequence. The key values contained in the partitions are specified when the partitioned table is created.
  - Hash partitioning: Data is mapped to each partition based on an internal hash algorithm. The number of partitions is specified when the partitioned table is created.

Creating a Partitioned Table

- Syntax

```
CREATE TABLE [ IF NOT EXISTS ] partition_table_name
( [
    { column_name data_type [ COLLATE collation ] [ column_constraint [ ... ] ]
    | table_constraint
    | LIKE source_table [ like_option [...] ] }[, ... ]
] )
    [ WITH ( {storage_parameter = value} [, ... ] ) ]
    [ COMPRESS | NOCOMPRESS ]
    [ TABLESPACE tablespace_name ]
   PARTITION BY {
            {RANGE (partition_key) [ INTERVAL ('interval_expr') [ STORE IN (tablespace_name [, ... ] ) ] ]
( partition_less_than_item [, ... ] )} |
            {RANGE (partition_key) [ INTERVAL ('interval_expr') [ STORE IN (tablespace_name [, ... ] ) ] ] (
partition_start_end_item [, ... ] )} |
            {LIST | HASH (partition_key) (PARTITION partition_name [VALUES (list_values_clause)] opt_table_space )}
     } [ { ENABLE | DISABLE } ROW MOVEMENT ];
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- partition_table_name
  - Specifies the name of a partitioned table.
- TABLESPACE tablespace_name
  - Specifies that the new table will be created in the **tablespace_name** tablespace. If the tablespace is not specified, the default tablespace is used.
- PARTITION BY RANGE(partition_key)
  - Creates a range partition. **partition_key** is the name of the partition key.
- { ENABLE | DISABLE } ROW MOVEMENT
  - Specifies whether to enable row movement. If the tuple value is updated on the partition key during the UPDATE operation, the partition where the tuple is located is altered. Setting this parameter enables error messages to be reported or movement of the tuple between partitions.
- Note
  - A partitioned table supports unique and primary key constraints. The constraint keys of these constraints must contain all partition keys.
  - Currently, hash partitioning and list partitioning support only single-column partitioning, and do not support multi-column partitioning.
  - When you have the **INSERT** permission for an interval partitioned table, partitions can be automatically created when you run **INSERT** to write data to the table.

Example

- Create a tablespace.

```
postgres=# CREATE TABLESPACE example1 RELATIVE LOCATION 'tablespace1/tablespace_1';
postgres=# CREATE TABLESPACE example2 RELATIVE LOCATION 'tablespace2/tablespace_2';
postgres=# CREATE TABLESPACE example3 RELATIVE LOCATION 'tablespace3/tablespace_3';
postgres=# CREATE TABLESPACE example4 RELATIVE LOCATION 'tablespace4/tablespace_4';
```

- Create a partitioned table.

```
postgres=# CREATE TABLE training(staff_id INTEGER NOT NULL, course_name CHAR(20), course_period
DATETIME,
exam_date DATETIME, score INTEGER)
TABLESPACE example1
PARTITION BY RANGE(staff_id)
(
   PARTITION training1 VALUES LESS THAN(100),
   PARTITION training2 VALUES LESS THAN(200),
   PARTITION training3 VALUES LESS THAN(300),
   PARTITION training4 VALUES LESS THAN(MAXVALUE) TABLESPACE example2
)
ENABLE ROW MOVEMENT;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **IF EXISTS**: Indicates whether the table exists. If it does, it is deleted. If it does not, no error is reported during the deletion.
- A foreign key is used to associate with other tables. A foreign key is the primary key of another table. Foreign keys can be duplicate or empty.

Modifying a Partitioned Table

- Syntax

```
ALTER TABLE [ IF EXISTS ] { table_name [*] | ONLY table_name | ONLY ( table_name )} action
[, ... ];
```

- Example
  - Modify the tablespace of the **training1** partition.

```
postgres=# ALTER TABLE training MOVE PARTITION training1 TABLESPACE example3;
```

  - Rename the **training1** partition to **training5**.

```
postgres=# ALTER TABLE training RENAME PARTITION training1 TO training5;
```

  - Merge the **training1** and **training2** partitions into **training3**.

```
postgres=# ALTER TABLE training MERGE PARTITION training1,training2 INTO PARTITION training3;
```

  - Delete the **training4** partition.

```
postgres=# ALTER TABLE training DROP PARTITION training4;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- **action** indicates the following clauses for maintaining partitions. For partition continuity, when multiple clauses are used for partition maintenance, openGauss performs **DROP PARTITION** and then **ADD PARTITION**, and finally runs the rest clauses in sequence: move_clause | exchange_clause | row_clause | merge_clause | modify_clause | split_clause | add_clause | drop_clause.
- The **move_clause** syntax is used to move the partition to a new tablespace.
  - MOVE PARTITION { partion_name | FOR (partition_value [, ...]) } TABLESPACE tablespacename
- The **exchange_clause** syntax is used to move the data from an ordinary table to a specified partition.
- EXCHANGE PARTITION { (partition_name) | FOR (partition_value [, ...]) } WITH TABLE {[ONLY] ordinary_table_name | ordinary_table_name * | ONLY (ordinary_table_name)} [{ WITH | WITHOUT } VALIDATION] [VERBOSE]
- The **row_clause** syntax is used to set row movement of a partitioned table.
- The **merge_clause** syntax is used to merge partitions into one.
- The **modify_clause** syntax is used to set whether a partitioned index is available.
- The **split_clause** syntax is used to split one partition into different partitions.
- The **add_clause** syntax is used to add one or more partitions to a specified partitioned table.
- The **drop_clause** syntax is used to remove a partition from a specified partitioned table.

Deleting a Partitioned Table

- Syntax

```
DROP TABLE [ IF EXISTS ] { [schema.]table_name } [, ...] [ CASCADE | RESTRICT ];
```

- Example
  - Delete a partitioned table.

```
postgres=# DROP TABLE training;
```

  - Delete a tablespace.

```
postgres=# DROP TABLESPACE example1;
postgres=# DROP TABLESPACE example2;
postgres=# DROP TABLESPACE example3;
postgres=# DROP TABLESPACE example4;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Defining an Index

- An index is a sequence of values in one or more columns in a database table. It is a data structure that improves the speed of data access to specific information in a database table. The following table lists the involved SQL statements.

Function	SQL Statement
Creating an index	CREATE INDEX
Modifying index attributes	ALTER INDEX
Deleting an index	DROP INDEX

- Category:
  - Indexes are classified into single-column indexes and multi-column indexes.
  - Indexes are also classified into common indexes, unique indexes, function indexes, and partitioned indexes, depending on how they are used.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Indexes can greatly speed up SQL search. For example, through the index of a dictionary, we can quickly locate the required information in the dictionary.
- Take an employee table with 200,000 data records as an example. If we want to query information about the employee with an ID of 10000, and if no index is available, the entire table must be traversed until the row with an ID of 10000 is found. After an index is created for the ID, you can search through the index. Indexes have been optimized by algorithms, greatly reducing the number of searches. Therefore, indexes can be used to quickly access data.
  - Single-column index: An index is created only for one column.
  - Multi-column index: A multi-column index is also called a composite index. An index is created for multiple columns. The index is used only when the first column during index creation is used in the query condition. In openGauss, a multi-column index supports a maximum of 16 columns and the total length cannot exceed 3900 bytes. It is calculated based on data types with the maximum length.
  - Common index: A B-Tree index is created by default.
  - Unique index: An index whose column values or column value combinations are unique. When a table is created, a unique index is automatically created on primary keys.
  - Function-based index: An index is created based on functions.
  - Partitioned index: A partitioned index is independently created on a table partition. Deleting a partition does not influence the use of other partitioned indexes.

Creating an Index

- Description
  - **CREATE INDEX** creates an index in a specified table. Indexes are primarily used to improve database query performance. However, be aware that inappropriate use may compromise performance.
- Note
  - To run this statement, you must have the **CREATE INDEX** or **CREATE ANY INDEX** system permission. Common users cannot create system users' objects.
- Syntax

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [schema_name.]index_name ] ON table_name [
USING method ]
({ { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [ ASC | DESC ] [ NULLS { FIRST
| LAST } ] }[, ...] )
[ WITH ( {storage_parameter = value} [, ... ] ) ]
[ TABLESPACE tablespace_name ]
[ WHERE predicate ];
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- The procedure for creating an index on a partitioned table is different to that used to create an index in a table. The following describes how to create an index in a partitioned table.
  - CREATE [UNIQUE] INDEX [[schema_name.]index_name] ON table_name [USING method] ({{ column_name | (expression) } [COLLATE collation] [opclass] [ASC | DESC] [NULLS LAST] }[, ...]) [LOCAL [({ PARTITION index_partition_name [TABLESPACE index_partition_tablespace] } [, ...])] | GLOBAL] [WITH ({ storage_parameter = value } [, ...])] [TABLESPACE tablespace_name];
- IF NOT EXISTS
  - Specifies that no error occurs if an index already exists. If the index does not exist, create an index.
  - Indexes consume storage and computing resources. Creating too many indexes compromises database performance (especially the performance of data import. As such, you are advised to import the data before creating indexes). Create indexes only when they are necessary.
- UNIQUE
  - Creates a unique index. In this way, the system checks whether new values are unique in the index column. Attempts to insert or update data which would result in duplicate values in the index column will generate an error.

Creating an Index

- Create an index in the **bank_card** table.

```
-- Create the ordinary table bank_card.
postgres=# CREATE TABLE bank_card( b_number NCHAR(30) PRIMARY KEY, b_type NCHAR(20),b_c_id INT NOT
NULL);
-- Create the idx_bank_card index.
postgres=# CREATE INDEX idx_bank_card ON bank_card(b_number ASC, b_type);
```

- Create a partitioned index in the partitioned table **education**.

```
-- Create the education partitioned table.
postgres=# CREATE TABLE education(staff_id INTEGER NOT NULL, higest_degree CHAR(8), graduate_school
VARCHAR(64),graduate_date DATETIME, education_note VARCHAR(70))
PARTITION BY LIST(higest_degree)
(
PARTITION doctor VALUES ('Doctor'),
PARTITION master VALUES ('Master'),
PARTITION undergraduate VALUES ('Bachelor')
);
-- Create a partitioned index with the name of the index partition specified.
postgres=# CREATE INDEX idx_education ON education(staff_id ASC, higest_degree) LOCAL (PARTITION doctor,
PARTITION master, PARTITION undergraduate);
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Modifying Index Attributes

- Syntax:

```
ALTER INDEX [ schema_name. ]index_name
{   rebuild_clauses
  | rename_clauses
  | modify_clauses
}
```

- Notes:
  - rebuild_clauses
    - **REBUILD [PARTITION index_partition_name]**: Rebuilds a table index or an index partition.
  - rename_clauses
    - **RENAME TO [schema_name.] index_name_new**: Specifies the name of the index to be renamed.
  - modify_clauses
    - **SET TABLESPACE tablespace_name**: Modifies the tablespace to which a table index belongs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example

- Example:
  - Rebuild an index.

    ```
    postgres=# ALTER INDEX idx_bank_card REBUILD;
    ```

  - Rename the existing index.

    ```
    postgres=# ALTER INDEX idx_bank_card RENAME TO idx_bank_card_temp;
    ```

  - Set the index to be unavailable.

    ```
    postgres=# ALTER INDEX idx_bank_card UNUSABLE;
    ```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deleting an Index

- Syntax

```
DROP INDEX [ CONCURRENTLY ] [ IF EXISTS ] index_name [, ...] [ CASCADE | RESTRICT ];
```

- Parameter Description
  - IF EXISTS
    - Does not report an error if an index does not exist.
  - index_name
    - Specifies the name of the index to be deleted.
  - CASCADE | RESTRICTON
    - **CASCADE**: Automatically deletes all objects that depend on the index to be deleted.
    - **RESTRICT** (default): Refuses to delete the index if any objects depend on it.
- Example

```
postgres=# DROP INDEX idx_bank_card;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- CONCURRENTLY
  - Deletes an index without locking it. In general, during index deletion, access of other statements to the tables that the index depends on is blocked. With this option, the statement does not lock the table during index deletion.
  - This parameter allows only one index name and does not support **CASCADE**.
  - The **DROP INDEX** statement can be run within a transaction, but **DROP INDEX CONCURRENTLY** cannot.

Defining a View

- A view is a virtual table exported from one or more base tables, and is used to control user access to data. The following table lists the related SQL statements.

Function	SQL Statement
Creating a view	CREATE VIEW
Deleting a view	DROP VIEW

- Notes:
  - A view is different from the base table. A database only stores the definition of a view and not its data, which is still stored in the original base table.
  - If data in the base table changes, the data queried from the view changes accordingly.
  - In this sense, a view is like a window through which users can see relevant data and changes within the database.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Functions of views:
  - Operations are simplified, and frequently used data is defined as a view. When you perform a query, the query statement may be long if many operations are defined within it. If this action occurs frequently, you can create a view. In this case, you only need to use the **SELECT* FROM view** statement.
  - Views can provide security, as users can only query and modify the data that can be viewed. A view is virtual and does not exist physically. As it only stores a set of data, important column information in the base table can be excluded from the view and users can only access other data through it. In addition, users cannot modify or delete the view, which further ensures data security.
  - Views can ensure logical independence, which shields the impact of the structure of the real table. Views can decouple (to some extent) applications from database tables. If there is no view, applications must be created in a table. Programs can be built in the view, so that the program and the database table are separated by the view.

Creating a View

- Syntax

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ] VIEW view_name [ ( column_name [, ...] ) ]
[ WITH ( {view_option_name [= view_option_value]} [, ... ] ) ] AS query;
```

- Example:
  - Create the **privilege_view** view. If the view exists, update it.

```
postgres=# CREATE OR REPLACE VIEW privilege_view AS SELECT b_number, b_type FROM
bank_card;
```

  - Query the data in the view. The syntax is the same as that for querying a table.

```
postgres=# SELECT * FROM privilege_view;
```

  - Query the view structure.

```
postgres=# \d privilege_view;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- OR REPLACE
  - Updates a view if the view to be created exists.
- TEMP | TEMPORARY
  - Creates a temporary view.
- view_name
  - Specifies a view name.
- column_name
  - Specifies a list of optional names to be used for columns in the view. If not provided, column names are deduced from the query.
- AS query
  - Specifies the **SELECT** or **VALUES** statement that will provide the columns and rows of the view.

Deleting a View

- Syntax

```
DROP VIEW [ IF EXISTS ] view_name [, ...] [ CASCADE | RESTRICT ];
```

- Parameter Description
  - IF EXISTS
    - Deletes a view if it exists.
  - view_name
    - Specifies the view to be deleted.
  - CASCADE | RESTRICT
    - **CASCADE**: Automatically deletes all objects that depend on the view (such as other views).
    - **RESTRICT**: Refuses to delete the view if any objects depend on it. This is the default value.
- Example

```
postgres=# DROP VIEW IF EXISTS privilige_view;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Defining a Sequence

- A sequence can generate numbers with the same interval, and these are used as primary key values. When a sequence value is generated, the sequence is incremented. The following table lists the SQL statements involved.

Function	SQL Statement
Creating a sequence	CREATE SEQUENCE
Modifying sequence attributes	ALTER SEQUENCE
Deleting a sequence	DROP SEQUENCE

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- A sequence can automatically generate sequence numbers for rows in a table, as well as a group of values (of numeric type) with equal intervals. A sequence is mainly used to generate the primary key value of a table. It can be referenced in an INSERT statement, checked by a query, or increased to the next value.

Creating a Sequence

- Description
  - Adds a sequencer to the current database. The current user is the owner of the sequencer.
- Syntax

```
CREATE SEQUENCE name
    [ INCREMENT [ BY ] increment ]
    [ MINVALUE minvalue | NO MINVALUE | NOMINVALUE ] [ MAXVALUE maxvalue | NO MAXVALUE | NOMAXVALUE]
    [ START [ WITH ] start ] [ CACHE cache ] [ [ NO ] CYCLE | NOCYCLE ]
    [ OWNED BY { table_name.column_name | NONE } ];
```

- Precautions
  - A sequence is a special table that stores arithmetic progressions. It has no actual meaning and is usually used to generate unique identifiers for rows or tables.
  - If a schema name is given, the sequence is created in the specified schema; otherwise, it is created in the current schema. The sequence name must be different from the names of other sequences, tables, indexes, views in the same schema.
  - After the sequence is created, functions **nextval()** and **generate_series(1,N)** insert data to the table. Make sure that the number of times **nextval** is invoked is greater than or equal to N + 1. Otherwise, errors will be reported because the number of times the **generate_series()** function is invoked is N + 1.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **name** specifies the name of the sequence to be created.
- **increment** specifies the step for a sequence. A positive number generates an ascending sequence, and a negative number generates a decreasing sequence. The default value is **1**.
- **MINVALUE** (**minvalue**)|**NO MINVALUE**|**NOMINVALUE**: specifies the minimum value of a sequence. If **minvalue** is not specified, or **NO MINVALUE** is specified, the default value of the ascending sequence is **1**, and that of the descending sequence is **–263 – 1**. **NOMINVALUE** is equivalent to **NO MINVALUE**.
- **MAXVALUE** (**maxvalue**)|**NO MAXVALUE**|**NOMAXVALUE**: Specifies the maximum value of a sequence. If **maxvalue** is not specified, or **NO MAXVALUE** is specified, the default value of the ascending sequence is **263 – 1**, and that of the descending sequence is **–1**. **NOMAXVALUE** is equivalent to **NO MAXVALUE**.
- **start** specifies the start value of a sequence. The default value for an ascending sequence is **minvalue** and that for a descending sequence is **maxvalue**.
- **cache** specifies the number of sequences stored in the memory for quick access purposes. Default value **1** indicates that one value can be generated each time and there is no cache.
- **CYCLE** recycles sequences after the number of sequences reaches the value of **maxvalue** or **minvalue**. If **NO CYCLE** is specified, any invocation of **nextval** will return an error after the number of sequences reaches the maximum number. **NOCYCLE** is equivalent to **NO CYCLE**. If **CYCLE** is specified, the uniqueness of the sequence cannot be ensured.
- **OWNED BY** associates a sequence with a specified column included in a table. In this way, the sequence will be deleted when you delete its associated column or the table where the column belongs. The associated table and sequence must be owned by the same user and in the same schema. **OWNED BY** only establishes the association between a table column and the sequence. Sequences on the column do not increase automatically when data is inserted.
- The default value **OWNED BY NONE** indicates that such an association does not exist.

Example

- Create the **seq_auto_extend** sequence starting with **10**, and with **increment** set to **2**, **MAXVALUE** set to **200**, and **CYCLE** specified.

```
postgres=# CREATE SEQUENCE seq_auto_extend START WITH 10 MAXVALUE 200 INCREMENT BY 2 CYCLE;
```

- Obtain the next value of the sequence.

```
postgres=# SELECT nextval ('seq_auto_extend');
```

- Use the sequence for ID auto-increment.

```
postgres=# CREATE SEQUENCE serial1
 START 101
 CACHE 20
;

CREATE TABLE test (id number(6) default nextval ('serial1'), name varchar(20),constraint ts_id primary
key(id));
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- **CACHE** specifies the number of sequences stored in the memory for quick access purposes. Default value **1** indicates that one value can be generated each time and there is no cache.

Modifying Sequence Attributes

- Syntax
  - Modify the owning column of a sequence.

```
ALTER SEQUENCE [ IF EXISTS ] name
    [MAXVALUE maxvalue | NO MAXVALUE | NOMAXVALUE]
    [ OWNED BY { table_name.column_name | NONE } ] ;
```

  - Change the owner of a sequence.

```
ALTER SEQUENCE [ IF EXISTS ] name OWNED TO new_owner;
```

- Example
  - Modify the **seq_auto_extend** sequence.

```
-- Set INCREMENT BY to 4 and MAXVALUE to 400.
postgres=# ALTER SEQUENCE seq_auto_extend MAXVALUE 400 INCREMENT BY 4 CYCLE;
```

  - The column to which the **seq_auto_extend** sequence belongs is changed to **b_number**.

```
postgres=# ALTER SEQUENCE seq_auto_extend OWNED BY bank_card.b_number;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Parameter Description
  - **name** specifies the name of the sequence to be modified.
  - **IF EXISTS** reports a notice instead of an error when a sequence that does not exist is modified.
  - **OWNED BY** associates a sequence with a specified column included in a table. In this way, the sequence will be deleted when you delete its associated column or the table where the column belongs.
  - If the sequence has been associated with another table before you use this option, the new association will overwrite the old one.
  - The associated table and sequence must be owned by the same user and in the same schema.
  - If **OWNED BY NONE** is used, all existing associations will be deleted.
  - **new_owner** specifies the username of the new owner. To change the owner, you must also be a direct or indirect member of the new role, and this role must have **CREATE** permission for the sequence's schema.

Deleting a Sequence

- Syntax

```
DROP SEQUENCE [ IF EXISTS ] {[schema.]sequence_name} [ , ... ] [ CASCADE | RESTRICT ];
```

- Parameter Description
  - IF EXISTS

    Does not report an error if the sequence to be deleted does not exist.
  - l [schema_name.] sequence_name

    Specifies the sequence to be deleted.
  - **CASCADE** automatically deletes objects that depend on the sequence to be deleted.
  - **RESTRICT** refuses to delete the sequence if any objects depend on it. This is the default action.
- Example
  - Delete the **seq_auto_extend** sequence.

```
postgres=# DROP SEQUENCE IF EXISTS seq_auto_extend;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Only the sequence owner or a user granted with the **DROP** permission can delete a sequence. The system administrator has this permission by default.

Quiz

1. (True or false) When an index is created for a table, the corresponding base table is deleted if the index is canceled.

   A. True

   B. False

2. (Single-choice) SQL integrates data query, data manipulation, data definition, and data control. Which of the following functions is implemented by the CREATE, DROP, and ALTER statements?

   A. Data query B. Data manipulation C. Data definition

   D. Data control

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - B
  - C

Quiz

3. (Multiple-choice) Create a decremental sequence named **seq_1**. The starting value is 400, the step is –4, and the minimum value is 100. When the sequence reaches the minimum value, it can be cyclically executed. Which of the following statements are correct?
   A. CREATE SEQUENCE seq_1 START WITH 400 MAXVALUE 100 INCREMENT BY -4 CYCLE;
   B. CREATE SEQUENCE seq_1 MAXVALUE 400 MINVALUE 100 INCREMENT BY -4 CYCLE;
   C. CREATE SEQUENCE seq_1 START WITH 400 MINVALUE 100 INCREMENT BY -4 CYCLE;
   D. CREATE SEQUENCE seq_1 START WITH 400 MINVALUE 100 MAXVALUE 400 INCREMENT BY -4 CYCLE;

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers:
  - BD

Contents

1. SQL Syntax Quick Start
2. SQL Syntax Classification
3. **Operators and Common Functions**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Logical Operators

- Common logical operators include AND, OR, and NOT. The operation result can be **TRUE**, **FALSE**, or **NULL** (which means unknown). Their priorities are NOT > AND > OR.

Operator	Function
and	The logical AND operator can be used in query conditions, such as **WHERE**, **ON**, and **HAVING**.
or	The logical OR operator can be used in query conditions, such as **WHERE**, **ON**, and **HAVING**.
not	The **NOT** keyword can be added before the condition expression after the **WHERE** or **HAVING** clause, in order to reverse the condition result. This keyword is used together with the relational operation, for example, **not in** and **not exists**.

```
-- Search the bank_card table for the information about a credit card whose customer ID is 1.
postgres=# SELECT * FROM bank_card where b_type='Credit Card' and b_c_id=1;
-- Search the bank_card table for the information about a credit card whose customer ID is 1.
postgres=# SELECT * FROM bank_card where b_type='Credit Card' or b_c_id=1;
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Syntax of the NOT operator: **SELECT* FROM table where/having not {condition};**
- The operand must be of the Boolean type and can be **TRUE**, **FALSE**, or **NULL**. **NULL** indicates that the operand is unknown.
- **AND**: If all operands are **TRUE** and not **NULL**, **t** is returned. When at least one operand is **FALSE**, **f** is returned. Otherwise, **NULL** is returned.
- **OR**: If both operands are not **NULL** and at least one operand is **TRUE**, **t** is returned. Otherwise, **f** is returned. If one operand is **NULL** and the other operand is **TRUE**, **t** is returned. Otherwise, **NULL** is returned. If both operands are **NULL**, **NULL** is returned.
- **NOT**: When the operand is **TRUE**, **f** is returned. When the operand is **FALSE**, **t** is returned. When the operand is **NULL**, **NULL** is returned.

Comparison Operators

- Comparison operators are available for the most data types and return Boolean values.

Operator	Description
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
=	Equal to
<> or !=	Not equal to

```
-- Search the bank_card table for information about a credit card.
postgres=# SELECT * FROM bank_card where b_type!='Credit Card' ;
-- Search the bank_card table for information about a non-credit card.
postgres=# SELECT * FROM bank_card where b_type<>'Credit Card';
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Comparison operators are available for all relevant data types. All comparison operators are binary operators that return Boolean type values. Expressions like **1 < 2 < 3** are invalid (as there is no comparison operator to compare a Boolean value with 3).
- The return value is of the Boolean type and can be **t**, **f**, or **NULL**.
- Comparison operators are typically used in the query conditions of SELECT statements to construct filter condition expressions.

Character Processing Functions and Operators

- **instr(string1,string2,int1,int2)** returns the text from **int1** to **int2** in **string1**. The first **int** indicates the start position for matching, and the second **int** indicates the number of matching times.

```
postgres=# SELECT instr( 'abcdabcdabcd', 'bcd', 2, 2 );

Instr
-------
6
```

- **overlay(string placing string FROM int [for int])** replaces a substring. **FROM int** indicates the start position of the replacement in the first string. **for int** indicates the number of characters replaced in the first string.

```
postgres=# SELECT overlay('hello' placing 'world' FROM 2 for 3 );
overlay
---------
hworldo
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- String processing functions and operators provided by openGauss are for concatenating strings with each other, concatenating strings with non-strings, and matching the patterns of strings.
- **CONCAT**: The input parameter is a string or an expression that can be converted into a string, and the return value is a string. This function can be used to concatenate one or more strings, and multiple strings are separated by commas (,). The return value is the string generated by concatenating parameters. If a parameter is **NULL**, the **CONCAT** function will ignore this parameter. However, if **NULL** is enclosed in single quotation marks ('), the **CONCAT** function will process **NULL** as a string.
- **CONCAT_WS**: The input parameter is a string or an expression that can be converted to a string, and the return value is a string. If a parameter is **NULL**, the **CONCAT_WS** function will ignore this parameter. However, if **NULL** is enclosed in single quotation marks ('), the **CONCAT_WS** function will process **NULL** as a string.
- The **CONCAT** and **CONCAT_WS** functions returns a maximum of 8000 bytes. If the returned value exceeds 8000 bytes, an error is reported. The **CONCAT** and **CONCAT_WS** functions can be nested.
- **HEX**: The input parameter **p1** can be of the numeric or string type, and the return value is a string.
- **HEX2BIN**: The input hexadecimal string must be prefixed with 0x, and the return value is of the binary type.

Character Processing Functions and Operators

- **substring(string [FROM int] [for int])** extracts a substring. **FROM int** indicates the start position of the truncation. **for int** indicates the number of characters truncated.

```
postgres=# SELECT substring('Thomas' FROM 2 for 3);
Substring
-----------
hom
(1 row)
```

- **replace(string text, FROM text, to text)** replaces all occurrences in **string** of substring **FROM** with substring **to**.

```
postgres=# SELECT replace('abcdefabcdef', 'cd', 'XXX');
    replace
----------------
 abXXXefabXXXef
(1 row)
```

100 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **insert** returns the string **str** and replaces it starting from the **pos** position and extending to length **len** with **newstr**. The input parameters **str** and **newstr** are expressions that can be converted into string values, while **pos** and **len** are expressions that can be converted into integer values. The return value is a string. If **pos** is not within the length of the string **str**, the original string is returned. If **len** is greater than the length of the remaining string starting from **pos**, all characters starting from **pos** will be replaced by **newstr**.
- **insert('quadratic', 5, 2, 'what')** indicates that two consecutive characters starting from the fifth character of the character string **quadratic** are replaced by **what**. It is equivalent to **replace('quadratic', 'ra', 'what')**.
- **replace** replaces the **src** substring in the **str** string with the **dst** substring. The input parameter **str** indicates the original string, **src** indicates the string to be replaced, and **dst** indicates the replacement string. The return value is a string. Note: 1. Currently, data of the CLOB or BLOB type cannot be processed. 2. The return value supports a maximum of 8000 bytes. If the value exceeds 8000 bytes, an error will be reported.
- **replace('123456','45','abds')** indicates that **45** in the string **12345** is replaced by **abds**. It is equivalent to **insert('123456',4,2,'abds')**.
- **INSTR**: **str1** is the source string, and str2 is the string to be searched for. **pos** is the index position, indicating the starting position in **str1** for searching for **str2**. **pos** is optional, and the value is an integer that is not 0. If this parameter is omitted, the default value is **1**. *n* indicates the number of times that **str2** occurs and is optional. The value is a positive integer. If the parameter is omitted, the

default value is **1**. In the source string **str1**, the **INSTR** function searches for the string **str2** from the |**pos**| position according to the sequence from left to right (**pos** is a positive integer) or from right to left (**pos** is a negative integer). When **str2** occurs for the *n*th time, the system returns the position of the first character of **str2** in the source string **str1**. If no **str2** is found for the *n*th time of its occurrence, the system returns **0**. Regardless of whether the search sequence is from left to right or from right to left, the position of the string to be searched for in the source string is calculated from left to right.

Common Character Processing Functions

Character Processing Functions	Function	Example
basicemailmasking(text, char DEFAULT 'x'::bpchar)	Masks text before the first at sign (@).	basicemailmasking('abcd@gmail.com') xxxx@gmail.com
char_length(string)	Specifies the number of characters in a string.	char_length('hello') 5
lengthb(text/bpchar)	Obtains the number of bytes in a specified string.	lengthb('hello') 5
left(str text, n int)	Returns the first *n* characters in a string. When *n* is negative, all but the last **|n|** characters are returned.	left('abcde', 2) ab
right(str text, n int)	Returns the last *n* characters in a string. When *n* is negative, all but the first **|n|** characters are returned.	right('abcde', 2) de
lpad(string text, length int [, fill text])	Fills up **string** to **length** by appending the character **fill** (a space by default). If **string** is already longer than **length**, it is truncated.	lpad('hi', 5, 'xyza') xyzhi
rpad(string text, length int [, fill text])	Fills up **string** to **length** by appending the character **fill** (a space by default). If **string** is already longer than **length**, it is truncated.	rpad('hi', 5, 'xy') hixyx
notlike(x bytea name text, y bytea text)	Compares x and y to check if they are inconsistent.	notlike(1,2) t notlike(1,1) f
rawcat(raw,raw)	Indicates the string concatenation functions.	rawcat('ab','cd') ABCD
reverse(str)	Returns reversed strings.	reverse('abcde') edcba

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **trim([leading |trailing |both] [characters] FROM string)** removes the longest string containing only the characters (a space by default) from the start, end, or both ends of the string.
- **rtrim(string [, characters])** removes the longest string containing only the characters (a space by default) from the end of the string.
- **ltrim(string [, characters])** removes the longest string containing only the characters (a space by default) from the start of string.
- **upper(string)** converts the string to uppercase.
- **lower(string)** converts the string to lowercase.
- **initcap(string)** converts the first letter of each word in the string to uppercase and other letters to lowercase.
- **ascii(string)** indicates the ASCII code of the first character in the string.
- **concat(str1,str2)** concatenates **str1** and **str2** and returns a string.

Arithmetic Functions and Operators

Arithmetic Operator	Description	Arithmetic Operator	Description
+	Addition	!!	Factorial (prefix operator)
-	Subtraction	|	Binary OR
*	Multiplication	&	Binary AND
/	Division (The result is not rounded.)	#	Binary XOR
%	Modulo	~	Binary NOT
@	Absolute value	^	Power (exponent calculation)
|/	Square root	<<	Left shift
||/	Cubic root	>>	Right shift

```
postgres=# SELECT 2+3,2*3, @ -5.0, 2.0^3.0, |/ 25.0, 91&15, 17#5,1<<4 AS RESULT;

2+3     |  2*3      | @ -5.0  | 2.0^3.0       | |/ 25.0  |  91&15   | 17#5   | 1<<4
---------+-----------+----------+---------------+---------+-----------+---------+------------
5       |    6      |   5.0   |  8.00000      |   5     |   11      |   20    | 16
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Example of the arithmetic operator syntax: SELECToperation as result FROM sys_dummy, for example, SELECT2+3 FROM dual;
- The value of the operation can be 2 + 3, 2 – 3, 2 x 3, or 4/2.
- The priority sequence is as follows: four arithmetic operations > left/right shift > bitwise AND> bitwise XOR > bitwise OR.
- When bitwise operators are used for execution, and if input parameter values have decimal digits, the values will first be rounded off before participating in the bitwise operation. In this scenario, if the **BITOR**, **BITAND**, and **BITXOR** functions are used, they will round down the input parameter values and then perform bitwise operation.
- When the above operators are executed, a null string will be returned if the input parameter values contain **NULL**.

Example

- **abs(exp)**, **cos(exp)**, **sin(exp)**, **acos(exp)**, and **asin(exp)** return the absolute value, cosine value, sine value, arc cosine value, and arc sine value of an expression, respectively.
  - The return value type of **abs(exp)** is the same as that of **exp**. The return values of the other parameters are numbers.
  - The input parameter **exp** of **asin** and **acos** is an expression that can be converted into a numeric value. The value range is [–1,1].

```
postgres=#  SELECT abs(-10),cos(0),sin(0),acos(1),asin(0);

ABS       |      COS       |     SIN      |      ACOS      |     ASIN
----------+----------------+--------------+----------------+----------------
   10     |        1       |       0      |        0       |      0

1 rows fetched.
```

103 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **abs**: **exp** must be an expression that can be converted into a numeric value. The absolute value of **exp** (including the INT, BIGINT, REAL, NUMBER, and DECIMAL types) is returned.
- **sin** and **cos**: The input parameter is an expression that can be converted into a numeric value.

Example

- **bitand(exp1,exp2)** performs an AND (&) operation on two digits.

```
postgres=# SELECT bitand(29,15);

BITAND(29,15)
--------------------
  13
```

- **round(number[, decimals])** truncates a number value to the left or right of a specified decimal point.

```
postgres=# SELECT round(1234.5678,-2),round(1234.5678,2);

ROUND(1234.5678,-2) |     ROUND(1234.5678,2)
----------------------------+---------------------------------
1200                                  1234.57
```

104 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- BITAND, BITOR, BITXOR: The input parameter is an expression that can be converted into a numeric value. The value range is [–9223372036854775808,9223372036854775807].
- round: returns the truncated value. *decimals* is optional. If it is ignored, all decimal parts are truncated and rounded off. The value range is [–2147483648,2147483647]. If *decimals* is negative, *number* is rounded off to the left of the decimal point and the right parts of the decimal point are truncated.

Common Arithmetic Functions

Arithmetic Functions	Function	Example
ceil(x)	Not smaller than the minimum integer of a parameter	ceil(-42.8) -42
floor(x)	Not larger than the maximum integer of a parameter	floor(-42.8) -43
log(x)	Logarithm with 10 as the base	log(100.0) 2.0000000000000000
div(y numeric, x numeric)	Integer part of y/x	div(9,4) 2
trunc(x)	Truncates and retains the integral part.	trunc(42.8) 42
cbrt(dp)	Cubic root	cbrt(27.0) 3
mod(x,y)	Remainder of x/y (modulus) If x equals to 0, **0** is returned.	mod(9,4) 1
power(a double precision, b double precision)	b power of a	power(9.0, 3.0) 729.0000000000000000

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **ceil**: The input parameter **n** is an expression that can be converted into a numeric value, and the return value is an integer.
- **rawtohex**: The input parameter is a value of the raw type, and the return value is a string.
- **sign**: The input parameter **exp** is of the numeric type.
- **sort**: The return value is a decimal.
- **trunk**: The return value is a number.
- **power**: If the value of **expn** is a decimal, the calculation is open. If the value of **base** is less than 0 and that of **expn** is a decimal, the **power** function will return an error indicating that the values are beyond the supported range.

Time and Date Operators (+)

- Time and date operators (+)

```
postgres=# SELECT date '2021-5-28' + integer '7' AS RESULT;
        result
---------------------------
2021-06-04 00:00:00
```

```
postgres=# SELECT date '2021-05-28' + interval '1 hour' AS RESULT;
        result
---------------------
2021-05-28 01:00:00
```

```
postgres=# SELECT date '2021-05-28' + time '03:00' AS RESULT;
        result
---------------------
2021-05-28 03:00:00
```

```
postgres=# SELECT interval '1 day' + interval '1 hour' AS RESULT;
        result
---------------
1 day 01:00:00
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Time and Date Operators (–)

- Time and date operators (–)

```
postgres=# SELECT date '2021-05-01' - date '2021-04-28' AS RESULT;
  result
------------
 3 days
```

```
postgres=# SELECT date '2021-05-01' - integer '7' AS RESULT;
  result
---------------------
2021-04-24 00:00:00
```

```
postgres=# SELECT date '2021-05-28' - interval '1 hour' AS RESULT;
  result
---------------------
2021-05-27 23:00:00
```

```
postgres=# SELECT time '05:00' - interval '2 hours' AS RESULT;
  result
--------------
03:00:00
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Time and Date Operators (* and /)

- Time and date operators (*)

```
postgres=# SELECT 900 * interval '1 second' AS RESULT;
   result
-------------
 00:15:00
```

```
postgres=# SELECT 21 * interval '1 day' AS RESULT;
   result
------------
 21 days
```

```
postgres=# SELECT double precision '3.5' * interval '1 hour' AS RESULT;
   result
-------------
 03:30:00
```

- Time and date operators (/)

```
postgres=# SELECT interval '1 hour' / double precision '1.5' AS RESULT;
   result
-------------
 00:40:00
```

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Time and Date Processing Functions

Time/Date Function	Function	Example
age(timestamp, timestamp)	Subtracts parameters, producing a result in YYYY-MM-DD format. If the result is negative, the returned result is also negative. The input parameters can also contain the timezone if required.	age(timestamp '2001-04-10', timestamp '1957-06-13') 43 years 9 mons 27 days
age(timestamp)	Subtracts the current time with the parameter. The input parameter can also contain the timezone if required.	age(timestamp '1957-06-13') 64 years 2 mons 18 days
current_date	Specifies the current date.	current_date 2021-05-01
date_part(text, timestamp)	Obtains the value of a subdomain in date or time (for example, the year or hour).	date_part('hour', timestamp '2001-02-16 20:38:40') 20
trunc(timestamp)	Truncates to day by default.	trunc(timestamp '2001-02-16 20:38:40') 2001-02-16 00:00:00
sysdate	Specifies the current date and time.	sysdate 2021-05-01 17:04:49
justify_days(interval)	Adjusts intervals to 30-day time periods, which are represented as months.	justify_days(interval '35 days') 1 mon 5 days
pg_sleep(seconds)	Specifies the delay time of the server thread, in seconds.	pg_sleep(10)
last_day(d)	Returns the date of the last day of the month that contains *date*.	last_day(to_date('2017-01-01', 'YYYY-MM-DD')) 2017-01-31 00:00:00

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Type Conversion Functions

- **to_char(int, text)**, **to_clob(str)**, **to_date(exp[, fmt])**, and **to_number(n[, fmt])** convert the specified input parameter to the CHAR, CLOB, DATE, and NUMBER types, respectively.

```
postgres=# SELECT to_char(125, '999'),to_clob('hello111'::CHAR(15)),to_date('05 Dec 2000', 'DD
Mon YYYY'), to_number('12,454.8-', '99G999D9S');

TO_CHAR | TO_CLOB    |   TO_DATE              | TO_NUMBER
--------+------------+------------------------+-----------
125     |    hello111   | 2000-12-05 00:00:00    |   -12454.8
```

110 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **to_char**: The input parameter is an expression that can be converted to a string, and the return value is a string. *fmt* specifies the value format of the DATE type, for example, **yyyy/mm/dd/hh/hh12/hh24/mi/ss/ff3**. If no format is specified, the current session time format is used by default.
- **to_clob** converts data specified by *str* to the CLOB type.
- **to_date** converts *expr* to the DATE type. *fmt* specifies the target format, for example, **yyyy/mm/dd/hh/hh12/hh24/mi/ss/ff3**. The input parameter is a string expression or a value type that can be implicitly converted to the DATE type. The return value is of the DATE type.
- **to_number** converts *n* to the NUMBER type. The input parameter *n* is an expression or string that can be converted to the NUMBER type. The *fmt* parameter is optional, indicating the target format of *n*. If this parameter is not set, the value is converted to a decimal number by default. The return value is a number. The *fmt* parameter in **to_number** supports the following two fixed formats:
  - **X** converts *n* to a hexadecimal string. In this format, *n* can only contain hexadecimal characters 0 to 9, A to F, and a to f. - In the format of **XX**, the number of Xs must be greater than or equal to the number of characters preceding them. - If the target format is **0X**, the value of *n* must be 0 or a positive integer.
  - **000.00** converts *n* to the decimal format. The number of digits before the decimal point indicates the number of integers. If the number of integers in *n* exceeds the number of integers in the format, an error is reported. The number after the decimal point in *fmt* indicates the number of digits following the decimal point after conversion.

Type Conversion Functions

Type Conversion Function	Function	Example
cast(x as y)	Converts x into the type specified by y.	cast('22-oct-1997' as timestamp) 1997-10-22 00:00:00
hextoraw(string)	Converts a string in hexadecimal format into binary format.	hextoraw('7D') 7D
numtoday(numeric)	Converts values of the NUMBER type into the timestamp of a specified type.	numtoday(2) 2 days
pg_systimestamp()	Obtains the system timestamp.	pg_systimestamp() 2021-05-14 11:21:28.317367+08
rawtohex(string)	Converts a string in binary format into hexadecimal format.	rawtohex('1234567') 31323334353637
to_bigint(varchar)	Converts the character type to the BIGINT type.	to_bigint('123364545554455') 123364545554455
to_timestamp(text, text)	Converts values of the string type into the timestamp of a specified type.	to_timestamp('05 Dec 2000', 'DD Mon YYYY') 2000-12-05 00:00:00
convert_to_nocase(text, text)	Converts a string into a specified encoding type.	convert_to_nocase('12345', 'GBK') \x3132333435

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **ascii**: The input parameter is a character string or a single character, which must be enclosed in single quotation marks (''). The return value is an ASCII code.
- **cast**: The CAST function can be used to convert data types in the following scenarios: (1) The expression to be converted can be implicitly converted to a specified data type. (2) The data type must be explicitly converted. In other scenarios, an error is reported.

Geometric Operators

Geometric Operator	Description	Example
+	Translation	box '((0,0),(1,1))' + point '(2.0,0)' (3,1),(2,0)
-	Translation	box '((0,0),(1,1))' - point '(2.0,0)' (-1,1),(-2,0)
*	Scaling out/Rotation	box '((0,0),(1,1))' * point '(2.0,0)' (2,2),(0,0)
/	Scaling in/Rotation	box '((0,0),(2,2))' / point '(2.0,0)' (1,1),(0,0)
#	Intersection	box '((1,-1),(-1,1))' # box '((1,1),(-2,-2))' (1,1),(-1,-1)
@-@	Length or circumference	@-@ path '((0,0),(1,0))' 2
@@	Center	@@ circle '((0,0),10)' (0,0)
<->	Distance	circle '((0,0),1)' <-> circle '((5,0),1)' 3
<^	Is below?	box '((0,0),(-3,-3))' <^ box '((0,0),(2,2))' t
>^	Is above?	box '((0,0),(2,2))' >^ box '((0,0),(-3,-3))' t
&&	Overlaps? (One point in common makes this true.)	box '((0,0),(1,1))' && box '((0,0),(2,2))' t
<<	Is strictly left of?	circle '((0,0),1)' << circle '((5,0),1)' t

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- >>: Is strictly right of (no common horizontal coordinate)?
  - Example: postgres=# SELECT circle '((5,0),1)' >> circle '((0,0) ,1)' AS RESULT; result -------- t (1 row)
- &<: Does not extend to the right of?
  - Example: postgres=# SELECT box '((0,0), (1,1))' &< box '((0,0), (2,2))' AS RESULT; result -------- t (1 row)
- &>: Does not extend to the left of?
  - Example: postgres=# SELECT box '((0,0), (3,3))' &> box '((0,0), (2,2))' AS RESULT; result -------- t (1 row)
- <<|: Is strictly below (no common horizontal coordinate)?
  - Example: postgres=# SELECT box '((0,0), (3,3))' <<| box '((3,4), (5,5))' AS RESULT; result -------- t (1 row)
- |>>: Is strictly above (no common horizontal coordinate)?
  - Example: postgres=# SELECT box '((3,4), (5,5))' |>> box'((0,0), (3,3))' AS RESULT; result -------- t (1 row)
- &<|: Does not extend above?
  - Example: postgres=# SELECT box '((0,0), (1,1))' &<| box '((0,0), (2,2))' AS RESULT; result -------- t (1 row)

Geometric Functions

Geometric Function	Description	Example
area(object)	Area	area(box '((0,0),(1,1))') 1
center(object)	Center	center(box '((0,0),(1,2))') (0.5,1)
diameter(circle)	Diameter of a circle	diameter(circle '((0,0),2.0)') 4
height(box)	Vertical size of a rectangle	height(box '((0,0),(1,1))') 1
isclosed(path)	Is it a closed path?	isclosed(path '((0,0),(1,1),(2,0))') t
isopen(path)	Is it an open path?	isopen(path '[(0,0),(1,1),(2,0)]') t
length(object)	Length calculation	length(path '((-1,0),(1,0))') 4
npoints(path)	Number of points in a path	npoints(path '[(0,0),(1,1),(2,0)]') 3
npoints(polygon)	Number of points in a polygon	npoints(polygon '((1,1),(0,0))') 2
pclose(path)	Converts a path to closed	pclose(path '[(0,0),(1,1),(2,0)]') ((0,0),(1,1),(2,0))
popen(path)	Converts a path to open	popen(path '((0,0),(1,1),(2,0))') [(0,0),(1,1),(2,0)]
radius(circle)	Radius of a circle	radius(circle '((0,0),2.0)') 2
width(box)	Horizontal size of a box	width(box '((0,0),(1,1))') 1

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Geometric Functions

Geometric Type Conversion Functions	Description	Example
box(circle)	Circle to box	box(circle '((0,0),2.0)') (1.41421356237309,1.41421356237309),(-1.41421356237309,-1.41421356237309)
box(point, point)	Point to box	box(point '(0,0)', point '(1,1)')--(1,1),(0,0)
box(polygon)	Polygon to box	box(polygon '((0,0),(1,1),(2,0))') --(2,1),(0,0)
circle(box)	Box to circle	circle(box '((0,0),(1,1))') <(0.5,0.5),0.707106781186548>
circle(point, double precision)	Center and radius to circle	circle(point '(0,0)', 2.0) <(0,0),2>
circle(polygon)	Polygon to circle	circle(polygon '((0,0),(1,1),(2,0))') <(1,0.333333333333333),0.924950591148529>
lseg(box)	Box diagonal to line segment	lseg(box '((-1,0),(1,0))') [(1,0),(-1,0)]
lseg(point, point)	Point to line segment	lseg(point '(-1,0)', point '(1,0)') [(-1,0),(1,0)]
path(polygon)	Polygon to path	path(polygon '((0,0),(1,1),(2,0))') ((0,0),(1,1),(2,0))
point(box)	Box center	point(box '((-1,0),(1,0))') (0,0)
point(circle)	Circle center	point(circle '((0,0),2.0)') (0,0)

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **point(lseg)**: Specifies the center of a line segment.
  - Example: postgres=# SELECT point(lseg '((-1,0), (1,0))') AS RESULT; result ------- (0,0) (1 row)
- **point(polygon)**: Specifies the center of a polygon.
  - Example: postgres=# SELECT point(polygon '((0,0), (1,1), (2,0))') AS RESULT; result ----------------------

    (1,0.333333333333333) (1 row)
- **polygon(box)**: Converts a box into a 4-point polygon.
  - Example: postgres=# SELECT polygon(box '((0,0), (1,1))') AS RESULT; result ---------------------------- ((0,0), (0,1), (1,1), (1,0)) (1 row)
- **polygon(path)**: Converts a path into a polygon.
  - Example: postgres=# SELECT polygon(path '((0,0), (1,1), (2,0))') AS RESULT; result ---------------------- ((0,0), (1,1), (2,0)) (1 row)

Quiz

1. (Single-choice) What is the result returned by the SQL statement **SELECT date '2021-4-28' + integer '7' AS RESULT;**?
   A. 2021-05-05
   B. 2021-05-05 01:00:00
   C. 2021-05-05 00:00:00
   D. 00:00:00
2. (Multiple-choice) Which of the following are arithmetic functions?
   A. length(str)
   B. sin(n)
   C. round(v numeric, s int)
   D. hex(p1)

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - A
  - BC sine value. The value of the NUMBER type is truncated to *decimals* places to the right of the decimal point.

Quiz

3. (Multiple-choice) Which of the following are logical operators?
   A. and
   B. or
   C. not
   D. not or
4. (Multiple-choice) Which three separate approaches to pattern matching are provided by a database?
   A. LIKE
   B. Same
   C. SIMILAR TO
   D. POSIX regular expressions

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Answers:
  - ABC
  - ACD

Summary

- This chapter describes the concepts, classification, and data types associated with SQL statements.
- This chapter describes the DQL, DML, DDL, and DCL in SQL statements, and provides the syntax, application scenarios, and examples of each statement.
- This chapter describes common system functions, operators, and typical examples of databases.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recommendations

- openGauss help document:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/GettingStarted/GettingStarted.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Acronyms and Abbreviations

Acronym and Abbreviation	Full Spelling
DCL	Data Control Language
DDL	Data Definition Language
DML	Data Manipulation Language
DQL	Data Query Language
I/O	Input/output
SQL	Structured Query Language

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.

HUAWEI

06 GaussDB



HUAWEI

Foreword

- Databases are essential tools and widely used throughout enterprises. GaussDB is one of the main application scenarios in the Kunpeng ecosystem. Database technology is the focus of continued innovation, with cloud-based, distributed, and multi-mode processing representing primary future trends. This chapter describes the key advantages, features, and application scenarios of GaussDB, as well as related tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completion of this course, you will be familiar with:
  - Basics of GaussDB
  - Features of GaussDB
  - Tools and services related to GaussDB

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **Introduction to GaussDB**
2. Enterprise-Level Features of GaussDB
3. Comprehensive Tools and Service-oriented Capabilities
4. Application Scenarios and Cases

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Introduction to GaussDB



GaussDB

- GaussDB is an enterprise-level distributed relational database developed by Huawei and based on the openGauss ecosystem. It features Hybrid Transactional/Analytical Processing (HTAP) capabilities and supports intra-city deployment across AZs, scale-out of more than 1,000 nodes, and storage for four petabytes of data to ensure zero data loss. It is highly available, reliable, secure, and scalable, and provides key capabilities including quick deployment, backup, restoration, monitoring, and alarm reporting for enterprises.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Differences from openGauss:
  - openGauss: open-source; GaussDB: cloud service featuring high scalability, high availability, and massive data
  - Architecture: standalone system & primary/standby nodes for openGauss; distributed system & centralized system for GaussDB

In-House, Full-Stack Development

- GaussDB is an in-house product based on Kunpeng, featuring full-stack, independent, and controllable capabilities. GaussDB continuously optimizes its performance to meet ever-increasing demands across a wide range of scenarios.
- Fully utilizing Huawei's accumulated R&D capabilities in database kernels, GaussDB combines the enterprise-level capabilities of traditional relational databases and the advantages of distributed databases on the Internet, and features high availability, scalability, and performance, as well as enhanced data security. In addition, GaussDB works with Huawei's powerful software and hardware R&D capabilities to achieve full-stack, independent, and controllable capabilities.
- Core technologies:
  - High performance: distributed execution framework, GTM-Lite technology, and NUMA-aware-based transaction processing
  - High availability: cross-AZ/region DR and parallel playback for ultimate Recovery Time Objective (RTO)
  - High scalability: scale-out online

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- GaussDB is a next-generation commercial enterprise-level distributed relational database developed by Huawei, and based on the company's more than 10 years of experience with databases.
- It is primarily based on the openGauss ecosystem led by Huawei. Its primary/standby deployment is completely open-source. For details, refer to the openGauss open-source community (website provided on the last slide for your reference).

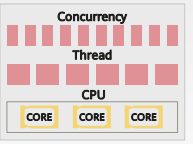
Product Positioning

Centralized Deployment
Focused on core enterprise transactions

GaussDB

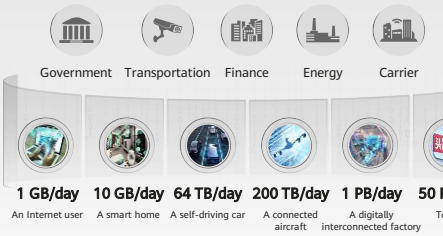
Kunpeng-powered
enterprise-level database





Huawei-developed kernel, maximum performance, and enhanced high availability

Distributed Deployment
Focused on massive transaction expansion



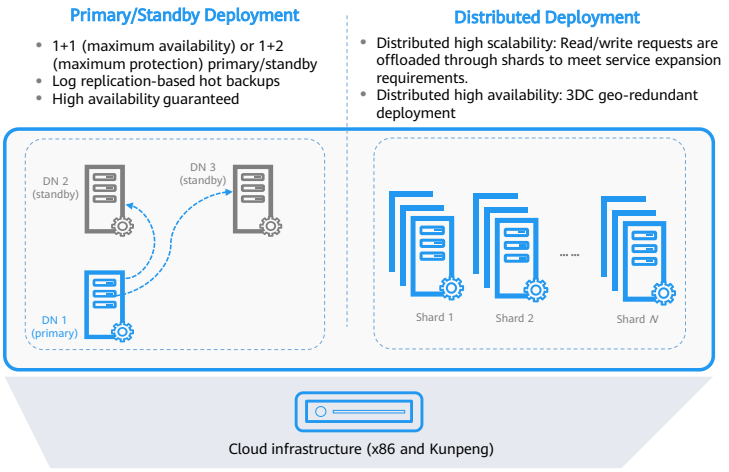
Data infrastructure

Chip | Storage | **Database** | Big data | AI

Heterogeneous computing framework: end-to-end multi-architecture computing power

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

High Availability - Flexible Deployment

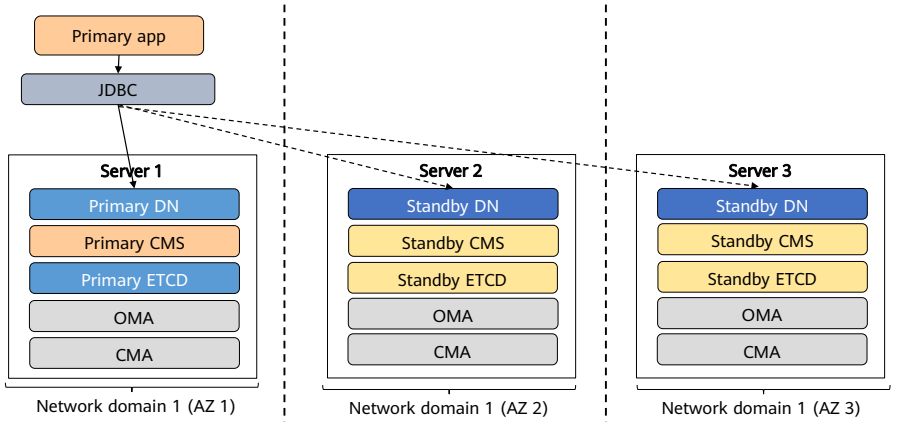


7 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 1+1 (maximum availability): Data is replicated to the standby node. However, if synchronous replication cannot be completed due to network problems, asynchronous replication will be implemented. After network recovery, synchronous replication will continue. If data is not synchronized, the switchover will lead to data loss.
- 1+2 (maximum protection): Data is synchronized to the standby node. A confirmation is needed before relevant messages are returned to the client. It boasts high reliability.
- To meet the high availability requirements of various scenarios, GaussDB supports different deployment modes. You can deploy just one database as a single node to support some simple applications and implement RAID redundancy between disks.
- You can also deploy the databases in primary/standby mode. A single AZ supports one primary node and two standby nodes, and multiple AZs support five replicas. This deployment mode provides intra-AZ or inter-AZ high availability.
- In scenarios where scale-out or resource sharing is required, you can use the distributed mode.

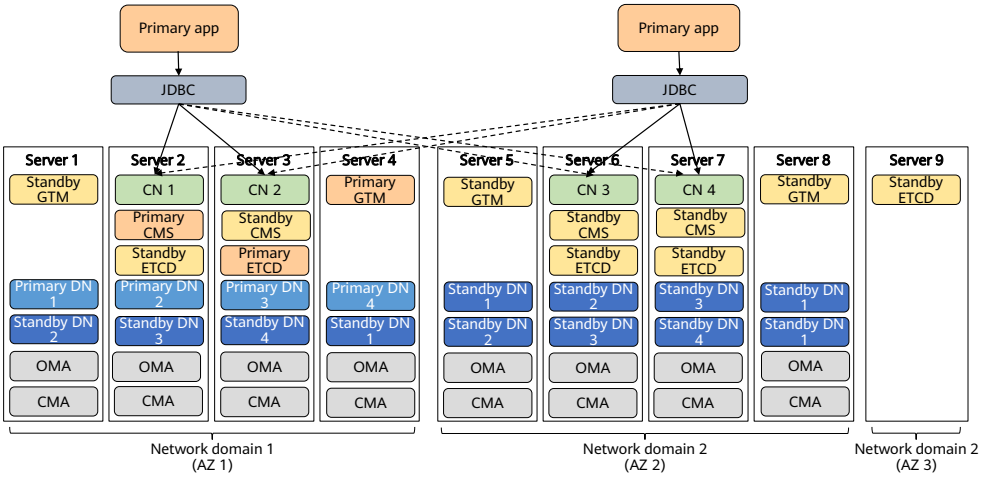
High Availability - Centralized Cluster Deployment



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This figure shows the deployment view and component relationships in primary/standby mode.
- Editable Text Configuration Daemon (ETCD) is a reliable distributed key-value storage product developed by CoreOS. It uses the Raft algorithm at the bottom layer to ensure consistency and is mainly used for shared configuration and service discovery. In a cluster, there must be three or more ETCDs and the number of ETCDs must be an odd number. The roles include leader and follower.

High Availability - Distributed Cluster in Intra-City Active-Active Mode

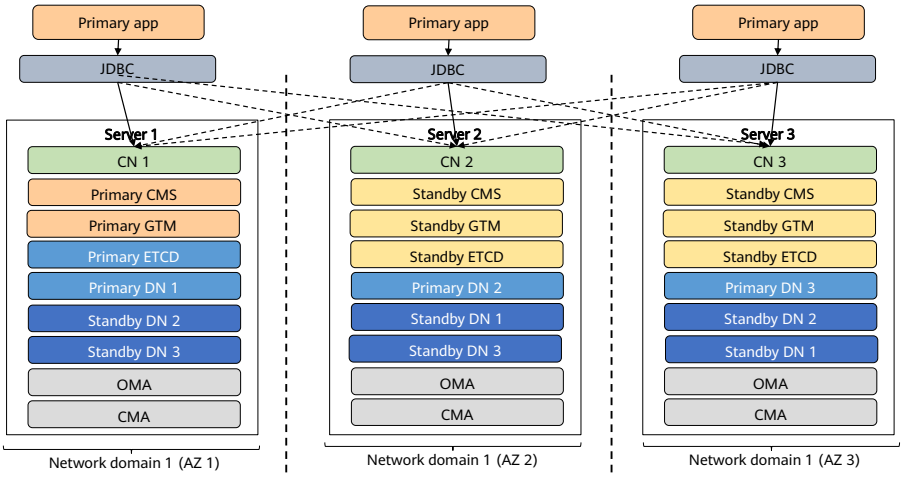


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- This figure shows the deployment view and component relationships in distributed mode.

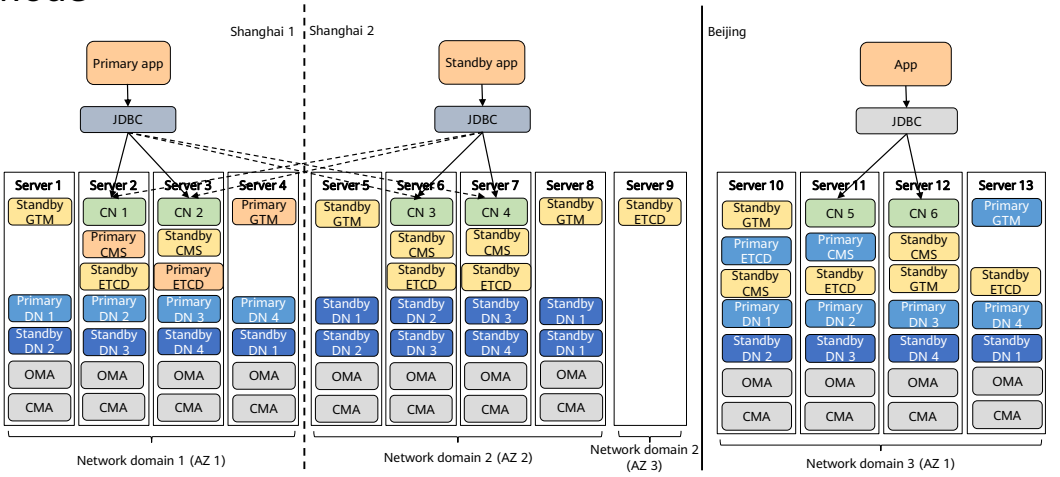
High Availability - Distributed Cluster in Intra-City 3-AZ Multi-active Mode



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- This figure shows the deployment view and component relationships in distributed mode.

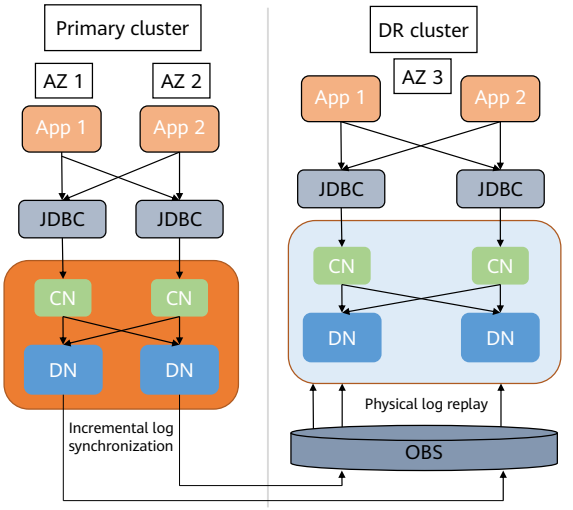
High Availability - Distributed Cluster in 3DC Geo-redundant Mode



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This figure shows the deployment view and component relationships in distributed mode.

High Availability - Remote DR



Features:

- A and B are deployed in intra-city active-active mode.
- C is a remote DR DC.
- Dual clusters are supported, causing little impact on production.

Pain points:

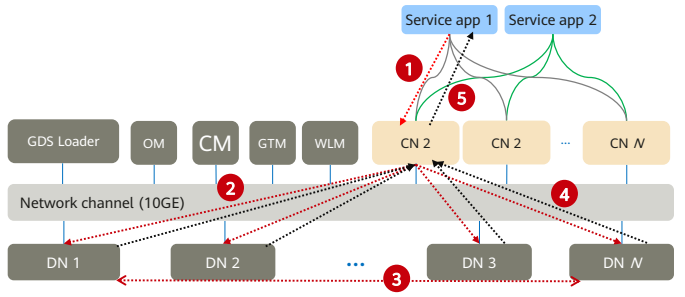
- Resources at the DR backup site are not fully utilized.

Specifications:

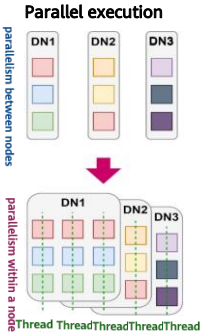
- RPO: 10 seconds
- RTO: 10 minutes

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

High Performance - Fully Parallel Distributed Execution



1. The service application sends an SQL statement to the CN. This SQL statement can be **INSERT**, **DELETE**/**DROP**, **UPDATE**, or **SELECT**.
2. The CN uses a database **optimizer** to generate an execution plan. DNs will process data according to this plan.
3. Data is evenly distributed on each DN using consistent hashing technology. As such, a DN may need to obtain data from other DNs during data processing. GaussDB provides three types of streams (**Broadcast**, **Gather**, and **Redistribution**) to reduce data transfer between DNs.
4. The DN returns result sets to the CN.
5. The CN summarizes and sends the results to the service application.



Multi-thread parallel execution

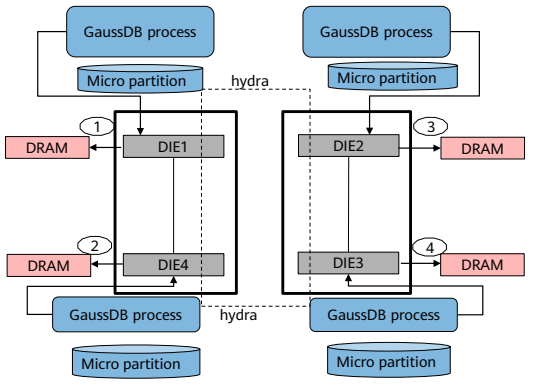
- The current multi-core feature is fully utilized and multi-thread parallel execution is used to improve the system throughput.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

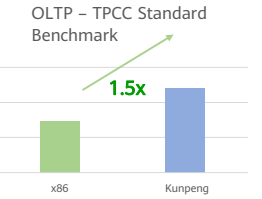
HUAWEI

- GaussDB adopts the massively parallel processing (MPP) architecture, which features parallel task execution, distributed data storage (localization), distributed computing, private resources (CPU, memory, disk, and network), scale-out, and shared nothing. As such, GaussDB can process massive amounts of concurrent data.
- In addition to the MPP architecture, GaussDB utilizes a streaming computing framework to enhance the data exchange capabilities between all compute nodes. Currently, almost all database operations, such as data scanning, table joining, and data aggregation, can be performed in fully parallel distributed mode. In addition, GaussDB supports scale-out of up to 256 shards. Compared with traditional databases, it offers far superior computing capabilities.
- Operation Manager (OM) provides management interfaces and tools for routine maintenance and configuration management of the cluster.
- Cluster Manager (CM) manages and monitors the running status of functional units and physical resources in the distributed system, ensuring stable running of the entire system.
- Global Transaction Manager (GTM) generates and maintains globally unique information, such as the global transaction ID, transaction snapshot, timestamp, and sequence.
- Coordinator Node (CN) receives access requests from the application layer, and returns the execution result to the client. The CN breaks down tasks and distributes task fragments to different DNs for parallel processing.
- Data Node (DN) stores service data by column or row or in the hybrid mode, executes data query tasks, and returns execution results to CNs.

High Performance - NUMA-Aware Technology



- Threads are bound to cores to prevent thread offset between cores.
- The NUMA-based data structure is reconstructed to reduce cross-core access.
- Data is partitioned to reduce thread access conflicts.
- The algorithm is adjusted to reduce single-point bottlenecks.
- ARM atomic instructions are used to reduce the computing overhead.



 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Binding threads to cores reduces the time spent thread switching between different CPU cores and reduces resource competition with other threads.
- NUMA is short for non-uniform memory access. The memory access time depends on the memory location of the processor. With NUMA, the processor accesses its own local memory faster than the non-local memory. You can use the numactl tool to query the CPU access distance.
- For NUMA atomic instructions, one instruction is used to replace the original four instructions in order to implement the same function, improving the execution efficiency.

High Performance - Other Features (1)

- High-speed parallel data loading
  - You can fully utilize the computing and I/O capabilities of all nodes to achieve the maximum parallel data import speed. Data is imported to GaussDB using a specific format (CSV, TEXT, or FIXED).
- Symmetric Multi-Processing (SMP) technology
  - The SMP technology of GaussDB uses the multi-core CPU architecture of a computer to implement multi-thread parallel computing, fully using CPU resources to improve query performance.
  - It significantly reduces the execution time of a single query.
  - It improves system throughput within the same period and effectively improves system resource utilization.

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- High-speed parallel data loading: CNs are only responsible for task planning and delivery and DNs are responsible for data import. In this way, CNs have resources to process external requests. As each DN imports data, the computing capability and network bandwidth of each node are fully utilized, improving overall data import performance.
- The SMP feature improves performance through operator parallelism and occupies more system resources. It is used to save time at the cost of resource consumption. SMP improves system performance in appropriate scenarios and when resources are sufficient.

High Performance - Other Features (2)

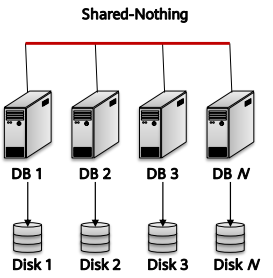
- Low Level Virtual Machine (LLVM) dynamic compilation technology
  - Based on the query execution plan tree, and with the library functions provided by LLVM, GaussDB moves the process of determining the actual execution path from the executor phase to the execution initialization phase. In this way, problems such as function calling, logic condition branch determination, and a large amount of data reading that are related to the original query execution are avoided, thereby improving the query performance.
- Adaptive compression
  - Adaptive compression chooses a suitable compression algorithm for data based on the data type and features, achieving high performance in compression ratio, import, and query.
  - GaussDB has implemented various compression algorithms, including RLE, DELTA, BYTEPACK/BITPACK, LZ4, ZLIB, and LOCAL DICTIONARY.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- LLVM provides an intermediate layer of a complete compilation system, and extracts and optimizes intermediate representation (IR) from a compiler. The optimized IR is then converted and linked to an assembly language of a target platform. LLVM can also generate relocatable code during compilation, linking, or even running.
- Adaptive compression applies to scenarios where large integer compression of mobile number-like character strings, large integer compression of the numeric type, and adjustment of the compression algorithm compression level are supported.

High Scalability - Shared-Nothing Architecture

- Shared-Nothing architecture
  - Each node (processing unit) in the cluster has its own independent CPU, memory, and storage, and does not share resources.
  - Each node (processing unit) processes its own local data. The processing result can be summarized to the upper layer or transferred between nodes through communication protocols.
  - Nodes are independent of each other and have strong scalability. The entire cluster has powerful parallel processing capabilities.

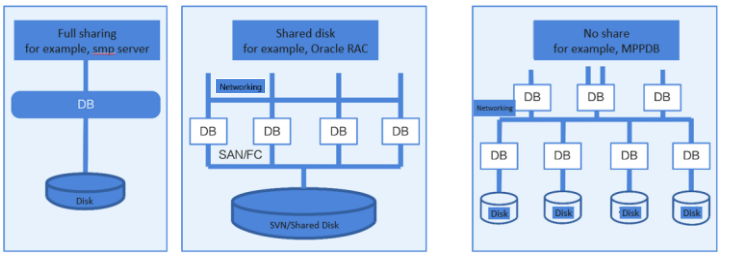


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- In the current hardware era, multiple processing units can be deployed on a node or a physical server. As such, the minimum unit of Shared-Nothing may not be a physical node but a logical virtual processing unit. For example, if a physical server has a 4-core CPU, four database instances can be deployed on the physical server. In other words, the physical server has four processing units.

High Scalability - Advantages of the Shared-Nothing Architecture

Architecture comparison:



Advantages of the Shared-Nothing architecture:

- High scalability
  - Provides high scalability for BI and data analysis to process high-concurrency and large-data-amount computing.
  - Supports the parallel processing mechanism.
- Automated internal parallel processing
  - Allows you to load and access data as you would in a common database.
  - Distributes data on all parallel nodes.
  - Each node processes only partial data.
- Optimal I/O processing
  - All nodes process data in parallel.
  - Nodes share nothing with each other and have no I/O conflicts.
- Performance expansion by adding nodes
  - The storage, query, and loading performance can be improved by adding nodes.

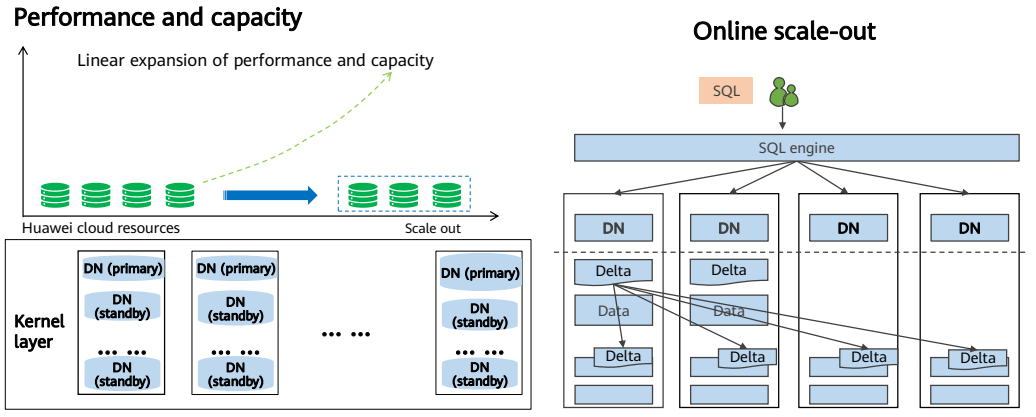
 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- GaussDB Kernel is designed based on the Shared-Nothing architecture. It consists of multiple independent logical nodes that do not share system resources, such as the CPU, memory, and storage units. In such an architecture, service data is stored on multiple compute nodes, and data query tasks are executed on the nearest nodes. A large amount of data can be processed in coordination with a CN for rapid data processing.
- **Shared-Everything**: Used for a single host. It transparently shares the CPU, memory, and I/O. Its parallel processing capabilities are the weakest, with SQL Server as a typical example.
- **Shared-Disk**: Each processing unit has its own CPU and memory, but the disk space is shared. Oracle RAC is a typical example. It permits data sharing, and also allows users to add nodes to improve parallel processing capabilities, providing outstanding scalability. While Oracle RAC is similar to the SMP mode, adding nodes cannot achieve a higher performance when storage interfaces are all occupied.
- **Shared-Nothing**: Each processing unit has its own CPU, memory, and hard disk, and these units do not share resources, similar to the MPP mode. These processing units communicate with each other using protocols, achieving better parallel processing and scalability. DB2 DPF and Hadoop are typical examples. Nodes are independent of each other and each node processes its own data. The processing results may be summarized and sent to the upper layer, or transferred among different nodes. Sharding in this case utilizes the Shared-Nothing architecture, and horizontally partitions a table from the physical storage perspective and allocates them to multiple servers (or instances). Each server can work independently and has a common schema (for example, the MySQL Proxy and Google architectures). You only need to increase the number of servers to improve the processing capabilities and capacity.
- In a pure Shared-Nothing system, you can achieve almost infinite expansion by simply adding some cheap computers as system nodes.

- The Shared-Nothing system usually distributes its data to different databases on multiple nodes (different computers process queries from different users) or requires each node to retain its own application data backup by using some coordination protocols. This is usually called database sharding.

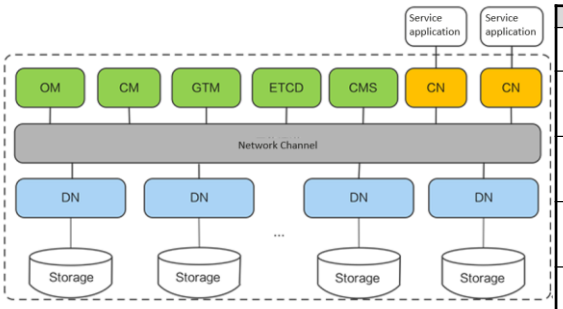
High Scalability



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- A single large-scale cluster supports a maximum of 256 DNs.
- A single cluster supports more than 100,000 concurrent access. Based on the thread pool technology, a single node supports 10,000 concurrent connections and a single cluster supports more than 100,000 concurrent connections.

GaussDB Distributed Architecture - Component Functions



Roles perceived by the management plane:

- DN
- CN
- GMT
- ETCD
- CMS

Name	Description
OM	Provides management interfaces and tools for routine maintenance and configuration management of the cluster.
CM	Manages and monitors the running status of functional units and physical resources in the distributed system, ensuring stable running of the entire system.
GTM	Generates and maintains globally unique information, such as the global transaction ID, transaction snapshot, timestamp, and sequence.
CN	Receives access requests from the application layer, and returns the execution result to the client. The CN breaks down tasks and distributes task fragments to different DNs for parallel processing.
DN	Stores service data by column or row or in the hybrid mode, executes data query tasks, and returns execution results to CNs.
ETCD	Used for sharing configurations and discovering services (service registration and search).
Storage	Functions as the server's local storage resources to store data permanently.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Introduction to GaussDB
2. **Enterprise-Level Features of GaussDB**
3. Comprehensive Tools and Service-oriented Capabilities
4. Application Scenarios and Cases

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Enterprise-Level Feature - Distributed Storage

- As a distributed database, GaussDB provides two-layer distributed storage capabilities.
  - For distributed storage of data between shards, both hash and replication distribution are supported. Hash distribution is mainly applied to user tables with a large amount of data. Hash calculation is performed on one or more distribution keys specified by a user, and data within the same user table is distributed and stored in different shards, thereby increasing the total data volume that can be supported by an entire database. In addition, distributed parallel processing capabilities and pruning processing capabilities are provided. Replication distribution is mainly used for user tables with a small amount of data. All of the replication distribution table's data is stored in each shard, improving the performance of distributed multi-table associated query.
  - Distributed storage of data in a shard: For each shard, quorum-based distributed multi-copy storage is supported to ensure high reliability and availability of the database. In addition, functions such as automatic primary/standby switchover, AZ switchover, and forcible promotion to the primary are provided, ensuring that the RPO and RTO are stable and meet expectations.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Key Feature - Data Partitioning (1)

- Data partitioning is a common function among database products. In GaussDB, data is partitioned horizontally with a user-specified policy. This operation splits a table into multiple partitions that do not overlap.
- GaussDB supports range partitioning. Records from one or more columns to be inserted into a table are divided into multiple ranges. A partition for each range is created to store data, and partition ranges do not overlap. If you specify the **PARTITION** parameter when running the **CREATE TABLE** statement, data in the table will be partitioned.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- GaussDB supports range partitioning, list partitioning, and hash partitioning.

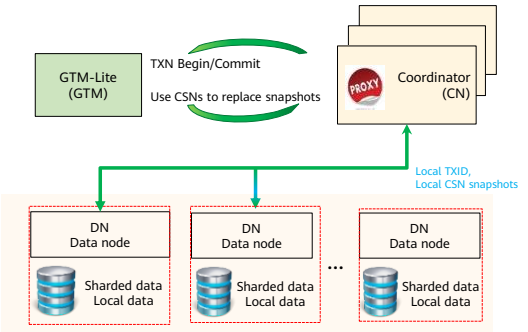
Key Feature - Data Partitioning (2)

- Advantages of data partitioning:
  - Improved manageability: Tables and indexes are divided into smaller and more manageable units. This helps database administrators manage data based on partitions, allowing maintenance to be performed for specific parts of a table.
  - Better deletion performance: Deleting a partition is more efficient and faster than deleting rows.
  - Stronger query performance: You can restrict the volume of data to be checked or operated on for stronger performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Distributed Transaction Processing Performance: GTM-Lite Technology

- GTM-Lite technology offers high-performance transaction processing and ensures strong global consistency, eliminating the performance bottlenecks associated with single GTM.



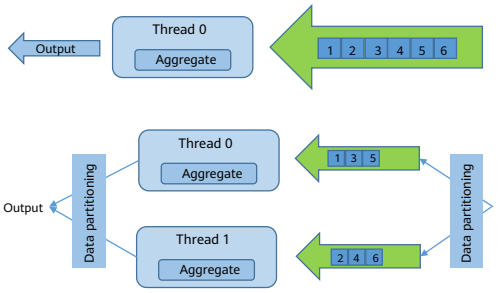
- High-performance transaction management: lockless, multi-version, highly concurrent transactions
- Strong consistency in distributed mode: The distributed GTM-Lite solution manages global transaction snapshots and commit IDs to achieve strong consistency, with no performance bottlenecks caused by central nodes.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- GTM-Lite: Traditional vendors maintain a global transaction list to ensure transaction consistency, and this can often become a bottleneck. Huawei has made a number of GTM innovations, and uses lightweight technologies offering the following advantages:
  - 1. When a Commit Sequence Number (CSN) is used to submit transaction IDs, the transaction list does not need to be traversed, enabling quick determination of transaction visibility.
  - 2. Lock-free atomic operations are used to provide CSNs, ensuring no single-point bottlenecks. Each transaction can obtain CSNs at the same time.
  - 3. Only one CSN is required for node interactions, greatly reducing the network overhead for transaction status synchronization between nodes.
- Consistency and performance represent the core of distributed transaction processing. Based on different service application scenarios, GaussDB comprehensively optimizes the distributed transaction processing performance and consistency. Currently, the following transaction processing modes are supported:
- In GTM-Lite mode, distributed transactions are strongly consistent. GTM is extremely lightweight and provides excellent performance in online transaction processing (OLTP) scenarios.
- In GTM-Free mode, distributed transactions are finally consistent. It does not connect to the GTM and does not provide real-time read consistency for distributed transactions. In perfect sharding scenarios of OLTP, optimal performance is achieved without central nodes.

SMP Technology



SMP parallel execution

- The SMP technology of GaussDB uses a computer's multi-core CPU architecture to implement multi-thread parallel computing, fully utilizing CPU resources to improve query performance.
  - SMP refers to the parallel execution of tasks at the **thread** level. Theoretically, the number of subtasks that can be concurrently executed can reach the upper limit of the number of cores on the physical server.
  - SMP parallel threads are in the same process, and **data can be exchanged directly through memory**, without occupying network connection and bandwidth. This reduces the impact of network factors that restrict the performance improvement of the MPP system.
  - As the parallel subtasks **do not need to be attached to another background worker thread** after being started, the utilization rate of system computing resources can be increased effectively.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The database requires not only MPP technology to achieve good horizontal scale-out, but also SMP technology to achieve better scale-up on a single node.
- SMP can improve database performance as follows:
  - It significantly reduces the execution time of a single query.
  - It improves system throughput within the same period and enhances system resource utilization.
- SMP technology leverages the multi-thread and multi-subtask parallel execution mechanism to comprehensively and efficiently utilize system computing resources. Consequently, SMP multi-thread lightweight execution can overcome the problems inherent in MPP architecture deployment.

Physical Backup

- GaussDB provides a physical backup function capable of backing up data from the entire cluster to Object Storage Service (OBS) in the internal format of databases, as well as restoring such data in homogeneous databases. Physical backup uses distributed parallel technology to physically back up data files of each DN, offering high backup and restoration performance. In addition, it also provides such advanced functions as compression, flow control, and resumable backup.
- Physical backup is classified as either full or incremental backup. Full backup includes the full data of the database at the backup time point, requires an extended timeframe (in direct proportion to the total data volume of the database), and enables a complete database to be restored. In contrast, an incremental backup involves only incremental data modified after a specified time point. It requires a shorter timeframe (in direct proportion to the incremental data volume and not related to the total data volume). However, a complete database can be restored only after both the incremental backup and full backup are performed. You can create a backup policy every week.
  - On Monday, perform a full backup of the database.
  - On Tuesday, perform an incremental backup based on Monday's full backup.
  - On Wednesday, perform an incremental backup based on Tuesday's incremental backup.
  - ...
  - On Sunday, perform an incremental backup based on Saturday's incremental backup.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Logical Backup

- GaussDB provides logical backup capabilities to back up data in user tables to local disk files and OBS in text or CSV format, and restore the data in homogeneous or heterogeneous databases. Logical backup uses the distributed parallel technology to extract user table records to be backed up from each DN in streaming mode, providing high backup and restoration performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Introduction to GaussDB
2. Enterprise-Level Features of GaussDB
3. **Comprehensive Tools and Service-oriented Capabilities**
4. Application Scenarios and Cases

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DAS

Data Admin Service (DAS) is a one-stop cloud database management platform that allows you to manage databases on a web-based console. It offers database development, O&M, intelligent diagnosis, and enterprise-level DevOps to facilitate your cloud database usage and maintenance.

Standard edition	Cloud DBA	Enterprise edition
Intended for developers	Intended for DBAs and O&M personnel	Intended for enterprise users
Provides a best-in-class experience. You no longer need to install clients locally for a visualized operation experience. Diverse database development functions are available, including online editing and intelligent prompts for SQL input.	Provides a wide range of functions, covering instance performance data, slow SQL analysis, SQL explorer, real-time performance analysis and diagnosis, as well as historical running data analysis, helping you to quickly locate every issue encountered during the lifetime of the database and avoid potential risks.	Offers a DevOps database product with a focus on database access, operation security, and communication of programmers, DBAs, and enterprise managers. It combines automatic identification of modification risks and the permission approval process to ensure data change security and improved development efficiency.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DRS

- Data Replication Service (DRS) is a stable, efficient, and easy-to-use cloud service for real-time database migration and synchronization.
- DRS simplifies data transmission between databases and reduces data movement costs. You can use DRS to migrate data between databases quickly in order to meet your service requirements.
- DRS provides multiple functions, including real-time migration, backup migration, real-time synchronization, data subscription, and real-time DR.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

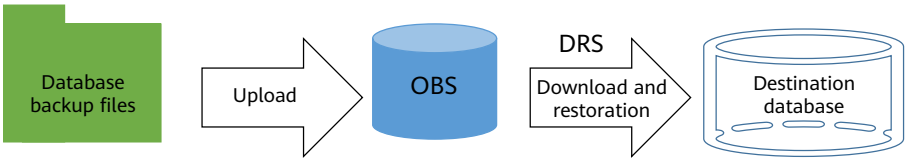
DRS Service Module - Real-Time Migration

- Real-time migration allows you to directly migrate data from the source database to the destination database on the condition that they are connected and the source database instances, destination database instances, and migration objects are configured.
- Real-time migration supports multiple types of networks, such as public networks, VPCs, VPNs, and Direct Connect. With these networks, migration can be performed between different cloud platforms, from on-premise databases to cloud databases, or on the cloud across regions.
- DRS supports incremental migration, which ensures your service continuity while minimizing the service downtime and migration impact. In this way, databases can be smoothly migrated to the cloud, and all database objects can be migrated.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DRS Service Module - Backup Migration

- It often becomes necessary to hide one of your database's real IP addresses for the sake of security. Migrating data through direct connections is an option, but can be costly. DRS supports backup migration, which allows you to export data from your source database for backup and upload the backup files to OBS. You can then restore the backup files to the destination database to complete the migration. Using this method, data migration can be achieved without exposing your source databases.
- Common scenario: on-premise to cloud migration. Currently, only SQL Server can be migrated to the cloud.
- Without connecting to your sources databases, DRS can help you complete data migration.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DRS Service Module – Real-Time Synchronization

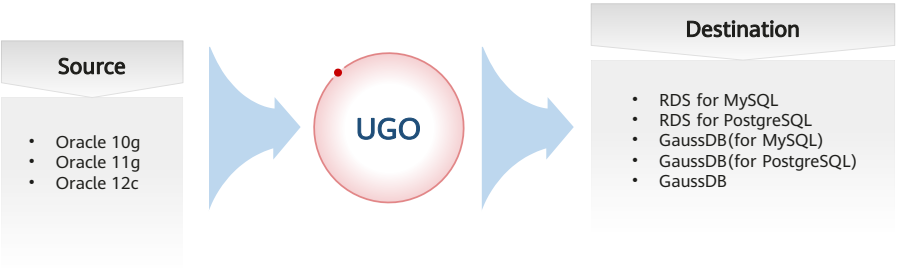
- Real-time synchronization refers to the real-time replication of key service data from the source database to the destination database through DRS, while ensuring consistency of the data.
- Different from data migration, which involves moving your overall database from one platform to another, synchronization refers to the continuous flow of data between different services.
- Common scenarios: real-time analysis, report system, and data warehouse environment
- DRS can satisfy various requirements, such as many-to-one, one-to-many synchronization, dynamic addition and deletion of synchronization tables, and synchronization between tables with different names.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

UGO Service

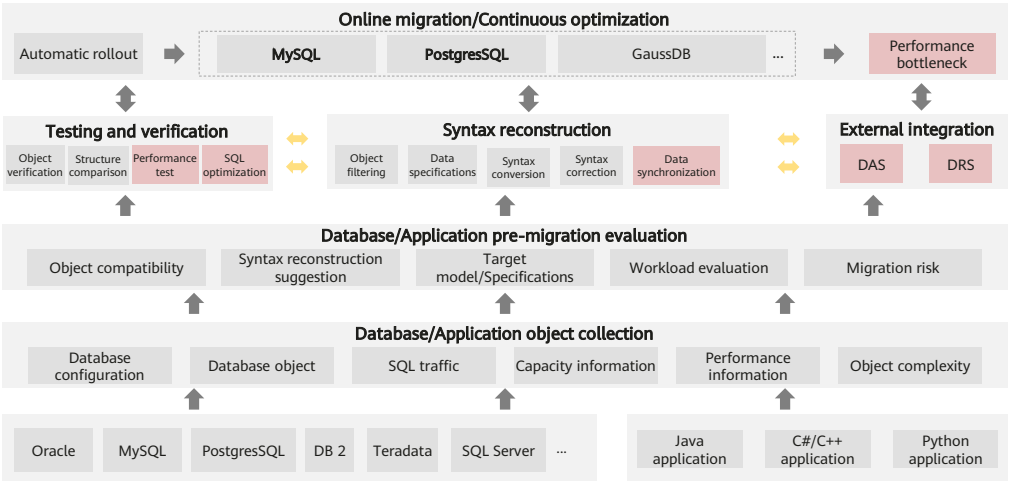
Database and Application Migration UGO, referred to as UGO, is a professional tool that focuses on **heterogeneous database object migration** and **application migration**. As such, it enables automatic migration of mainstream commercial databases to HUAWEI CLOUD databases. Data is **easily migrated to the cloud**, and **databases can be switched with just a few clicks**.

Application Scenarios



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

UGO Product Migration Solution



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

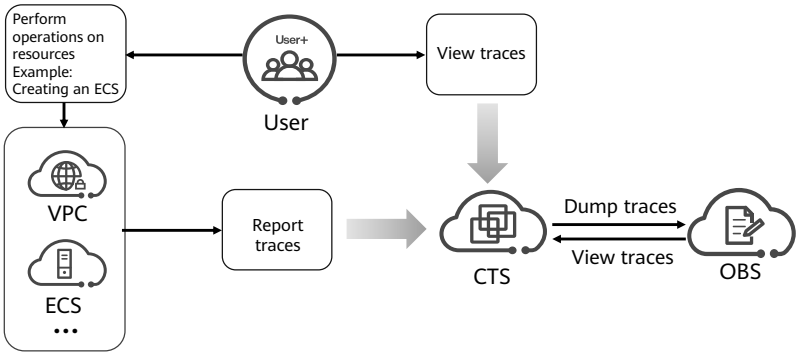
UGO Core Functions

Source database profiling	Destination database selection and specification recommendation	Destination database compatibility analysis
Migration workload evaluation	Database structure migration	Application SQL migration

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CTS

- Cloud Trace Service (CTS) is a log audit service that is available on HUAWEI CLOUD, and has been designed to offer enhanced cloud security. It allows you to collect, store, and query cloud resource operation records, and then use these records for security analysis, compliance auditing, resource tracking, and fault locating.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- The log audit module is a core component necessary for information security auditing and an important information system providing security risk management and control for enterprises and organizations. As the information system is migrating to the cloud, information and data security management departments around the world, including Standardization Administration of the People's Republic of China/Technical Committee (SAC/TC), have released multiple standards, such as ISO IEC27000, GB/T 20945-2013, COSO, COBIT, ITIL, and NISTSP800.

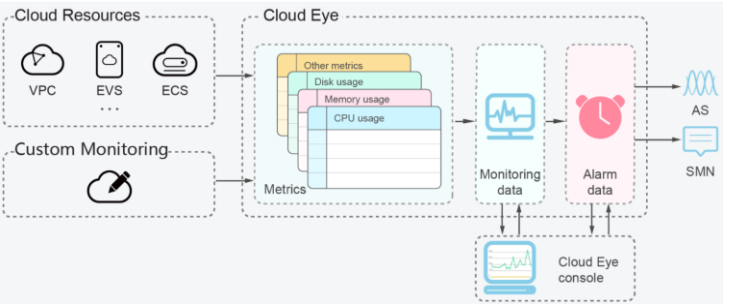
CTS Functions

- Trace recording: CTS records operations performed on the management console or by calling APIs, as well as operations triggered by each interconnected service.
- Trace query: Operation records from the last seven days can be queried on the management console from multiple dimensions, such as trace type, trace source, resource type, filter, operator, and trace level.
- Trace dumping: Traces are delivered to OBS buckets on a regular basis for long-term storage. Through this process, traces are compressed into trace files by service.
- Trace file encryption: Trace files are encrypted using keys provided by Key Management Service (KMS) during dumping.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CES (1)

- Cloud Eye Service (CES) is a comprehensive monitoring platform for resources such as the ECS and bandwidth. You can use CES to monitor the utilization of service resources, track the running status of cloud services, configure alarm rules and notifications, and quickly respond to resource changes.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CES (2)

- CES provides the following functions: automatic monitoring, server monitoring, real-time notification, monitoring panel, OBS dumping, resource grouping, log monitoring, and trace monitoring.
- GaussDB(openGauss) can use CES to monitor the operating status of the system. By monitoring system resource usage when the database is running, you can identify periods of high resource usage. To optimize database performance, you can analyze which SQL statements may have had performance issues by checking error logs or slow logs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

DBSS

- Database Security Service (DBSS) leverages Huawei's 30 years of experience in database security practices. It offers two subservices, database security audit and database protection, which deliver such functions as data breach prevention, database firewall, and database security audit to protect your databases and assets on the cloud.
- Database security audit
  - Database security audit provides an audit function in bypass mode, enabling the system to generate real-time alarms for risky operations and attack behaviors. In addition, database security audit generates compliance reports that meet data security standards. In this way, it locates internal violations and improper operations, detects and blocks external intrusions, and protects data assets. Functions such as user behavior detection and auditing, multi-dimensional lead analysis, real-time alarms, and reports are all supported.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Security Audit (1)

- User behavior detection and audit
  - Access operations at the application layer and database layer can be associated to help customers trace the identities and behaviors of users.
- Multi-dimensional lead analysis
  - Risk leads: This function can analyze SQL behaviors such as SQL injection, blacklist statements, and authorization violation at high, medium, and low risk levels.
  - Session leads: This function supports analysis from multiple perspectives, including time, user, IP address, and client.
  - Detailed statement leads: This function supports searching by user, client IP address, access time, operation object, and operation type.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Security Audit (2)

- Real-time alarms for abnormal operations, SQL injection, as well as blacklist and whitelist hits
  - Abnormal behavior detection: This function monitors access behaviors based on multiple fine-grained factors such as the client IP address, database IP address, database account, and risk severity.
  - SQL injection: This function provides a systematic SQL injection database and SQL injection description based on regular expressions or syntax abstraction, and generates alarms when detecting database exceptions.
  - Blacklist and whitelist: SQL statements for accessing the system are described accurately and abstractly, and alarms are generated in real time when the SQL statements appear.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Database Security Audit (3)

- Fine-grained reports for various abnormal behaviors:
  - Session behavior: login failure and session analysis reports
  - SQL behavior: new SQL, SQL statement execution history, and SQL failure reports
  - Risky behavior: alarm, notification, SQL injection, and batch data access behavior reports
  - Compliance report: compliance reports that meet data security standards (such as Sarbanes-Oxley)

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

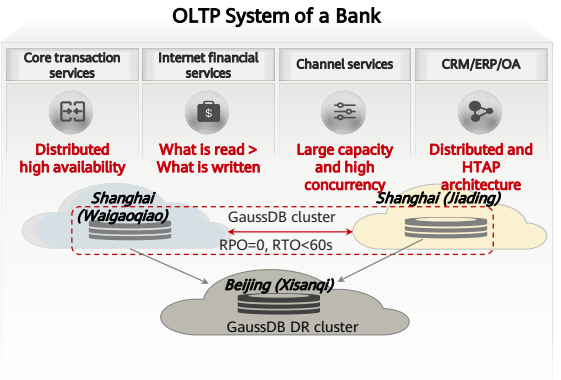
Contents

1. Introduction to GaussDB
2. Enterprise-Level Features of GaussDB
3. Comprehensive Tools and Service-oriented Capabilities
4. **Application Scenarios and Cases**

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

HUAWEI CLOUD GaussDB Supports In-House Distributed Reconstruction of a Bank's Core Service System (1)



Challenges

- **Capacity bottleneck**: Rapid development of Internet financial services, poor scalability of traditional centralized databases, and urgent need for distributed reconstruction have caused bottlenecks.
- **High availability in finance**: Core financial services are migrated to an open platform. Benchmarking DB2 is required to provide high availability in intra-city and active- active DR and 3DC geo-redundant DR in finance..

Solutions

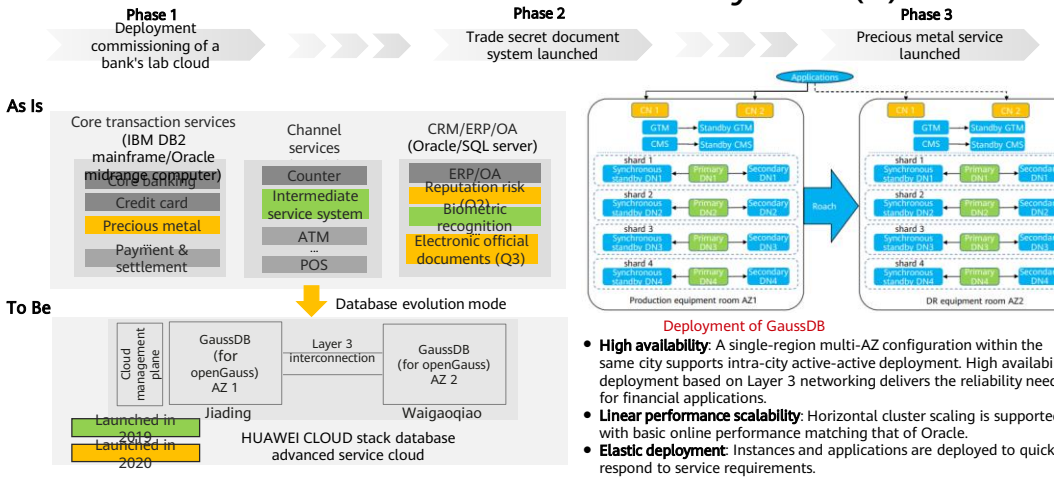
- **Distributed high scalability**: Distributed expansion capabilities and ACID protection for distributed transactions are provided.
- **3DC geo-redundant DR**: A single cluster is deployed across AZs in intra-city active-active mode to achieve an RPO of zero and an RTO of less than 60s. Dual-cluster DR is deployed cross regions.

Customer Benefits

- **Large capacity and high scalability**: Terabyte- or petabyte-level data for a single database and online scale-out are supported with no sharding, simplifying application development.
- **Financial high availability**: Intra-city active-active deployment enables services from two DCs within the same city to be accessed at the same time. If one DC becomes faulty, services can be recovered within seconds.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI CLOUD GaussDB Supports In-House Distributed Reconstruction of a Bank's Core Service System (2)

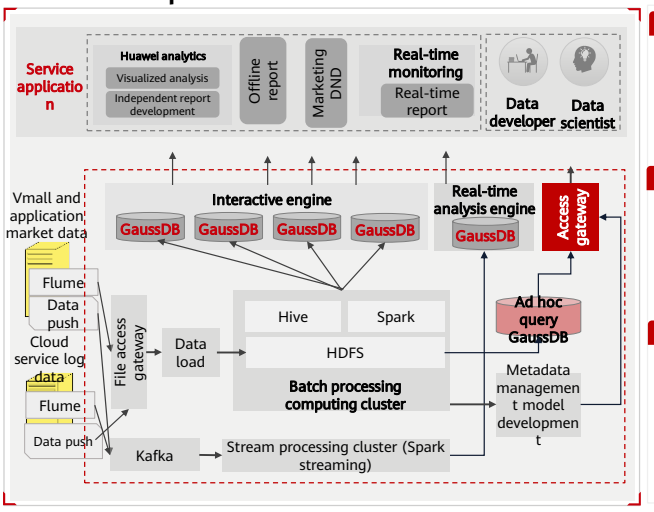


Deployment of GaussDB

- **High availability**: A single-region multi-AZ configuration within the same city supports intra-city active-active deployment. High availability deployment based on Layer 3 networking delivers the reliability needed for financial applications.
- **Linear performance scalability**: Horizontal cluster scaling is supported, with basic online performance matching that of Oracle.
- **Elastic deployment**: Instances and applications are deployed to quickly respond to service requirements.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

GaussDB Helps Huawei Consumer Cloud Implement Smart Service Operation



Challenges

Huawei Consumer Cloud Big Data platform centrally stores and manages service data, including the **fitness and health service**, **cloud space service**, and **geographic big data service**, by using databases in a hybrid architecture. However, it must overcome the following challenges: Fitness services expand rapidly, with the amount of data expanding by over 30% every year.

Solutions

After the preceding services are migrated to GaussDB:

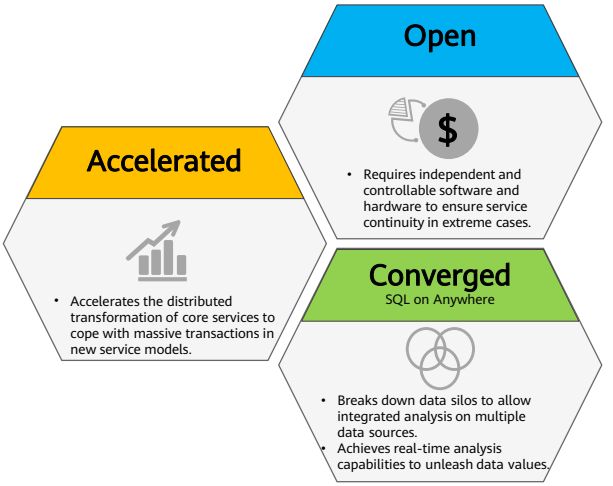
1. On-demand sharding and elastic scaling sustain rapid service expansion.
2. Powerful HTAP capabilities support real-time service analysis.

Customer Benefits

1. On-demand scaling conserves resources without interrupting services.
2. After the new data analysis model is brought online, analysis results can be obtained in real time. Marketing precision is improved by more than 50% and the analysis efficiency is boosted to over 80%.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

GaussDB Enables Smooth Transformation of Customer Services in the Financial Industry



Service Requirements and Challenges

Customers in the financial industry in China require the following:

1. Independent and controllable core services
2. Transformation to a distributed transaction system

Solutions

1. HUAWEI CLOUD provides a full-stack, independent, and self-controllable solution for software and hardware.
2. Various migration tools are used to efficiently and cost-effectively complete online service migration and distributed reconstruction.
3. Powerful HTAP capabilities, desirable storage project compatibility, and real-time service analysis are supported.

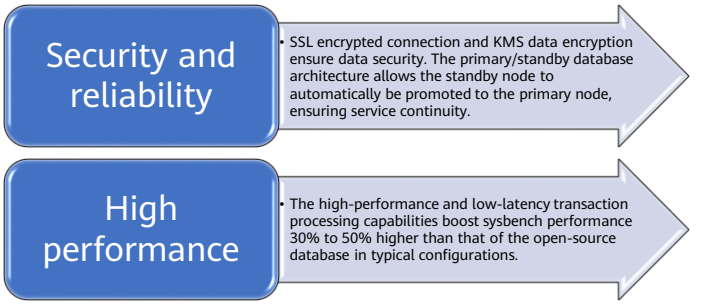
Customer Benefits

1. In-house independent and controllable capabilities are achieved.
2. A solid foundation is laid for continuous service expansion.
3. Data values are unleashed more effectively.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Financial Internet Transaction

- GaussDB is applicable to the Internet transaction systems of small- and medium-sized banks, such as mobile apps and websites. It is compatible with mainstream commercial database ecosystems, features high performance, security, and reliability, and is recommended to be deployed in active/standby mode.
- Advantages:



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-answer) In the GaussDB distributed architecture, which of the following components is used for sharing configuration and service discovery (service registration and search)?
   A. CN
   B. DN
   C. GTM
   D. ETCD
2. (Short-answer) What are the core functions of UGO for database and application migration?

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 1. D
- 2. Source database profiling, destination database selection and specification recommendation, destination database compatibility analysis, migration workload evaluation, database structure migration, and application SQL migration

Summary

- This chapter describes the positioning of GaussDB, as well as its key features, such as high performance, high availability, high security, and high scalability.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recommendations

- openGauss help document:
  - https://docs.opengauss.org/en/docs/6.0.0/docs/GettingStarted/GettingStarted.html
- openGauss code repository:
  - https://gitee.com/opengauss

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Acronyms and Abbreviations

Acronym and Abbreviation	Full Spelling
AZ	Availability Zone
CBO	Rule-Based Optimization
CM	Cluster Manager
CN	Coordinator Node
CTS	Cloud Trace Service
DAS	Data Admin Service
DBSS	Database Security Service
DN	Data Node
DRS	Data Replication Service
GTM	Global Transaction Manager
OM	Operation Manager
RBO	Rule-Based Optimization
SMP	Symmetric Multi-Processing
UGO	Database and Application Migration UGO

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. 2026.
All Rights Reserved.

Without prior written permission of Huawei, no one is allowed to reproduce, modify, integrate, adapt, translate this Document, or provide, sell, transfer, sublicense this Document or any part of this Document or its derivative to any third party for any other purpose otherwise permitted by Huawei.

HUAWEI